

Aprenda a Pensar Como un Programador

con Python

Aprenda a Pensar Como un Programador

con Python

Allen Downey
Jeffrey Elkner
Chris Meyers

Traducido por
Miguel Ángel Vilella
Ángel Arnal
Iván Juanes
Litza Amurrio
Efrain Andia
César Ballardini

Green Tea Press

Wellesley, Massachusetts

Copyright © 2002 Allen Downey, Jeffrey Elkner, y Chris Meyers.

Corregido por Shannon Turlington y Lisa Cutler.

Diseño de la cubierta por Rebecca Gimenez.

Green Tea Press
1 Grove St.
P.O. Box 812901
Wellesley, MA 02482

Se permite copiar, distribuir, y/o modificar este documento bajo los términos de la GNU Free Documentation License, Versión 1.1 o cualquier versión posterior publicada por la Free Software Foundation; siendo las Secciones Invariantes “Prólogo”, “Prefacio”, y “Lista de Colaboradores”, sin texto de cubierta, y sin texto de contracubierta. Se incluye una copia de la licencia en el apéndice titulado “GNU Free Documentation License”.

La GNU Free Documentation License está disponible en www.gnu.org o escribiendo a la Free Software Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA 02111-1307, USA.

La forma original de este libro es código fuente L^AT_EX. La compilación de este fuente L^AT_EX tiene el efecto de generar una representación independiente del dispositivo de un libro de texto, que puede convertirse a otros formatos e imprimirse.

El fuente L^AT_EX de este libro y más información sobre el proyecto de Libro de Texto de Código Abierto están disponibles en

<http://www.thinkpython.com>

La composición de este libro se realizó utilizando L^AT_EX y LyX. Las ilustraciones se hicieron con xfig. Todos ellos son programas gratuitos de código abierto.

Historia de la impresión:

Abril 2002: Primera edición.

ISBN 0-9716775-0-6

Prólogo

Por David Beazley

Como educador, investigador, y autor de libros, estoy encantado de ver la finalización de este libro. Python es un lenguaje de programación divertido y extremadamente fácil de usar que en los últimos años se ha hecho muy popular. Desarrollado hace diez años por Guido van Rossum, su sintaxis simple y la sensación general se deriva en gran parte del ABC, un lenguaje desarrollado en los 1980s para la enseñanza. Sin embargo, Python también se creó para resolver problemas reales y presenta una variedad amplia de características de lenguajes de programación como C++, Java, Modula-3 y Scheme. Debido a esto, una de las características notables de Python es su atractivo para los desarrolladores profesionales de programación, científicos, investigadores, artistas, y educadores.

A pesar del atractivo de Python para muchas comunidades diferentes, puede que aún se pregunte “¿por qué Python?” o “¿por qué enseñar programación con Python?” No es tarea fácil responder a estas preguntas, en especial cuando la opinión popular está del lado de alternativas más masoquistas como C++ y Java. Sin embargo, pienso que la respuesta más directa es que la programación en Python es simplemente más divertida y más productiva.

Cuando imparto cursos de informática, quiero cubrir conceptos importantes, hacer el material interesante y enganchar a los estudiantes. Desgraciadamente, hay una tendencia en los cursos de introducción a la programación a prestar demasiada atención a la abstracción matemática que hace que los estudiantes se frustren con problemas farragosos relacionados con detalles nimios de la sintaxis, compilación, y la aplicación de reglas aparentemente arcanas. Aunque tal abstracción y formalismo son importantes para ingenieros profesionales de la programación y estudiantes que planean continuar sus estudios de informática, decidirse por este enfoque en un curso introductorio sólo tiene éxito en hacer aburrida la informática. Cuando imparto un curso, no quiero tener un aula de estudiantes sin inspiración. Quisiera verlos intentando resolver problemas interesantes, explorando ideas diferentes, probando enfoques no convencionales,

rompiendo las reglas, y aprendiendo de sus errores. Al hacerlo, no quiero perder la mitad del semestre tratando de sortear problemas con una sintaxis abstrusa, mensajes de error del compilador incomprensibles, o los varios cientos de maneras que un programa puede generar un error de protección general.

Una de las razones por las que me gusta Python es por que proporciona un equilibrio muy bueno entre lo práctico y lo conceptual. Puesto que Python es un lenguaje interpretado, los principiantes pueden tomar el lenguaje y empezar a hacer cosas interesantes casi inmediato, sin perderse en los problemas de compilación y enlazado. Además, Python viene con una gran biblioteca de módulos que se pueden usar para hacer toda clase de tareas que abarcan desde programación para web a gráficos. Este enfoque práctico es una buena manera de enganchar a estudiantes y permite que completen proyectos significativos. Sin embargo, Python también puede servir como una base excelente para introducir conceptos importantes de informática. Puesto que Python soporta completamente procedimientos y clases, los estudiantes pueden introducirse gradualmente en temas como abstracción procedural, estructuras de datos, y programación orientada objetos, que son aplicables a cursos posteriores en Java o C++. Python incluso toma prestada cierta cantidad de características de lenguajes de programación funcionales y puede usarse para introducir conceptos que pudieran ser cubiertos en mas detalle en cursos de Scheme o Lisp.

Leendo, el prefacio de Jeffrey, me sorprenden sus comentarios sobre que Python le permite ver un “más alto nivel de éxito y un bajo nivel de frustración” que puede “avanzar rápido con mejores resultados”. Aunque estos comentarios se refieren a sus cursos introductorios, a veces uso Python por estas mismas razones en cursos de informática avanzada en la Universidad de Chicago. En estos cursos me enfrento constantemente con la desalentadora tarea de cubrir un montón de material difícil en un agotador trimestre de nueve semanas. Aunque es ciertamente posible para mí infligir mucho dolor y sufrimiento usando un lenguaje como C++, he visto a menudo que ese estilo es ineficaz, especialmente cuando el curso se trata de un asunto sin relación apenas con la “programación”. Encuentro que usar Python me permite dedicarme más al asunto en cuestión mientras permito a los estudiantes completar proyectos útiles.

Aunque Python es todavía un lenguaje joven y en desarrollo, creo que tiene un futuro brillante en la educación. Este libro es un paso importante en esa dirección.

David Beazley
Universidad de Chicago
Autor de *Python Essential Reference*

Prefacio

Por Jeff Elkner

Este libro debe su existencia a la colaboración hecha posible por la Internet y al movimiento de software libre. Sus tres autores, un profesor universitario, un profesor de instituto y un programador profesional, todavía tienen que conocerse cara a cara, pero hemos sido capaces de colaborar estrechamente y hemos recibido la ayuda de mucha gente maravillosa que han donado su tiempo y esfuerzo para ayudar a mejorar este libro.

Creemos que este libro es un testamento a los beneficios y futuras posibilidades de este tipo de colaboración, cuyo marco han establecido Richard Stallman y la Free Software Foundation.

Cómo y por qué vine a usar Python

En 1999, el examen de Advanced Placement (AP) de Ciencias de la Computación del Claustro Escolar se realizó por primera vez en C++. Como en muchos institutos en todo el país, la decisión de cambiar de lenguaje tuvo un impacto directo sobre el curriculum de informática en el Insituto de Yorktown en Arlington, Virginia, donde doy clase. Hasta ese momento, el lenguaje de enseñanza era Pascal tanto en nuestro curso de primer año como en el AP. Al seguir con la práctica anterior de dar a los estudiantes dos años de exposición al mismo lenguaje, tomamos la decisión de cambiar a C++ en el aula de primer año del curso 1997-98 de modo que estaríamos en sintonía con el cambio del Claustro Escolar para el curso AP del año siguiente.

Dos años más tarde, me convencí de que C++ era una mala elección para iniciar a los estudiantes en la informática. Aunque es un lenguaje de programación muy poderoso, también es extremadamente difícil de aprender y enseñar. Me encontré luchando constantemente con la difícil sintaxis de C++ y sus múltiples formas de hacer las cosas, y como consecuencia perdía muchos estudiantes sin

necesidad. Convencido de que debía de haber una elección mejor para el lenguaje de nuestro curso de primer año, me puse a buscar una alternativa para C++.

Necesitaba un lenguaje que funcionase tanto en las máquinas de nuestro laboratorio de Linux como en las plataformas Windows y Macintosh que la mayoría de los estudiantes tenían en casa. Quería que fuera de código abierto, para que los estudiantes pudieran usarlo en casa sin importar su nivel económico. Quería un lenguaje utilizado por programadores profesionales, y que tuviera una comunidad activa de desarrolladores a su alrededor. Tenía que soportar tanto la programación procedural como la orientada a objetos. Y lo más importante, tenía que ser fácil de aprender y de enseñar. Cuando investigué las opciones con estos objetivos en mente, Python destacó como el mejor candidato.

Pedí a uno de los estudiantes más talentosos de Yorktown, Matt Ahrens, que probase Python. En dos meses, no sólo había aprendido el lenguaje, sino que escribió una aplicación llamada pyTicket que permitía a nuestro personal informar de problemas tecnológicos via Web. Sabía que Matt no podía terminar una aplicación de tal escala en tan poco tiempo con C++, y este logro, combinado con la positiva valoración de Python por parte de Matt, sugería que Python era la solución que buscaba.

Encontrar un libro de texto

Una vez decidido a usar Python tanto en mis clases de informática básica como en el año siguiente, el problema más acuciante era la falta de un libro de texto disponible.

El contenido libre vino al rescate. Anteriormente en ese año, Richard Stallman me presentó a Allen Downey. Ambos habíamos escrito a Richard expresando nuestro interés en desarrollar contenidos educativos libres. Allen ya había escrito un libro de texto de informática de primer año, *How to Think Like a Computer Scientist*. Cuando leí ese libro, supe inmediatamente que quería usarlo en mi clase. Era el libro de informática más claro y práctico que había visto. Ponía el énfasis en los procesos de pensamiento involucrados en la programación más que en las características de un lenguaje en particular. Su lectura me hizo inmediatamente un maestro mejor.

How to Think Like a Computer Scientist no era sólo un libro excelente, sino que se publicó bajo la licencia pública GNU, lo que significaba que podía usarse y modificarse libremente para ajustarse a las necesidades de su usuario. Una vez que decidí usar Python, se me ocurrió que podría traducir la versión original en Java del libro de Allen al nuevo lenguaje. Aunque no hubiera sido capaz de escribir un libro de texto por mi cuenta, tener el libro de Allen para trabajar a

partir de él me hizo posible hacerlo, mostrando al mismo tiempo que el modelo cooperativo de desarrollo que tan buenos resultados había dado en el software podía funcionar también para el contenido educativo.

El trabajo en este libro durante los dos últimos años ha sido gratificante para mis estudiantes y para mí, y mis estudiantes desempeñaron un importante papel en el proceso. Como podía hacer cambios instantáneos cuando alguien encontraba un error ortográfico o un pasaje difícil, los animé a buscar errores en el libro dándoles un punto extra cada vez que hacían una sugerencia que terminaba como un cambio en el texto. Esto tuvo el doble beneficio de animarlos a leer el texto con más atención y tener el texto revisado en profundidad por sus críticos más importantes: los estudiantes que lo usan para aprender informática.

Para la segunda mitad del libro, acerca de la programación orientada a objetos, sabía que necesitaría a alguien con más experiencia real en programación de la que yo tenía para hacerlo bien. El libro se estancó en un estado inacabado durante buena parte de un año hasta que la comunidad de código abierto de nuevo proporcionó los medios necesarios para su terminación.

Recibí un correo electrónico de Chris Meyers expresando su interés en el libro. Chris es un programador profesional que empezó a impartir un curso de programación con Python el año pasado en el Colegio de Lane Community, en Eugene, Oregon. La perspectiva de impartir el curso llevó a Chris hasta el libro, y empezó a colaborar con él inmediatamente. Hacia el final del año escolar había creado un proyecto complementario en nuestro sitio web en <http://www.ibiblio.org/obp> llamado *Python for Fun* y estaba trabajando con algunos de mis estudiantes aventajados como profesor magistral, dirigiéndolos más allá de donde yo podía llevarles.

Presentando la programación con Python

El proceso de traducir y usar *How to Think Like a Computer Scientist* durante los dos últimos años ha confirmado la idoneidad de Python para enseñar a estudiantes principiantes. Python simplifica enormemente los ejemplos de programación y facilita la enseñanza de los conceptos importantes en programación.

El primer ejemplo del texto ilustra esta cuestión. Es el tradicional programa “hola, mundo”, que en la versión C++ del libro es así:

```
#include <iostream.h>

void main()
{
    cout << "Hola, mundo" << endl;
}
```

en la versión Python se convierte en:

```
print "Hola, Mundo"
```

Aunque es un ejemplo trivial, destacan las ventajas de Python. El curso de Informática I en Yorktown no tiene prerequisites, así que muchos de los estudiantes que ven este ejemplo están mirando su primer programa. Algunos de ellos están sin duda un poco nerviosos, tras haber oído que programar computadores es algo difícil de aprender. La versión C++ siempre me ha obligado a elegir entre dos opciones insatisfactorias: explicar las sentencias `#include`, `void main()`, `{`, `y` `}` y arriesgarme a confundir o intimidar a algunos estudiantes desde el principio, o decirles “No te preocupes de todo eso ahora, hablaremos de ello más tarde”, y arriesgarme a lo mismo. Los objetivos educativos en este momento del curso son exponer a los estudiantes a la idea de una sentencia de programación y llevarles a escribir su primer programa, presentándoles de esta forma el entorno de programación. La programación con Python tiene exactamente lo que necesito para hacer estas cosas, y nada más.

La comparación del texto explicativo de este programa para cada versión del libro ilustra mejor lo que esto significa para los estudiantes principiantes. Hay trece párrafos de explicación de “¡Hola, mundo!” en la versión C++. En la versión Python sólo hay dos. Aún más importante: los once párrafos que faltan no tocan las “grandes ideas” de la programación de computadores, sino las minucias de la sintaxis de C++. Encontré que esto mismo sucedía por todo el libro. Párrafos enteros desapareciendo de la versión Python del texto porque la sintaxis clara de Python los hace innecesarios.

El uso de un lenguaje de muy alto nivel como Python permite que el profesor deje para más tarde hablar sobre los detalles de bajo nivel de la máquina hasta que los estudiantes tengan el fondo necesario para entender los detalles. De este modo crea la habilidad de poner pedagógicamente “antes lo primero”. Uno de los mejores ejemplos de ello es la manera en la cual Python maneja las variables. En C++ una variable es un nombre para un lugar que contiene una cosa. Las variables deben declararse según su tipo en parte porque el tamaño del lugar al que apuntan tiene que determinarse de antemano. Así, la idea de una variable está ligada al hardware de la máquina. El concepto poderoso y fundamental de

lo que es una variable ya es suficientemente difícil para estudiantes principiantes (tanto de informática como de álgebra). Octetos y direcciones no ayudan a la comprensión. En Python una variable es un nombre que señala una cosa. Este es un concepto mucho más intuitivo para estudiantes principiantes y está más cerca del significado de “variable” que aprendieron en su clase de matemáticas. Este año tuve muchas menos dificultades enseñando lo que son las variables que en el anterior, y pasé menos tiempo ayudándoles con los problemas derivados de su uso.

Otro ejemplo de cómo Python ayuda en la enseñanza y aprendizaje de la programación es en su sintaxis para las funciones. Mis estudiantes siempre han tenido una gran dificultad comprendiendo las funciones. El problema principal se centra alrededor de la diferencia entre la definición de una función y la llamada a una función, y la distinción asociada entre un parámetro y un argumento. Python viene al rescate con una sintaxis a la que no le falta belleza. La definición de una función empieza con la palabra clave `def`, y simplemente digo a mis estudiantes: “cuando definas una función, empieza con `def`, seguido del nombre de la función que estés definiendo; cuando llames a una función, simplemente di (escribe) su nombre”. Los parámetros van con las definiciones; los argumentos con las llamadas. No hay tipo de retorno, tipos de parámetros, o parámetro por referencia y valor de por medio, por lo que ahora soy capaz de enseñar funciones en la mitad de tiempo que antes, con mejor comprensión.

El uso de Python ha mejorado la eficacia de nuestro programa de informática para todos los estudiantes. Veo un mayor nivel general de éxito y un menor nivel de frustración del que experimenté durante los dos años que enseñé C++. Avanzo más rápido con mejores resultados. Más estudiantes terminan el curso con la habilidad de crear programas útiles y con la actitud positiva hacia la experiencia de programación que esto engendra.

Formar una comunidad

He recibido correos electrónicos de todos los rincones del planeta de parte de gente que usa este libro para aprender o enseñar a programar. Ha empezando a surgir una comunidad de usuarios, y muchas personas han contribuido al proyecto mandando materiales a través del sitio web complementario <http://www.thinkpython.com>.

Con la publicación de este libro en forma impresa, espero que continúe y se acelere el crecimiento de la comunidad de usuarios. La emergencia de esta comunidad de usuarios y la posibilidad que sugiere para colaboraciones similares entre educadores han sido para mí las partes más excitantes de trabajar en este proyecto. Trabajando juntos, podemos incrementar la calidad de los materiales

disponibles para nuestro uso y ahorrar un tiempo valioso. Les invito a unirse a nuestra comunidad y espero con impaciencia saber algo de ustedes. Por favor, escriban a los autores a feedback@thinkpython.com.

Jeffrey Elkner
Escuela Secundaria Yortown
Arlington, Virginia

Lista de Colaboradores

Parafraseando la filosofía de la Free Software Foundation, este libro es libre como la libre expresión, pero no necesariamente gratis como la pizza gratis. Se hizo realidad a causa de una colaboración que no habría sido posible sin la GNU Free Documentation License. Así que queremos agradecer a la Free Software Foundation por desarrollar esta licencia y, por supuesto, ponerla a nuestra disposición.

También nos gustaría dar las gracias a los más de cien lectores de aguda vista que se han preocupado de enviarnos sugerencias y correcciones en los dos últimos años. Siguiendo el espíritu del software libre, decidimos expresar nuestra gratitud en la forma de una lista de colaboradores. Desgraciadamente, esta lista no está completa, pero hacemos lo que podemos para mantenerla actualizada.

Si se toma el tiempo de echar un vistazo a la lista, verá que cada una de las personas que aparecen le ha ahorrado a usted y a los lectores que le sucedan la confusión de un error técnico o una explicación poco clara simplemente enviándonos una nota.

Pos imposible que parezca tras tantas correcciones, todavía puede haber errores en el libro. Si se encontrara con una, esperamos que se tome un minuto para ponerse en contacto con nosotros. La dirección de correo es feedback@thinkpython.com. Si cambiamos algo a partir de su sugerencia, aparecerá en la siguiente versión de la lista de colaboradores (a no ser que pida quedar omitido). ¡Gracias!

- Lloyd Hugh Allen envió una corrección de la Sección 8.4.
- Yvon Boulianne envió una corrección de un error semántico en el Capítulo 5.
- Fred Bremmer comunicó una corrección de la Sección 2.1.
- Jonah Cohen escribió los scripts en Perl para convertir la fuente L^AT_EX del libro en hermoso HTML.

- Michael Conlon envió una corrección gramatical del Capítulo 2 y una mejora del estilo del Capítulo 1, e inició una discusión sobre aspectos técnicos de los intérpretes.
- Benoit Girard envió una corrección de un divertido error de la Sección 5.6.
- Courtney Gleason y Katherine Smith escribieron `horsebet.py`, que se usó como un caso de estudio en una versión temprana del libro. Su programa puede encontrarse en el sitio web.
- Lee Harr comunicó más correcciones de las que tenemos sitio para enumerar aquí, y de verdad debería aparecer como uno de los principales editores del texto.
- James Kaylin es un estudiante que usó el texto. Envío numerosas correcciones.
- David Kershaw arregló la función `catTwice` que no funcionaba en la Sección 3.10.
- Eddie Lam ha enviado numerosas correcciones de los Capítulos 1, 2 y 3. También arregló el Makefile de forma que crea un índice la primera vez que se ejecuta y nos ayudó a preparar un esquema de versiones.
- Man-Yong Lee envió una corrección del código de ejemplo de la Sección 2.4.
- David Mayo señaló que la palabra “unconscientemente” en el Capítulo 1 debía cambiarse por “subconscientemente”.
- Chris McAloon envió varias correcciones de las Secciones 3.9 y 3.10.
- Matthew J. Moelter ha sido un colaborador durante mucho tiempo y ha enviado numerosas correcciones y sugerencias.
- Simon Dicon Montford informó de una definición de función faltante y varios errores tipográficos en el Capítulo 3. También encontró errores en la función `incrementa` del Capítulo 13.
- John Ouzts corrigió la definición de “valor de retorno” del Capítulo 3.
- Kevin Parks envió valiosos comentarios y sugerencias acerca de cómo mejorar la distribución del libro.
- David Pool envió un error tipográfico en el glosario del Capítulo 1, y también amables palabras de ánimo.

- Michael Schmitt envió una corrección del Capítulo sobre archivos y excepciones.
- Robin Shaw señaló un error en la Sección 13.1, donde la función `imprimeHora` se usaba en un ejemplo sin haberla definido.
- Paul Sleigh encontró un error en el Capítulo 7 y un error en el script Perl de Jonah Cohen que, a partir de $\text{\LaTeX}$ genera, el HTML.
- Craig T. Snyder está poniendo a prueba el texto en un curso en la Universidad de Drew. Ha contribuido con varias sugerencias y correcciones de importancia.
- Ian Thomas y sus estudiantes usan el texto en un curso de programación. Son los primeros en probar los Capítulos de la segunda mitad del libro, y han hecho numerosas correcciones y sugerencias.
- Keith Verheyden envió una corrección del Capítulo 3.
- Peter Winstanley nos hizo saber de un persistente error en nuestro latín del Capítulo 3.
- Chris Wrobel hizo correcciones al código del Capítulo sobre E/S de archivos y excepciones.
- Moshe Zadka ha hecho contribuciones inestimables al proyecto. Además de escribir el primer borrador del Capítulo sobre diccionarios, proporcionó una guía continuada en las primeras etapas del libro.
- Christoph Zwerschke envió varias correcciones y sugerencias pedagógicas, y explicó la diferencia entre *gleich* y *selbe*.
- James Mayer envió un cargamento de errores tipográficos y ortográficos, incluyendo dos en la lista de colaboradores.
- Hayden McAfee pilló una inconsistencia potencialmente confusa entre dos ejemplos.
- Ángel Arnal es parte de un equipo internacional de traductores que trabajan en la versión en español del texto. También ha encontrado varios errores en la versión inglesa.
- Tauhidul Hoque y Lex Berezhny crearon las ilustraciones del Capítulo 1 y mejoraron muchas de las otras ilustraciones.
- Dr. Michele Alzetta pilló un error en el Capítulo 8 y envió varios comentarios y sugerencias pedagógicas interesantes sobre Fibonacci y La Mona.

- Andy Mitchell pilló un error tipográfico en el Capítulo 1 y un ejemplo erróneo en el Capítulo 2.
- Kalin Harvey sugirió una clarificación al Capítulo 7 y detectó varios errores tipográficos.
- Christopher P. Smith encontró varios errores tipográficos y nos está ayudando a preparar la actualización del libro para Python 2.2.
- David Hutchins pilló un error tipográfico en el Prólogo.
- Gregor Lingl enseña Python en un instituto de Viena, Austria. Está trabajando en una traducción del libro al alemán, y pilló un par de errores graves en el Capítulo 5.
- Julie Peters encontró un error tipográfico en el Prefacio.

Índice general

Prólogo	v
Prefacio	vii
Lista de Colaboradores	xiii
1. El Camino del Programa	1
1.1. El lenguaje de programación Python	1
1.2. ¿Qué es un programa?	3
1.3. ¿Qué es la depuración (debugging)?	4
1.4. Lenguajes formales y lenguajes naturales	6
1.5. El primer programa	8
1.6. Glosario	9
2. Variables, expresiones y sentencias	11
2.1. Valores y tipos	11
2.2. Variables	12
2.3. Nombres de variables y palabras reservadas	13
2.4. Sentencias	14
2.5. Evaluar expresiones	15
2.6. Operadores y expresiones	16

2.7.	El orden de las operaciones	17
2.8.	Las operaciones sobre cadenas	17
2.9.	Composición	18
2.10.	Los comentarios	19
2.11.	Glosario	19
3.	Funciones	21
3.1.	Llamadas a funciones	21
3.2.	Conversión de tipos	22
3.3.	Coerción de tipos	22
3.4.	Funciones matemáticas	23
3.5.	Composición	24
3.6.	Añadir funciones nuevas	24
3.7.	Las definiciones y el uso	26
3.8.	Flujo de ejecución	27
3.9.	Parámetros y argumentos	28
3.10.	Las variables y los parámetros son locales	29
3.11.	Diagramas de pila	30
3.12.	Funciones con resultado	31
3.13.	Glosario	32
4.	Condicionales y recursividad	35
4.1.	El operador módulo	35
4.2.	Expresiones booleanas	36
4.3.	Operadores lógicos	36
4.4.	Ejecución condicional	37
4.5.	Ejecución alternativa	37
4.6.	Condiciones encadenadas	38

4.7.	Condiciones anidadas	39
4.8.	La sentencia return	40
4.9.	Recursividad	40
4.10.	Diagramas de pila para funciones recursivas	42
4.11.	Recursividad infinita	43
4.12.	Entrada por teclado	44
4.13.	Glosario	45
5.	Funciones productivas	47
5.1.	Valores de retorno	47
5.2.	Desarrollo de programas	48
5.3.	Composición	51
5.4.	Funciones booleanas	52
5.5.	Más recursividad	53
5.6.	Acto de fe	55
5.7.	Un ejemplo más	56
5.8.	Comprobación de tipos	57
5.9.	Glosario	58
6.	Iteración	61
6.1.	Asignación múltiple	61
6.2.	La sentencia while	62
6.3.	Tablas	64
6.4.	Tablas de dos dimensiones	66
6.5.	Encapsulado y generalización	67
6.6.	Más encapsulación	68
6.7.	Variables locales	69
6.8.	Más generalización	70
6.9.	Funciones	71
6.10.	Glosario	72

7. Cadenas	75
7.1. Un tipo de datos compuesto	75
7.2. Longitud	76
7.3. Recorrido y el bucle for	76
7.4. Porciones de cadenas	78
7.5. Comparación de cadenas	78
7.6. Las cadenas son inmutables	79
7.7. Una función “encuentra”	80
7.8. Bucles y conteo	80
7.9. El módulo “string”	81
7.10. Clasificación de caracteres	82
7.11. Glosario	83
 8. Listas	 85
8.1. Valores de una lista	85
8.2. Acceso a los elementos	86
8.3. Longitud (tamaño) de una lista	87
8.4. Pertenencia a una lista	88
8.5. Listas y bucles for	88
8.6. Operaciones con listas	89
8.7. Porciones (slices)	90
8.8. Las listas son mutables	90
8.9. Borrado en una lista	91
8.10. Objetos y valores	91
8.11. Alias (poner sobrenombres)	92
8.12. Clonar listas	93
8.13. Listas como parámetros	94
8.14. Listas anidadas	95

8.15.	Matrices	95
8.16.	Cadenas y listas	96
8.17.	Glosario	97
9.	Tuplas	99
9.1.	Mutabilidad y tuplas	99
9.2.	Asignación de tuplas	100
9.3.	Tuplas como valor de retorno	101
9.4.	Números aleatorios	101
9.5.	Lista de números aleatorios	102
9.6.	Conteo	103
9.7.	Muchos baldes	104
9.8.	Una solución en una sola pasada	106
9.9.	Glosario	107
10.	Diccionarios	109
10.1.	Operaciones sobre diccionarios	110
10.2.	Métodos del diccionario	111
10.3.	Asignación de alias y copiado	112
10.4.	Matrices dispersas	112
10.5.	Pistas	113
10.6.	Enteros largos	115
10.7.	Contar letras	116
10.8.	Glosario	116
11.	Archivos y excepciones	119
11.1.	Archivos de texto	121
11.2.	Escribir variables	123
11.3.	Directorios	125

11.4. Encurtido	125
11.5. Excepciones	126
11.6. Glosario	128
12. Clases y objetos	131
12.1. Tipos compuestos definidos por el usuario	131
12.2. Atributos	132
12.3. Instancias como parámetro	133
12.4. Mismidad	134
12.5. Rectángulos	135
12.6. Instancias como valores de retorno	136
12.7. Los objetos son mudables	136
12.8. Copiado	137
12.9. Glosario	139
13. Clases y funciones	141
13.1. Hora	141
13.2. Funciones puras	142
13.3. Modificadores	143
13.4. ¿Qué es mejor?	144
13.5. Desarrollo de prototipos frente a planificación	145
13.6. Generalización	146
13.7. Algoritmos	146
13.8. Glosario	147
14. Clases y métodos	149
14.1. Características de la orientación a objetos	149
14.2. <code>imprimeHora</code>	150
14.3. Otro ejemplo	151

14.4. Un ejemplo más complicado	152
14.5. Argumentos opcionales	153
14.6. El método de inicialización	154
14.7. Revisión de los Puntos	155
14.8. Sobrecarga de operadores	156
14.9. Polimorfismo	158
14.10. Glosario	160
15. Conjuntos de objetos	161
15.1. Composición	161
15.2. Objetos <code>Carta</code>	161
15.3. Atributos de clase y el método <code>__str__</code>	163
15.4. Comparación de naipes	164
15.5. Mazos de naipes	165
15.6. Impresión del mazo de naipes	166
15.7. Barajar el mazo	167
15.8. Eliminación y reparto de los naipes	168
15.9. Glosario	169
16. Herencia	171
16.1. Herencia	171
16.2. Una mano de cartas	172
16.3. El reparto de los naipes	173
16.4. Mostremos la mano	174
16.5. La clase <code>JuegoDeCartas</code>	175
16.6. La clase <code>ManoDeLaMona</code>	176
16.7. La clase <code>JuegoDeLaMona</code>	177
16.8. Glosario	181

17.Listas enlazadas	183
17.1. Referencias incrustadas	183
17.2. La clase <code>Nodo</code>	183
17.3. Listas como colecciones	185
17.4. Listas y recursividad	186
17.5. Listas infinitas	187
17.6. Teorema fundamental de la ambigüedad	188
17.7. Modificar listas	189
17.8. Envoltorios y ayudantes	190
17.9. La clase <code>ListaEnlazada</code>	190
17.10. Invariantes	191
17.11. Glosario	192
18.Pilas	195
18.1. Tipos abstractos de datos	195
18.2. El TAD Pila	196
18.3. Cómo implementar pilas con listas de Python	196
18.4. Uso de <code>push</code> y <code>pop</code>	197
18.5. Usar una pila para evaluar postfijo	198
18.6. Análisis sintáctico	199
18.7. Evaluar un postfijo	199
18.8. Clientes y proveedores	200
18.9. Glosario	201
19.Colas	203
19.1. El TAD Cola	203
19.2. Cola Enlazada	204
19.3. Rendimiento típico	205

19.4. Cola Enlazada Mejorada	205
19.5. Cola priorizada	207
19.6. La clase <code>Golfista</code>	209
19.7. Glosario	210
20. Árboles	211
20.1. Crear árboles	212
20.2. Recorrer árboles	213
20.3. Árboles de expresión	213
20.4. Recorrido de un árbol	214
20.5. Construir un árbol de expresión	216
20.6. Manejar errores	220
20.7. El árbol de animales	221
20.8. Glosario	224
A. Depuración	225
A.1. Errores de sintaxis	225
A.2. Errores en tiempo de ejecución	227
A.3. Errores semánticos	231
B. Crear un nuevo tipo de datos	235
B.1. Multiplicación de fracciones	236
B.2. Suma de fracciones	237
B.3. Algoritmo de Euclides	238
B.4. Comparar fracciones	239
B.5. Forzando la máquina	240
B.6. Glosario	241

C. Listados Completos de Python	243
C.1. Clase Punto	243
C.2. Clase Hora	244
C.3. Cartas, mazos y juegos	245
C.4. Lists Enlazadas	249
C.5. Clase Pila	250
C.6. Colas y colas priorizadas	251
C.7. Árboles	253
C.8. Árboles de expresión	254
C.9. Adivina el animal	255
C.10. Fraction class	256
D. Lecturas recomendadas	259
D.1. Libros y sitios web sobre Python	260
D.2. Libros recomendados sobre informática en general	261
E. GNU Free Documentation License	263
E.1. Applicability and Definitions	264
E.2. Verbatim Copying	265
E.3. Copying in Quantity	265
E.4. Modifications	266
E.5. Combining Documents	268
E.6. Collections of Documents	269
E.7. Aggregation with Independent Works	269
E.8. Translation	269
E.9. Termination	270
E.10. Future Revisions of This License	270
E.11. Addendum: How to Use This License for Your Documents	270

Capítulo 1

El Camino del Programa

El objetivo de este libro es enseñarle a pensar como lo hacen los científicos informáticos. Esta manera de pensar combina las mejores características de la matemática, la ingeniería, y las ciencias naturales. Como los matemáticos, los científicos informáticos usan lenguajes formales para designar ideas (específicamente, computaciones). Como los ingenieros, ellos diseñan cosas, ensamblando sistemas a partir de componentes y evaluando ventajas y desventajas de cada una de las alternativas. Como los científicos, ellos observan el comportamiento de sistemas complejos, forman hipótesis, y prueban sus predicciones.

La habilidad más importante del científico informático es **la solución de problemas**. La solución de problemas incluye poder formular problemas, pensar en la solución de manera creativa, y expresar una solución con claridad y precisión. Como se verá, el proceso de aprender a programar es la oportunidad perfecta para desarrollar la habilidad de resolver problemas. Por esa razón este capítulo se llama “El Camino del programa”.

A cierto nivel, usted aprenderá a programar, lo cual es una habilidad muy útil por sí misma. A otro nivel, usted utilizará la programación para obtener algún resultado. Ese resultado se verá más claramente durante el proceso.

1.1. El lenguaje de programación Python

El lenguaje de programación que aprenderá es Python. Python es un ejemplar de un **lenguaje de alto nivel**; otros ejemplos de lenguajes de alto nivel son C, C++, Perl y Java.

Como se puede deducir de la nomenclatura “lenguaje de alto nivel”, también existen **lenguajes de bajo nivel**, a los que también se califica como lenguajes de máquina o lenguajes ensambladores. A propósito, los computadores sólo ejecutan programas escritos en lenguajes de bajo nivel. Los programas de alto nivel tienen que traducirse antes de ejecutarse. Esta traducción lleva tiempo, lo cual es una pequeña desventaja de los lenguajes de alto nivel.

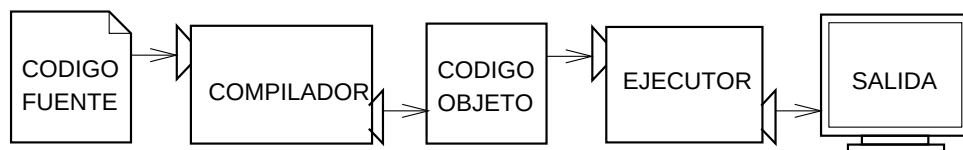
Aun así las ventajas son enormes. En primer lugar, la programación en lenguajes de alto nivel es mucho más fácil; escribir programas en un lenguaje de alto nivel toma menos tiempo, los programas son más cortos y más fáciles de leer, y es más probable que estos programas sean correctos. En segundo lugar, los lenguajes de alto nivel son **portables**, lo que significa que pueden ejecutarse en tipos diferentes de computadores sin modificación alguna o con pocas modificaciones. Los programas escritos en lenguajes de bajo nivel sólo pueden ser ejecutarse en un tipo de computador y deben reescribirse para ejecutarlos en otro.

Debido a estas ventajas, casi todos los programas se escriben en un lenguaje de alto nivel. Los lenguajes de bajo nivel sólo se usan para unas pocas aplicaciones especiales.

Hay dos tipos de programas que traducen lenguajes de alto nivel a lenguajes de bajo nivel: **intérpretes** y **compiladores**. Un intérprete lee un programa de alto nivel y lo ejecuta, lo que significa que lleva a cabo lo que indica el programa. Traduce el programa poco a poco, leyendo y ejecutando cada comando.



Un compilador lee el programa y lo traduce todo al mismo tiempo, antes de ejecutar cualquiera de las instrucciones. En este caso, al programa de alto nivel se le llama el **código fuente**, y al programa traducido el **código de objeto** o el **código ejecutable**. Una vez compilado el programa, puede ejecutarlo repetidamente sin volver a traducirlo.



Python se considera como lenguaje interpretado porque los programas de Python se ejecutan por medio de un intérprete. Existen dos maneras de usar el

intérprete: modo de comando y modo de guión. En modo de comando se escriben sentencias en el lenguaje Python y el intérprete muestra el resultado.

```
$ python
Python 1.5.2 (#1, Feb 1 2000, 16:32:16)
Copyright 1991-1995 Stichting Mathematisch Centrum, Amsterdam
>>> print 1 + 1
2
```

La primera línea de este ejemplo es el comando que pone en marcha el intérprete Python. Las dos líneas siguientes son mensajes del intérprete. La tercera línea comienza con `>>>`, que es la invitación del intérprete para indicar que está listo. Escribimos `print 1 + 1` y el intérprete contestó 2.

Alternativamente, se puede escribir el programa en un archivo y usar el intérprete para ejecutar el contenido de dicho archivo. El archivo se llama, en este caso, un **guión**. Por ejemplo, en un editor de texto se puede crear un archivo `latoya.py` que contenga esta línea:

```
print 1 + 1
```

Por acuerdo unánime, los archivos que contienen programas de Python tienen nombres que terminan con `.py`.

Para ejecutar el programa, se le tiene que indicar el nombre del guión al intérprete.

```
$ python latoya.py
2
```

En otros entornos de desarrollo los detalles de la ejecución de programas pueden ser diferentes. Además, la mayoría de programas son más interesantes que el mencionado.

La mayoría de ejemplos en este libro se ejecutan desde en la línea de comando. La línea de comando es muy apropiada para el desarrollo de programas y para pruebas rápidas porque se pueden teclear las instrucciones de Python y se pueden ejecutar inmediatamente. Una vez que un programa está completo, puede archivar en un guión para ejecutarlo o modificarlo en el futuro.

1.2. ¿Qué es un programa?

Un **programa** es una secuencia de instrucciones que especifican cómo ejecutar una computación. La computación puede ser algo matemático, como solucionar

un sistema de ecuaciones o determinar las raíces de un polinomio, pero también puede ser una computación simbólica, como buscar y reemplazar el texto de un documento o (aunque parezca raro) compilar un programa.

Las instrucciones (comandos, órdenes) tienen una apariencia diferente en lenguajes de programación diferentes, pero existen algunas funciones básicas que se presentan en casi todo lenguaje:

entrada: Recibir datos del teclado, o un archivo u otro aparato.

salida: Mostrar datos en el monitor o enviar datos a un archivo u otro aparato.

matemáticas: Ejecutar operaciones básicas de matemáticas como la adición y la multiplicación.

operación condicional: Probar la veracidad de alguna condición y ejecutar una secuencia de instrucciones apropiada.

repetición: Ejecutar alguna acción repetidas veces, normalmente con alguna variación.

Lo crea o no, eso es todo. Todos los programas que existen, por complicados que sean, están formulados exclusivamente con tales instrucciones. Así, una manera de describir la programación es: El proceso de romper una tarea en tareas cada vez más pequeñas hasta que estas tareas sean suficientemente simples para ser ejecutadas con una de estas instrucciones simples.

Quizás esta descripción sea un poco ambigua. No se preocupe. Lo explicaremos con más detalle con el tema de los **algoritmos**.

1.3. ¿Qué es la depuración (debugging)?

La programación es un proceso complejo y, por ser realizado por humanos, a menudo desemboca en errores. Por razones caprichosas, esos errores se llaman **bugs** y el proceso de buscarlos y corregirlos se llama **depuración** (en inglés “debugging”).

Hay tres tipos de errores que pueden ocurrir en un programa, de sintaxis, en tiempo de ejecución y semánticos. Es muy útil distinguirlos para encontrarlos mas rápido.

1.3.1. Errores sintácticos

Python sólo puede ejecutar un programa si el programa es correcto sintácticamente. En caso contrario, es decir si el programa no es correcto sintácticamente, el proceso falla y devuelve un mensaje de error. El término **sintaxis** se refiere a la estructura de cualquier programa y a las reglas de esa estructura. Por ejemplo, en español la primera letra de toda oración debe ser mayúscula, y todas las oraciones deben terminar con un punto. esta oración tiene un **error sintáctico**. Esta oración también

Para la mayoría de lectores, unos pocos errores sintácticos no son significativos, y por eso pueden leer la poesía de e. e. cummings sin anunciar errores de sintaxis. Python no es tan permisivo. Si hay aunque sea un solo error sintáctico en el programa, Python mostrará un mensaje de error y abortará la ejecución del programa. Durante las primeras semanas de su carrera como programador pasará, seguramente, mucho tiempo buscando errores sintácticos. Sin embargo, tal como adquiera experiencia tendrá menos errores y los encontrará mas rápido.

1.3.2. Errores en tiempo de ejecución

El segundo tipo de error es un error en tiempo de ejecución. Este error no aparece hasta que se ejecuta el programa. Estos errores también se llaman **excepciones** porque indican que algo excepcional (y malo) ha ocurrido.

Con los programas que vamos a escribir al principio, los errores en tiempo de ejecución ocurrirán con poca frecuencia, así que puede pasar bastante tiempo hasta que vea uno.

1.3.3. Errores semánticos

El tercer tipo de error es el **error semántico**. Si hay un error de lógica en su programa, el programa se ejecutará sin ningún mensaje de error, pero el resultado no será el deseado. Será cualquier otra cosa. Concretamente, el programa hará lo que usted le dijo.

A veces ocurre que el programa escrito no es el programa que se tenía en mente. El sentido o significado del programa (su semántica) no es correcto. Es difícil hallar errores de lógica, porque requiere trabajar al revés, observando el resultado del programa para averiguar lo que hace.

1.3.4. Depuración experimental

Una de las técnicas más importantes que usted aprenderá es la depuración. Aunque a veces es frustrante, la depuración es una de las partes más intelectualmente ricas, interesantes y estimulantes de la programación.

La depuración es una actividad parecida a la tarea de un investigador: se tienen que estudiar las claves para inducir los procesos y eventos llevaron a los resultados que tiene a la vista.

La depuración también es una ciencia experimental. Una vez que se tiene la idea de cuál es el error, se modifica el programa y se intenta nuevamente. Si su hipótesis fue la correcta se pueden predecir los resultados de la modificación y estará más cerca de un programa correcto. Si su hipótesis fue errónea tendrá que idearse otra hipótesis. Como dijo Sherlock Holmes, “Cuando se ha descartado lo imposible, lo que queda, no importa cuan inverosímil, debe ser la verdad.” (A. Conan Doyle, *The Sign of Four*)

Para algunas personas, la programación y la depuración son lo mismo: la programación es el proceso de depurar un programa gradualmente hasta que haga lo que usted quiera. La idea es que debería usted comenzar con un programa que haga **algo** y hacer pequeñas modificaciones, depurándolas sobre la marcha, de modo que siempre tenga un programa que funcione.

Por ejemplo, Linux es un sistema operativo que contiene miles de líneas de código, pero Linus Torvalds lo comenzó como un programa para explorar el microprocesador Intel 80836. Según Larry Greenfield, “Uno de los proyectos tempranos de Linus fue un programa que alternaba la impresión de AAAA con BBBB. Este programa evolucionó en Linux” (de *The Linux Users' Guide* Versión Beta 1).

Otros capítulos tratarán más acerca del tema de depuración y otras técnicas de programación.

1.4. Lenguajes formales y lenguajes naturales

Los **lenguajes naturales** son los lenguajes hablados por seres humanos, como el español, el inglés y el francés. No los han diseñados personas (aunque se intente poner cierto orden en ellos), sino que se han desarrollado naturalmente.

Los **lenguajes formales** son lenguajes diseñados por humanos y que tienen aplicaciones específicas. La notación matemática, por ejemplo, es un lenguaje formal ya que se presta a la representación de las relaciones entre números y símbolos. Los químicos utilizan un lenguaje formal para representar la estructura química de las moléculas. Y lo más importante:

Los lenguajes de programación son lenguajes formales desarrollados para expresar computaciones.

Los lenguajes formales casi siempre tienen reglas sintácticas estrictas. Por ejemplo, $3 + 3 = 6$ es una expresión matemática correcta, pero $3 = +6\$$ no lo es. De la misma manera, H_2O es una nomenclatura química correcta, pero $_2Zz$ no lo es.

Existen dos clases de reglas sintácticas, en cuanto a **unidades** y estructura. Las unidades son los elementos básicos de un lenguaje, como lo son las palabras, los números y los elementos químicos. Por ejemplo, en $3=+6\$$, $\$$ no es una unidad matemática aceptada (al menos hasta donde nosotros sabemos). Similarmente, $_2Xx$ no es formal porque no hay ningún elemento con la abreviatura Zz .

La segunda clase de regla sintáctica está relacionada con la estructura de un elemento; o sea, el orden de las unidades. La estructura de la sentencia $3=+6\$$ no se acepta porque no se puede escribir el símbolo de igualdad seguido de un símbolo positivo. Similarmente, las fórmulas moleculares tienen que mostrar el número de subíndice después del elemento, no antes.

A manera de práctica, trate de producir una oración con estructura aceptada pero que esté compuesta de unidades irreconocibles. Luego escriba otra oración con unidades aceptables pero con estructura no válida.

Al leer una oración, sea en un lenguaje natural o una sentencia en un lenguaje técnico, se debe discernir la estructura de la oración. En un lenguaje natural este proceso, llamado **análisis sintáctico** ocurre subconscientemente.

Por ejemplo cuando usted escucha la oración “El otro zapato cayó”, entiende que “el otro zapato” es el sujeto y “cayó” es el verbo. Cuando se ha analizado la oración sintácticamente, se puede deducir el significado, o la semántica, de la oración. Suponiendo que sepa lo que es un zapato y lo que es caer, entenderá el significado de la oración.

Aunque existen muchas cosas en común entre los lenguajes naturales y los lenguajes formales—por ejemplo las unidades, la estructura, la sintaxis y la semántica—también existen muchas diferencias:

ambigüedad: Los lenguajes naturales tienen muchísimas ambigüedades, que los hablantes sortean usando claves contextuales y otra información. Los lenguajes formales se diseñan para estar completamente libres de ambigüedades o tanto como sea posible, lo que quiere decir que cualquier sentencia tiene sólo un significado, sin importar el contexto.

redundancia: Para reducir la ambigüedad y los malentendidos, las lenguas naturales utilizan bastante redundancia. Como resultado suelen ser prolijos. Los lenguajes formales son menos redundantes y más concisos.

literalidad: Los lenguajes naturales tienen muchas metáforas y frases hechas. El significado de un dicho, por ejemplo “Estirar la pata”, es diferente al significado de sus sustantivos y verbos. En este ejemplo, la oración no tiene nada que ver con un pie y significa ‘morirse’. Los lenguajes formales no difieren de su significado literal.

Los que aprenden a hablar un lenguaje natural—es decir, todo el mundo—muchas veces tienen dificultad en adaptarse a los lenguajes formales. A veces la diferencia entre los lenguajes formales y los naturales es comparable a la diferencia entre la prosa y la poesía:

Poesía: Se utiliza una palabra por su cualidad auditiva tanto como por su significado. El poema, en su totalidad, produce un efecto o reacción emocional. La ambigüedad no es solo común sino utilizada a propósito.

Prosa: El significado literal de la palabra es más importante y la estructura da más significado aún. La prosa se presta al análisis más que la poesía, pero todavía contiene ambigüedad.

Programas: El significado de un programa es inequívoco y literal, y es entendido en su totalidad analizando las unidades y la estructura.

He aquí unas sugerencias para la lectura de un programa (y de otros lenguajes formales). Primero, recuerde que los lenguajes formales son mucho más densos que los lenguajes naturales, y por consiguiente lleva más tiempo leerlos. También, la estructura es muy importante, así que entonces no es una buena idea leerlo de pies a cabeza, de izquierda a derecha. En vez de eso, aprenda a separar las diferentes partes en su mente, identificar las unidades e interpretar la estructura. Finalmente, ponga atención a los detalles. Los fallos de puntuación y la ortografía, que puede obviar en el lenguaje natural, pueden suponer una gran diferencia en un lenguaje formal.

1.5. El primer programa

Tradicionalmente el primer programa en un lenguaje nuevo se llama “Hola, mundo” (Hello world!) porque sólo muestra las palabras “Hola a todo el mundo”. En Python es así:

```
print "Hola, mundo"
```

Este es un ejemplo de una **sentencia print**, la cual no imprime nada en papel, más bien muestra un valor. En este caso, el resultado es las palabras

```
Hola, mundo
```

Las comillas señalan el comienzo y el final del valor; no aparecen en el resultado.

Alguna gente evalúa la calidad de un lenguaje de programación por la simplicidad del programa “Hola, mundo”. Si seguimos ese criterio, Python cumple con todas sus metas.

1.6. Glosario

solución de problemas: El proceso de formular un problema, hallar la solución y expresar esa solución.

lenguaje de alto nivel: Un lenguaje como Python diseñado para ser fácil de leer y escribir para la gente.

lenguaje de bajo nivel: Un lenguaje de programación diseñado para ser fácil de ejecutar para un computador; también se lo llama “lenguaje de máquina” o “lenguaje ensamblador”.

portabilidad: La cualidad de un programa que le permite ser ejecutado en más de un tipo de computador.

interpretar: Ejecutar un programa escrito en un lenguaje de alto nivel traduciéndolo línea por línea

compilar: Traducir un programa escrito en un lenguaje de alto nivel a un lenguaje de bajo nivel todo al mismo tiempo, en preparación para la ejecución posterior.

código fuente: Un programa escrito en un lenguaje de alto nivel antes de ser compilado.

código de objeto: La salida del compilador una vez que ha traducido el programa.

programa ejecutable: Otro nombre para el código de objeto que está listo para ejecutarse.

guión: Un programa archivado (que va a ser interpretado).

programa: Un conjunto de instrucciones que especifica una computación.

algoritmo: Un proceso general para resolver una clase completa de problemas.

error (bug): Un error en un programa.

depuración: El proceso de hallazgo y eliminación de los tres tipos de errores de programación.

sintaxis: La estructura de un programa.

error sintáctico: Un error en un programa que hace que el programa sea imposible de analizar sintácticamente (e imposible de interpretar).

error en tiempo de ejecución: Un error que no ocurre hasta que el programa ha comenzado a ejecutarse e impide que el programa continúe.

excepción: Otro nombre para un error en tiempo de ejecución.

error semántico: Un error en un programa que hace que ejecute algo que no era lo deseado.

semántica: El significado de un programa.

language natural: Cualquier lenguaje hablado que evolucionó de forma natural.

lenguaje formal: Cualquier lenguaje diseñado por humanos que tiene un propósito específico, como la representación de ideas matemáticas o programas de computadores; todos los lenguajes de programación son lenguajes formales.

unidad: Uno de los elementos básicos de la estructura sintáctica de un programa, análogo a una palabra en un lenguaje natural.

análisis sintáctico: La examinación de un programa y el análisis de su estructura sintáctica.

sentencia print: Una instrucción que causa que el intérprete Python muestre un valor en el monitor.

Capítulo 2

Variables, expresiones y sentencias

2.1. Valores y tipos

El **valor** es uno de los elementos fundamentales (como por ejemplo una letra o un número) que manipula un programa. Los valores que hemos visto hasta el momento son 2 (el resultado de sumar $1 + 1$) y `Hola, mundo`.

Estos valores son de distintos **tipos**: 2 es un entero y `Hola, mundo` es una **cadena**, llamada así porque contiene una “cadena” de letras. Usted (y el intérprete) puede identificar las cadenas porque están encerradas entre comillas.

La sentencia `print` también funciona con enteros:

```
>>> print 4
4
```

Si no está seguro del tipo que tiene un determinado valor, puede preguntárselo al intérprete de Python.

```
>>> type("Hola, mundo")
<type 'string'>
>>> type(17)
<type 'int'>
```

No es sorprendente que las cadenas sean de tipo `string` (cadena en inglés) y los enteros sean de tipo `int` (por *integer* en inglés). De forma menos obvia, los

números con decimales (separados por medio de un punto en inglés) son de tipo **float** debido a la representación de estos números en el formato llamado **de coma flotante** (floating-point).

```
>>> type(3.2)
<type 'float'>
```

¿Qué ocurre con los valores como "17" y "3.2"? Parecen números, pero están entre comillas como las cadenas.

```
>>> type("17")
<type 'string'>
>>> type("3.2")
<type 'string'>
```

Son cadenas.

Cuando escriba un entero largo, podría estar tentado de usar comas entre grupos de tres dígitos, como en 1,000,000. Éste no es un entero legal en Python, pero es una expresión legal:

```
>>> print 1,000,000
1 0 0
```

En fin, no era eso lo que queríamos. Python interpreta 1,000,000 como una lista de tres números que debe imprimir. Así que recuerde no insertar comas en sus enteros.¹

2.2. Variables

Una de las características más potentes de los lenguajes de programación es la capacidad de manipular **variables**. Una variable es un nombre que hace referencia a un valor.

La **sentencia de asignación** crea nuevas variables y les asigna un valor:

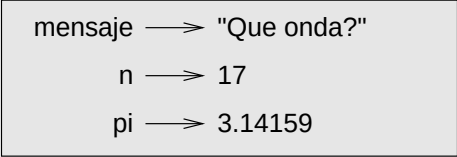
```
>>> mensaje = "Que onda?"
>>> n = 17
>>> pi = 3.14159
```

Este ejemplo muestra tres asignaciones. La primera de ellas asigna el valor "Que onda?" a una variable nueva, de nombre **mensaje**. La segunda le da el

¹El uso de la coma y el punto en número es en inglés el contrario al uso español, como se apuntó en una nota anterior

valor entero 17 a `n`, y la tercera le da el valor de número en coma flotante 3.14159 a `pi`.

Una forma habitual de representar variables sobre el papel es escribir el nombre con una flecha señalando al valor de la variable. Este tipo de representación se llama **diagrama de estado**, ya que muestra en qué estado se halla cada una de las variables (considérelo como el “estado de ánimo” de la variable). El siguiente diagrama muestra el efecto de las tres sentencias de asignación anteriores:



```
mensaje —> "Que onda?"  
n —> 17  
pi —> 3.14159
```

La sentencia `print` también funciona con variables.

```
>>> print mensaje  
"Que onda?"  
>>> print n  
17  
>>> print pi  
3.14159
```

En cada caso, el resultado es el valor de la variable. Las variables también tienen tipo. De nuevo, podemos preguntar al intérprete lo que son.

```
>>> type(mensaje)  
<type 'string'>  
>>> type(n)  
<type 'int'>  
>>> type(pi)  
<type 'float'>
```

El tipo de una variable es el tipo del valor al que se refiere.

2.3. Nombres de variables y palabras reservadas

Como norma general, los programadores eligen nombres significativos para sus variables: esto permite documentar para qué se usa la variable.

Los nombres de las variables pueden tener una longitud arbitraria. Pueden estar formados por letras y números, pero deben comenzar con una letra. Aunque es

aceptable usar mayúsculas, por convención no lo hacemos. Si lo hace, recuerde que la distinción es importante: **Bruno** y **bruno** son dos variables diferentes.

El guión bajo (`_`) también es legal y se utiliza a menudo para separar nombres con múltiples palabras, como `mi_nombre` o `precio_del_cafe_colombiano`.

Si intenta darle a una variable un nombre ilegal, obtendrá un error de sintaxis.

```
>>> 76trombones = "gran desfile"
SyntaxError: invalid syntax
>>> mas$ = 1000000
SyntaxError: invalid syntax
>>> class = "Curso de Programación 101"
SyntaxError: invalid syntax
```

`76trombones` es ilegal porque no comienza por una letra. `mas$` es ilegal porque contiene un carácter ilegal, el signo del dólar. Pero ¿qué tiene de malo `class`?

Resulta que `class` es una de las **palabras reservadas** de Python. El lenguaje usa las palabras reservadas para definir sus reglas y estructura, y no pueden usarse como nombres de variables.

Python tiene 28 palabras reservadas:

<code>and</code>	<code>continue</code>	<code>else</code>	<code>for</code>	<code>import</code>	<code>not</code>	<code>raise</code>
<code>assert</code>	<code>def</code>	<code>except</code>	<code>from</code>	<code>in</code>	<code>or</code>	<code>return</code>
<code>break</code>	<code>del</code>	<code>exec</code>	<code>global</code>	<code>is</code>	<code>pass</code>	<code>try</code>
<code>class</code>	<code>elif</code>	<code>finally</code>	<code>if</code>	<code>lambda</code>	<code>print</code>	<code>while</code>

Tal vez quiera mantener esta lista a mano. Si el intérprete se queja de alguno de sus nombres de variable, y usted no sabe por qué, compruebe si está en esta lista.

2.4. Sentencias

Una sentencia es una instrucción que puede ejecutar el intérprete de Python. Hemos visto dos tipos de sentencias: `print` y la asignación.

Cuando usted escribe una sentencia en la línea de comandos, Python la ejecuta y muestra el resultado, si lo hay. El resultado de una sentencia `print` es un valor. Las sentencias de asignación no entregan ningún resultado.

Normalmente un guión contiene una secuencia de sentencias. Si hay más de una sentencia, los resultados aparecen de uno en uno tal como se van ejecutando las sentencias.

Por ejemplo, el guión

```
print 1
x = 2
print x
```

presenta la salida

```
1
2
```

De nuevo, la sentencia de asignación no produce ninguna salida.

2.5. Evaluar expresiones

Una expresión es una combinación de valores, variables y operadores. Si teclea una expresión en la línea de comandos, el intérprete la **evalúa** y muestra el resultado:

```
>>> 1 + 1
2
```

Un valor, y también una variable, se considera una expresión por sí mismo.

```
>>> 17
17
>>> x
2
```

Para complicar las cosas, evaluar una expresión no es del todo lo mismo que imprimir un valor.

```
>>> mensaje = "Que onda?"
>>> mensaje
"Que onda?"
>>> print mensaje
Que onda?
```

Cuando Python muestra el valor de una expresión, usa el mismo formato que usted usaría para introducir un valor. En el caso de las cadenas, eso significa que incluye las comillas. Pero la sentencia `print` imprime el valor de la expresión, lo que en este caso es el contenido de la cadena.

En un guión, una expresión sin más es una sentencia válida, pero no hace nada. El guión

```
17
3.2
"Hola, mundo"
1 + 1
```

no presenta ninguna salida. ¿Cómo cambiaría usted el guión para mostrar los valores de estas cuatro expresiones?

2.6. Operadores y expresiones

Los **operadores** son símbolos especiales que representan cálculos simples, como la suma y la multiplicación. Los valores que usa el operador se llaman **operandos**.

Las siguientes expresiones son legales en Python y su significado es más o menos claro:

```
20+32   hora-1   hora*60+minuto   minuto/60   5**2   (5+9)*(15-7)
```

Los símbolos `+`, `-`, `/`, y el uso de los paréntesis para el agrupamiento, se usan todos de la misma forma que en matemáticas. El asterisco (`*`) es el signo de multiplicación y `**` el símbolo para exponenciación.

Cuando aparece el nombre de una variable en el lugar de un operando, se sustituye con su valor antes de realizar la operación.

La suma, resta, multiplicación y exponenciación hacen lo esperado, pero la división le puede sorprender. La operación que sigue tiene un resultado inesperado:

```
>>> minuto = 59
>>> minuto/60
0
```

El valor de la variable `minuto` es 59, y 59 dividido entre 60 es 0.98333 y no 0. El motivo de la discrepancia reside en que Python está llevando a cabo una **división de enteros**.

Cuando ambos operandos son enteros, el resultado ha de ser también un entero; por convención, la división de enteros siempre se redondea *a la baja*, incluso en casos como estos en los que el siguiente entero está muy próximo.

Una alternativa posible en este caso es el cálculo de un porcentaje y no el de una fracción:

```
>>> minuto*100/60
98
```

De nuevo se redondea el resultado a la baja, pero al menos ahora la respuesta es aproximadamente correcta. Otra alternativa es la división de coma flotante, que veremos en el Capítulo 3.

2.7. El orden de las operaciones

Cuando aparece más de un operador en una expresión, el orden de evaluación depende de las **reglas de precedencia**. Python sigue las mismas reglas de precedencia que los propios matemáticos para sus operaciones matemáticas. Los ingleses usan el acrónimo PEMDAS como regla para recordar el orden de las operaciones:

- **Paréntesis**: tienen la precedencia más alta y pueden usarse para forzar que una expresión se evalúe en el orden que queramos nosotros. Puesto que las expresiones entre paréntesis se evalúan primero, $2 * (3-1)$ es igual a 4, y $(1+1)**(5-2)$ es igual a 8. También puede usar paréntesis para que una expresión sea más legible; por ejemplo $(\text{minuto} * 100) / 60$, aunque el resultado no cambie de todas formas.
- **Exponenciación** tiene la siguiente precedencia más alta; así pues $2**1+1$ es igual a 3 y no a 4, y $3*1**3$ es igual a 3 y no a 27.
- La **Multiplicación** y la **División** tienen la misma precedencia, que es más alta que la de la **Adición** y la **Sustracción**, que tienen también la misma precedencia. Por tanto $2*3-1$ devuelve 5 y no 4, y $2/3-1$ da -1, y no 1 (recuerde que en la división de enteros $2/3$ da 0).
- Los operadores que tienen la misma precedencia se evalúan de izquierda a derecha. Así, en la expresión $\text{minuto}*100/60$, tiene lugar primero la multiplicación, devolviendo $5900/60$, que a su vez da como resultado 98. Si las operaciones se hubiesen realizado de derecha a izquierda, el resultado habría sido $59/1$ que da 59, y que es incorrecto.

2.8. Las operaciones sobre cadenas

En general no es posible realizar operaciones matemáticas con cadenas, incluso si las cadenas parecen números. Las siguientes sentencias son ilegales (suponiendo que `mensaje` sea de tipo `string`)

```
mensaje-1    "Hola"/123    mensaje*"Hola"    "15"+2
```

Es curioso que el operador `+` funcione con cadenas, aunque no haga exactamente lo que usted esperaría. Para cadenas, el operador `+` representa la **concatenación**, lo que significa que se unen los dos operandos uniéndolos extremo con extremo. Por ejemplo:

```
fruta = "plátano"
bizcochoBueno = " pan de leche"
print fruta + bizcochoBueno
```

La salida del programa es `plátano pan de leche`. El espacio delante de `pan` es parte de la cadena, y es necesario para introducir el espacio que separa las cadenas concatenadas.

El operador `*` también funciona con cadenas; lleva a cabo la **repetición**. Por ejemplo `'Chiste'*3` es `'ChisteChisteChiste'`. Uno de los operandos ha de ser una cadena, el otro ha de ser un entero.

Por un lado, esta interpretación de `+` y `*` cobra sentido por analogía con la suma y la multiplicación. Igual que `4*3` es equivalente a `4+4+4`, esperamos que `'Chiste'*3` sea lo mismo que `'Chiste'+'Chiste'+'Chiste'`, y así es. Por otro lado, la concatenación y la repetición son de alguna manera muy diferentes de la adición y la multiplicación de enteros. ¿Puede encontrar una propiedad que tienen la suma y la multiplicación de enteros y que no tengan la concatenación y la repetición de cadenas?

2.9. Composición

Hasta ahora hemos examinado los elementos de un programa (variables, expresiones y sentencias) por separado, sin hablar de cómo combinarlos.

Una de las características más útiles de los lenguajes de programación es su capacidad de tomar pequeños bloques de construcción y **ensamblarlos**. Por ejemplo, sabemos cómo sumar números y cómo imprimirlos; resulta que podemos hacer las dos cosas a un tiempo:

```
>>> print 17 + 3
20
```

En realidad, no debemos decir “al mismo tiempo”, puesto que en realidad la suma tiene que realizarse antes de la impresión, pero la cuestión es que cualquier expresión relacionada con números, cadenas y variables puede usarse dentro de una sentencia `print`. Ya hemos visto un ejemplo de ello:

```
print "Número de minutos desde la medianoche: ", hora*60+minuto
```

Y también puede poner expresiones arbitrarias en la parte derecha de una sentencia de asignación:

```
porcentaje = (minuto * 100) / 60
```

Esta capacidad puede no resultar muy sorprendente, pero ya veremos otros ejemplos donde la composición hace posible expresar cálculos complejos con limpieza y brevedad.

ATENCIÓN: Hay límites al lugar donde pueden usarse ciertas expresiones. Por ejemplo, la parte izquierda de una sentencia de asignación tiene que ser un nombre de *variable*, no una expresión. Por tanto es ilegal lo siguiente: `minute+1 = hour`.

2.10. Los comentarios

Conforme los programas van creciendo de tamaño y complicándose, se vuelven más complicados de leer. Los lenguajes formales son densos y con frecuencia es difícil observar un trozo de código y averiguar lo que hace, o por qué lo hace.

Por ello es una buena idea añadir notas a su programa que expliquen, en un lenguaje natural, qué hace el programa. Estas notas se llaman **comentarios** y se marcan con el símbolo `#`:

```
# calcula el porcentaje de la hora que ha pasado ya
porcentaje = (minuto * 100) / 60
```

En este caso, el comentario aparece en una línea propia. También puede poner comentarios al final de otras líneas:

```
porcentaje = (minuto * 100) / 60      # ojo: división de enteros
```

Todo lo que va del `#` al final de la línea se ignora (no tiene efecto sobre el programa). El mensaje está destinado al programador, o a futuros programadores que podrían tener que usar el código. En este caso avisa al lector sobre el sorprendente comportamiento de la división de enteros.

2.11. Glosario

valor: un número o cadena (o cualquier otra cosa que se especifique posteriormente) que puede almacenarse en una variable o calcularse en una expresión.

tipo: un conjunto de valores. El tipo de un valor determina cómo puede usarse en las expresiones. Hasta ahora, los tipos que hemos visto son enteros (tipo `int`), números de coma flotante (tipo `float`) y cadenas (tipo `string`).

coma flotante: un formato para representar números con decimales.

variable: nombre que hace referencia a un valor.

sentencia: es una porción de código que representa una orden o acción. Hasta ahora, las sentencias que hemos vistos son las asignaciones y las sentencias `print`.

asignación: sentencia que asigna un valor a una variable.

diagrama de estado: representación gráfica de un conjunto de variables y de los valores a los que se refiere.

palabra reservada: es una palabra clave que usa el compilador para analizar sintácticamente los programas. No pueden usarse palabras reservadas, por ejemplo `if`, `def` y `while` como nombres de variables.

operador: un símbolo especial que representa un cálculo sencillo, como la suma, la multiplicación o la concatenación de cadenas.

operando: uno de los valores sobre los que actúa un operador.

expresión: una combinación de variables, operadores y valores. Dicha combinación representa un único valor como resultado.

evaluar: simplificar una expresión ejecutando las operaciones para entregar un valor único.

división de enteros: es una operación que divide un entero entre otro y devuelve un entero. La división de enteros devuelve sólo el número entero de veces que el numerador es divisible por el denominador, y descarta el resto.

reglas de precedencia: la serie de reglas que especifican el orden en el que las expresiones con múltiples operadores han de evaluarse.

concatenar: unir dos operandos extremo con extremo.

composición: la capacidad de combinar expresiones sencillas y sentencias hasta crear sentencias y expresiones compuestas, con el fin de representar cálculos complejos de forma concisa.

comentario: un segmento de información en un programa, destinado a otros programadores (o cualquiera que lea el código fuente) y que no tiene efecto sobre la ejecución del programa.

Capítulo 3

Funciones

3.1. Llamadas a funciones

Ya hemos visto un ejemplo de una **llamada a una función**:

```
>>> type("32")
<type 'string'>
```

El nombre de la función es **type**, y muestra el tipo de un valor o de una variable. El valor o variable, llamado el **argumento** de la función, ha de estar encerrado entre paréntesis. Es habitual decir que una función “toma” un argumento y “devuelve” un resultado. El resultado se llama **valor de retorno**.

En lugar de imprimir el valor de retorno, podemos asignárselo a una variable.

```
>>> nereida = type("32")
>>> print nereida
<type 'string'>
```

Otro ejemplo más: la función **id** toma como argumento un valor o una variable y devuelve un entero que actúa como identificador único de ese valor.

```
>>> id(3)
134882108
>>> yanira = 3
>>> id(yanira)
134882108
```

Cada valor tiene un **id**, que es un valor único relacionado con dónde se almacena en la memoria del computador. El **id** de una variable es el **id** del valor al que hace referencia.

3.2. Conversión de tipos

Python proporciona una colección de funciones internas que convierten valores de un tipo a otro. La función `int` toma un valor y lo convierte a un entero, si es posible, o da un error si no es posible.

```
>>> int("32")
32
>>> int("Hola")
ValueError: invalid literal for int(): Hola
```

`int` también convierte valores de coma flotante a enteros, pero recuerde que siempre redondea hacia abajo:

```
>>> int(3.99999)
3
```

La función `float` que convierte enteros y cadenas en números en coma flotante:

```
>>> float(32)
32.0
>>> float("3.14159")
3.14159
```

Finalmente, está la función `str`, que convierte a tipo `string`:

```
>>> str(32)
'32'
>>> str(3.14149)
'3.14149'
```

Pudiera parecer extraño que Python distinga entre el valor entero 1 y el valor de coma flotante 1.0. Tal vez representen el mismo número, pero pertenecen a tipos distintos. El motivo es que se representan de forma distinta dentro del computador.

3.3. Coerción de tipos

Ahora que ya sabemos convertir entre tipos, tenemos otra forma de enfrentarnos a la división de enteros. Volviendo al ejemplo del capítulo anterior, suponga que queremos calcular qué fracción de una hora había transcurrido. La expresión más obvia, `minuto / 60`, realiza una división de enteros, por lo que el resultado es siempre 0, incluso 59 minutos después de la hora.

Una alternativa es convetir `minuto` a tipo `float` (coma flotante) y luego efectuar una división de coma flotante:


```
>>> minuto = 59
>>> float(minuto) / 60.0
0.983333333333
```

O bien podemos sacar provecho de las reglas de la conversión automática de tipos, llamada **coerción** de tipos. Para los operadores matemáticos, si uno de los operandos matemáticos es tipo `float`, el otro se convierte automáticamente en `float`.

```
>>> minuto = 59
>>> minuto / 60.0
0.983333333333
```

Al usar un denominador que es `float`, obligamos a Python a hacer división de coma flotante.

3.4. Funciones matemáticas

Es posible que ya haya visto usted en matemáticas funciones como `sin` (seno) y `log`, y que haya aprendido a evaluar expresiones como `sin(pi/2)` y `log(1/x)`. Primero evalúa la expresión entre paréntesis, (el argumento). Por ejemplo, `pi/2` es aproximadamente 1.571, y `1/x` es 0.1 (si `x` es igual a 10.0).

Luego evalúa la función en sí misma, bien mirándola en una tabla, bien llevando a cabo diversos cálculos. El `sin` (seno) de 1.571 es 1, y el `log` de 0.1 es -1 (suponiendo que `log` indique el logaritmo de base 10).

Este proceso puede aplicarse repetidamente para evaluar expresiones más complicadas como `log(1/sin(pi/2))`. Primero evaluamos el argumento de la función más interna, luego se evalúa la función, y así sucesivamente.

Python dispone de un módulo matemático que proporciona la mayoría de las funciones matemáticas habituales. Un **módulo** es un archivo que contiene una colección de funciones agrupadas juntas.

Antes de poder usar las funciones de un módulo, tenemos que importarlo:

```
>>>import math
```

Para llamar a una de las funciones, tenemos que especificar el nombre del módulo y el nombre de la función, separados por un punto. A esto se le llama notación de punto:

```
decibelio = math.log10 (17.0)
angulo = 1.5
altura = math.sin(angulo)
```

La primera sentencia da a **decibelio** el valor del logaritmo de 17, en base 10. Hay también una función llamada **log** que toma logaritmos en base **e**.

La tercera sentencia halla el seno del valor de la variable **angulo**. **sin** y las otras funciones trigonométricas (**cos**, **tan**, etc.) toman sus argumentos en *radianes*. Para convertir de grados a radianes, puede dividir por 360 y multiplicar por **2*pi**. Por ejemplo, para hallar el seno de 45 grados, calcule primero el ángulo en radianes y luego halle el seno:

```
grados = 45
angulo = grados * 2 * math.pi / 360.0
math.sin(angulo)
```

La constante **pi** también es parte del módulo **math**. Si se sabe la geometría, puede verificar el resultado comparándolo con el de la raíz cuadrada de 2, dividida entre 2.

```
>>> math.sqrt(2) / 2.0
0.707106781187
```

3.5. Composición

Igual que con las funciones matemáticas, las funciones de Python se pueden componer; eso quiere decir que se usa una expresión como parte de otra. Por ejemplo, puede usar cualquier expresión como argumento de una función:

```
x = math.cos(angulo + pi/2)
```

Esta sentencia toma el valor de **pi**, lo divide entre dos y le añade el resultado al valor de **angulo**. La suma se pasa luego como argumento a la función **cos**.

También puede tomar el resultado de una función y pasárselo como argumento a otra:

```
x = math.exp(math.log(10.0))
```

Esta sentencia encuentra el logaritmo en base **e** de 10 y luego eleva **e** a ese exponente. El resultado queda asignado a **x**.

3.6. Añadir funciones nuevas

Hasta ahora sólo hemos usado las funciones que vienen incluidas con Python, pero también es posible añadir nuevas funciones. La creación de nuevas funciones para resolver sus problemas particulares es una de las cosas más útiles de los lenguajes de programación de propósito general.

En contextos de programación, **función** es una secuencia de instrucciones con nombre, que lleva a cabo la operación deseada. Esta operación se especifica en una **definición de función**. Las funciones que hemos usado hasta ahora las han definido por nosotros, y esas definiciones están ocultas. Eso es bueno, ya que nos permite usar funciones sin preocuparnos sobre los detalles de sus definiciones.

La sintaxis de la definición de una función es:

```
def NOMBRE( LISTA DE PARAMETROS ):  
    SENTENCIAS
```

Puede inventarse el nombre que desee para su función, con la excepción de que no puede usar las palabras reservadas de Python. La lista de parámetros especifica qué información, en caso de haberla, ha de proporcionar para usar la función nueva.

Puede haber cualquier número de sentencias dentro de la función, pero tienen que estar indentadas desde el margen izquierdo. En los ejemplos de este libro se usará una indentación de dos espacios.

El primer par de funciones que escribiremos no tienen parámetros, de manera que su sintaxis es:

```
def nueva_linea():  
    print
```

Esta función se llama `nueva_linea`. Los paréntesis vacíos indican que no tiene parámetros. Contiene una única sentencia, que muestra como salida un carácter de nueva línea (es lo que sucede cuando utiliza una orden `print` sin argumentos).

Llamamos entonces a la función nueva usando la misma sintaxis que usamos para las funciones internas:

```
print "Primera linea."  
nueva_linea()  
print "Segunda linea."
```

The output of this program is

```
Primera linea.
```

```
Segunda linea.
```

Observe el espacio añadido que hay entre las dos líneas. Si quisiéramos más espacios, entre las líneas, ¿qué haríamos? Podemos llamar varias veces a la misma función:

```
print "Primera linea."  
nueva_linea()  
nueva_linea()  
nueva_linea()  
print "Segunda linea."
```

O bien podemos escribir una nueva función que llamaremos `tresLineas`, y que imprima tres nuevas líneas:

```
def tresLineas():
    nueva_linea()
    nueva_linea()
    nueva_linea()

print "Primera Linea."
tresLineas()
print "Segunda Linea."
```

Esta función contiene tres sentencias, las cuales están todas indentadas con dos espacios. Puesto que la siguiente sentencia no está indentada, Python sabe que no es parte de la función.

Observe los siguientes puntos con respecto a este programa:

1. Se puede llamar al mismo procedimiento repetidamente. De hecho es bastante útil hacerlo, además de habitual.
2. Se puede llamar a una función desde dentro de otra función: en este caso `tresLineas` llama a `nueva_linea`.

Hasta ahora puede no haber quedado claro por qué vale la pena crear todas estas funciones nuevas. En realidad hay muchísimas razones, pero este ejemplo demuestra dos:

- Crear una nueva función le da la oportunidad de dar un nombre a un grupo de sentencias. Las funciones simplifican su programa al ocultar cálculos complejos detrás de órdenes sencillas, y usar palabras de su propia lengua en vez de código arcano.
- Crear una nueva función hace que el programa sea más pequeño, al eliminar código repetitivo. Por ejemplo, una manera de imprimir nueve líneas consecutivas es llamar a `tresLineas` tres veces.

Como actividad, escriba una función llamada `nueveLineas` que use `tresLineas` para imprimir nueve líneas en blanco. ¿Cómo imprimiría 27 líneas nuevas?

3.7. Las definiciones y el uso

Juntando los fragmentos de código de la sección anterior, el programa completo queda de la siguiente manera:

```
def nueva_linea():  
    print  
  
def tresLineas():  
    nueva_linea()  
    nueva_linea()  
    nueva_linea()  
  
print "Primera Linea."  
tresLineas()  
print "Segunda Linea."
```

El presente programa contiene dos definiciones de funciones: `nueva_linea` y `tresLineas`. Las definiciones de funciones se ejecutan como el resto de sentencias, pero el efecto es crear una nueva función. Las sentencias del interior de la función no se ejecutan hasta que se llama a la función, y la definición de la función no genera salida.

Como era de esperar, tiene que crear una función antes de poder ejecutarla. En otras palabras, la definición de la función tiene que ejecutarse antes de la primera vez que se la invoque.

Como actividad, pruebe a ejecutar este programa moviendo las tres últimas sentencias al principio del programa. Registre qué mensaje de error obtiene usted.

Como segunda actividad, pruebe a tomar la versión del programa que funcionaba y a mover la definición de `nueva_linea` más abajo que la definición de `tresLineas`. ¿Qué ocurre cuando ejecuta el programa?

3.8. Flujo de ejecución

Para asegurarse de que una función se define antes de su primer uso, tiene que conocer el orden en el que se ejecutan las sentencias; a esto se le llama **flujo de ejecución**.

La ejecución comienza siempre por la primera sentencia del programa. Las sentencias se ejecutan a razón de una cada vez, en orden, hasta que se alcanza una llamada a una función.

Las definiciones de funciones no alteran el flujo de ejecución del programa, pero recuerde que las sentencias que hay dentro de la función no se ejecutan hasta que se hace la llamada a la función. Aunque no es habitual, puede definir una

función dentro de otra. En este caso, la definición de función interior no se ejecuta hasta que no se llama a la función exterior.

Las llamadas a funciones son como un desvío en el flujo de ejecución. En lugar de ir a la siguiente sentencia, el flujo salta hasta la primera línea de la función a la que se llama, ejecuta todas las sentencias que encuentre allí, y vuelve a retomar la ejecución en el punto donde lo dejó.

Esto suena bastante sencillo... hasta que se acuerda de que una función puede llamar a otra. Mientras estamos en medio de una función, podríamos vernos obligados a abandonarla e ir a ejecutar sentencias en otra función más. Pero mientras estamos en esta nueva función, ¿podríamos salirnos y ejecutar otra función más!

Afortunadamente, a Python se le da bien tomar nota de dónde está, de manera que cada vez que se completa una función, el programa retoma el punto en donde lo dejó en la función que hizo la llamada. Cuando llega al final del programa, termina.

¿Cuál es la moraleja de toda esta historia? Cuando esté leyendo un programa, no lo lea desde la parte superior a la inferior. En lugar de eso, siga el flujo de ejecución.

3.9. Parámetros y argumentos

Algunas de las funciones internas que hemos usado precisan de argumentos, los valores que controlan cómo la función lleva a cabo su tarea. Por ejemplo, si desea encontrar el seno de un número, tiene que indicar de qué número se trata. Así pues, `sin` toma como argumento un valor numérico.

Algunas funciones toman más de un argumento, como `pow`, que toma dos argumentos: la base y el exponente. Dentro de la función, los valores que se le han pasado se asignan a variables llamadas **parámetros**.

He aquí un ejemplo de una función definida por el usuario, que toma un parámetro:

```
def imprimeDoble(paso):  
    print paso, paso
```

Esta función toma un único argumento y se lo asigna a un parámetro llamado `paso`. El valor del parámetro (en este punto todavía no tenemos ni idea de cuál será) se imprime dos veces, seguido por un carácter de nueva línea. El nombre `paso` se eligió para sugerir que el nombre que le dé a un parámetro depende de usted, pero en general es mejor que elija un nombre más ilustrativo que `paso`.

La función `imprimeDoble` sirve con cualquier tipo (de dato) que se pueda imprimir:

```
>>> imprimeDoble('Jamón')
Jamón Jamón
>>> imprimeDoble(5)
5 5
>>> imprimeDoble(3.14159)
3.14159 3.14159
```

En la primera llamada a la función, el argumento es una cadena; en la segunda es un entero, y en la tercera es un número de coma flotante.

Las mismas reglas de composición que se aplican a las funciones internas se aplican también a las funciones definidas por el usuario, así que puede usar cualquier tipo de expresión como argumento de `imprimeDoble`.

```
>>> imprimeDoble('Jamón'*4)
JamónJamónJamónJamón JamónJamónJamónJamón
>>> imprimeDoble(math.cos(math.pi))
-1.0 -1.0
```

Como de costumbre, se evalúa la expresión antes de ejecutar la función, de modo que `imprimeDoble` devuelve `JamónJamónJamónJamón JamónJamónJamónJamón` en lugar de `'Jamón'*4` `Jamón'*4`.

Asimismo podemos usar una variable como argumento:

```
>>> latoya = 'Dafne, es mitad laurel mitad ninfa'
>>> imprimeDoble(latoya)
Dafne, es mitad laurel mitad ninfa. Dafne, es mitad laurel mitad ninfa.
```

Observe un aspecto realmente importante en este caso: el nombre de la variable que pasamos como argumento (`latoya`) no tiene nada que ver con el nombre del parámetro (`paso`). No importa cómo se llamaba el valor en su lugar original (el lugar desde donde se invocó); aquí en `imprimeDoble` llamamos a todo el mundo `paso`.

3.10. Las variables y los parámetros son locales

Cuando crea una variable dentro de una función, sólo existe dentro de dicha función, y no puede usarla fuera de ella. Por ejemplo, la función

```
>>> def catDoble(parte1, parte2):
...     cat = parte1 + parte2
...     imprimeDoble(cat)
...
>>>
```

toma dos argumentos, los concatena y luego imprime el resultado dos veces. Podemos llamar a la función con dos cadenas:

```
>>> cantus1 = "Die Jesu domine, "
>>> cantus2 = "Dona eis requiem."
>>> catDoble(cantus1, cantus2)
Die Jesu domine, Dona eis requiem. Die Jesu domine, Dona eis requiem.
```

Cuando `catDoble` termina, la variable `cat` se destruye. Si tratásemos de imprimirla, obtendríamos un error:

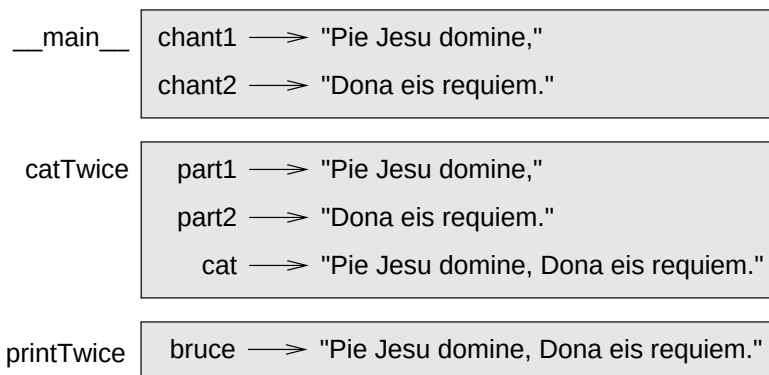
```
>>> print cat
NameError: cat
```

Los parámetros también son locales. Por ejemplo, una vez fuera de la función `imprimeDoble`, no existe nada llamado `paso`. Si trata de usarla, Python se quejará.

3.11. Diagramas de pila

Para mantener el rastro de qué variables pueden usarse y dónde, a veces es útil dibujar un **diagrama de pila**. Como los diagramas de estado, los diagramas de pila muestran el valor de cada variable, pero también muestran la función a la que cada variable pertenece.

Cada función se representa por una caja con el nombre de la función junto a él. Los parámetros y variables que pertenecen a una función van dentro. Por ejemplo, el diagrama de stack para el programa anterior tiene este aspecto:



El orden de la pila muestra el flujo de ejecución. `imprimeDoble` fue llamado por `catDoble` y a `catDoble` lo invocó `__main__`, que es un nombre especial de la

función más alta. Cuando crea una variable fuera de cualquier función, pertenece a `__main__`.

En cada caso, el parámetro se refiere al mismo valor que el argumento correspondiente. Así que `parte1` en `catDoble` tiene el mismo valor que `cantus1` en `__main__`.

Si sucede un error durante la llamada a una función, Python imprime el nombre de la función y el nombre de la función que la llamó, y el nombre de la función que llamó a *ésta*, y así hasta `__main__`.

Por ejemplo, si intentamos acceder a `cat` desde `imprimeDoble`, provocaremos un `NameError`:

```
Traceback (innermost last):
  File "test.py", line 13, in __main__
    catDoble(cantus1, cantus2)
  File "test.py", line 5, in catDoble
    imprimeDoble(cat)
  File "test.py", line 9, in imprimeDoble
    print cat
NameError: cat
```

Esta lista de funciones de llama **traceback** (traza inversa). Le dice a usted en qué archivo de programa sucedió el error, y en qué línea, y qué funciones se ejecutaban en ese momento. También muestra la línea de código que causó el error.

Fíjese en la similitud entre la traza inversa y el diagrama de pila. No es una coincidencia.

3.12. Funciones con resultado

Seguramente ha notado ya que algunas de las funciones que estamos usando, igual que las funciones matemáticas, devuelven un resultado. Otras funciones, como `nueva_linea`, llevan a cabo una acción pero no devuelven un valor. Ello suscita varias preguntas:

1. ¿Qué sucede si llama usted a una función y no hace nada con el resultado (es decir, no lo asigna a una variable ni lo usa como parte de una expresión más amplia)?
2. ¿Qué sucede si usa una función sin resultado como parte de una expresión, por ejemplo `nueva_linea() + 7`?

3. ¿Se pueden escribir funciones que devuelvan resultados, o debemos limitarnos a funciones simples como `nueva_linea` e `imprimeDoble`?

La respuesta a la tercera pregunta es “sí, puede escribir funciones que devuelvan valores”, y lo haremos en el capítulo 5.

Como actividad final, consteste a las otras dos preguntas intentando hacerlas en la práctica. Cada vez que tenga una duda sobre lo que es legal o ilegal en Python, preguntar al intérprete será una buena manera de averiguarlo.

3.13. Glosario

llamada a función: Una sentencia que ejecuta una función. Está compuesta por el nombre de la función más una lista de argumentos encerrados entre paréntesis.

argumento: Valor que se le pasa a una función cuando se la llama. El valor se asigna al parámetro correspondiente de la función.

valor de retorno: Es el resultado de una función. Si se usa una llamada a función a modo de expresión, el valor de retorno es el valor de la expresión.

conversión de tipo: Una sentencia explícita que toma un valor de un tipo y calcula el valor correspondiente de otro tipo.

coerción: Conversión tipos que ocurre automáticamente de acuerdo con las reglas de coerción de Python.

módulo: Fichero que contiene una colección de funciones y clases relacionadas.

notación de punto: La sintaxis para llamar a una función de otro módulo, especificando el nombre del módulo, seguido por un punto y el nombre de la función.

función: Secuencia de sentencias etiquetadas que llevan a cabo determinada operación de utilidad. Las funciones pueden tomar parámetros o no, y pueden producir un resultado o no.

definición de función: Sentencia que crea una nueva función, especificando su nombre, parámetros y las sentencias que ejecuta.

flujo de ejecución: Orden en el que se ejecutan las sentencias durante la ejecución de un programa.

parámetro: Nombre que se usa dentro de una función para referirse a el valor que se le pasa como argumento.

variable local: variable definida dentro de una función. Las variables locales sólo pueden usarse dentro de su función.

diagrama de pila: Representación gráfica de una pila de funciones, sus variables y los valores a los que se refieren.

traza inversa: (traceback en inglés) Una lista de las funciones en curso de ejecución, presentadas cuando sucede un error en tiempo de ejecución.

notación de punto traza inversa

Capítulo 4

Condicionales y recursividad

4.1. El operador módulo

El **operador módulo** funciona con enteros (y expresiones enteras), y devuelve el resto de dividir el primer operando entre el segundo. En Python, el operador de módulo es el signo de tanto por ciento (%). La sintaxis es la misma de los otros operadores:

```
>>> cociente = 7 / 3
>>> print cociente
2
>>> resto = 7 % 3
>>> print resto
1
```

Así, 7 dividido entre 3 da 2 con 1 de resto.

El operador de módulo resulta ser sorprendentemente útil. Por ejemplo, puede comprobar si un número es divisible entre otro: si $x \% y$ es cero, entonces x es divisible entre y .

También puede usar el operador módulo para extraer el dígito más a la derecha de un número. Por ejemplo, $x \% 10$ devuelve el dígito más a la derecha de x (en base 10). De forma similar, $x \% 100$ devuelve los dos últimos dígitos.

4.2. Expresiones booleanas

Una **expresión booleana** es una expresión que es cierta o falsa. En Python, una expresión que es cierta tiene el valor 1, y una expresión que es falsa tiene el valor 0.

El operador `==` compara dos valores y entrega una expresión booleana:

```
>>> 5 == 5
1
>>> 5 == 6
0
```

En la primera sentencia, los dos operandos son iguales, así que la expresión se evalúa como 1 (verdadero); en la segunda sentencia, 5 no es igual a 6, así que obtenemos 0 (falso).

El operador `==` es uno de los **operadores de comparación**; los otros son:

<code>x != y</code>	# x no es igual a y
<code>x > y</code>	# x es mayor que y
<code>x < y</code>	# x es menor que y
<code>x >= y</code>	# x es mayor o igual que y
<code>x <= y</code>	# x es menor o igual que y

Aunque probablemente estas operaciones le resulten familiares, los símbolos en Python son diferentes de los matemáticos. Un error habitual es utilizar un signo igual sencillo (`=`) en lugar del doble (`==`). Recuerde que `=` es un operador de asignación y `==` es un operador de comparación. Además, no existen `=<` ni `=>`.

4.3. Operadores lógicos

Hay tres **operadores lógicos**: `and`, `or`, y `not`. La semántica (significado) de estos operadores es similar a sus significados en inglés. Por ejemplo, `x > 0 and x < 10` es verdadero sólo si `x` es mayor que 0 *y* menor que 10.

`n%2 == 0 or n%3 == 0` es verdadero si *cualquiera* de las condiciones es verdadera, o sea, si el número es divisible por 2 *o* por 3.

Finalmente, el operador `not` niega una expresión booleana, de forma que `not(x > y)` es cierto si `(x > y)` es falso, o sea, si `x` es menor o igual que `y`.

Hablando estrictamente, los operandos de los operadores lógicos deberían ser expresiones booleanas, pero Python no es muy estricto. Cualquier número que no sea cero se interpreta como “verdadero”.

```
>>> x = 5
>>> x and 1
1
>>> y = 0
>>> y and 1
0
```

En general, este tipo de cosas no se considera buen estilo. Si quiere comparar un valor con cero, debería hacerlo explícitamente.

4.4. Ejecución condicional

Para escribir programas útiles, casi siempre necesitamos la capacidad de comprobar ciertas condiciones y cambiar el comportamiento del programa en consonancia. Las **sentencias condicionales** nos dan esta capacidad. La forma más sencilla es la sentencia `if`:

```
if x > 0:
    print "x es positivo"
```

La expresión booleana tras el `if` se llama **condición**. Si es verdadera, entonces la sentencia indentada se ejecuta. Si la condición no es verdadera, no pasa nada.

Como otras sentencias compuestas, `if` consta de una cabecera y un bloque de sentencias:

```
CABECERA:
    PRIMERA SENTENCIA
    ...
    ULTIMA SENTENCIA
```

La cabecera comienza con una nueva línea y termina con el signo de dos puntos. Los elementos indentados que siguen se llaman **bloque** de la sentencia. La primera sentencia no indentada marca el fin del bloque. Un bloque de sentencias dentro de una sentencia compuesta recibe el nombre de **cuerpo** de la sentencia.

No hay límite a la cantidad de sentencias que pueden aparecer en el cuerpo de una sentencia `if`, pero debe haber al menos una. A veces, es útil tener un cuerpo sin sentencias, (normalmente como reserva de espacio para algo de código que todavía no ha escrito). En tales casos, puede usted utilizar la sentencia `pass`, que no hace nada.

4.5. Ejecución alternativa

Una segunda forma de la sentencia `if` es la ejecución alternativa, en la que hay dos posibilidades, y la condición determina cuál de ellas se ejecuta. La sintaxis

tiene este aspecto:

```
if x%2 == 0:
    print x, "es par"
else:
    print x, "es impar"
```

Si el resto cuando se divide x entre 2 es cero, entonces sabemos que x es par, y este programa muestra un mensaje a tal efecto. Si la condición es falsa, se ejecuta el segundo lote de sentencias. Puesto que la condición debe ser verdadera o falsa, se ejecutará exactamente una de las alternativas. Llamamos **ramas** a las posibilidades porque son ramas del flujo de ejecución.

Como un aparte, si piensa que querrá comprobar con frecuencia la paridad de números, quizá desee “envolver” este código en una función:

```
def imprimeParidad(x):
    if x%2 == 0:
        print x, "es par"
    else:
        print x, "es impar"
```

Ahora tiene una función llamada `imprimeParidad` que muestra el mensaje apropiado para cada número entero que usted le pase. Llame a esta función de la manera siguiente:

```
>>> imprimeParidad(17)
>>> imprimeParidad(y+1)
```

4.6. Condiciones encadenadas

A veces hay más de dos posibilidades y necesitamos más de dos ramas. Una forma de expresar tal computación es un **conditional encadenado**:

```
if x < y:
    print x, "es menor que", y
elif x > y:
    print x, "es mayor que", y
else:
    print x, "y", y, "son iguales"
```

`elif` es una abreviatura de “else if”. De nuevo, sólo se ejecutará una rama. No hay límite al número de sentencias `elif`, pero sólo se permite una sentencia `else` (que puede omitirse) y debe ser la última rama de la sentencia:

```
if eleccion == 'A':
    funcionA()
elif eleccion == 'B':
```



```
funcionB()
elif eleccion == 'C':
    funcionC()
else:
    print "Eleccion no valida."
```

Las condiciones se comprueban en orden. Si la primera es falsa, se comprueba la siguiente, y así. Si una de ellas es cierta, se ejecuta la rama correspondiente y termina la sentencia. Incluso si es cierta más de una condición, sólo se ejecuta la primera rama verdadera.

Como ejercicio, envuelva estos ejemplos en funciones llamadas `compara(x, y)` y `resuelve(eleccion)`.

4.7. Condiciones anidadas

Una condición puede estar anidada dentro de otra. Podíamos haber escrito así el ejemplo de tricotomía:

```
~~if x == y:
~~~~print x, "y", y, "son iguales"
~~else:
~~~~if x < y:
~~~~~print x, "es menor que", y
~~~~else:
~~~~~print x, "es mayor que", y
```

La condición externa que contiene dos ramas. La primera rama contiene una sentencia simple de salida. La segunda rama contiene otra sentencia `if`, que tiene dos ramas en sí misma. Estas dos ramas son ambas sentencias de salida de datos, aunque podrían ser igualmente sentencias condicionales.

Aunque la indentación de las sentencias hace la estructura evidente, las condiciones anidadas en seguida se vuelven difíciles de leer. En general es una buena idea evitarlas cuando pueda.

Los operadores lógicos suelen facilitar un modo de simplificar las sentencias condicionales anidadas. Por ejemplo, podemos reescribir el código siguiente con un sólo condicional:

```
if 0 < x:
    if x < 10:
        print "x es un número positivo de un dígito."
```

La sentencia `print` sólo se ejecuta si conseguimos superar ambos condicionales, así que podemos usar el operador `and`:

```
if 0 < x and x < 10:  
    print "x es un número positivo de un dígito."
```

Estos tipos de condiciones son habituales, por lo que Python nos proporciona una sintaxis alternativa similar a la notación matemática:

```
if 0 < x < 10:  
    print "x es un número positivo de un dígito."
```

Esta condición es semánticamente la misma que la expresión booleana compuesta y que el condicional anidado.

4.8. La sentencia return

La sentencia `return` le permite terminar la ejecución de una función antes de alcanzar su final. Una razón para usarla es detectar una condición de error:

```
import math  
  
def imprimeLogaritmo(x):  
    if x <= 0:  
        print "Solo numeros positivos, por favor."  
        return  
  
    result = math.log(x)  
    print "El log de x es", result
```

La función `imprimeLogaritmo` toma un parámetro llamado `x`. Lo primero que hace es comprobar si `x` es menor o igual que cero, en cuyo caso muestra un mensaje de error y luego usa `return` para salir de la función. El flujo de la ejecución vuelve inmediatamente al llamante y no se ejecutan las líneas restantes de la función.

Recuerde que para usar una función del módulo `math` tiene que importarlo.

4.9. Recursividad

Ya mencionamos que es legal que una función llame a otra, y de ello hemos visto ya varios ejemplos. Olvidamos mencionar que también es legal que una función se llame a sí misma. Puede no resultar evidente por qué es bueno esto, pero viene a resultar una de las cosas más interesantes y curiosas que puede hacer un programa. Examine por ejemplo la siguiente función:

```
def cuenta_atras(n):  
    if n == 0:  
        print "Despegando!"  
    else:  
        print n  
        cuenta_atras(n-1)
```

`cuenta_atras` espera que su parámetro, `n`, sea un entero positivo. Si `n` el parámetro es cero, muestra la palabra “Despegando!”. En otro caso, muestra `n` y luego llama a la función llamada `cuenta_atras` (ella misma) pasándole como argumento `n-1`.

¿Qué sucede si llamamos a la función de la siguiente manera?

```
>>> cuenta_atras(3)
```

La ejecución de `cuenta_atras` comienza con `n=3`, y puesto que `n` no es cero, da como salida el valor 3, y luego se llama a sí misma ...

La ejecución de `cuenta_atras` comienza con `n=2`, y puesto que `n` no es cero, muestra el valor 2 y luego se llama a sí misma ...

La ejecución de `cuenta_atras` comienza con `n=1`, y puesto que `n` no es cero, muestra el valor 1, y luego se llama a sí misma...

La ejecución de `cuenta_atras` comienza con `n=0`, y puesto que `n` es cero, muestra la palabra “Despegando!” y luego retorna.

La `cuenta_atras` que dio `n=1` retorna.

La `cuenta_atras` que dio `n=2` retorna.

La `cuenta_atras` que dio `n=3` retorna.

Y entonces ya está de vuelta en `__main__` (menudo viaje). De manera que la salida completa presenta el siguiente aspecto:

```
3  
2  
1  
Despegando!
```

Como segundo ejemplo, consideremos de nuevo las funciones `nuevaLinea` and `tresLineas`.

```
def nuevaLinea():  
    print
```

```
def tresLineas():  
    nuevaLinea()  
    nuevaLinea()  
    nuevaLinea()
```

Aunque todas funcionan, no serían de mucha ayuda si quisiera mostrar 2 líneas nuevas o 106. Una mejor alternativa será:

```
def nLineas(n):  
    if n > 0:  
        print  
        nLineas(n-1)
```

Este programa es parecido a `cuenta_atras`; mientras `n` sea mayor que cero, muestra una nueva línea, y luego se llama a sí misma para mostrar `>n-1` nuevas líneas más. De esta manera, el número total de nuevas líneas es $1 + (n-1)$, que si rescata su álgebra verá que es `n`.

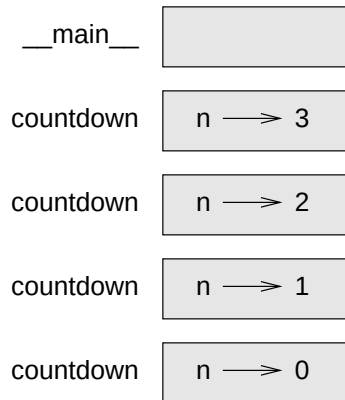
El proceso por el que una función se llama a sí misma se llama **recursividad**, y dichas funciones se denominan recursivas.

4.10. Diagramas de pila para funciones recursivas

En la Sección 3.11 utilizamos un diagrama de pila para representar el estado de un programa durante la llamada de una función. El mismo tipo de diagrama puede hacer más fácil interpretar una función recursiva.

Cada vez que se llama a una función, Python crea un nuevo marco para la función, que contiene sus variables locales y parámetros. En el caso de una función recursiva, puede haber más de un marco en la pila al mismo tiempo.

La figura muestra un diagrama de pila para `cuenta_atras`, invocada con `n = 3`:



Como es habitual, en lo alto de la pila está el marco de `__main__`. Está vacío porque no hemos creado ninguna variable sobre `__main__` ni le hemos pasado ningún parámetro.

Los cuatro marcos de `cuenta_atras` tienen valores diferentes para el parámetro `n`. El fondo de la pila, donde `n=0`, se llama **caso base**. No hace una llamada recursiva, de manera que no hay más marcos.

Como actividad, dibuje un diagrama de pila para `nLineas`, invocada con el parámetro `n=4`.

4.11. Recursividad infinita

Si una recursión no alcanza nunca el caso base, seguirá haciendo llamadas recursivas para siempre y nunca terminará. Esta circunstancia se conoce como **recursividad infinita**, y generalmente no se la considera una buena idea. Este es un programa mínimo con recursividad infinita:

```
def recurre():
    recurre()
```

En la mayoría de los entornos de programación, un programa con recursividad infinita no se ejecutará realmente para siempre. Python informará de un mensaje de error cuando se alcance el nivel máximo de recursividad:

```
File "<stdin>", line 2, in recurre
(98 repetitions omitted)
File "<stdin>", line 2, in recurre
RuntimeError: Maximum recursion depth exceeded
```

Esta traza inversa es un poco mayor que la que vimos en el capítulo anterior. ¡Cuando sucede el error, hay 100 marcos **recurre** en la pila!

Como actividad, escriba una función con recursividad infinita y ejecútela en el intérprete de Python.

4.12. Entrada por teclado

Los programas que hemos escrito hasta ahora son un poco maleducados en el sentido de que no aceptan entradas de datos del usuario. Simplemente hacen lo mismo siempre.

Python proporciona funciones internas que obtienen entradas desde el teclado. La más sencilla se llama `raw_input`. Cuando llamamos a esta función, el programa se detiene y espera a que el usuario escriba algo. Cuando el usuario pulsa la tecla **Return** o **Enter**, el programa se reanuda y `raw_input` devuelve lo que el usuario escribió como tipo `string`:

```
>>> entrada = raw_input ()
A qué estás esperando?
>>> print entrada
A qué estás esperando?
```

Antes de llamar a `raw_input` es conveniente mostrar un mensaje que le pida al usuario el dato solicitado. Este mensaje se llama **indicador** (prompt en inglés). Puede proporcionarle un indicador a `raw_input` como argumento:

```
>>> nombre = raw_input ("Cómo te llamas? ")
Cómo te llamas? Héctor, héroe de los Troyanos!
>>> print nombre
Héctor, héroe de los Troyanos!
```

Si espera que la entrada sea un entero, utilice la función `input`. Por ejemplo:

```
>>> indicador = \
... "Cuál es la velocidad de una golondrina sin carga?\n"
>>> velocidad = input (indicador)
```

Si el usuario teclea una cadena de números, se convertirá en un entero y se asignará a `velocidad`. Por desgracia, si el usuario escribe algo que no sea un dígito, el programa dará un error:

```
>>> velocidad = input (indicador)
Cuál es la velocidad de una golondrina sin carga?
Se refiere usted a la golondrina europea o a la africana?
SyntaxError: invalid syntax
```

Para evitar este tipo de error, generalmente es buena idea usar `raw_input` para obtener una cadena y usar entonces las funciones de conversión para convertir a otros tipos.

4.13. Glosario

operador módulo: Operador, señalado con un signo de tanto por ciento (%), que trabaja sobre enteros y devuelve el resto cuando un número se divide entre otro.

expresión booleana: Una expresión que es cierta o falsa.

operador de comparación: Uno de los operadores que comparan dos valores: `==`, `!=`, `>`, `<`, `>=` y `<=`.

operador lógico: Uno de los operadores que combinan expresiones booleanas: `and`, `or` y `not`.

sentencia condicional: Sentencia que controla el flujo de ejecución de un programa dependiendo de cierta condición.

condición: La expresión booleana de una sentencia condicional que determina qué rama se ejecutará.

sentencia compuesta: Estructura de Python que está formado por una cabecera y un cuerpo. La cabecera termina en dos puntos (:). El cuerpo tiene una sangría con respecto a la cabecera.

bloque: Grupo sentencias consecutivas con el mismo sangrado.

cuerpo: En una sentencia compuesta, el bloque de sentencias que sigue a la cabecera de la sentencia.

anidamiento: Una estructura de programa dentro de otra; por ejemplo, una sentencia condicional dentro de una o ambas ramas de otra sentencia condicional.

recursividad: El proceso de volver a llamar a la función que se está ejecutando en ese momento.

caso base: En una función recursiva, la rama de una sentencia condicional que no ocasiona una llamada recursiva.

recursividad infinita: Función que se llama a sí misma recursivamente sin alcanzar nunca el caso base. A la larga una recursión infinita provocará un error en tiempo de ejecución.

indicador: indicador visual que invita al usuario a introducir datos.

Capítulo 5

Funciones productivas

5.1. Valores de retorno

Algunas de las funciones internas que hemos usado, como las funciones `math` o funciones matemáticas, han producido resultados. Llamar a la función genera un nuevo valor, que normalmente asignamos a una variable para usar como parte de una expresión.

```
import math

e = math.exp(1.0)
altura = radio * math.sin(angulo)
```

Pero hasta ahora, ninguna de las funciones que hemos escrito ha devuelto un valor.

En este capítulo escribiremos funciones que devuelvan valores, que llamaremos **funciones productivas**, a falta de un nombre mejor. El primer ejemplo es `area`, que devuelve el área de un círculo con un radio dado:

```
import math

def area(radio):
    temporal = math.pi * radio**2
    return temporal
```

Ya hemos visto antes la sentencia `return`, pero en una función productiva la sentencia `return` incluye un valor de retorno. Esta sentencia quiere decir “retorna inmediatamente de la función y usa la siguiente expresión como valor de retorno”. La expresión dada puede ser arbitrariamente complicada; así pues, podríamos haber escrito esta función más concisamente:

```
def area(radio):
    return math.pi * radio**2
```

Por otra parte, las **variables temporales** como `temporal` suelen hacer más fácil el depurado.

A veces es útil disponer de varias sentencias de retorno, una en cada rama de una condición:

```
def valorAbsoluto(x):
    if x < 0:
        return -x
    else:
        return x
```

Puesto que estas sentencias `return` están en una condición alternativa, sólo se ejecutará una de ellas. En cuanto se ejecuta una de ellas, la función termina sin ejecutar ninguna de las sentencias siguientes.

El código que aparece después de una sentencia `return` o en cualquier otro lugar donde el flujo de ejecución no pueda llegar, recibe el nombre de **código muerto**.

En una función productiva es una buena idea asegurarse de que cualquier posible recorrido del programa alcanza una sentencia `return`. Por ejemplo:

```
def valorAbsoluto(x):
    if x < 0:
        return -x
    elif x > 0:
        return x
```

Este programa no es correcto porque si resulta que `x` vale 0, entonces no se cumple ninguna de ambas condiciones y la función termina sin alcanzar la sentencia `return`. En este caso, el valor de retorno es un valor especial llamado **None**:

```
>>> print valorAbsoluto(0)
None
```

*Como actividad, escriba una función **comparar** que devuelva 1 si $x > y$, 0 si $x == y$, y -1 si $x < y$.*

5.2. Desarrollo de programas

Llegados a este punto, tendría que poder mirar a funciones Python completas y adivinar qué hacen. También, si ha hecho los ejercicios, habrá escrito algunas funcioncillas. Tal como vaya escribiendo funciones mayores puede empezar

a experimentar más dificultades, especialmente con los errores en tiempo de ejecución y los semánticos.

Para lidiar con programas de complejidad creciente, vamos a sugerirle una técnica que llamaremos **desarrollo incremental**. El objetivo del desarrollo incremental es sustituir largas sesiones de depuración por la adición y prueba de pequeñas porciones de código en cada vez.

Por ejemplo, supongamos que desea encontrar la distancia entre dos puntos, dados por las coordenadas (x_1, y_1) y (x_2, y_2) . Por el teorema de Pitágoras, podemos escribir la distancia es:

$$\text{distancia} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \quad (5.1)$$

El primer paso es considerar qué aspecto tendría una función **distancia** en Python. En otras palabras, ¿cuáles son las entradas (parámetros) y cuál es la salida (valor de retorno)?

En este caso, los dos puntos son los parámetros, que podemos representar usando cuatro parámetros. El valor de retorno es la distancia, que es un valor en coma flotante.

Ya podemos escribir un bosquejo de la función:

```
def distancia(x1, y1, x2, y2):  
    return 0.0
```

Obviamente, la función no calcula distancias; siempre devuelve cero. Pero es sintácticamente correcta y se ejecutará, lo que significa que podemos probarla antes de complicarla más.

Para comprobar la nueva función, tenemos que llamarla con valores de ejemplo:

```
>>> def distancia(x1, y1, x2, y2):  
...     return 0.0  
...  
>>> distancia(1, 2, 4, 6)  
0.0  
>>>
```

Elegimos estos valores de tal forma que la distancia horizontal sea igual a 3 y la distancia vertical sea igual a 4; de esa manera el resultado es 5 (la hipotenusa del triángulo 3-4-5). Cuando se comprueba una función, es útil saber la respuesta correcta.

Hasta el momento, hemos comprobado que la función es sintácticamente correcta, así que podemos empezar a añadir líneas de código. Después de cada cambio

incremental, comprobamos de nuevo la función. Si en un momento dado aparece un error, sabremos dónde está exactamente: en la última línea que hayamos añadido.

El siguiente paso en el cálculo es encontrar las diferencias entre $x_2 - x_1$ y $y_2 - y_1$. Almacenaremos dichos valores en variables temporales llamadas `dx` y `dy` y las imprimiremos.

```
def distancia(x1, y1, x2, y2):
    dx = x2 - x1
    dy = y2 - y1
    print "dx es", dx
    print "dy es", dy
    return 0.0
```

Si la función funciona, valga la redundancia, las salidas deberían ser 3 y 4. Si es así, sabemos que la función recibe correctamente los parámetros y realiza correctamente el primer cálculo. Si no, sólo hay unas pocas líneas que revisar.

Ahora calculamos la suma de los cuadrados de `dx` y `dy`:

```
def distancia(x1, y1, x2, y2):
    dx = x2 - x1
    dy = y2 - y1
    dalcuadrado = dx**2 + dy**2
    print "dalcuadrado es: ", dalcuadrado
    return 0.0
```

Fíjese en que hemos eliminado las sentencias `print` que escribimos en el paso anterior. Este código se llama **andamiaje** porque es útil para construir el programa pero no es parte del producto final.

De nuevo queremos ejecutar el programa en este estado y comprobar la salida (que debería dar 25).

Por último, si hemos importado el módulo `math`, podemos usar la función `sqrt` para calcular y devolver el resultado:

```
def distancia(x1, y1, x2, y2):
    dx = x2 - x1
    dy = y2 - y1
    dalcuadrado = dx**2 + dy**2
    resultado = math.sqrt(dalcuadrado)
    return resultado
```

Si esto funciona correctamente, ha terminado. Si no, podría ser que quisiera usted imprimir el valor de `resultado` antes de la sentencia `return`.

Al principio, debería añadir solamente una o dos líneas de código cada vez. Conforme vaya ganando experiencia, puede que se encuentre escribiendo y depurando trozos mayores. Sin embargo, el proceso de desarrollo incremental puede ahorrarle mucho tiempo de depurado.

Los aspectos fundamentales del proceso son:

1. Comience con un programa que funcione y hágale pequeños cambios incrementales. Si hay un error, sabrá exactamente dónde está.
2. Use variables temporales para mantener valores intermedios, de tal manera que pueda mostrarlos por pantalla y comprobarlos.
3. Una vez que el programa esté funcionando, tal vez prefiera eliminar parte del andamiaje o aglutinar múltiples sentencias en expresiones compuestas, pero sólo si eso no hace que el programa sea difícil de leer.

*Como actividad, utilice el desarrollo incremental para escribir una función de nombre **hipotenusa** que devuelva la longitud de la hipotenusa de un triángulo rectángulo, dando como parámetros los dos catetos. Registre cada estado del desarrollo incremental según vaya avanzando.*

5.3. Composición

Como seguramente a estas alturas ya supondrá, se puede llamar a una función desde dentro de otra. Esta habilidad se llama *composición*.

Como ejemplo, escribiremos una función que tome dos puntos, el centro del círculo y un punto del perímetro, y calcule el área del círculo.

Supongamos que el punto central está almacenado en las variables `xc` e `yc`, y que el punto del perímetro lo está en `xp` e `yp`. El primer paso es hallar el radio del círculo, que es la distancia entre los dos puntos. Afortunadamente hay una función, `distancia`, que realiza esta tarea:

```
radio = distancia(xc, yc, xp, yp)
```

El segundo paso es encontrar el área de un círculo con ese radio y devolverla:

```
resultado = area(radio)
return resultado
```

Envolviendo todo esto en una función, obtenemos:

```
def area2(xc, yc, xp, yp):
    radio = distancia(xc, yc, xp, yp)
    resultado = area(radio)
    return resultado
```

Hemos llamado a esta función `area2` para distinguirla de la función `area` definida anteriormente. Sólo puede haber una única función con un determinado nombre dentro de un módulo.

Las variables temporales `radio` y `area` son útiles para el desarrollo y el depurado, pero una vez que el programa está funcionando, podemos hacerlo más conciso integrando las llamadas a las funciones en una sola línea:

```
def area2(xc, yc, xp, yp):
    return area(distancia(xc, yc, xp, yp))
```

Como actividad, escriba una función `pendiente(x1, y1, x2, y2)` que devuelva la pendiente de la línea que atraviesa los puntos $(x1, y1)$ y $(x2, y2)$. Luego use esta función en una función que se llame `intercepta(x1, y1, x2, y2)` que devuelva la `[[y-intercepta]]` de la línea a través de los puntos $(x1, y1)$ y $(x2, y2)$.

5.4. Funciones booleanas

Las funciones pueden devolver valores booleanos, lo que a menudo es conveniente para ocultar complicadas comprobaciones dentro de funciones. Por ejemplo:

```
def esDivisible(x, y):
    if x % y == 0:
        return 1      # it's true
    else:
        return 0      # it's false
```

La función lleva por nombre `esDivisible`. Es habitual dar a las funciones booleanas nombres que suenan como preguntas sí/no. Devuelve 1 ó 0 para indicar si la `x` es o no divisibel por `y`.

Podemos reducir el tamaño de la función aprovechándonos del hecho de que la sentencia condicional que hay después del `if` es en sí misma una expresión booleana. Podemos devolverla directamente, evitando a la vez la sentencia `if`:

```
def esDivisible(x, y):
    return x % y == 0
```

La siguiente sesión muestra a la nueva función en acción:

```
>>> esDivisible(6, 4)
0
>>> esDivisible(6, 3)
1
```

El uso más común para las funciones booleanas es dentro de sentencias condicionales:

```
if esDivisible(x, y):
    print "x es divisible entre y"
else:
    print "x no es divisible entre y"
```

Puede parecer tentador escribir algo como:

```
if esDivisible(x, y) == 1:
```

Pero la comparación extra es innecesaria.

Como actividad, escriba una función `estaEntre(x, y, z)` que devuelva 1 en caso de que $y \leq x \leq z$ y que devuelva 0 en cualquier otro caso.

5.5. Más recursividad

Hasta ahora, usted ha aprendido solamente un pequeño subconjunto de Python, pero puede que le interese saber que ese subconjunto es ya un lenguaje de programación *completo*; con esto queremos decir que cualquier cosa que pueda computarse se puede expresar en este lenguaje. Cualquier programa que se haya escrito alguna vez puede reescribirse utilizando únicamente las características del lenguaje que ha aprendido hasta el momento (de hecho, necesitaría algunas órdenes para controlar dispositivos como el teclado, el ratón, los discos, etc, pero eso es todo).

Probar tal afirmación es un ejercicio nada trivial, completado por primera vez por Alan Turing, uno de los primeros científicos informáticos (algunos argumentarán que era un matemático, pero muchos de los científicos informáticos pioneros comenzaron como matemáticos). En correspondencia, se la conoce como la tesis de Turing. Si estudia un curso de Teoría de la Computación, tendrá oportunidad de ver la prueba.

Para darle una idea de lo que puede hacer con las herramientas que ha aprendido hasta ahora, evaluaremos una serie de funciones matemáticas que se definen recursivamente. Una definición recursiva es semejante a una definición circular, en el sentido de que la definición contiene una referencia a lo que se define. Una definición verdaderamente circular no es muy útil:

frangoso: adjetivo que describe algo que es frangoso

Si usted viera esa definición en el diccionario, se quedaría confuso. Por otra parte, si ha buscado la definición de la función matemática factorial, habrá visto algo semejante a lo siguiente:

$$0! = 1$$
$$n! = n \cdot (n - 1)!$$

Esta definición establece que el factorial de 0 es 1, y que el factorial de cualquier otro valor, n , es n multiplicado por el factorial de $n - 1$.

Así pues, $3!$ es 3 veces $2!$, que es 2 veces $1!$, que es una vez $0!$. Juntándolos todos, $3!$ es igual a 3 veces 2 veces 1 vez 1, que es 6.

Si puede escribir una definición recursiva de algo, normalmente podrá escribir un programa de Python para evaluarlo. El primer paso es decidir cuáles son los parámetros para esta función. Con poco esfuerzo llegará a la conclusión de que `factorial` toma un único parámetro:

```
def factorial(n):
```

Si resultase que el argumento fuese 0, todo lo que hemos de hacer es devolver 1:

```
def factorial(n):
    if n == 0:
        return 1
```

En otro caso, y he aquí la parte interesante, tenemos que hacer una llamada recursiva para hallar el factorial de $n - 1$ y luego multiplicarlo por n :

```
def factorial(n):
    if n == 0:
        return 1
    else:
        recursivo = factorial(n-1)
        resultado = n * recursivo
        return resultado
```

El flujo de ejecución de este programa es similar al de `cuenta_atras` de la Sección 4.9. Si llamamos a `factorial` con el valor 3:

Puesto que 3 no es 0, tomamos la segunda rama y calculamos el factorial de $n-1$...

Puesto que 2 no es 0, tomamos la segunda rama y calculamos el factorial de $n-1$...

Puesto que 1 no es 0, tomamos la segunda rama y calculamos el factorial de $n-1$...

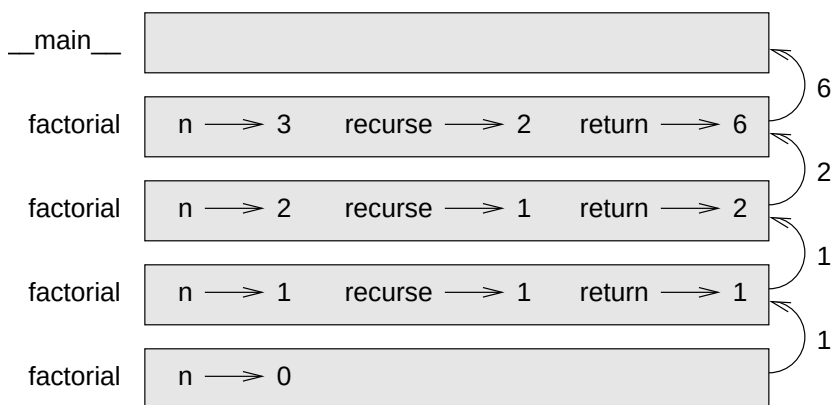
Puesto que 0 es 0, tomamos la primera rama y devolvemos el valor 1 sin hacer más llamadas recursivas.

El valor de retorno (1) se multiplica por n , que es 1, y se devuelve el resultado.

El valor de retorno (1) se multiplica por n , que es 2, y se devuelve el resultado.

El valor de retorno (2) se multiplica por n , que es 3, y el resultado 6, se convierte en el valor de retorno de la llamada a la función que comenzó todo el proceso.

He aquí el aspecto que tiene el diagrama de pila para esta secuencia de llamadas a función:



Los valores de retorno se muestran según se pasan hacia la parte superior de la pila. En cada marco, el valor de retorno es el valor de **resultado**, que es el producto de n por **recursivo**.

Nótese que en el último marco las variables locales **recursivo** y **resultado** no existen porque la rama que las crea no se ejecuta.

5.6. Acto de fe

Seguir el flujo de ejecución es una de las maneras de leer programas; pero puede volverse rápidamente una tarea laberíntica. La alternativa es lo que llamamos el “acto de fe”. Cuando llegamos a una función, en lugar de seguir el flujo de ejecución, *damos por sentado* que la función trabaja correctamente y devuelve el valor apropiado.

De hecho, usted ya practica dicho salto de fe cuando usa funciones internas. Cuando llama a `math.cos` o a `math.exp`, no examina la implementación de

dichas funciones. Simplemente da por sentado que funcionan porque los que escribieron las bibliotecas internas de Python son buenos programadores.

Lo mismo se aplica cuando llama a una de las funciones programadas por usted. Por ejemplo en la Sección 5.4, escribimos una función llamada **esDivisible** que determina si un número es divisible por otro. Una vez que nos hayamos convencido de que dicha función es correcta, comprobando y examinando el código, podremos usar la función sin tener siquiera que volver a mirar el código otra vez.

Lo mismo vale para los programas recursivos. Cuando llegue a la llamada recursiva, en lugar de seguir el flujo de ejecución, tendría que dar por supuesto que la llamada recursiva funciona (es decir, devuelve el resultado correcto) y luego preguntarse: “suponiendo que puedo hallar el factorial de $n - 1$, ¿puedo hallar el factorial de n ?” En este caso, está claro que sí puede, multiplicándolo por n .

Por supuesto, es un tanto extraño dar por supuesto que la función está bien cuando ni siquiera ha acabado de escribirla, pero precisamente por eso se llama acto de fe.

5.7. Un ejemplo más

En el ejemplo anterior, usamos variables temporales para ir apuntando los resultados y para hacer que el código fuese más fácil de depurar, pero podríamos habernos ahorrado unas cuantas líneas:

```
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n-1)
```

De ahora en adelante, tenderemos a usar la versión más concisa, pero le recomendamos que utilice la versión más explícita mientras se halle desarrollando código. Cuando lo tenga funcionando, lo podrá acortar, si se siente inspirado.

Después de **factorial**, el ejemplo más común de una función matemática recursivamente definida es **fibonacci**, que presenta la siguiente definición:

$$\begin{aligned} \text{fibonacci}(0) &= 1 \\ \text{fibonacci}(1) &= 1 \\ \text{fibonacci}(n) &= \text{fibonacci}(n-1) + \text{fibonacci}(n-2); \end{aligned}$$

Traducido a Python, es como sigue:

```
def fibonacci (n):  
    if n == 0 or n == 1:  
        return 1  
    else:  
        return fibonacci(n-1) + fibonacci(n-2)
```

Si intenta seguir el flujo de ejecución aquí, incluso para valores relativamente pequeños de n , le puede dar un dolor de cabeza. Pero si confiamos en el acto de fe, si da por supuesto que las dos llamadas recursivas funcionan correctamente, entonces estará claro que obtiene el resultado correcto al sumarlas juntas.

5.8. Comprobación de tipos

¿Qué sucede si llamamos a `factorial` y le damos 1.5 como argumento?

```
>>> factorial (1.5)  
RuntimeError: Maximum recursion depth exceeded
```

Tiene todo el aspecto de una recursión infinita. Pero, ¿cómo ha podido ocurrir? Hay una condición de salida o caso base: cuando `n == 0`. El problema es que el valor de `n` yerra el caso base.

En la primera llamada recursiva, el valor de `n` es 0.5. En la siguiente vez su valor es -0.5. A partir de ahí, se vuelve más y más pequeño, pero nunca será 0.

Tenemos dos opciones. Podemos intentar generalizar la función `factorial` para que trabaje con números de coma flotante, o podemos hacer que `factorial` compruebe el tipo de su parámetro. La primera opción se llama función gamma, y está más allá del objetivo de este libro. Así pues, tomemos la segunda.

Podemos usar la función `type` para comparar el tipo del parámetro con el tipo de un valor entero conocido (por ejemplo 1). Ya que estamos en ello, podemos asegurarnos de que el parámetro sea positivo:

```
def factorial (n):  
    if type(n) != type(1):  
        print "El factorial está definido sólo para enteros."  
        return -1  
    elif n < 0:  
        print "El factorial está definido sólo para enteros\  
positivos."  
        return -1  
    elif n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)
```

Ahora tenemos tres condiciones de salida o casos base. El primero filtra los números no enteros. El segundo evita los enteros negativos. En ambos casos, se muestra un mensaje de error y se devuelve un valor especial, -1, para indicar a quien hizo la llamada a la función que algo fue mal:

```
>>> factorial (1.5)
El factorial esta definido solo para enteros.
-1
>>> factorial (-2)
El factorial esta definido solo para enteros positivos.
-1
>>> factorial ("paco")
El factorial esta definido solo para enteros.
-1
```

Si pasamos ambas comprobaciones, entonces sabemos que n es un entero positivo y podemos probar que la recursión termina.

Este programa muestra un patrón que se llama a veces **guardián**. Las primeras dos condicionales actúan como guardianes, protegiendo al código que sigue de los valores que pudieran causar errores. Los guardianes hacen posible demostrar la corrección del código.

5.9. Glosario

función productiva: Función que devuelve un valor de retorno.

valor de retorno: El valor obtenido como resultado de una llamada a una función.

variable temporal: Variable utilizada para almacenar un valor intermedio en un cálculo complejo.

código muerto: Parte de un programa que no podrá ejecutarse nunca, a menudo debido a que aparece tras una sentencia de **return**.

None: Valor especial de Python que devuelven funciones que o bien no tienen sentencia de **return** o bien tienen una sentencia de **return** sin argumento.

desarrollo incremental: Un método de desarrollo de programas que busca evitar el depurado añadiendo y probando una pequeña cantidad de código en cada paso.

andamiaje: El código que se usa durante el desarrollo del programa pero que no es parte de la versión final.

guardián: Una condición que comprueba y maneja circunstancias que pudieran provocar un error.

Capítulo 6

Iteración

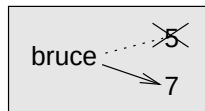
6.1. Asignación múltiple

Es posible que haya descubierto que es posible hacer más de una asignación a una misma variable. El efecto de la nueva asignación es redirigir la variable de manera que deje de remitir al valor antiguo y empiece a remitir al valor nuevo.

```
bruno = 5  
print bruno,  
bruno = 7  
print bruno
```

La salida del programa es 5 7, ya que la primera vez que imprimimos **bruno** su valor es 5, y la segunda vez su valor es 7. La coma al final de la primera sentencia **print** impide que se imprima una nueva línea en ese punto, por eso ambos valores aparecen en la misma línea.

He aquí el aspecto de una asignación múltiple en un diagrama de estado:



Cuando hay asignaciones múltiples a una variable, es especialmente importante distinguir entre una sentencia de asignación y una sentencia de igualdad. Puesto que Python usa el símbolo **=** para la asignación, es tentador interpretar una sentencia como **a = b** como sentencia de igualdad. Pero no lo es.

Para empezar, la igualdad es conmutativa, y la asignación no lo es. Por ejemplo en matemáticas si $a = 7$ entonces $7 = a$. Pero en Python la sentencia `a = 7` es legal, y `7 = a` no lo es.

Y lo que es más, en matemáticas, una sentencia de igualdad es verdadera todo el tiempo. Si $a = b$ ahora, entonces a siempre será igual a b . En Python, una sentencia de asignación puede hacer que dos variables sean iguales, pero no tienen por qué quedarse así.

```
a = 5
b = a    # a y b son ahora iguales
a = 3    # a y b ya no son iguales
```

La tercera línea cambia el valor de `a` pero no cambia el valor de `b`, y por lo tanto ya dejan de ser iguales. En algunos lenguajes de programación, se usa para la asignación un símbolo distinto, como `<-` o como `:=`, para evitar la confusión.

Aunque la asignación múltiple es útil a menudo, debe usarla con cuidado. Si los valores de las variables van cambiando constantemente en distintas partes del programa, podría suceder que el código sea difícil de leer y mantener.

6.2. La sentencia `while`

Una de las tareas para las que los computadores se usan con frecuencia es la automatización de tareas repetitivas. Repetir tareas similares o idénticas es algo que los computadores hacen bien y las personas no hacen tan bien.

Hemos visto dos programas, `nLineas` y `cuenta_atras`, que usan la recursividad para llevar a cabo la repetición, que también se llama **iteración**. Por ser la iteración tan habitual, Python proporciona como lenguaje varias características que la hacen más fácil. La primera característica que vamos a considerar es la sentencia `while`.

Éste es el aspecto de `cuenta_atras` con una sentencia `while`:

```
def cuenta_atras(n):
    while n > 0:
        print n
        n = n-1
    print "Despegando!"
```

Como eliminamos la llamada recursiva, esta función no es recursiva.

Casi podría leer una sentencia `while` como si fuera inglés (castellano “mientras”). Quiere decir que “Mientras `n` sea mayor que cero, continúa mostrando el valor de `n` y después restándole 1 al valor de `n`. Cuando llegues a cero, muestra la palabra “Despegando!”.

Más formalmente, el flujo de ejecución de una sentencia **while** es el siguiente:

1. Evaluar la condición, devolviendo 0 o 1.
2. Si la condición es falsa (0), salir de la sentencia **while** y continuar la ejecución en la siguiente sentencia.
3. Si la condición es verdadera (1), ejecutar cada una de las sentencias en el cuerpo del bucle **while**, y luego volver al paso 1.

El cuerpo está formado por todas las sentencias bajo el encabezado que tienen el mismo sangrado.

Este tipo de flujo de llama **bucle** porque el tercer paso vuelve de nuevo arriba. Nótese que si la condición es falsa la primera vez que se atraviesa el bucle, las sentencias del interior del bucle no se ejecutan nunca.

El cuerpo del bucle debe cambiar el valor de una o más variables de manera que, llegado el momento, la condición sea falsa y el bucle termine. En caso contrario, el bucle se repetirá para siempre, que es lo que se llama **bucle infinito**. Una infinita fuente de diversión para los científicos informáticos es la observación de que las instrucciones del champú “lavar, aclarar, repetir”, son un bucle infinito.

En el caso de **cuenta_atras**, podemos probar que el bucle terminará porque sabemos que el valor de **n** es finito, y podemos ver que el valor de **n** disminuye cada vez que se atraviesa el bucle (cada iteración), de manera que ea la larga tenemos que llegar a cero. En otros casos no es tan fácil decirlo:

```
def secuencia(n):  
    while n != 1:  
        print n,  
        if n%2 == 0:           # n es par  
            n = n/2  
        else:                  # n es impar  
            n = n*3+1
```

La condición de este bucle es **n != 1**, de manera que el bucle continuará hasta que **n** sea 1, que hará que la condición sea falsa.

En cada iteración, el programa muestra como salida el valor de **n** y luego comprueba si es par o impar. Si es par, el valor de **n** se divide entre dos. Si es impar, el valor se sustituye por $3n+1$. Por ejemplo, si el valor de comienzo (el argumento pasado a la secuencia) es 3, la secuencia resultante es 3, 10, 5, 16, 8, 4, 2, 1.

Puesto que **n** a veces aumenta y a veces disminuye, no hay una prueba obvia de que **n** alcance alguna vez el valor 1, o de que el programa vaya a terminar. Para algunos valores particulares de **n**, podemos probar la terminación. Por ejemplo,

si el valor de inicio es una potencia de dos, entonces el valor de `n` será par cada vez que se pasa a través del bucle, hasta que lleguemos a 1. El ejemplo anterior acaba con dicha secuencia, empezando por 16.

Dejando aparte valores particulares, la pregunta interesante es si podemos probar que este programa terminará para *todos* los valores de `n`. Hasta la fecha, nadie ha sido capaz de probarlo o negarlo.

Como actividad, reescriba la función `nLines` de la sección 4.9 utilizando iteración en lugar de recursividad.

6.3. Tablas

Una de las cosas para las que resultan buenos los bucles es para generar datos tabulares. Antes de que los computadores estuvieran disponibles de forma masiva, la gente tenía que calcular a mano logaritmos, senos, cosenos y otras funciones matemáticas. Para facilitarlos, los libros de matemáticas contenían largas tablas donde aparecían los valores de estas funciones. Confeccionar estas tablas era una tarea lenta y pesada, y el resultado estaba lleno de erratas.

Cuando los computadores aparecieron en escena, una de las primeras reacciones fue “¡Qué bueno! Podemos usar los computadores para generar las tablas, así no habrá errores”. Resultó cierto (casi), pero no se vio más allá. Poco después los computadores y las calculadoras científicas se hicieron tan ubicuas que las tablas resultaron obsoletas.

Bueno, casi. Resulta que para ciertas operaciones, los computadores usan tablas para obtener un valor aproximado, y luego ejecutan cálculos para mejorar la aproximación. En algunos casos, ha habido errores en las tablas subyacentes; el más famoso estaba en la tabla que el Pentium de Intel usaba para llevar a cabo la división de coma flotante.

Aunque una tabla logarítmica ya no es tan útil como lo fuera antaño, todavía constituye un buen ejemplo de iteración. El siguiente programa muestra una secuencia de valores en la columna izquierda y sus logaritmos en la columna derecha:

```
x = 1.0
while x < 10.0:
    print x, '\t', math.log(x)
    x = x + 1.0
```

El `\t` representa un carácter de **tabulación**.

Tal como se van mostrando en la pantalla caracteres y cadenas, un señalador invisible llamado **cursor** lleva la cuenta de dónde irá el próximo carácter. Tras una sentencia `print`, el cursor va normalmente al principio de la línea siguiente.

El carácter de tabulación hace que el cursor se desplace a la derecha hasta que alcance uno de los marcadores de tabulación. Los tabuladores son útiles para alinear columnas de texto, como en la salida del programa anterior:

1.0	0.0
2.0	0.69314718056
3.0	1.09861228867
4.0	1.38629436112
5.0	1.60943791243
6.0	1.79175946923
7.0	1.94591014906
8.0	2.07944154168
9.0	2.19722457734

Si estos valores le parecen raros, recuerde que la función `log` usa como base `e`. Debido a que las potencias de dos son muy importantes en las ciencias de la computación, generalmente queremos hallar los logaritmos en relación con la base dos. Para llevarlo a cabo, podemos usar la siguiente fórmula:

$$\log_2 x = \frac{\log_e x}{\log_e 2} \quad (6.1)$$

Cambiar la sentencia de salida a:

```
print x, '\t', math.log(x)/math.log(2.0)
```

devuelve

1.0	0.0
2.0	1.0
3.0	1.58496250072
4.0	2.0
5.0	2.32192809489
6.0	2.58496250072
7.0	2.80735492206
8.0	3.0
9.0	3.16992500144

Podemos ver que 1, 2, 4 y 8 son potencias de dos, porque sus logaritmos de base 2 son números enteros. Si quisiéramos encontrar los logaritmos de otras potencias de dos, podríamos modificar el programa de la siguiente manera:

```
x = 1.0
while x < 100.0:
```

```
print x, '\t', math.log(x)/math.log(2.0)
x = x * 2.0
```

Ahora, en lugar de añadir algo a `x` cada vez que atravesamos el bucle, que devuelve una secuencia aritmética, multiplicamos `x` por algo, devolviendo una secuencia geométrica. El resultado es:

```
1.0    0.0
2.0    1.0
4.0    2.0
8.0    3.0
16.0   4.0
32.0   5.0
64.0   6.0
```

Debido a que usamos caracteres de tabulación entre las columnas, la posición de la segunda columna no depende del número de dígitos de la primera columna.

Las tablas logarítmicas quizás ya no sean útiles, pero conocer las potencias de dos no ha dejado de serlo para los científicos informáticos.

Como actividad, modifique el programa para que muestre las potencias de dos hasta 65536 (es decir, 2^{16}). Imprímala y memorícela.

El carácter de barra invertida en `'\t'` indica el principio de una **secuencia de escape**. Las secuencias de escape se usan para representar caracteres invisibles como tabulaciones y retornos de carro. La secuencia `\n` representa un retorno de carro.

Una sentencia de escape puede aparecer en cualquier lugar de una cadena; en el ejemplo, la secuencia de escape del tabulador es la única de la cadena.

¿Cómo cree que puede representar una barra invertida en una cadena?

Como ejercicio, escriba un única cadena que

```
presente
      esta
          salida.
```

6.4. Tablas de dos dimensiones

Una tabla de dos dimensiones es una tabla en la que Usted elige una fila y una columna y lee el valor de la intersección. Un buen ejemplo es una tabla de

multiplicar. Supongamos que desea imprimir una tabla de multiplicar para los valores del 1 al 6.

Una buena manera de comenzar es escribir un bucle sencillo que imprima los múltiplos de 2, todos en una línea.

```
i = 1
while i <= 6:
    print 2*i, '\t',
    i = i + 1
print
```

La primera línea inicializa una variable llamada `i`, que actuará como contador, o **variable de bucle**. Conforme se ejecuta el bucle, el valor de `i` se incrementa de 1 a 6. Cuando `i` vale 7, el bucle termina. Cada vez que se atraviesa el bucle, imprimimos el valor `2*i` seguido por tres espacios.

De nuevo, la coma de la sentencia `print` suprime el salto de línea. Después de completar el bucle, la segunda sentencia `print` crea una línea nueva.

La salida de este programa es:

```
2      4      6      8      10     12
```

Hasta ahora, bien. El siguiente paso es **encapsular** y **generalizar**.

6.5. Encapsulado y generalización

Por “encapsulado” generalmente se entiende tomar una pieza de código y envolverla en una función, permitiéndole obtener las ventajas de todo aquello para lo que valen las funciones. Hemos visto dos ejemplos de encapsulado, cuando escribimos `imprimeParidad` en la Sección 4.5 y `esDivisible` en la Sección 5.4.

Por “generalización” entendemos tomar algo específico, como imprimir los múltiplos de 2, y hacerlo más general, como imprimir los múltiplos de cualquier entero.

He aquí una función que encapsula el bucle de la sección anterior y la generaliza para imprimir múltiplos de `n`.

```
def imprimeMultiplos(n):
    i = 1
    while i <= 6:
        print n*i, '\t',
        i = i + 1
    print
```

Para encapsular, todo lo que hubimos de hacer fue añadir la primera línea, que declara el nombre de la función y la lista de parámetros. Para generalizar, todo lo que tuvimos que hacer fue sustituir el valor 2 por el parámetro `n`.

Si llamamos a esta función con el argumento 2, obtenemos la misma salida que antes. Con el argumento 3, la salida es:

```
3      6      9      12     15     18
```

y con argumento 4, la salida es

```
4      8      12     16     20     24
```

A estas alturas es probable que haya adivinado cómo vamos a imprimir una tabla de multiplicación: llamaremos a `imprimeMultiplos` repetidamente con diferentes argumentos. De hecho, podemos a usar otro bucle:

```
i = 1
while i <= 6:
    imprimeMultiplos(i)
    i = i + 1
```

Observe hasta qué punto este bucle es similar al que hay en el interior de `imprimeMultiplos`. Todo lo que hicimos fue sustituir la sentencia `print` por una llamada a una función.

La salida de este programa es una tabla de multiplicación:

```
1      2      3      4      5      6
2      4      6      8     10     12
3      6      9     12     15     18
4      8     12     16     20     24
5     10     15     20     25     30
6     12     18     24     30     36
```

6.6. Más encapsulación

Para dar más ejemplos de encapsulación, tomaremos el código del final de la Sección 6.5 y lo envolveremos en una función:

```
def imprimeTablaMult():
    i = 1
    while i <= 6:
        imprimeMultiplos(i)
        i = i + 1
```

El proceso que mostramos aquí es un **plan de desarrollo** habitual. Se desarrolla gradualmente el código añadiendo líneas fuera de cualquier función o en

el intérprete. Cuando conseguimos que funcionen, se extraen y se envuelven en una función.

Este plan de desarrollo es especialmente si, al comenzar a escribir, no sabe cómo dividir el programa en funciones. Este enfoque le permite diseñarlo sobre la marcha.

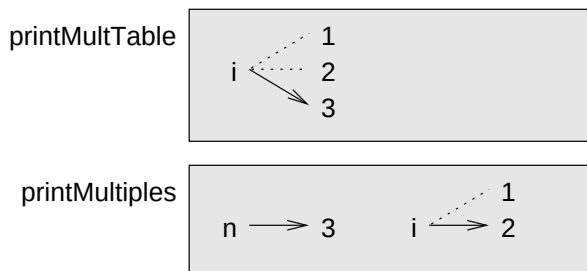
6.7. Variables locales

Quizá se esté preguntando cómo podemos usar la misma variable tanto en `imprimeMultiplos` como en `imprimeTablaMult`. ¿No habrá problemas cuando una de las funciones cambie los valores de la variable?

La respuesta es no, ya que la variable `i` en `imprimeMultiplos` y la variable `i` in `imprimeTablaMult` *no* son la misma variable.

Las variables creadas dentro de una función son locales. No puede acceder a una variable local fuera de su función “huésped”. Eso significa que es posible tener múltiples variables con el mismo nombre, siempre que no estén en la misma función.

El diagrama de pila de esta función muestra claramente que las dos variables llamadas `i` no son la misma variable. Pueden referirse a diferentes valores, y cambiar uno no afecta al otro.



El valor de `i` en `imprimeTablaMult` va desde 1 hasta 6. En el diagrama, resulta ser 3. El próximo recorrido del bucle será 4. En cada recorrido del bucle, `imprimeTablaMult` llama a `imprimeMultiplos` con el valor actual de `i` como argumento. Ese valor se asigna al parámetro `n`.

Dentro de `imprimeMultiplos`, el valor de `i` va desde 1 hasta 6. En el diagrama, resulta ser 2. Los cambios en esta variable no tienen ningún efecto sobre el valor de `i` en `imprimeTablaMult`.

Es habitual y perfectamente legal tener diferentes variables locales con el mismo nombre. En especial, los nombres `i`, `j` y `k` se suelen usar como variables de

bucle. Si evita usarlas en una función porque las utilizó en algún otro lugar, probablemente consiga que el programa sea más difícil de leer.

6.8. Más generalización

Como otro ejemplo de generalización, imagine que desea un programa que imprima una tabla de multiplicación de cualquier tamaño, y no sólo la tabla de 6x6. Podría añadir un parámetro a `imprimeTablaMult`:

```
def imprimeTablaMult(mayor):
    i = 1
    while i <= mayor:
        imprimeMultiplos(i)
        i = i + 1
```

Hemos sustituido el valor 6 con el parámetro `mayor`. Si ahora se llama a `imprimeTablaMult` con el argumento 7, obtenemos:

1	2	3	4	5	6
2	4	6	8	10	12
3	6	9	12	15	18
4	8	12	16	20	24
5	10	15	20	25	30
6	12	18	24	30	36
7	14	21	28	35	42

lo que es correcto, excepto por el hecho de que seguramente queremos que la tabla esté cuadrada, con el mismo número de filas que de columnas. Para hacerlo, añadimos otro parámetro a `imprimeMultiplos`, a fin de especificar cuántas columnas tendría que tener la tabla.

Sólo para fastidiar, llamaremos también a este parámetro `mayor`, para demostrar que diferentes funciones pueden tener parámetros con el mismo nombre (al igual que las variables locales). Aquí tenemos el programa completo:

```
def imprimeMultiplos(n, mayor):
    int i = 1
    while i <= mayor:
        print n*i, '\t',
        i = i + 1
    print

def imprimeTablaMult(mayor):
    int i = 1
    while i <= mayor:
```



```
imprimeMultiplos(i, mayor)
i = i + 1
```

Nótese que al añadir un nuevo parámetro, tuvimos que cambiar la primera línea de la función (el encabezado de la función), y tuvimos también que cambiar el lugar donde se llama a la función en `imprimeTablaMult`.

Según lo esperado, este programa genera una tabla cuadrada de 7x7:

1	2	3	4	5	6	7
2	4	6	8	10	12	14
3	6	9	12	15	18	21
4	8	12	16	20	24	28
5	10	15	20	25	30	35
6	12	18	24	30	36	42
7	14	21	28	35	42	49

Cuando generaliza correctamente una función, a menudo se encuentra con que el programa resultante tiene capacidades que Usted no pensaba. Por ejemplo, quizá observe que la tabla de multiplicación es simétrica, porque $ab = ba$, de manera que todas las entradas de la tabla aparecen dos veces. Puede ahorrar tinta imprimiendo sólo la mitad de la tabla. Para hacerlo, sólo tiene que cambiar una línea de `imprimeTablaMult`. Cambie

```
imprimeMultiplos(i, mayor)
```

por

```
imprimeMultiplos(i, i)
```

y obtendrá

1						
2	4					
3	6	9				
4	8	12	16			
5	10	15	20	25		
6	12	18	24	30	36	
7	14	21	28	35	42	49

Como actividad, siga o trace la ejecución de esta nueva versión de `imprimeTablaMult` para hacerse una idea de cómo funciona.

6.9. Funciones

Hasta el momento hemos mencionado en alguna ocasión “todas las cosas para las que sirven las funciones”. Puede que ya se esté preguntando qué cosas son exactamente. He aquí algunas de las razones por las que las funciones son útiles:

- Al dar un nombre a una secuencia de sentencias, hace que su programa sea más fácil de leer y depurar.
- Dividir un programa largo en funciones le permite separar partes del programa, depurarlas aisladamente, y luego recomponerlas en un todo.
- Las funciones facilitan tanto la recursividad como la iteración.
- Las funciones bien diseñadas son generalmente útiles para más de un programa. Una vez escritas y depuradas, pueden reutilizarse.

6.10. Glosario

asignación múltiple: Hacer más de una asignación a la misma variable durante la ejecución de un programa.

iteración: La ejecución repetida de un conjunto de sentencias por medio de una llamada recursiva a una función o un bucle.

bucle: Sentencia o grupo de sentencias que se ejecutan repetidamente hasta que se cumple una condición de terminación.

bucle infinito: Bucle cuya condición de terminación nunca se cumple.

cuerpo: Las sentencias que hay dentro de un bucle.

variable de bucle: Variable que se usa para determinar la condición de terminación de un bucle.

tabulador: Carácter especial que hace que el cursor se mueva hasta la siguiente marca de tabulación en la línea actual.

nueva línea: Un carácter especial que hace que el cursor se mueva al inicio de la siguiente línea.

cursor: Un marcador invisible que sigue el rastro de dónde se imprimirá el siguiente carácter.

secuencia de escape: Carácter de escape (\) seguido por uno o más caracteres imprimibles, que se usan para designar un carácter no imprimible.

encapsular: Dividir un programa largo y complejo en componentes (como las funciones) y aislar los componentes unos de otros (por ejemplo usando variables locales).

generalizar: Sustituir algo innecesariamente específico (como es un valor constante) con algo convenientemente general (como es una variable o parámetro). La generalización hace el código más versátil, más apto para reutilizarse y algunas veces incluso más fácil de escribir.

plan de desarrollo: Proceso para desarrollar un programa. En este capítulo, hemos mostrado un estilo de desarrollo basado en desarrollar código para hacer cosas simples y específicas, y luego encapsularlas y generalizarlas.

Capítulo 7

Cadenas

7.1. Un tipo de datos compuesto

Hasta el momento hemos visto tres tipos: `int`, `float`, y `string`. Las cadenas son cuantitativamente diferentes de los otros dos porque están hechas de piezas menores: caracteres.

Los tipos que comprenden piezas menores se llaman **tipos de datos compuestos**. Dependiendo de qué estemos haciendo, podemos querer tratar un tipo compuesto como una única cosa o acceder a sus partes. Esta ambigüedad es útil.

El operador corchete selecciona un carácter suelto de una cadena.

```
>>> fruta = "banana"
>>> letra = fruta[1]
>>> print letra
```

La expresión `fruta[1]` selecciona el carácter número 1 de `fruta`. La variable `letra` apunta al resultado. Cuando mostramos `letra`, nos encontramos con una sorpresa:

a

La primera letra de `"banana"` no es `a`. A no ser que usted sea un programador. Por perversas razones, los científicos de la computación siempre empiezan a contar desde cero. La 0-sima letra (“cerósima”) de `"banana"` es `b`. La 1-ésima (“unésima”) es `a`, y la 2-ésima (“dosésima”) letra es `n`.

Si quiera la cerósima letra de una cadena, simplemente pone 0, o cualquier expresión de valor 0, entre los corchetes:

```
>>> letra = fruta[0]
>>> print letra
b
```

A la expresión entre corchetes se le llama **índice**. Un índice identifica a un miembro de un conjunto ordenado, en este caso el conjunto de caracteres de la cadena. El índice *indica* cuál quiere usted, de ahí el nombre. Puede ser cualquier expresión entera.

7.2. Longitud

La función `len` devuelve el número de caracteres de una cadena:

```
>>> fruta = "banana"
>>> len(fruta)
6
```

Para obtener la última letra de una cadena puede sentirse tentado a probar algo como esto:

```
longitud = len(fruta)
ultima = fruta[longitud]          # ERROR!
```

Eso no funcionará. Provoca un error en tiempo de ejecución `IndexError: string index out of range`. La razón es que no hay una sexta letra en "banana". Como empezamos a contar por cero, las seis letras están numeradas del 0 al 5. Para obtener el último carácter tenemos que restar 1 de `longitud`:

```
longitud = len(fruta)
ultima = fruta[longitud-1]
```

De forma alternativa, podemos usar índices negativos, que cuentan hacia atrás desde el final de la cadena. La expresión `fruta[-1]` nos da la última letra. `fruta[-2]` nos da la penúltima, y así.

7.3. Recorrido y el bucle for

Muchos cálculos incluyen el proceso de una cadena carácter a carácter. A menudo empiezan por el principio, seleccionan cada carácter por turno, hacen algo con él y siguen hasta el final. Este patrón de proceso se llama **recorrido**. Una forma de codificar un recorrido es una sentencia `while`:

```
indice = 0
while indice < len(fruta):
    letra = fruta[indice]
```

```
print letra
indice = indice + 1
```

Este bucle recorre la cadena y muestra cada letra en una línea distinta. La condición del bucle es `indice < len(fruta)`, de modo que cuando `indice` es igual a la longitud de la cadena, la condición es falsa y no se ejecuta el cuerpo del bucle. El último carácter al que se accede es el que tiene el índice `len(fruta)-1`, que es el último carácter de la cadena.

Como ejercicio, escriba una función que tome una cadena como argumento y entregue las letras en orden inverso, una por línea.

Es tan habitual usar un índice para recorrer un conjunto de valores que Python facilita una sintaxis alternativa más simple: el bucle `for`:

```
for car in fruta:
    print car
```

Cada vez que recorremos el bucle, se asigna a la variable `car` el siguiente carácter de la cadena. El bucle continúa hasta que no quedan caracteres.

El ejemplo siguiente muestra cómo usar la concatenación junto a un bucle `for` para generar una serie abecedárica. “Abecedárica” es la serie o lista en la que cada uno de los elementos aparece en orden alfabético. Por ejemplo, en el libro de Robert McCloskey *Make Way for Ducklings (Dejad paso a los patitos)*, los nombres de los patitos son Jack, Kack, Lack, Mack, Nack, Ouack, Pack, y Quack. Este bucle saca esos nombres en orden:

```
prefijos = "JKLMNPOQ"
sufijo = "ack"
```

```
for letra in prefijos:
    print letra + sufijo
```

La salida del programa es:

```
Jack
Kack
Lack
Mack
Nack
Ouack
Pack
Quack
```

Por supuesto, esto no es del todo correcto, porque “Ouack” y “Quack” no están correctamente escritos.

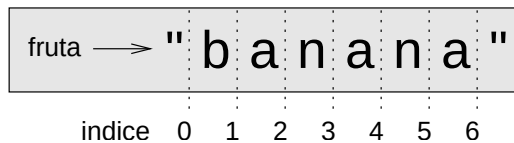
Como ejercicio, modifique el programa para corregir este error.

7.4. Porciones de cadenas

Llamamos **porción** a un segmento de una cadena. La selección de una porción es similar a la selección de un carácter:

```
>>> s = "Pedro, Pablo, y María"
>>> print s[0:5]
Pedro
>>> print s[7:12]
Pablo
>>> print s[15:20]
María
```

El operador `[n:m]` devuelve la parte de la cadena desde el *n*ésimo carácter hasta el *m*ésimo, incluyendo el primero pero excluyendo el último. Este comportamiento contradice a nuestra intuición; tiene más sentido si imagina los índices señalando *entre* los caracteres, como en el siguiente diagrama:



Si omite el primer índice (antes de los dos puntos), la porción comienza al principio de la cadena. Si omite el segundo índice, la porción llega al final de la cadena. Así:

```
>>> fruta = "banana"
>>> fruta[:3]
'ban'
>>> fruta[3:]
'ana'
```

¿Qué cree usted que significa `s[:]`?

7.5. Comparación de cadenas

Los operadores de comparación trabajan sobre cadenas. Para ver si dos cadenas son iguales:


```
if palabra == "banana":  
    print "Sí, no tenemos bananas!"
```

Otras operaciones de comparación son útiles para poner palabras en orden alfabético:

```
if palabra < "banana":  
    print "Tu palabra," + palabra + ", va antes de banana."  
elif palabra > "banana":  
    print "Tu palabra," + palabra + ", va después de banana."  
else:  
    print "Sí, no tenemos bananas!"
```

Sin embargo, debería usted ser consciente de que Python no maneja las mayúsculas y minúsculas como lo hace la gente. Todas las mayúsculas van antes de la minúsculas. Como resultado de ello:

Tu palabra, Zapato, va antes de banana.

Una forma común de abordar este problema es convertir las cadenas a un formato estándar, como pueden ser las minúsculas, antes de realizar la comparación. Un problema mayor es hacer que el programa se dé cuenta de que los zapatos no son frutas.

7.6. Las cadenas son inmutables

Es tentador usar el operador `[]` en el lado izquierdo de una asignación, con la intención de cambiar un carácter en una cadena. Por ejemplo:

```
saludo = "Hola, mundo"  
saludo[0] = 'M'           # ERROR!  
print saludo
```

En lugar de presentar la salida `Mola, mundo`, este código presenta el siguiente error en tiempo de ejecución `TypeError: object doesn't support item assignment`.

Las cadenas son **inmutables**, lo que significa que no puede cambiar una cadena existente. Lo más que puede hacer es crear una nueva cadena que sea una variación de la original:

```
saludo = "Hola, mundo"  
nuevoSaludo = 'M' + saludo[1:]  
print nuevoSaludo
```

Aquí la solución es concatenar una nueva primera letra a una porción de **saludo**. Esta operación no tiene efectos sobre la cadena original.

7.7. Una función “encuentra”

¿Qué hace la siguiente función?

```
def encuentra(cad, c):  
    indice = 0  
    while indice < len(cad):  
        if cad[indice] == c:  
            return indice  
        indice = indice + 1  
    return -1
```

En cierto sentido, **encuentra** es lo contrario del operador `[]`. En lugar de tomar un índice y extraer el carácter correspondiente, toma un carácter y encuentra el índice donde aparece el carácter. Si el carácter no se encuentra, la función devuelve `-1`.

Este es el primer ejemplo que hemos visto de una sentencia **return** dentro de un bucle. Si `cad[indice] == c`, la función vuelve inmediatamente, escapando del bucle prematuramente.

Si el carácter no aparece en la cadena, el programa sale del bucle normalmente y devuelve `-1`.

Este patrón de computación se llama a veces un recorrido “eureka” porque en cuanto encontramos lo que buscamos, podemos gritar “¡Eureka!” y dejar de buscar.

*A modo de ejercicio, modifique la función **encuentra** para que acepte un tercer parámetro, el índice de la cadena donde debería empezar a buscar.*

7.8. Bucles y conteo

El programa que sigue cuenta el número de veces que la letra **a** aparece en una cadena:

```
fruta = "banana"
cuenta = 0
for car in fruta:
    if car == 'a':
        cuenta = cuenta + 1
print cuenta
```

Este programa muestra otro patrón de computación llamado **contador**. La variable `cuenta` se inicializa a 0 y luego se incrementa cada vez que se encuentra una **a**. (**Incrementar** es aumentar en uno; es lo contrario de **decrementar**, y sin relación alguna con “excremento”, que es un nombre.) Al salir del bucle, `cuenta` contiene el resultado – el número total de **a**s.

Como ejercicio, encapsule este código en una función llamada `cuentaLetras`, y generalícela de forma que acepte la cadena y la letra como parámetros.

Como un segundo ejercicio, reescriba esta función para que en lugar de recorrer la cadena, use la versión de tres parámetros de `encuentra` del anterior.

7.9. El módulo “string”

El módulo `string` contiene funciones útiles para manipular cadenas. Como es habitual, tenemos que importar el módulo antes de poder usarlo:

```
>>> import string
```

El módulo `string` incluye una función llamada `find` que hace lo mismo que la función `encuentra` que escribimos. Para llamarla debemos especificar el nombre del módulo y el nombre de la función por medio de la notación de punto.

```
>>> fruta = "banana"
>>> indice = string.find(fruta, "a")
>>> print indice
1
```

Este ejemplo demuestra uno de los beneficios de los módulos: ayudan a evitar las colisiones entre los nombres de las funciones predefinidas y las definidas por el usuario. Al usar la notación de punto podríamos especificar qué versión de `find` queremos en caso de haberle dado un nombre en inglés a nuestra función.

En realidad, `string.find` es más general que nuestra versión. Para empezar, puede encontrar subcadenas, no sólo caracteres:

```
>>> string.find("banana", "na")
2
```

Además, acepta un argumento adicional que especifica el índice en el que debería comenzar:

```
>>> string.find("banana", "na", 3)
4
```

O puede tomar dos argumentos adicionales que especifican un intervalo de índices:

```
>>> string.find("sus", "s", 1, 2)
-1
```

En este ejemplo, la búsqueda falla porque la letra *s* no aparece en el intervalo de índices desde 1 hasta 2 (sin incluir 2).

7.10. Clasificación de caracteres

A menudo viene bien examinar un carácter y comprobar si es una letra mayúscula o minúscula, o si es un carácter o un dígito. El módulo `string` proporciona varias constantes que son útiles para estos menesteres.

La cadena `string.lowercase` contiene todas las letras que el sistema considera como minúsculas. De forma similar, `string.uppercase` contiene todas las mayúsculas. Pruebe lo que sigue y vea qué obtiene:

```
>>> print string.lowercase
>>> print string.uppercase
>>> print string.digits
```

Podemos usar estas constantes y `find` para clasificar caracteres. Por ejemplo, si `find(lowercase, c)` devuelve un valor que no sea `-1`, entonces `c` es una minúscula:

```
def esMinuscula(c):
    return find(string.lowercase, c) != -1
```

Alternativamente, podemos aprovecharnos del operador `in`, que determina si un carácter aparece en una cadena:

```
def esMinuscula(c):
    return c in string.lowercase
```

Como una alternativa más, podemos usar el operador de comparación, aunque esta solución sólo sea práctica para el alfabeto inglés:

```
def esMinuscula(c):  
    return 'a' <= c <= 'z'
```

Si *c* está entre *a* y *z*, tiene que ser una minúscula.

Como ejercicio, explique qué versión de `esMinuscula` cree que es más rápida. ¿Puede pensar en otras razones aparte de la velocidad para preferir una sobre la otra?

Otra constante definida en el módulo `string` puede sorprenderle cuando la imprima:

```
>>> print string.whitespace
```

Los caracteres de `whitespace` mueven el cursor sin imprimir nada. Crean los espacios en blanco entre los caracteres visibles (al menos sobre papel blanco). La constante `string.whitespace` contiene todos los caracteres de espacio en blanco, incluidos espacio, tabulador (`\t`), y salto de línea (`\n`).

Hay otras funciones útiles en el módulo `string`, pero este libro no pretende ser un manual de referencia. Por otra parte, la *Referencia de la Biblioteca de Python* sí lo es. Junto con un montón más de documentación, está disponible en el sitio web de Python, www.python.org.

7.11. Glosario

tipo de datos compuesto: Un tipo de datos en el que los valores están hechos de componentes o elementos que son a su vez valores.

recorrer: Realizar de forma iterativa una operación similar sobre cada uno de los elementos de un conjunto.

índice: Una variable o valor usado para seleccionar un miembro de un conjunto ordenado, como puede ser un carácter de una cadena.

porción: Una parte de una cadena especificada por un intervalo de índices.

mutable: Un tipo de datos compuesto a cuyos elementos se les puede asignar nuevos valores.

contador: Una variable usada para contar algo, normalmente inicializado a cero e incrementado posteriormente.

incrementar: Aumentar el valor de una variable en una unidad.

decrementar: Disminuir el valor de una variable en una unidad.

espacio en blanco: Cualquiera de los caracteres que mueven el cursor sin imprimir caracteres visibles. La constante `string.whitespace` contiene todos los caracteres de espacio en blanco.

Capítulo 8

Listas

Una **lista** es un conjunto ordenado de valores, en el cual cada valor va identificado por un índice. Los valores que constituyen una lista son sus **elementos**. Las listas son similares a las cadenas de texto (*strings*), que son conjuntos ordenados de caracteres, excepto en que los elementos de una lista pueden ser de cualquier tipo. Las listas y las cadenas, y otras cosas que se comportan como conjuntos ordenados, se llaman **secuencias**.

8.1. Valores de una lista

Hay varias maneras de crear una nueva lista; la más sencilla es encerrar sus elementos entre corchetes:

```
[10, 20, 30, 40]
["spam", "elástico", "golondrina"]
```

El primer ejemplo es una lista de cuatro enteros. El segundo es una lista de tres cadenas de texto. Los elementos de una lista no tienen por qué ser del mismo tipo. La siguiente lista contiene una cadena, un número con decimales y un entero, y, maravilla de las maravillas, otra lista:

```
["hola", 2.0, 5, [10, 20]]
```

Se dice que una lista dentro de otra lista está **anidada**.

Las listas que contienen números enteros consecutivos son comunes, de manera que Python proporciona una manera sencilla de crearlas:

```
>>> range(1,5)
[1, 2, 3, 4]
```

La función **range** toma dos argumentos y devuelve una lista que contiene todos los enteros entre el primero y el segundo, incluyendo el primero pero no el segundo!

Hay dos formas alternativas para **range**. Con un solo argumento, crea una lista que empieza desde 0:

```
>>> range(10)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Si hay un tercer argumento, especificará el espacio entre dos valores sucesivos; a esto se le llama *paso* (*step*). Este ejemplo cuenta de 1 a 10 de dos en dos (con pasos de 2).

```
>>> range(1, 10, 2)
[1, 3, 5, 7, 9]
```

Para terminar, hay una lista especial que no contiene elementos. Se la llama lista vacía y se representa [].

Con todas estas maneras para crear listas, sería decepcionante que no pudiéramos asignar valores de listas a variables o pasar listas como parámetros a funciones. Por supuesto que podemos.

```
vocabulario = ["mejorar", "castigar", "defenestrar"]
numeros = [17, 123]
vacio = []
print vocabulario, numeros, vacio
['mejorar', 'castigar', 'defenestrar'] [17, 123] []
```

8.2. Acceso a los elementos

La sintaxis para acceder a los elementos de una lista es la misma que para acceder a los caracteres de una cadena: el operador corchetes []. La expresión dentro de los corchetes especifica el índice. Recuerde que los índices siempre comienzan en cero:

```
print numeros[0]
numeros[1] = 5
```

El operador [] puede aparecer en cualquier parte de una expresión. Cuando aparece a la izquierda de una asignación, cambia uno de los elementos de la lista, de manera que el “unésimo” elemento de **numeros**, que era 123, ahora es 5.

Se puede usar como índice cualquier expresión entera.


```
>>> numeros[3-2]
```

```
5
```

```
>>> numeros[1.0]
```

```
TypeError: sequence index must be integer
```

Si intenta acceder (leer o modificar) un elemento que no existe, obtendrá un error en tiempo de ejecución:

```
>>> numeros[2] = 5
```

```
IndexError: list assignment index out of range
```

Si se da un índice negativo, se cuenta hacia atrás desde el final de la lista.

```
>>> numeros[-1]
```

```
5
```

```
>>> numeros[-2]
```

```
17
```

```
>>> numeros[-3]
```

```
IndexError: list index out of range
```

`numeros[-1]` es el último elemento de la lista, `numeros[-2]` es el penúltimo, y `numeros[-3]` no existe.

Es muy habitual usar una variable de bucle como índice para una lista:

```
jinetes = ["guerra", "hambre", "peste", "muerte"]
i = 0
while i < 4:
    print jinetes[i]
    i = i + 1
```

Este bucle `while` cuenta desde 0 hasta 4. Cuando la variable de bucle vale 4, la condición falla y acaba el bucle. Por tanto, el cuerpo del bucle sólo se ejecuta cuando `i` es 0, 1, 2 y 3.

Cada vez que recorremos el bucle, la variable `i` se usa como índice de la lista, imprimiendo el elemento `i`-ésimo. Esta plantilla de computación se llama **recorrido de lista**.

8.3. Longitud (tamaño) de una lista

La función `len` toma una lista y devuelve su tamaño. Es una buena idea usar este valor como límite superior de un bucle, en lugar de una constante. De esta manera, si el tamaño de la lista cambia, no habrá que estar haciendo cambios en todos los bucles; funcionarán correctamente con cualquier tamaño de lista.

```
jinetes = ["guerra", "hambre", "peste", "muerte"]
i = 0
```

```
while i < len(jinetes):  
    print jinetes[i]  
    i = i + 1
```

La última vez que se ejecuta el cuerpo del bucle, `i` es `len(jinetes) - 1`, que es el índice del último elemento. Cuando `i` se iguala a `len(jinetes)`, la condición falla y no se ejecuta el cuerpo, lo que es una cosa buena, ya que `len(jinetes)` no es un índice legal.

Aunque una lista puede contener otra lista como elemento, la lista anidada cuenta como un elemento sencillo. El tamaño de esta lista es 4:

```
['spam!', 1, ['Brie', 'Roquefort', 'Pol le Veq'], [1, 2, 3]]
```

Como ejercicio, escriba un bucle que recorra la lista anterior e imprima la longitud de cada elemento. ¿qué ocurre si envía un entero a `len`?

8.4. Pertenencia a una lista

`in` es un operador booleano que comprueba la pertenencia a una secuencia. Lo usamos en la Sección 7.10 con las cadenas, pero también funciona con las listas y otras secuencias:

```
>>> jinetes = ['guerra', 'hambre', 'peste', 'muerte']  
>>> 'peste' in jinetes  
1  
>>> 'libertinaje' in jinetes  
0
```

Como “peste” es un miembro de la lista `jinetes`, el operador `in` devuelve verdadero. Como “libertinaje” no está en la lista, `in` devuelve falso.

Podemos usar `not` en combinación con `in` para comprobar si un elemento no es miembro de una lista:

```
>>> 'libertinaje' not in jinetes  
1
```

8.5. Listas y bucles for

El bucle `for` que vimos en la Sección 7.3 también funciona con las listas. La sintaxis generalizada de un bucle `for` es:

```
for VARIABLE in LISTA:
    CUERPO
```

Esta sentencia es equivalente a:

```
i = 0
while i < len(LISTA):
    VARIABLE = LISTA[i]
    CUERPO
    i = i + 1
```

El bucle `for` es más conciso porque podemos eliminar la variable de bucle, `i`. Aquí tenemos el bucle anterior con un bucle `for`:

```
for jinete in jinetes:
    print jinete
```

Más aún, casi se lee igual que en español, “Para (cada) jinete en (la lista de) jinetes, imprime (el nombre del) jinete”.

Se puede usar cualquier expresión de lista en un bucle `for`:

```
for numero in range(20):
    if numero % 2 == 0:
        print numero

for fruta in ["plátano", "manzana", "membrillo"]:
    print "Me gusta comer " + fruta + "s!"
```

El primer ejemplo imprime todos los números pares entre el 0 y el 19. El segundo ejemplo expresa su entusiasmo por diferentes frutas.

8.6. Operaciones con listas

El operador `+` concatena listas:

```
>>> a = [1, 2, 3]
>>> b = [4, 5, 6]
>>> c = a + b
>>> print c
[1, 2, 3, 4, 5, 6]
```

De forma similar, el operador `*` repite una lista un número dado de veces:

```
>>> [0] * 4
[0, 0, 0, 0]
>>> [1, 2, 3] * 3
[1, 2, 3, 1, 2, 3, 1, 2, 3]
```

En el primer ejemplo la lista `[0]` contiene un solo elemento que es repetido cuatro veces. En el segundo ejemplo, la lista `[1, 2, 3]` se repite tres veces.

8.7. Porciones (slices)

Las operaciones de porciones que vimos en la Sección 7.4 también funcionan en sobre las listas:

```
>>> lista = ['a', 'b', 'c', 'd', 'e', 'f']
>>> lista[1:3]
['b', 'c']
>>> lista[:4]
['a', 'b', 'c', 'd']
>>> lista[3:]
['d', 'e', 'f']
>>> lista[:]
['a', 'b', 'c', 'd', 'e', 'f']
```

8.8. Las listas son mutables

A diferencia de las cadenas, las listas son mutables, lo que significa que podemos cambiar sus elementos. Podemos modificar uno de sus elementos usando el operador corchetes en el lado izquierdo de una asignación:

```
>>> fruta = ["plátano", "manzana", "membrillo"]
>>> fruta[0] = "pera"
>>> fruta[-1] = "naranja"
>>> print fruta
['pera', 'manzana', 'naranja']
```

Con el operador de porción podemos reemplazar varios elementos a la vez:

```
>>> lista = ['a', 'b', 'c', 'd', 'e', 'f']
>>> lista[1:3] = ['x', 'y']
>>> print lista
['a', 'x', 'y', 'd', 'e', 'f']
```

Además, puede eliminar elementos de una lista asignándoles la lista vacía:

```
>>> lista = ['a', 'b', 'c', 'd', 'e', 'f']
>>> lista[1:3] = []
>>> lista
['a', 'd', 'e', 'f']
```

Y puede añadir elementos a la lista embutiéndolos en una porción vacía en la posición deseada:

```
>>> lista = ['a', 'd', 'f']
>>> lista[1:1] = ['b', 'c']
>>> print lista
```

```
['a', 'b', 'c', 'd', 'f']
>>> lista[4:4] = ['e']
>>> print lista
['a', 'b', 'c', 'd', 'e', 'f']
```

8.9. Borrado en una lista

El uso de porciones para borrar elementos de una lista puede ser extraño, y por ello propicio a los errores. Python nos da una alternativa que resulta más legible.

`del` elimina un elemento de una lista:

```
>>> a = ['uno', 'dos', 'tres']
>>> del a[1]
>>> a
['uno', 'tres']
```

Como podría esperar, `del` maneja índices negativos y provoca un error en tiempo de ejecución sin el índice está fuera de límites.

Puede usar una porción como índice para `del`:

```
>>> lista = ['a', 'b', 'c', 'd', 'e', 'f']
>>> del lista[1:5]
>>> print lista
['a', 'f']
```

Como es habitual, las porciones seleccionan todos los elementos hasta, pero no incluido, el segundo índice.

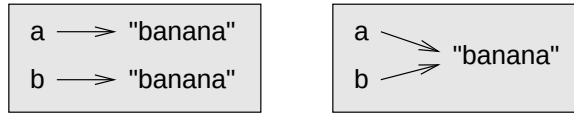
8.10. Objetos y valores

Si ejecutamos estas sentencias de asignación:

```
a = "banana"
b = "banana"
```

Está claro que `a` y `b` apuntan ambos a cadenas con las letras "banana". Pero no podemos saber si están apuntando a *la misma* cadena.

Hay dos posibles estados:



En un caso, **a** y **b** se refieren a dos cosas diferentes que tienen el mismo valor. En el segundo caso, se refieren a la misma cosa. Estas “cosas” tienen nombres; se les denomina **objetos**. Un objeto es una cosa a la que se puede referir una variable.

Cada objeto tiene un **identificador** único, que podemos obtener por medio de la función `id`. Imprimiendo los identificadores de **a** y **b** podemos saber si apuntan al mismo objeto.

```
>>> id(a)
135044008
>>> id(b)
135044008
```

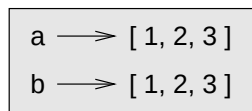
En este caso, las dos veces obtenemos el mismo identificador, lo que significa que Python sólo creó una cadena y ambas variables, **a** y **b**, apuntan a ella.

Como las cadenas de texto son inmutables, no hay diferencia práctica entre los dos posibles estados. Para tipos mutables como las listas, sí que importa.

Curiosamente, las listas se comportan de otra manera. Cuando crea dos listas, obtiene dos objetos:

```
>>> a = [1, 2, 3]
>>> b = [1, 2, 3]
>>> id(a)
135045528
>>> id(b)
135041704
```

De manera que el diagrama de estado sería tal como éste:



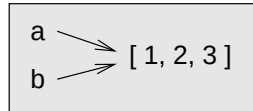
a y **b** tienen el mismo valor, pero no se refieren al mismo objeto.

8.11. Alias (poner sobrenombres)

Como las variables apuntan a objetos, si asigna una variable a otra, ambas variables se refieren al mismo objeto:

```
>>> a = [1, 2, 3]
>>> b = a
```

En este caso, el diagrama de estados sería como éste:



Como la misma lista tiene dos nombres diferentes, **a** y **b**, podemos decir que se le ha puesto un **alias**. Los cambios hechos a un alias afectan al otro:

```
>>> b[0] = 5
>>> print a
[5, 2, 3]
```

Aunque este comportamiento puede ser útil, a veces es inesperado o indeseable. En general, es más seguro evitar los alias cuando trabajemos con objetos mutables. Por supuesto, no hay problema con los objetos inmutables. Por ello Python se toma la libertad de poner alias a las cadenas cuando ve una oportunidad de economizar.

8.12. Clonar listas

Si queremos modificar una lista y mantener una copia del original, necesitaremos ser capaces de hacer una copia de la lista en sí, no sólo de su referencia. Este proceso a veces se denomina **clonado**, para evitar la ambigüedad de la palabra “copia”.

La forma más fácil de clonar una lista es por medio del operador de porción:

```
>>> a = [1, 2, 3]
>>> b = []
>>> b[:] = a[:]
>>> print b
[1, 2, 3]
```

La extracción de una porción de **a** crea una nueva lista. En este caso, la porción consta de la lista completa.

Ahora tenemos libertad de hacer cambios en **b** sin preocuparnos de **a**:

```
>>> b[0] = 5
>>> print a
[1, 2, 3]
```

*Como ejercicio, dibuje un diagrama de estado de **a** y **b** antes y después del cambio.*

8.13. Listas como parámetros

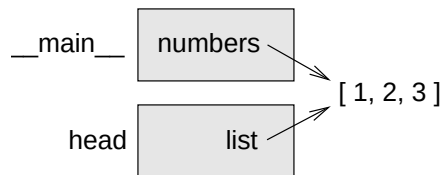
Cuando se pasa una lista como argumento, en realidad se pasa una referencia a ella, no una copia de la lista. Por ejemplo, la función `cabeza` toma una lista como parámetro y devuelve el primer elemento.

```
def cabeza(lista):
    return lista[0]
```

Así es como se usa.

```
>>> numeros = [1,2,3]
>>> cabeza(numeros)
1
```

El parámetro `lista` y la variable `numeros` son alias de un mismo objeto. El diagrama de estado es así:



Como el objeto `lista` está compartido por dos marcos, lo dibujamos entre ambos.

Si la función modifica una lista pasada como parámetro, el que hizo la llamada verá el cambio. `borra_cabeza` elimina el primer elemento de una lista.

```
def borra_cabeza(lista):
    del lista[0]
```

Aquí vemos el uso de `borra_cabeza`:

```
>>> numeros = [1,2,3]
>>> borra_cabeza(numeros)
>>> print numeros
[2, 3]
```

Si una función devuelve una lista, devuelve una referencia a la lista. Por ejemplo, `cola` devuelve una lista que contiene todos los elementos de una lista dada, excepto el primero.

```
def cola(lista):
    return lista[1:]
```

Aquí vemos cómo se usa `cola`:

```
>>> numeros = [1,2,3]
>>> resto = cola(numeros)
>>> print resto
>>> [2, 3]
```


Como el valor de retorno se creó con una porción, es una lista. La creación de **rest**, así como cualquier cambio posterior en **rest**, no afectará a **numbers**.

8.14. Listas anidadas

Una lista anidada es una lista que aparece como elemento dentro de otra lista. En esta lista, el tri-ésimo elemento es una lista anidada:

```
>>> lista = ["hola", 2.0, 5, [10, 20]]
```

Si imprimimos `lista[3]`, obtendremos `[10, 20]`. Para extraer los elementos de la lista anidada, podemos proceder en dos pasos:

```
>>> elt = lista[3]
>>> elt[0]
10
```

O podemos combinarlos:

```
>>> lista[3][1]
20
```

Los operadores corchete se evalúan de izquierda a derecha, así que esta expresión saca el tri-ésimo elemento de `lista` y luego extrae el unésimo elemento de ella.

8.15. Matrices

Es común usar listas anidadas para representar matrices. Por ejemplo, la matriz:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

puede ser representada como:

```
>>> matriz = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

`matriz` es una lista con tres elementos, siendo cada elemento una fila de la matriz. Podemos elegir una fila entera de la matriz de la forma normal:

```
>>> matriz[1]
[4, 5, 6]
```

O tomar sólo un elemento de la matriz usando la forma de doble índice:

```
>>> matriz[1][1]
5
```

El primer índice escoge la fila y el segundo la columna. Aunque esta manera de representar matrices es común, no es la única posibilidad. Una pequeña variación consiste en usar una lista de columnas en lugar de filas. Más adelante veremos una alternativa más radical usando un diccionario.

8.16. Cadenas y listas

Dos de las funciones más útiles del módulo `string` tienen que ver con listas de cadenas. La función `split` divide una cadena en una lista de palabras. Por defecto, cualquier número de caracteres de espacio en blanco se considera un límite de palabra:

```
>>> import string
>>> cancion = "La lluvia en Sevilla..."
>>> string.split(cancion)
['La', 'lluvia', 'en', 'Sevilla...']
```

Se puede usar un argumento opcional llamado **delimitador** para especificar qué caracteres se usarán como límites de palabra. El siguiente ejemplo usa la cadena `ll` como delimitador:

```
>>> string.split(cancion, 'll')
['La ', 'uvia en Sevi', 'a...']
```

Observe que el delimitador no aparece en la lista.

La función `join` es la inversa de `split`. Toma una lista de cadenas y concatena los elementos con un espacio entre cada par:

```
>>> lista = ['La', 'lluvia', 'en', 'Sevilla...']
>>> string.join(lista)
'La lluvia en Sevilla...'
```

Como `split`, `join` acepta un delimitador opcional que se inserta entre los elementos. El delimitador por defecto es el espacio.

```
>>> string.join(lista, '_')
'La_lluvia_en_Sevilla...'
```

A modo de ejercicio, describa la relación que hay entre `string.join(string.split(cancion))` y `cancion`. ¿Es la misma para todas las cadenas? ¿Cuándo sería diferente?

8.17. Glosario

lista: Una colección de objetos con nombre, en la que cada objeto es identificado por un índice.

índice: Una variable o valor enteros que se usan para indicar un elemento de una lista.

elemento: Uno de los valores de una lista (u otra secuencia). El operador corchete selecciona elementos de una lista.

secuencia: Cualquier tipo de datos que consita en un conjunto ordenado de elementos, con cada elemento identificado por un índice.

lista anidada: Una lista que es elemento de otra lista.

recorrido de lista: Acceso secuencial a cada elemento de una lista.

objeto: Una cosa a la que se puede referir una variable.

alias: Múltiples variables que contienen referencias al mismo objeto.

clonar: Crear un objeto nuevo que tiene el mismo valor que un objeto ya existente. Copiar una referencia a un objeto crea un alias, pero no clona el objeto.

delimitador: Un carácter o cadena utilizado para indicar dónde debe cortarse una cadena.

Capítulo 9

Tuplas

9.1. Mutabilidad y tuplas

Hasta ahora, ha visto dos tipos compuestos: cadenas, que están hechas de caracteres, y listas, que están hechas de elementos de cualquier tipo. Una de las diferencias que señalamos es que los elementos de una lista se pueden modificar, pero los caracteres de una cadena no. En otras palabras, las cadenas son **inmutables** y las listas son **mutables**.

En Python hay otro tipo llamado **tupla** que es similar a una lista salvo en que es inmutable. Sintácticamente, una tupla es una lista de valores separados por comas:

```
>>> tupla = 'a', 'b', 'c', 'd', 'e'
```

Aunque no es necesario, la convención dice que hay que encerrar las tuplas entre paréntesis:

```
>>> tupla = ('a', 'b', 'c', 'd', 'e')
```

Para crear una tupla con un solo elemento, debemos incluir una coma final:

```
>>> t1 = ('a',)
>>> type(t1)
<type 'tuple'>
```

Sin la coma, Python trata ('a') como una cadena entre paréntesis:

```
>>> t2 = ('a')
>>> type(t2)
<type 'string'>
```

Dejando a un lado las cuestiones de sintaxis, las operaciones sobre las tuplas son las mismas que sobre las listas. El operador índice selecciona un elemento de la tupla.

```
>>> tupla = ('a', 'b', 'c', 'd', 'e')
>>> tupla[0]
'a'
```

Y el operador de porción selecciona un intervalo de elementos.

```
>>> tupla[1:3]
('b', 'c')
```

Pero si intentamos modificar uno de los elementos de la tupla provocaremos un error:

```
>>> tupla[0] = 'A'
TypeError: object doesn't support item assignment
```

Por supuesto, incluso aunque no podamos modificar los elementos de una tupla, podemos sustituir una tupla por otra diferente:

```
>>> tupla = ('A',) + tupla[1:]
>>> tupla
('A', 'b', 'c', 'd', 'e')
```

9.2. Asignación de tuplas

De vez en cuando, es útil intercambiar los valores de dos variables. Para hacerlo con sentencias de asignación convencionales debemos usar una variable temporal. Por ejemplo, para intercambiar `a` y `b`:

```
>>> temp = a
>>> a = b
>>> b = temp
```

Si tenemos que hacer esto a menudo, esta aproximación resulta aparatosa. Python proporciona una forma de **asignación de tuplas** que soluciona este problema elegantemente:

```
>>> a, b = b, a
```

El lado izquierdo es una tupla de variables, el lado derecho es una tupla de valores. Cada valor se asigna a su respectiva variable. Todas las expresiones del lado derecho se evalúan antes de las asignaciones. Esta característica hace de la asignación de tuplas algo muy versátil.

Naturalmente, el número de variables a la izquierda y el número de valores a la derecha deben ser iguales:

```
>>> a, b, c, d = 1, 2, 3
ValueError: unpack tuple of wrong size
```

9.3. Tuplas como valor de retorno

Las funciones pueden devolver tuplas como valor de retorno. Por ejemplo, podríamos escribir una función que intercambie dos parámetros:

```
def intercambio(x, y):
    return y, x
```

Luego podemos asignar el valor de retorno a una tupla con dos variables:

```
a, b = intercambio(a, b)
```

En este caso, no hay ninguna ventaja en convertir `intercambio` en una función. De hecho, existe un peligro al intentar encapsular `intercambio`, y es el tentador error que sigue:

```
def intercambio(x, y):          # versión incorrecta
    x, y = y, x
```

Si llamamos a esta función así:

```
intercambio(a, b)
```

`a` y `x` son alias del mismo valor. Cambiar `x` dentro de `intercambio` hace que `x` se refiera a un valor diferente, pero no tiene efecto alguno sobre `a` en `__main__`. De forma similar, cambiar `y` no tiene efecto sobre `b`.

Esta función se ejecuta sin generar un mensaje de error, pero no hace lo que intentamos. Este es un ejemplo de error semántico.

A modo de ejercicio, dibuje un diagrama de estados para esta función de manera que pueda ver por qué no trabaja como usted quiere.

9.4. Números aleatorios

La mayor parte de los programas hacen lo mismo cada vez que los ejecutamos, por lo que se dice que son **deterministas**. Normalmente el determinismo es una cosa buena, ya que esperamos que un cálculo nos dé siempre el mismo

resultado. Para algunas aplicaciones, sin embargo, queremos que el computador sea impredecible. El ejemplo obvio son los juegos, pero hay más.

Hacer que un programa sea realmente no determinista resulta no ser tan sencillo, pero hay formas de que al menos parezca no determinista. Una de ellas es generar números aleatorios y usarlos para determinar el resultado del programa. Python proporciona una función interna que genera números **pseudoaleatorios**, que no son verdaderamente aleatorios en un sentido matemático, pero servirán para nuestros propósitos.

El módulo **random** contiene una función llamada **random** que devuelve un número en coma flotante entre 0,0 y 1,0. Cada vez que usted llama a **random** obtiene el siguiente número de una larga serie. Para ver un ejemplo, ejecute este bucle:

```
import random

for i in range(10):
    x = random.random()
    print x
```

Para generar un número aleatorio entre 0,0 y un límite superior como **maximo**, multiplique **x** por **maximo**.

Como ejercicio, genere un número aleatorio entre minimo y maximo.

Como ejercicio adicional, genere un número aleatorio entero entre minimo y maximo, incluyendo ambos extremos.

9.5. Lista de números aleatorios

El primer paso es generar una lista de valores aleatorios. **listaAleatorios** acepta un parámetro entero y devuelve una lista de números aleatorios de la longitud dada. Comienza con una lista de **n** ceros. Cada vez que ejecuta el bucle, sustituye uno de los elementos con un número aleatorio. El valor de retorno es una referencia a la lista completa:

```
def listaAleatorios(n):
    s = [0] * n
    for i in range(n):
        s[i] = random.random()
    return s
```

Vamos a probar esta función con una lista de ocho elementos. A la hora de depurar es una buena idea empezar con algo pequeño.


```
>>> listaAleatorios(8)
0.15156642489
0.498048560109
0.810894847068
0.360371157682
0.275119183077
0.328578797631
0.759199803101
0.800367163582
```

Se supone que los números generados por **random** están distribuidos uniformemente, lo que significa que cada valor es igualmente probable.

Si dividimos el intervalo de valores posibles en “baldes” de igual tamaño y contamos el número de veces que un valor cae en cada balde, deberíamos tener más o menos el mismo número en todos.

Podemos contrastar esta teoría escribiendo un programa que divida el intervalo en baldes y contando el número de valores en cada uno.

9.6. Conteo

Un buen enfoque sobre problemas como éste es dividir el problema en subproblemas que encajen en un esquema computacional que hayamos visto antes.

En este caso, queremos recorrer una lista de números y contar el número de veces que un valor cae en un intervalo dado. Eso nos suena. En la Sección 7.8 escribimos un programa que recorría una cadena de texto y contaba el número de veces que aparecía una letra determinada.

Así, podemos hacerlo copiando el programa viejo y adaptándolo al problema actual. El programa original era:

```
cuenta = 0
for car in fruta:
    if car == 'a':
        cuenta = cuenta + 1
print cuenta
```

El primer paso es sustituir **fruta** con **lista** y **car** con **num**. Esto no cambia el programa, sólo lo hace más legible.

El segundo paso es cambiar la comprobación. No estamos interesados en encontrar letras. Queremos ver si **num** está entre los valores de **minimo** y **maximo**.

```
cuenta = 0
for num in lista:
    if minimo < num < maximo:
        cuenta = cuenta + 1
print cuenta
```

El último paso es encapsular este código en una función llamada `enElBalde`. Los parámetros son la lista y los valores `minimo` y `maximo`.

```
def enElBalde(lista, minimo, maximo):
    cuenta = 0
    for num in lista:
        if minimo < num < maximo:
            cuenta = cuenta + 1
    return cuenta
```

Copiar y modificar un programa existente nos facilita escribir esta función rápidamente y nos ahorra un montón de tiempo de depuración. Este plan de desarrollo se llama **coincidencia de esquemas**. Si se encuentra trabajando en un problema que ya solucionó, reutilice la solución.

9.7. Muchos baldes

Tal como aumenta el número de baldes, `enElBalde` se hace un tanto difícil de manejar. Con dos baldes, no está mal:

```
bajo = enElBalde(a, 0.0, 0.5)
alto = enElBalde(a, 0.5, 1)
```

Pero con cuatro baldes ya es aparatoso.

```
balde1 = enElBalde(a, 0.0, 0.25)
balde2 = enElBalde(a, 0.25, 0.5)
balde3 = enElBalde(a, 0.5, 0.75)
balde4 = enElBalde(a, 0.75, 1.0)
```

Hay dos problemas. Uno es que tenemos que inventar nuevos nombres de variables para cada resultado. El otro es que tenemos que calcular el intervalo de cada balde.

Empezaremos por solucionar el segundo problema. Si el número de baldes es `numBaldes`, la anchura de cada balde es $1.0 / \text{numBaldes}$.

Usaremos un bucle para calcular el intervalo de cada balde. La variable del bucle, `i`, cuenta de 1 a `numBaldes-1`:

```
anchuraBalde = 1.0 / numBaldes
for i in range(numBaldes):
    minimo = i * anchuraBalde
    maximo = minimo + anchuraBalde
    print minimo, "hasta", maximo
```

Para calcular el límite inferior de cada balde, multiplicamos la variable de bucle por la anchura de balde. El límite superior está a tan sólo una `anchuraBalde`.

Con `numBaldes = 8`, la salida es:

```
0.0 hasta 0.125
0.125 hasta 0.25
0.25 hasta 0.375
0.375 hasta 0.5
0.5 hasta 0.625
0.625 hasta 0.75
0.75 hasta 0.875
0.875 hasta 1.0
```

Puede confirmar que todos los bucles tienen la misma anchura, que no se solapan y que cubren todo el intervalo entre 0,0 y 1,0.

Volvamos ahora al primer problema. Necesitamos un modo de almacenar ocho enteros, usando la variable de bucle para señalarlos uno por uno. En estos momentos debería usted estar pensando “¡Lista!”.

Debemos crear la lista de baldes fuera del bucle, porque sólo queremos hacerlo una vez. Dentro del bucle, podemos llamar repetidamente a `enElBalde` y actualizar el *i*-ésimo elemento de la lista:

```
numBaldes = 8
baldes = [0] * numBaldes
anchuraBalde = 1.0 / numBaldes
for i in range(numBaldes):
    minimo = i * anchuraBalde
    maximo = minimo + anchuraBalde
    baldes[i] = enElBalde(lista, minimo, maximo)
print baldes
```

Con una lista de 1000 valores, este código genera esta lista de baldes:

```
[138, 124, 128, 118, 130, 117, 114, 131]
```

Estos números son razonablemente próximos a 125, que es lo que esperábamos. Por lo menos, están lo bastante cerca como para que podamos pensar que el generador de números aleatorios funciona.

Como ejercicio, compruebe esta función con listas más largas, y vea si el número de valores en cada balde tiende a equilibrarse.

9.8. Una solución en una sola pasada

Aunque este programa funciona, no es tan eficiente como podría ser. Cada vez que llama a `enElBalde` recorre la lista entera. Con el aumento del número de baldes, llega a ser un montón de recorridos.

Sería mejor hacer una sola pasada por la lista y calcular para cada valor el índice del balde en el que cae. Luego podemos incrementar el contador apropiado.

En la sección anterior tomamos un índice, `i`, y lo multiplicamos por la `anchuraBalde` para hallar el límite inferior de un balde dado. Ahora queremos tomar un valor del intervalo 0,0 a 1,0 y hallar el índice del balde en el que cae.

Como el problema es el inverso del anterior, podemos suponer que deberíamos dividir por `anchuraBalde` en lugar de multiplicar. La suposición es correcta.

Como `anchuraBalde = 1.0 / numBaldes`, dividir por `anchuraBalde` es lo mismo que multiplicar por `numBaldes`. Si multiplicamos un número del intervalo que va de 0,0 a 1,0 por `numBaldes`, obtenemos un número del intervalo entre 0,0 y `numBaldes`. Si redondeamos ese número al entero inferior obtendremos exactamente lo que estamos buscando, un índice de balde:

```
numBaldes = 8
baldes = [0] * numBaldes
for i in lista:
    indice = int(i * numBaldes)
    baldes[indice] = baldes[indice] + 1
```

Usamos la función `int` para convertir un número en coma flotante en un entero.

¿Es posible que este cálculo genere un índice que esté fuera del intervalo (tanto negativo como mayor que `len(baldes)-1`)?

Una lista como `baldes` que contiene conteos del número de valores en cada intervalo se llama **histograma**.

Como ejercicio, escriba una función llamada `histograma` que tome como parámetros una lista y un número de baldes y devuelva un histograma con el número dado de baldes.

9.9. Glosario

tipo immutable: Un tipo en el cual los elementos no se puede modificar. Las asignaciones de elementos o porciones de tipos inmutables provocan un error.

tipo mutable: Un tipo de datos en el cual los elementos pueden ser modificados. Todos los tipos mutables son compuestos. Las listas y diccionarios son tipos de datos mutables, las cadenas y las tuplas no.

tupla: Un tipo de secuencia que es similar a una lista excepto en que es immutable. Las tuplas se pueden usar donde quiera que se necesite un tipo immutable, como puede ser la clave de un diccionario.

asignación de tuplas: Una asignación de todos los elementos de una tupla usando una única sentencia de asignación. La asignación de tuplas sucede más bien en paralelo que secuencialmente, haciéndola útil para intercambiar valores.

determinista: Un programa que hace lo mismo todas las veces que se ejecuta.

pseudoaleatorio: Una secuencia de números que parece ser aleatoria pero que en realidad es el resultado de un cálculo determinista.

histograma: Una lista de enteros en la que cada elemento cuenta el número de veces que ocurre algo.

coincidencia de esquemas: Un plan de desarrollo de programas que implica la identificación de un esquema computacional conocido y el copiado de la solución para un problema similar.

Capítulo 10

Diccionarios

Los tipos compuestos que ha visto hasta ahora (cadenas, listas y tuplas) usan enteros como índices. Si intenta usar cualquier otro tipo como índice provocará un error.

Los **diccionarios** son similares a otros tipos compuestos excepto en que pueden usar como índice cualquier tipo inmutable. A modo de ejemplo, crearemos un diccionario que traduzca palabras inglesas al español. En este diccionario, los índices son **strings** (cadenas).

Una forma de crear un diccionario es empezar con el diccionario vacío y añadir elementos. El diccionario vacío se expresa como {}:

```
>>> ing\_a\_esp = {}
>>> ing\_a\_esp['one'] = 'uno'
>>> ing\_a\_esp['two'] = 'dos'
```

La primera asignación crea un diccionario llamado **ing_a_esp**; las otras asignaciones añaden nuevos elementos al diccionario. Podemos presentar el valor actual del diccionario del modo habitual:

```
>>> print ing\_a\_esp
{'one': 'uno', 'two': 'dos'}
```

Los elementos de un diccionario aparecen en una lista separada por comas. Cada entrada contiene un índice y un valor separado por dos puntos (:). En un diccionario, los índices se llaman **claves**, por eso los elementos se llaman **pares clave-valor**.

Otra forma de crear un diccionario es dando una lista de pares clave-valor con la misma sintaxis que la salida del ejemplo anterior:

```
>>> ing\_a\_esp = {'one': 'uno', 'two': 'dos', 'three': 'tres'}
```

Si volvemos a imprimir el valor de `ing_a_esp`, nos llevamos una sorpresa:

```
>>> print ing\_a\_esp
{'one': 'uno', 'three': 'tres', 'two': 'dos'}
```

¡Los pares clave-valor no están en orden! Afortunadamente, no necesitamos preocuparnos por el orden, ya que los elementos de un diccionario nunca se indexan con índices enteros. En lugar de eso, usamos las claves para buscar los valores correspondientes:

```
>>> print ing\_a\_esp['two']
'dos'
```

La clave `'two'` nos da el valor `'dos'` aunque aparezca en el tercer par clave-valor.

10.1. Operaciones sobre diccionarios

La sentencia `del` elimina un par clave-valor de un diccionario. Por ejemplo, el diccionario siguiente contiene los nombres de varias frutas y el número de esas frutas en el almacén:

```
>>> inventario = {'manzanas': 430, 'bananas': 312,
...               'naranjas': 525, 'peras': 217}
>>> print inventario
{'naranjas': 525, 'manzanas': 430, 'peras': 217, 'bananas': 312}
```

Si alguien compra todas las peras, podemos eliminar la entrada del diccionario:

```
>>> del inventario['peras']
>>> print inventario
{'naranjas': 525, 'manzanas': 430, 'bananas': 312}
```

O si esperamos recibir más peras pronto, podemos simplemente cambiar el inventario asociado con las peras:

```
>>> inventario['peras'] = 0
>>> print inventario
{'naranjas': 525, 'manzanas': 430, 'peras': 0, 'bananas': 312}
```

La función `len` también funciona con diccionarios; devuelve el número de pares clave-valor:

```
>>> len(inventario)
```

4

10.2. Métodos del diccionario

Un **método** es similar a una función, acepta parámetros y devuelve un valor, pero la sintaxis es diferente. Por ejemplo, el método **keys** acepta un diccionario y devuelve una lista con las claves que aparecen, pero en lugar de la sintaxis de la función `keys(ing_a_esp)`, usamos la sintaxis del método `ing_a_esp.keys()`.

```
>>> ing\_a\_esp.keys()
['one', 'three', 'two']
```

Esta forma de notación de punto especifica el nombre de la función, **keys**, y el nombre del objeto al que se va a aplicar la función, **ing_a_esp**. Los paréntesis indican que este método no admite parámetros.

La llamada a un método se denomina **invocación**; en este caso, diríamos que estamos invocando **keys** sobre el objeto **ing_a_esp**.

El método **values** es similar; devuelve una lista de los valores del diccionario:

```
>>> ing\_a\_esp.values()
['uno', 'tres', 'dos']
```

El método **items** devuelve ambos, una lista de tuplas con los pares clave-valor del diccionario:

```
>>> ing\_a\_esp.items()
[('one', 'uno'), ('three', 'tres'), ('two', 'dos')]
```

La sintaxis nos proporciona información muy útil acerca del tipo de datos. Los corchetes indican que es una lista. Los paréntesis indican que los elementos de la lista son tuplas.

Si un método acepta un argumento, usa la misma sintaxis que una llamada a una función. Por ejemplo, el método **has_key** acepta una clave y devuelve verdadero (1) si la clave aparece en el diccionario:

```
>>> ing\_a\_esp.has_key('one')
1
>>> ing\_a\_esp.has_key('deux')
0
```

Si usted invoca un método sin especificar un objeto, provoca un error. En este caso, el mensaje de error no es de mucha ayuda:

```
>>> has_key('one')
NameError: has_key
```

10.3. Asignación de alias y copiado

Debe usted estar atento a los alias a causa de la mutabilidad de los diccionarios. Si dos variables se refieren al mismo objeto los cambios en una afectan a la otra.

Si quiere modificar un diccionario y mantener una copia del original, use el método `copy`. Por ejemplo, `opuestos` es un diccionario que contiene pares de opuestos:

```
>>> opuestos = {'arriba': 'abajo', 'derecho': 'torcido',
...             'verdadero': 'falso'}
>>> alias = opuestos
>>> copia = opuestos.copy()
```

`alias` y `opuestos` se refieren al mismo objeto; `copia` hace referencia a una copia nueva del mismo diccionario. Si modificamos `alias`, `opuestos` también resulta cambiado:

```
>>> alias['derecho'] = 'sentado'
>>> opuestos['derecho']
'sentado'
```

Si modificamos `copia`, `opuestos` no varía:

```
>>> copia['derecho'] = 'privilegio'
>>> opuestos['derecho']
'sentado'
```

10.4. Matrices dispersas

En la Sección 8.14 usamos una lista de listas para representar una matriz. Es una buena opción para una matriz en la que la mayoría de los valores es diferente de cero, pero piense en una matriz como ésta:

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 \end{bmatrix}$$

La representación de la lista contiene un montón de ceros:

```
matriz = [ [0,0,0,1,0],
            [0,0,0,0,0],
```

```
[0,2,0,0,0],  
[0,0,0,0,0],  
[0,0,0,3,0] ]
```

Una posible alternativa es usar un diccionario. Como claves, podemos usar tuplas que contengan los números de fila y columna. Ésta es la representación de la misma matriz por medio de un diccionario:

```
matriz = {(0,3): 1, (2, 1): 2, (4, 3): 3}
```

Sólo hay tres pares clave-valor, una para cada elemento de la matriz diferente de cero. Cada clave es una tupla, y cada valor es un entero.

Para acceder a un elemento de la matriz, podemos usar el operador []:

```
matriz[0,3]  
1
```

Fíjese en que la sintaxis para la representación por medio del diccionario no es la misma de la representación por medio de la lista anidada. En lugar de dos índices enteros, usamos un índice que es una tupla de enteros.

Hay un problema. Si apuntamos a un elemento que es cero, se produce un error porque en el diccionario no hay una entrada con esa clave:

```
>>> matriz[1,3]  
KeyError: (1, 3)
```

El método `get` soluciona este problema:

```
>>> matriz.get((0,3), 0)  
1
```

El primer argumento es la clave; el segundo argumento es el valor que debe devolver `get` en caso de que la clave no esté en el diccionario:

```
>>> matriz.get((1,3), 0)  
0
```

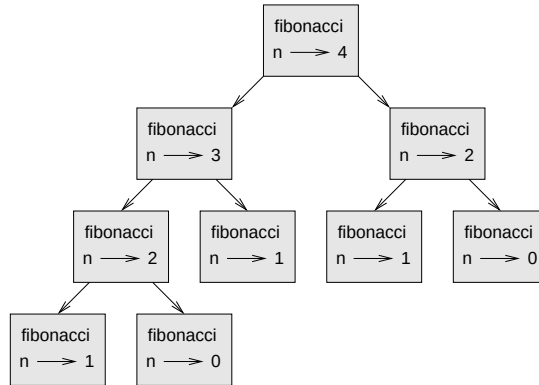
`get` mejora sensiblemente la semántica del acceso a una matriz dispersa. Lástima de sintaxis.

10.5. Pistas

Si estuvo jugando con la función `fibonacci` de la Sección 5.7, es posible que haya notado que cuanto más grande es el argumento que le da, más tiempo le

cuesta ejecutarse. Más aún, el tiempo de ejecución aumenta muy rápidamente. En nuestra máquina, `fibonacci(20)` acaba instantáneamente, `fibonacci(30)` tarda más o menos un segundo, y `fibonacci(40)` tarda una eternidad.

Para entender por qué, observe este **gráfico de llamadas** de `fibonacci` con `n=4`:



Un gráfico de llamadas muestra un conjunto de cajas de función con líneas que conectan cada caja con las cajas de las funciones a las que llama. En lo alto del gráfico, `fibonacci` con `n=4` llama a `fibonacci` con `n=3` y `n=2`. A su vez, `fibonacci` con `n=3` llama a `fibonacci` con `n=2` y `n=1`. Y así sucesivamente.

Cuente cuántas veces se llama a `fibonacci(0)` y `fibonacci(1)`. Es una solución ineficaz al problema, y empeora mucho tal como crece el argumento.

Una buena solución es llevar un registro de los valores que ya se han calculado almacenándolos en un diccionario. A un valor que ya ha sido calculado y almacenado para un uso posterior se le llama **pista**. Aquí hay una implementación de `fibonacci` con pistas:

```

anteriores = {0:1, 1:1}

def fibonacci(n):
    if anteriores.has_key(n):
        return anteriores[n]
    else:
        nuevoValor = fibonacci(n-1) + fibonacci(n-2)
        anteriores[n] = nuevoValor
        return nuevoValor

```

El diccionario llamado `anteriores` mantiene un registro de los valores de Fibonacci que ya conocemos. El programa comienza con sólo dos pares: 0 corresponde a 1 y 1 corresponde a 1.

Siempre que se llama a `fibonacci` comprueba si el diccionario contiene el resultado ya calculado. Si está ahí, la función puede devolver el valor inmediatamente sin hacer más llamadas recursivas. Si no, tiene que calcular el nuevo valor. El nuevo valor se añade al diccionario antes de que la función vuelva.

Con esta versión de `fibonacci`, nuestra máquina puede calcular `fibonacci(40)` en un abrir y cerrar de ojos. Pero cuando intentamos calcular `fibonacci(50)`, nos encontramos con otro problema:

```
>>> fibonacci(50)
OverflowError: integer addition
```

La respuesta, como verá en un momento, es 20.365.011.074. El problema es que este número es demasiado grande para caber en un entero de Python. Se **desborda**. Afortunadamente, hay una solución fácil para este problema.

10.6. Enteros largos

Python proporciona un tipo llamado `long int` que puede manejar enteros de cualquier tamaño. Hay dos formas de crear un valor `long int`. Una es escribir un entero con una *L* mayúscula al final:

```
>>> type(1L)
<type 'long int'>
```

La otra es usar la función `long` para convertir un valor en `long int`. `long` acepta cualquier tipo numérico e incluso cadenas de dígitos:

```
>>> long(1)
1L
>>> long(3.9)
3L
>>> long('57')
57L
```

Todas las operaciones matemáticas funcionan sobre los `long ints`, así que no tenemos que hacer mucho para adaptar `fibonacci`:

```
>>> previous = {0:1L, 1:1L}
>>> fibonacci(50)
20365011074L
```

Simplemente cambiando el contenido inicial de `anteriores` cambiamos el comportamiento de `fibonacci`. Los primeros dos números de la secuencia son `long ints`, así que todos los números subsiguientes lo serán también.

Como ejercicio, modifique `factorial` de forma que produzca un `long int` como resultado.

10.7. Contar letras

En el capítulo 7 escribimos una función que contaba el número de apariciones de una letra en una cadena. Una versión más genérica de este problema es crear un histograma de las letras de la cadena, o sea, cuántas veces aparece cada letra.

Ese histograma podría ser útil para comprimir un archivo de texto. Como las diferentes letras aparecen con frecuencias distintas, podemos comprimir un archivo usando códigos cortos para las letras más habituales y códigos más largos para las que aparecen con menor frecuencia.

Los diccionarios facilitan una forma elegante de generar un histograma:

```
>>> cuentaLetras = {}
>>> for letra in "Mississippi":
...     cuentaLetras[letra] = cuentaLetras.get (letra, 0) + 1
...
>>> cuentaLetras
{'M': 1, 's': 4, 'p': 2, 'i': 4}
>>>
```

Inicialmente, tenemos un diccionario vacío. Para cada letra de la cadena, buscamos el recuento actual (posiblemente cero) y lo incrementamos. Al final, el diccionario contiene pares de letras y sus frecuencias.

Puede ser más atractivo mostrar el histograma en orden alfabético. Podemos hacerlo con los métodos `items` y `sort`:

```
>>> itemsLetras = cuentaLetras.items()
>>> itemsLetras.sort()
>>> print itemsLetras
[('M', 1), ('i', 4), ('p', 2), ('s', 4)]
```

Ya había visto usted el método `items`, pero `sort` es el primer método aplicable a listas que hemos visto. Hay varios más, como `append`, `extend`, y `reverse`. Consulte la documentación de Python para ver los detalles.

10.8. Glosario

diccionario: Una colección de pares clave-valor que establece una correspondencia entre claves y valores. Las claves pueden ser de cualquier tipo inmutable, los valores pueden ser de cualquier tipo.

clave: Un valor que se usa para buscar una entrada en un diccionario.

par clave-valor: Uno de los elementos de un diccionario, también llamado “asociación”.

método: Un tipo de función al que se llama con una sintaxis diferente y al que se invoca “sobre” un objeto.

invocar: Llamar a un método.

pista: Almacenamiento temporal de un valor precalculado para evitar cálculos redundantes.

desbordamiento: Un resultado numérico que es demasiado grande para representarse en formato numérico.

Capítulo 11

Archivos y excepciones

Cuando un programa se está ejecutando, sus datos están en la memoria. Cuando un programa termina, o se apaga el computador, los datos de la memoria desaparecen. Para almacenar los datos de forma permanente debe usted ponerlos en un **archivo**. Normalmente los archivos se guardan en un disco duro, disquete o CD-ROM.

Cuando hay un gran número de archivos, suelen estar organizados en **directorios** (también llamados “carpetas”). Cada archivo se identifica con un nombre único, o una combinación de nombre de archivo y nombre de directorio.

Leyendo y escribiendo archivos, los programas pueden intercambiar información entre ellos y generar formatos imprimibles como PDF.

Trabajar con archivos se parece mucho a trabajar con libros. Para usar un libro, tiene que abrirlo. Cuando ha terminado, tiene que cerrarlo. Mientras el libro está abierto, puede escribir en él o leer de él. En cualquier caso, sabe en qué lugar del libro se encuentra. Casi siempre lee el libro según su orden natural, pero también puede ir saltando de una página a otra.

Todo esto sirve también para los archivos. Para abrir un archivo, especifique su nombre e indique si quiere leer o escribir.

La apertura de un archivo crea un objeto archivo. En este ejemplo, la variable `f` apunta al nuevo objeto archivo.

```
>>> f = open("test.dat","w")
>>> print f
<open file 'test.dat', mode 'w' at fe820>
```

La función `open` toma dos argumentos. El primero es el nombre del archivo y el segundo es el modo. El modo `'w'` (write) significa que lo estamos abriendo para escribir.

Si no hay un archivo llamado `test.dat` se creará. Si ya hay uno, el archivo que estamos escribiendo lo reemplazará.

Al imprimir el objeto archivo, vemos el nombre del archivo, el modo y la localización del objeto.

Para meter datos en el archivo invocamos al método `write` sobre el objeto archivo:

```
>>> f.write("Ya es hora")
>>> f.write("de cerrar el archivo")
```

El cierre del archivo le dice al sistema que hemos terminado de escribir y deja el archivo listo para leer:

```
>>> f.close()
```

Ya podemos abrir el archivo de nuevo, esta vez para lectura, y poner su contenido en una cadena. Esta vez el argumento de modo es `'r'` (read) para lectura:

```
>>> f = open("test.dat", "r")
```

Si intentamos abrir un archivo que no existe, recibimos un mensaje de error:

```
>>> f = open("test.cat", "r")
IOError: [Errno 2] No such file or directory: 'test.cat'
```

Como era de esperar, el método `read` lee datos del archivo. Sin argumentos, lee el archivo completo:

```
>>> text = f.read()
>>> print text
Ya es horade cerrar el archivo
```

No hay un espacio entre “hora” y “de” porque no escribimos un espacio entre las cadenas.

`read` también puede aceptar un argumento que le indica cuántos caracteres leer:

```
>>> f = open("test.dat", "r")
>>> print f.read(7)
Ya es h
```

Si no quedan suficientes caracteres en el archivo, `read` devuelve los que haya. Cuando llegamos al final del archivo, `read` devuelve una cadena vacía:

```
>>> print f.read(1000006)
orade cerrar el archivo
>>> print f.read()
```

```
>>>
```

La siguiente función copia un archivo, leyendo y escribiendo los caracteres de cincuenta en cincuenta. El primer argumento es el nombre del archivo original; el segundo es el nombre del archivo nuevo:

```
def copiaArchivo(archViejo, archNuevo):
    f1 = open(archViejo, "r")
    f2 = open(archNuevo, "w")
    while 1:
        texto = f1.read(50)
        if texto == "":
            break
        f2.write(texto)
    f1.close()
    f2.close()
    return
```

La sentencia **break** es nueva. Su ejecución interrumpe el bucle; el flujo de la ejecución pasa a la primera sentencia tras el bucle.

En este ejmplo, el bucle **while** es infinito porque el valor **1** siempre es verdadero. La *única* forma de salir del bucle es ejecutar **break**, lo que sucede cuando **texto** es una cadena vacía, lo que sucede cuando llegamos al final del archivo.

11.1. Archivos de texto

Un **archivo de texto** es un archivo que contiene caracteres imprimibles y espacios organizados en líneas separadas por caracteres de salto de línea. Como Python está diseñado específicamente para procesar archivos de texto, proporciona métodos que facilitan la tarea.

Para hacer una demostración, crearemos un archivo de texto con tres líneas de texto separadas por saltos de línea:

```
>>> f = open("test.dat", "w")
>>> f.write("línea uno\nlínea dos\nlínea tres\n")
>>> f.close()
```

El método **readline** lee todos los caracteres hasta e inclusive el siguiente salto de línea:

```
>>> f = open("test.dat","r")
>>> print f.readline()
línea uno

>>>
```

`readlines` devuelve todas las líneas que queden como una lista de cadenas:

```
>>> print f.readlines()
['línea dos\n', 'línea tres\n']
```

En este caso, la salida está en forma de lista, lo que significa que las cadenas aparecen con comillas y el carácter de salto de línea aparece como la secuencia de escape `\n`.

Al final del archivo, `readline` devuelve una cadena vacía y `readlines` devuelve una lista vacía:

```
>>> print f.readline()

>>> print f.readlines()
[]
```

Lo que sigue es un ejemplo de un programa de proceso de líneas. `filtraArchivo` hace una copia de `archViejo`, omitiendo las líneas que comienzan por `#`:

```
def filtraArchivo(archViejo, archNuevo):
    f1 = open(archViejo, "r")
    f2 = open(archNuevo, "w")
    while 1:
        texto = f1.readline()
        if texto == "":
            break
        if texto[0] == '#':
            continue
        f2.write(texto)
    f1.close()
    f2.close()
    return
```

La sentencia `continue` termina la iteración actual del bucle, pero sigue haciendo bucles. El flujo de ejecución pasa al principio del bucle, comprueba la condición y continúa en consecuencia.

Así, si `texto` es una cadena vacía, el bucle termina. Si el primer carácter de `texto` es una almohadilla, el flujo de ejecución va al principio del bucle. Sólo si ambas condiciones fallan copiamos `texto` en el archivo nuevo.

11.2. Escribir variables

El argumento de `write` debe ser una cadena, así que si queremos poner otros valores en un archivo, tenemos que convertirlos antes en cadenas. La forma más fácil de hacerlo es con la función `str`:

```
>>> x = 52
>>> f.write (str(x))
```

Una alternativa es usar el **operador de formato** `%`. Cuando aplica a enteros, `%` es el operador de módulo. Pero cuando el primer operando es una cadena, `%` es el operador de formato.

El primer operando es la **cadena de formato**, y el segundo operando es una tupla de expresiones. El resultado es una cadena que contiene los valores de las expresiones, formateados de acuerdo a la cadena de formato.

A modo de ejemplo simple, la **secuencia de formato** `'%d'` significa que la primera expresión de la tupla debería formatearse como un entero. Aquí la letra *d* quiere decir “decimal”:

```
>>> motos = 52
>>> "%d" % motos
'52'
```

El resultado es la cadena `'52'`, que no debe confundirse con el valor entero 52.

Una secuencia de formato puede aparecer en cualquier lugar de la cadena de formato, de modo que podemos incrustar un valor en una frase:

```
>>> motos = 52
>>> "En julio vendimos %d motos." % motos
'En julio vendimos 52 motos.'
```

La secuencia de formato `'%f'` formatea el siguiente elemento de la tupla como un número en coma flotante, y `'%s'` formatea el siguiente elemento como una cadena:

```
>>> "En %d días ingresamos %f millones de %s." \
    % (34,6.1,'dólares')
'En 34 días ingresamose 6.100000 millones de dólares.'
```

Por defecto, el formato de coma flotante imprime seis decimales.

El número de expresiones en la tupla tiene que coincidir con el número de secuencias de formato de la cadena. Igualmente, los tipos de las expresiones deben coincidir con las secuencias de formato:

```
>>> "%d %d %d" % (1,2)
TypeError: not enough arguments for format string
>>> "%d" % 'dólares'
TypeError: illegal argument type for built-in operation
```

En el primer ejemplo, no hay suficientes expresiones; en el segundo, la expresión es de un tipo incorrecto.

Para tener más control sobre el formato de los números, podemos detallar el número de dígitos como parte de la secuencia de formato:

```
>>> "%6d" % 62
'   62'
>>> "%12f" % 6.1
'  6.100000'
```

El número tras el signo de porcentaje es el número mínimo de espacios que ocupará el número. Si el valor necesita menos dígitos, se añaden espacios en blanco delante del número. Si el número de espacios es negativo, se añaden los espacios tras el número:

```
>>> "%-6d" % 62
'62   '
```

También podemos especificar el número de decimales para los números en coma flotante:

```
>>> "%12.2f" % 6.1
'  6.10'
```

En este ejemplo, el resultado ocupa doce espacios e incluye dos dígitos tras la coma. Este formato es útil para imprimir cantidades de dinero con las comas alineadas.

Imagine, por ejemplo, un diccionario que contiene los nombres de los estudiantes como clave y las tarifas horarias como valores. He aquí una función que imprime el contenido del diccionario como un informe formateado:

```
def informe (tarifas) :
    estudiantes = tarifas.keys()
    estudiantes.sort()
```

```
for estudiante in estudiantes :  
    print "%-20s %12.02f" % (estudiante, tarifas[estudiante])
```

Para probar la función, crearemos un pequeño diccionario e imprimiremos el contenido:

```
>>> tarifas = {'maría': 6.23, 'josé': 5.45, 'jesús': 4.25}  
>>> informe (tarifas)  
josé                5.45  
jesús               4.25  
maría               6.23
```

Controlando la anchura de cada valor nos aseguramos de que las columnas van a quedar alineadas, siempre que los nombres tengan menos de veintiún caracteres y las tarifas sean menos de mil millones la hora.

11.3. Directorios

Cuando usted crea un archivo nuevo abriéndolo y escribiendo, el nuevo archivo va al directorio en uso (aquél en el que estuviese al ejecutar el programa). Del mismo modo, cuando abre un archivo para leerlo, Python lo busca en el directorio en uso.

Si quiere abrir un archivo de cualquier otro sitio, tiene que especificar la **ruta** del archivo, que es el nombre del directorio (o carpeta) donde se encuentra éste:

```
>>> f = open("/usr/share/dict/words", "r")  
>>> print f.readline()  
Aarhus
```

Este ejemplo abre un archivo llamado **words** que está en un directorio llamado **dict**, que está en **share**, que está en **usr**, que está en el directorio de nivel superior del sistema, llamado **/**.

No puede usar **/** como parte del nombre de un archivo; está reservado como delimitador entre nombres de archivo y directorios.

El archivo **/usr/share/dict/words** contiene una lista de palabras en orden alfabético, la primera de las cuales es el nombre de una universidad danesa.

11.4. Encurtido

Para poner valores en un archivo, debe convertirlos en cadenas. Ya ha visto cómo hacerlo con **str**:

```
>>> f.write (str(12.3))
>>> f.write (str([1,2,3]))
```

El problema es que cuando vuelve usted a leer el valor, obtiene una cadena. Ha perdido la información del tipo de dato original. En realidad, no puede distinguir dónde termina un valor y comienza el siguiente:

```
>>> f.readline()
'12.3[1, 2, 3]'
```

La solución es el **encurtido**, llamado así porque “conserva” estructuras de datos. El módulo `pickle` contiene las órdenes necesarias. Para usarlo, importe `pickle` y luego abra el archivo de la forma habitual:

```
>>> import pickle
>>> f = open("test.pck","w")
```

Para almacenar una estructura de datos, use el método `dump` y luego cierre el archivo de la forma habitual:

```
>>> pickle.dump(12.3, f)
>>> pickle.dump([1,2,3], f)
>>> f.close()
```

Ahora podemos abrir el archivo para leer y cargar las estructuras de datos que volcamos ahí:

```
>>> f = open("test.pck","r")
>>> x = pickle.load(f)
>>> x
12.3
>>> type(x)
<type 'float'>
>>> y = pickle.load(f)
>>> y
[1, 2, 3]
>>> type(y)
<type 'list'>
```

Cada vez que invocamos `load` obtenemos un valor del archivo, completo con su tipo original.

11.5. Excepciones

Siempre que ocurre un error en tiempo de ejecución, se crea una **excepción**. Normalmente el programa se para y Python presenta un mensaje de error.

Por ejemplo, la división por cero crea una excepción:

```
>>> print 55/0
ZeroDivisionError: integer division or modulo
```

Un elemento no existente en una lista hace lo mismo:

```
>>> a = []
>>> print a[5]
IndexError: list index out of range
```

O el acceso a una clave que no está en el diccionario:

```
>>> b = {}
>>> print b['qué']
KeyError: qué
```

En cada caso, el mensaje de error tiene dos partes: el tipo de error antes de los dos puntos y detalles sobre el error después de los dos puntos. Normalmente Python también imprime una traza de dónde se encontraba el programa, pero la hemos omitido en los ejemplos.

A veces queremos realizar una operación que podría provocar una excepción, pero no queremos que se pare el programa. Podemos **manejar** la excepción usando las sentencias **try** y **except**.

Por ejemplo, podemos preguntar al usuario por el nombre de un archivo y luego intentar abrirlo. Si el archivo no existe, no queremos que el programa se pare; queremos manejar la excepción.

```
nombreArch = raw_input('Introduce un nombre de archivo: ')
try:
    f = open(nombreArch, "r")
except:
    print 'No hay ningún archivo que se llame', nombreArch
```

La sentencia **try** ejecuta las sentencias del primer bloque. Si no se produce ninguna excepción, pasa por alto la sentencia **except**. Si ocurre cualquier excepción, ejecuta las sentencias de la rama **except** y después continúa.

Podemos encapsular esta capacidad en una función: **existe** acepta un nombre de archivo y devuelve verdadero si el archivo existe y falso si no:

```
def existe(nombreArch):
    try:
        f = open(nombreArch)
        f.close()
```

```
    return 1
except:
    return 0
```

Puede usar múltiples bloques **except** para manejar diferentes tipos de excepciones. El *Manual de Referencia de Python* contiene los detalles.

Si su programa detecta una condición de error, puede hacer que lance (**raise** en inglés) una excepción. Aquí tiene usted un ejemplo que acepta una entrada del usuario y comprueba si es 17. Suponiendo que 17 no es una entrada válida por cualquier razón, lanzamos una excepción.

```
def tomaNumero () :                # Recuerde, los acentos están
    x = input ('Elige un número: ') # prohibidos en los nombres
    if x == 17 :                    # de funciones y variables!
        raise 'ErrorNúmeroMalo', '17 es un mal número'
    return x
```

La sentencia **raise** acepta dos argumentos: el tipo de excepción e información específica acerca del error. **ErrorNúmeroMalo** es un nuevo tipo de excepción que hemos inventado para esta aplicación.

Si la función llamada **tomaNumero** maneja el error, el programa puede continuar; en caso contrario, Python imprime el mensaje de error y sale:

```
>>> tomaNumero ()
Elige un número: 17
ErrorNúmeroMalo: 17 es un mal número
```

El mensaje de error incluye el tipo de excepción y la información adicional que usted proporcionó.

*Como ejercicio, escriba una función que use **tomaNumero** para leer un número del teclado y que maneje la excepción **ErrorNúmeroMalo**.*

11.6. Glosario

archivo: Una entidad con nombre, normalmente almacenada en un disco duro, disquete o CD-ROM, que contiene una secuencia de caracteres.

directorio: Una colección, con nombre, de archivos, también llamado carpeta.

ruta: Una secuencia de nombres de directorio que especifica la localización exacta de un archivo.

archivo de texto: Un archivo que contiene caracteres imprimibles organizados en líneas separadas por caracteres de salto de línea.

sentencia break: Una sentencia que provoca que el flujo de ejecución salga de un bucle.

sentencia continue: Una sentencia que provoca que termine la iteración actual de un bucle. El flujo de la ejecución va al principio del bucle, evalúa la condición, y procede en consecuencia.

operador de formato: El operador % toma una cadena de formato y una tupla de expresiones y entrega una cadena que incluye las expresiones, formateadas de acuerdo con la cadena de formato.

cadena de formato: Una cadena que contiene caracteres imprimibles y secuencias de formato que indican cómo formatear valores.

secuencia de formato: Una secuencia de caracteres que comienza con % e indica cómo formatear un valor.

encurtir: Escribir el valor de un dato en un archivo junto con la información sobre su tipo de forma que pueda ser reconstituido más tarde.

excepción: Un error que ocurre en tiempo de ejecución.

manejar: Impedir que una excepción detenga un programa utilizando las sentencias `try` y `except`.

lanzar: Señalar una excepción usando la sentencia `raise`.

Capítulo 12

Clases y objetos

12.1. Tipos compuestos definidos por el usuario

Una vez utilizados algunos de los tipos internos de Python, estamos listos para crear un tipo definido por el usuario: el **Punto**.

Piense en el concepto de un punto matemático. En dos dimensiones, un punto es dos números (coordenadas) que se tratan colectivamente como un solo objeto. En notación matemática, los puntos suelen escribirse entre paréntesis con una coma separando las coordenadas. Por ejemplo, $(0, 0)$ representa el origen, y (x, y) representa el punto x unidades a la derecha e y unidades hacia arriba desde el origen.

Una forma natural de representar un punto en Python es con dos valores en coma flotante. La cuestión es, entonces, cómo agrupar esos dos valores en un objeto compuesto. La solución rápida y burda es utilizar una lista o tupla, y para algunas aplicaciones esa podría ser la mejor opción.

Una alternativa es que el usuario defina un nuevo tipo compuesto, también llamado una **clase**. Esta aproximación exige un poco más de esfuerzo, pero tiene sus ventajas que pronto se harán evidentes.

Una definición de clase se parece a esto:

```
class Punto:
    pass
```

Las definiciones de clase pueden aparecer en cualquier lugar de un programa, pero normalmente están al principio (tras las sentencias **import**). Las reglas

sintácticas de la definición de clases son las mismas que para cualesquiera otras sentencias compuestas. (ver la Sección 4.4).

Esta definición crea una nueva clase llamada **Punto**. La sentencia **pass** no tiene efectos; sólo es necesaria porque una sentencia compuesta debe tener algo en su cuerpo.

Al crear la clase **Punto** hemos creado un nuevo tipo, que también se llama **Punto**. Los miembros de este tipo se llaman **instancias** del tipo u **objetos**. La creación de una nueva instancia se llama **instanciación**. Para instanciar un objeto **Punto** ejecutamos una función que se llama (lo ha adivinado) **Punto**:

```
blanco = Punto()
```

A la variable **blanco** se le asigna una referencia a un nuevo objeto **Punto**. A una función como **Punto** que crea un objeto nuevo se le llama **constructor**.

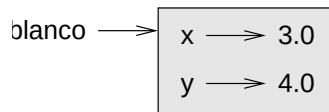
12.2. Atributos

Podemos añadir nuevos datos a una instancia utilizando la notación de punto:

```
>>> blanco.x = 3.0
>>> blanco.y = 4.0
```

Esta sintaxis es similar a la sintaxis para seleccionar una variable de un módulo, como **math.pi** o **string.uppercase**. En este caso, sin embargo, estamos seleccionando un dato de una instancia. Estos ítemes con nombre se llaman **atributos**.

El diagrama de estados que sigue muestra el resultado de esas asignaciones:



La variable **blanco** apunta a un objeto **Punto**, que contiene dos atributos. Cada atributo apunta a un número en coma flotante.

Podemos leer el valor de un atributo utilizando la misma sintaxis:

```
>>> print blanco.y
4.0
>>> x = blanco.x
>>> print x
3.0
```

La expresión `blanco.x` significa, “ve al objeto al que apunta `blanco` y toma el valor de `x`”. En este caso, asignamos ese valor a una variable llamada `x`. No hay conflicto entre la variable `x` y el atributo `x`. El propósito de la notación de punto es identificar de forma inequívoca a qué variable se refiere.

Puede usted usar la notación de punto como parte de cualquier expresión. Así, las sentencias que siguen son correctas:

```
print '(' + str(blanco.x) + ', ' + str(blanco.y) + ')\n'
distanciaAlCuadrado = blanco.x * blanco.x + blanco.y * blanco.y
```

La primera línea presenta (3.0, 4.0); la segunda línea calcula el valor 25.0.

Puede tentarle imprimir el propio valor de `blanco`:

```
>>> print blanco
<__main__.Punto instance at 80f8e70>
```

El resultado indica que `blanco` es una instancia de la clase `Punto` que se definió en `__main__`. `80f8e70` es el identificador único de este objeto, escrito en hexadecimal. Probablemente no es esta la manera más clara de mostrar un objeto `Punto`. En breve verá cómo cambiarlo.

Como ejercicio, cree e imprima un objeto `Punto` y luego use `id` para imprimir el identificador único del objeto. Traduzca el número hexadecimal a decimal y asegúrese de que coinciden.

12.3. Instancias como parámetro

Puede usted pasar una instancia como parámetro de la forma habitual. Por ejemplo:

```
def imprimePunto(p):
    print '(' + str(p.x) + ', ' + str(p.y) + ')\n'
```

`imprimePunto` acepta un punto como argumento y lo muestra en formato estándar. Si llama a `imprimePunto(blanco)`, el resultado es (3.0, 4.0).

Como ejercicio, reescriba la función `distancia` de la Sección 5.2 de forma que acepte dos `Puntos` como parámetros en lugar de cuatro números.

12.4. Mismidad

El significado de la palabra “mismo” parece totalmente claro hasta que uno se para un poco a pensarlo, y entonces se da cuenta de que hay algo más de lo que suponía.

Por ejemplo, si dice “Pepe y yo tenemos la misma moto”, lo que quiere decir es que su moto y la de usted son de la misma marca y modelo, pero que son dos motos distintas. Si dice “Pepe y yo tenemos la misma madre”, quiere decir que su madre y la de usted son la misma persona¹. Así que la idea de “identidad” es diferente según el contexto.

Cuando habla de objetos, hay una ambigüedad parecida. Por ejemplo, si dos `Puntos` son el mismo, ¿significa que contienen los mismos datos (coordenadas) o que son de verdad el mismo objeto?

Para averiguar si dos referencias se refieren al mismo objeto, utilice el operador `==`. Por ejemplo:

```
>>> p1 = Punto()
>>> p1.x = 3
>>> p1.y = 4
>>> p2 = Punto()
>>> p2.x = 3
>>> p2.y = 4
>>> p1 == p2
0
```

Aunque `p1` y `p2` contienen las mismas coordenadas, no son el mismo objeto. Si asignamos `p1` a `p2`, las dos variables son alias del mismo objeto:

```
>>> p2 = p1
>>> p1 == p2
1
```

Este tipo de igualdad se llama **igualdad superficial** porque sólo compara las referencias, pero no el contenido de los objetos.

Para comparar los contenidos de los objetos (**igualdad profunda**) podemos escribir una función llamada `mismoPunto`:

```
def mismoPunto(p1, p2) :
    return (p1.x == p2.x) and (p1.y == p2.y)
```

¹No todas las lenguas tienen el mismo problema. Por ejemplo, el alemán tiene palabras diferentes para los diferentes tipos de identidad. “Misma moto” en este contexto sería “gleiche Motorrad” y “misma madre” sería “selbe Mutter”.

Si ahora creamos dos objetos diferentes que contienen los mismos datos podremos usar `mismoPunto` para averiguar si representan el mismo punto:

```
>>> p1 = Punto()
>>> p1.x = 3
>>> p1.y = 4
>>> p2 = Punto()
>>> p2.x = 3
>>> p2.y = 4
>>> mismoPunto(p1, p2)
1
```

Por supuesto, si las dos variables apuntan al mismo objeto `mismoPunto` devuelve verdadero.

12.5. Rectángulos

Digamos que queremos una clase que represente un rectángulo. La pregunta es, ¿qué información tenemos que proporcionar para definir un rectángulo? Para simplificar las cosas, supongamos que el rectángulo está orientado vertical u horizontalmente, nunca en diagonal.

Tenemos varias posibilidades: podemos señalar el centro del rectángulo (dos coordenadas) y su tamaño (anchura y altura); o podemos señalar una de las esquinas y el tamaño; o podemos señalar dos esquinas opuestas. Un modo convencional es señalar la esquina superior izquierda del rectángulo y el tamaño.

De nuevo, definiremos una nueva clase:

```
class Rectangulo: # Prohibidos los acentos fuera de las cadenas!
    pass
```

Y la instanciamos:

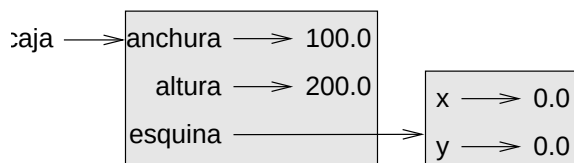
```
caja = Rectangulo()
caja.anchura = 100.0
caja.altura = 200.0
```

Este código crea un nuevo objeto `Rectangulo` con dos atributos en coma flotante. ¡Para señalar la esquina superior izquierda podemos incrustar un objeto dentro de otro!

```
caja.esquina = Punto()
caja.esquina.x = 0.0;
caja.esquina.y = 0.0;
```

El operador punto compone. La expresión `caja.esquina.x` significa “ve al objeto al que se refiere `caja` y selecciona el atributo llamado `esquina`; entonces ve a ese objeto y selecciona el atributo llamado `x`”.

La figura muestra el estado de este objeto:



12.6. Instancias como valores de retorno

Las funciones pueden devolver instancias. Por ejemplo, `encuentraCentro` acepta un `Rectangulo` como argumento y devuelve un `Punto` que contiene las coordenadas del centro del `Rectangulo`:

```
def encuentraCentro(caja):
    p = Punto()
    p.x = caja.esquina.x + caja.anchura/2.0
    p.y = caja.esquina.y + caja.altura/2.0
    return p
```

Para llamar a esta función, pase `caja` como argumento y asigne el resultado a una variable:

```
>>> centro = encuentraCentro(caja)
>>> imprimePunto(centro)
(50.0, 100.0)
```

12.7. Los objetos son mudables

Podemos cambiar el estado de un objeto efectuando una asignación sobre uno de sus atributos. Por ejemplo, para cambiar el tamaño de un rectángulo sin cambiar su posición, podemos cambiar los valores de `anchura` y `altura`:

```
caja.anchura = caja.anchura + 50
caja.altura = caja.altura + 100
```

Podemos encapsular este código en un método y generalizarlo para agrandar el rectángulo en cualquier cantidad:

```
def agrandaRect(caja, danchura, daltura) :  
    caja.anchura = caja.anchura + danchura  
    caja.altura = caja.altura + daltura
```

Las variables `danchura` y `daltura` indican cuánto debe agrandarse el rectángulo en cada dirección. Invocar este método tiene el efecto de modificar el `Rectángulo` que se pasa como argumento.

Por ejemplo, podemos crear un nuevo `Rectángulo` llamado `bob` y pasárselo a `agrandaRect`:

```
>>> bob = Rectangulo()  
>>> bob.anchura = 100.0  
>>> bob.altura = 200.0  
>>> bob.esquina = Punto()  
>>> bob.esquina.x = 0.0;  
>>> bob.esquina.y = 0.0;  
>>> agrandaRect(bob, 50, 100)
```

Mientras `agrandaRect` se está ejecutando, el parámetro `caja` es un alias de `bob`. Cualquier cambio que haga a `caja` afectará también a `bob`.

A modo de ejercicio, escriba una función llamada `mueveRect` que tome un `Rectángulo` y dos parámetros llamados `dx` y `dy`. Tiene que cambiar la posición del rectángulo añadiendo `dx` a la coordenada `x` de `esquina` y añadiendo `dy` a la coordenada `y` de `esquina`.

12.8. Copiado

El uso de alias puede hacer que un programa sea difícil de leer, porque los cambios hechos en un lugar pueden tener efectos inesperados en otro lugar. Es difícil estar al tanto de todas las variables a las que puede apuntar un objeto dado.

Copiar un objeto es, muchas veces, una alternativa a la creación de un alias. El módulo `copy` contiene una función llamada `copy` que puede duplicar cualquier objeto:

```
>>> import copy  
>>> p1 = Punto()  
>>> p1.x = 3  
>>> p1.y = 4  
>>> p2 = copy.copy(p1)  
>>> p1 == p2
```

```

0
>>> mismoPunto(p1, p2)
1

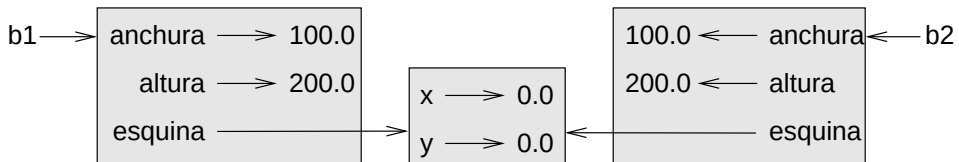
```

Una vez que hemos importado el módulo `copy`, podemos usar el método `copy` para hacer un nuevo `Punto`. `p1` y `p2` no son el mismo punto, pero contienen los mismos datos.

Para copiar un objeto simple como un `Punto`, que no contiene objetos incrustados, `copy` es suficiente. Esto se llama **copiado superficial**.

Para algo como un `Rectangulo`, que contiene una referencia a un `Punto`, `copy` no lo hace del todo bien. Copia la referencia al objeto `Punto`, de modo que tanto el `Rectangulo` viejo como el nuevo apuntan a un único `Punto`.

Si creamos una caja, `b1`, de la forma habitual y entonces hacemos una copia, `b2`, usando `copy`, el diagrama de estados resultante se ve así:



Es casi seguro que esto no es lo que queremos. En este caso, la invocación de `agrandarRect` sobre uno de los `Rectangulos` no afectaría al otro, ¡pero la invocación de `moverRect` sobre cualquiera afectaría a ambos! Este comportamiento es confuso y propicia los errores.

Afortunadamente, el módulo `copy` contiene un método llamado `deepcopy` que copia no sólo el objeto sino también cualesquiera objetos incrustados. No le sorprenderá saber que esta operación se llama **copia profunda** (deep copy).

```
>>> b2 = copy.deepcopy(b1)
```

Ahora `b1` y `b2` son objetos totalmente independientes.

Podemos usar `deepcopy` para reescribir `agrandarRect` de modo que en lugar de modificar un `Rectangulo` existente, cree un nuevo `Rectangulo` que tiene la misma localización que el viejo pero nuevas dimensiones:

```

def agrandarRect(caja, danchura, daltura) :
    import copy
    nuevaCaja = copy.deepcopy(caja)
    nuevaCaja.anchura = nuevaCaja.anchura + danchura
    nuevaCaja.altura = nuevaCaja.altura + daltura
    return nuevaCaja

```

Como ejercicio, rescriba `mueveRect` de modo que cree y devuelva un nuevo `Rectangulo` en lugar de modificar el viejo.

12.9. Glosario

clase: Un tipo compuesto definido por el usuario. También se puede pensar en una clase como una plantilla para los objetos que son instancias de la misma.

instanciar: Crear una instancia de una clase.

instancia: Un objeto que pertenece a una clase.

objeto: Un tipo de dato compuesto que suele usarse para representar una cosa o concepto del mundo real.

constructor: Un método usado para crear nuevos objetos.

atributo: Uno de los elementos de datos con nombre que constituyen una instancia.

igualdad superficial: Igualdad de referencias, o dos referencias que apuntan al mismo objeto.

igualdad profunda: Igualdad de valores, o dos referencias que apuntan a objetos que tienen el mismo valor.

copia superficial: Copiar el contenido de un objeto, incluyendo cualquier referencia a objetos incrustados; implementada por la función `copy` del módulo `copy`.

copia profunda: Copiar el contenido de un objeto así como cualesquiera objetos incrustados, y los incrustados en estos, y así sucesivamente; implementada por la función `deepcopy` del módulo `copy`.

Capítulo 13

Clases y funciones

13.1. Hora

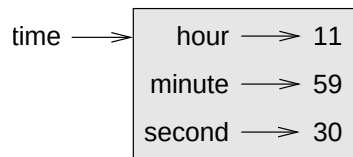
Como otro ejemplo de un tipo definido por el usuario, definiremos una clase llamada **Hora** que registra la hora del día. La definición de la clase es como sigue:

```
class Hora:  
    pass
```

Podemos crear un nuevo objeto **Hora** y asignar atributos para contener las horas, minutos y segundos:

```
hora = Hora()  
hora.horas = 11  
hora.minutos = 59  
hora.segundos = 30
```

El diagrama de estado del objeto **Hora** es así:



*A modo de ejercicio, escriba una función **imprimeHora** que acepte un objeto **Hora** como argumento y lo imprima en el formato **horas:minutos:segundos**.*

*Como un segundo ejercicio, escriba una función booleana **despues** que tome dos objetos **Hora**, **t1** y **t2**, como argumentos y devuelva verdadero (1) si **t1** sigue cronológicamente a **t2** y falso (0) en caso contrario.*

13.2. Funciones puras

En las próximas secciones, escribiremos dos versiones de una función llamada **sumaHora** que calcule la suma de dos Horas. Mostrarán dos tipos de funciones: funciones puras y modificadores.

Éste es un esbozo de **sumaHora**:

```
def sumaHora(t1, t2):
    suma = Hora()
    suma.horas = t1.horas + t2.horas
    suma.minutos = t1.minutos + t2.minutos
    suma.segundos = t1.segundos + t2.segundos
    return suma
```

La función crea un nuevo objeto **Hora**, inicializa sus atributos y devuelve una referencia al nuevo objeto. A esto se le llama **función pura** porque no modifica ninguno de los objetos que se le pasan y no tiene efectos laterales, como mostrar un valor o tomar una entrada del usuario.

Aquí tiene un ejemplo de cómo usar esta función. Crearemos dos objetos **Hora**: **horaActual**, que contiene la hora actual, y **horaPan**, que contiene la cantidad de tiempo que necesita un panadero para hacer pan. Luego usaremos **sumaHora** para averiguar cuándo estará hecho el pan. Si aún no ha terminado de escribir **imprimeHora**, eche un vistazo a la Sección 14.2 antes de probar esto:

```
>>> horaActual = Hora()
>>> horaActual.horas = 9
>>> horaActual.minutos = 14
>>> horaActual.segundos = 30

>>> horaPan = Hora()
>>> horaPan.horas = 3
>>> horaPan.minutos = 35
>>> horaPan.segundos = 0

>>> horaHecho = sumaHora(horaActual, horaPan)
>>> imprimeHora(horaHecho)
```


La salida de este programa es 12:49:30, lo que es correcto. Por otra parte, hay casos en los que el resultado no es correcto. ¿Puede imaginar uno?

El problema es que esta función no trata los casos en los que el número de segundos o minutos suma más que sesenta. Cuando ocurre eso, debemos “llevar” los segundos sobrantes a la columna de los minutos o los minutos extras a la columna de las horas.

He aquí una versión corregida de la función:

```
def sumaHora(t1, t2):
    suma = Hora()
    suma.horas = t1.horas + t2.horas
    suma.minutos = t1.minutos + t2.minutos
    suma.segundos = t1.segundos + t2.segundos

    if suma.segundos >= 60:
        suma.segundos = suma.segundos - 60
        suma.minutos = suma.minutos + 1

    if suma.minutos >= 60:
        suma.minutos = suma.minutos - 60
        suma.horas = suma.horas + 1

    return suma
```

Aunque esta función es correcta, empieza a ser grande. Más adelante sugeriremos una aproximación alternativa que nos dará un código más corto.

13.3. Modificadores

Hay veces en las que es útil que una función modifique uno o más de los objetos que recibe como parámetros. Normalmente, el llamante conserva una referencia a los objetos que pasa, así que cualquier cambio que la función haga será visible para el llamante. Las funciones que trabajan así se llaman **modificadores**.

incremento, que añade un número dado de segundos a un objeto `Hora`, se escribiría de forma natural como un modificador. Un esbozo rápido de la función podría ser éste:

```
def incremento(hora, segundos):
    hora.segundos = hora.segundos + segundos

    if hora.segundos >= 60:
        hora.segundos = hora.segundos - 60
        hora.minutos = hora.minutos + 1

    if hora.minutos >= 60:
        hora.minutos = hora.minutos - 60
        hora.horas = hroa.horas + 1
```

La primera línea realiza la operación básica, las restantes tratan con los casos especiales que vimos antes.

¿Es correcta esta función? ¿Qué ocurre si el parámetro **segundos** es mucho mayor que sesenta? En tal caso, no es suficiente con acarrear una vez; debemos seguir haciéndolo hasta que **segundos** sea menor que sesenta. Una solución es sustituir las sentencias **if** por sentencias **while**:

```
def incremento(hora, segundos):
    hora.segundos = hora.segundos + segundos

    while hora.segundos >= 60:
        hora.segundos = hora.segundos - 60
        hora.minutos = hora.minutos + 1

    while hora.minutos >= 60:
        hora.minutos = hora.minutos - 60
        hora.horas = hroa.horas + 1
```

Ahora esta función es correcta, pero no es la solución más eficiente.

Como ejercicio, reescriba esta función de modo que no contenga tantos bucles.

*Como un segundo ejercicio, reescriba **incremento** como una función pura, y escriba una función que llame a ambas versiones.*

13.4. ¿Qué es mejor?

Todo lo que se pueda hacer con modificadores puede hacerse también con funciones puras. En realidad, algunos lenguajes de programación sólo permiten funciones puras. Hay ciertas evidencias de que los programas que usan funciones puras son más rápidos de desarrollar y menos propensos a los errores que

los programas que usan modificadores. Sin embargo, a veces los modificadores son útiles, y en algunos casos los programas funcionales son menos eficientes.

En general, recomendamos que escriba funciones puras siempre que sea razonable hacerlo así y recurra a los modificadores sólo si hay una ventaja convincente. Este enfoque podría llamarse **estilo funcional de programación**.

13.5. Desarrollo de prototipos frente a planificación

En este capítulo mostramos una aproximación al desarrollo de programas a la que llamamos **desarrollo de prototipos**. En cada caso, escribimos un esbozo basto (o prototipo) que realizaba el cálculo básico y luego lo probamos sobre unos cuantos casos, corrigiendo los fallos tal como los encontrábamos.

Aunque este enfoque puede ser efectivo, puede conducirnos a código que es innecesariamente complicado, ya que trata con muchos casos especiales, y poco fiable, porque es difícil saber si encontró todos los errores.

Una alternativa es el **desarrollo planificado**, en el que una comprensión del problema en profundidad puede hacer la programación mucho más fácil. En este caso, el enfoque es que un objeto `Hora` es en realidad ¡un número de tres dígitos en base 60! El componente **segundo** es la “columna de unidades”, el componente **minuto** es la “columna de las sesentenas” y el componente **hora** es la “columna de las tresmilseiscientenas”.

Cuando escribimos `sumaHora` e `incremento`, en realidad estábamos haciendo una suma en base 60, que es por lo que debíamos acarrear de una columna a la siguiente.

Esta observación sugiere otro enfoque para el problema. Podemos convertir un objeto `Hora` en un simple número y sacar provecho del hecho de que la máquina sabe la aritmética necesaria. La siguiente función convierte un objeto `Hora` en un entero:

```
def convierteASegundos(t):  
    minutos = t.horas * 60 + t.minutos  
    segundos = minutos * 60 + t.segundos  
    return segundos
```

Ahora, sólo necesitamos una forma de convertir un entero en un objeto `Hora`:

```
def haceHora(segundos):  
    hora = Hora()  
    hora.horas = segundos/3600
```

```
segundos = segundos - hora.horas * 3600
hora.minutos = segundos/60
segundos = segundos - hora.minutos * 60
hora.segundos = segundos
return hora
```

Puede que tenga usted que pensar un poco para convencerse de que esta técnica para convertir de una base a otra es correcta. Suponiendo que está usted convencido, puede usar estas funciones para reescribir `sumaHora`:

```
def sumaHora(t1, t2):
    segundos = convierteASegundos(t1) + convierteASegundos(t2)
    return haceHora(segundos)
```

Esta versión es mucho más corta que la original, y es mucho más fácil de demostrar que es correcta (suponiendo, como es habitual, que las funciones a las que llama son correctas).

Como ejercicio, reescriba `incremento` de la misma forma.

13.6. Generalización

De algún modo, convertir de base 60 a base 10 y de vuelta es más difícil que simplemente manejarse con las horas. La conversión de base es más abstracta; nuestra intuición para tratar con las horas es mejor.

Pero si tenemos la comprensión para tratar las horas como números en base 60, y hacer la inversión de escribir las funciones de conversión (`convierteASegundos` y `haceHora`), obtenemos un programa que es más corto, más fácil de leer y depurar y más fiable.

También es más fácil añadir funcionalidades más tarde. Por ejemplo, imagine restar dos `Horas` para hallar el intervalo entre ellas. La aproximación ingenua sería implementar la resta con acarreo. Con el uso de las funciones de conversión será más fácil y con mayor probabilidad, correcto.

Irónicamente, a veces hacer un problema más complejo (o más general) lo hace más fácil (porque hay menos casos especiales y menos oportunidades de error).

13.7. Algoritmos

Cuando escribe una solución general para una clase de problemas, en contraste con una solución específica a un problema concreto, ha escrito un **algoritmo**.

Mencionamos esta palabra antes pero no la definimos con precisión. No es fácil de definir, así que probaremos un par de enfoques.

Primero, piense en algo que no es un algoritmo. Cuando usted aprendió a multiplicar números de una cifra, probablemente memorizó la tabla de multiplicar. En efecto, memorizó 100 soluciones específicas. Ese tipo de conocimiento no es algorítmico.

Pero si usted era “haragán” probablemente hizo trampa aprendiendo algunos trucos. Por ejemplo, para encontrar el producto de n por 9, puede escribir $n - 1$ como el primer dígito y $10 - n$ como el segundo dígito. Este truco es una solución general para multiplicar cualquier número de una cifra por 9. ¡Eso es un algoritmo!

De forma similar, las técnicas que aprendió para la suma y la resta con acarreo y la división larga son todas algoritmos. Una de las características de los algoritmos es que no requieren inteligencia para llevarse a cabo. Son procesos mecánicos en los que cada paso sigue al anterior de acuerdo a un conjunto simple de reglas.

En nuestra opinión, es un poco vergonzoso que los humanos pasen tanto tiempo en la escuela aprendiendo a ejecutar algoritmos que, de forma bastante similar, no exigen inteligencia.

Por otra parte, el proceso de diseñar algoritmos es interesante, un desafío intelectual y una parte primordial de lo que llamamos programar.

Algunas de las cosas que la gente hace naturalmente, sin dificultad ni pensamiento consciente, son las más difíciles de expresar algorítmicamente. Entender el lenguaje natural es un buen ejemplo. Todos lo hacemos, pero hasta el momento nadie ha sido capaz de explicar *cómo* lo hacemos, al menos no en la forma de un algoritmo.

13.8. Glosario

función pura: Una función que no modifica ninguno de los objetos que recibe como parámetros. La mayoría de las funciones puras son rentables.

modificador: Una función que modifica uno o más de los objetos que recibe como parámetros. La mayoría de los modificadores no entregan resultado.

estilo funcional de programación: Un estilo de programación en el que la mayoría de las funciones son puras.

desarrollo de prototipos: Una forma de desarrollar programas empezando con un prototipo y probándolo y mejorándolo gradualmente.

desarrollo planificado: Una forma de desarrollar programas que implica una comprensión de alto nivel del problema y más planificación que desarrollo incremental o desarrollo de prototipos.

algoritmo: Un conjunto de instrucciones para solucionar una clase de problemas por medio de un proceso mecánico sin intervención de inteligencia.

Capítulo 14

Clases y métodos

14.1. Características de la orientación a objetos

Python es un **lenguaje de programación orientado a objetos**, lo que significa que proporciona características que apoyan la **programación orientada a objetos**.

No es fácil definir la programación orientada a objetos, pero ya hemos visto algunas de sus características:

- Los programas se hacen a base de definiciones de objetos y definiciones de funciones, y la mayor parte de la computación se expresa en términos de operaciones sobre objetos.
- Cada definición de un objeto se corresponde con un objeto o concepto del mundo real, y las funciones que operan en ese objeto se corresponden con las formas en que interactúan los objetos del mundo real.

Por ejemplo, la clase `Hora` definida en el Capítulo 13 se corresponde con la forma en la que la gente registra la hora del día, y las funciones que definimos se corresponden con el tipo de cosas que la gente hace con las horas. De forma similar, las clases `Punto` y `Rectángulo` se corresponden con los conceptos matemáticos de un punto y un rectángulo.

Hasta ahora, no nos hemos aprovechado de las características que Python nos ofrece para dar soporte a la programación orientada a objetos. Hablando estrictamente, estas características no son necesarias. En su mayoría, proporcionan una sintaxis alternativa para cosas que ya hemos hecho, pero en muchos casos,

la alternativa es más concisa y expresa con más precisión a la estructura del programa.

Por ejemplo, en el programa `Hora` no hay una conexión obvia entre la definición de la clase y las definiciones de las funciones que siguen. Observando bien, se hace patente que todas esas funciones toman al menos un objeto `Hora` como parámetro.

Esta observación es la que motiva los **métodos**. Ya hemos visto varios métodos, como **keys** y **values**, que se invocan sobre diccionarios. Cada método está asociado con una clase y está pensado para invocarse sobre instancias de esa clase.

Los métodos son como las funciones, con dos diferencias:

- Los métodos se definen dentro de una definición de clase para explicitar la relación entre la clase y el método.
- La sintaxis para invocar un método es diferente de la de una llamada a una función.

En las próximas secciones tomaremos las funciones de los capítulos anteriores y las transformaremos en métodos. Esta transformación es puramente mecánica; puede hacerla simplemente siguiendo una secuencia de pasos. Si se acostumbra a convertir de una forma a la otra será capaz de elegir la mejor forma de hacer lo que quiere.

14.2. `imprimeHora`

En el Capítulo 13, definimos una clase llamada `Hora` y escribimos una función llamada `imprimeHora`, que debería ser parecida a esto:

```
class Hora:
    pass

def imprimeHora(hora):
    print str(hora.horas) + ":" +
          str(hora.minutos) + ":" +
          str(hora.segundos)
```

Para llamar a esta función, pasábamos un objeto `Hora` como parámetro:

```
>>> horaActual = Hora()
>>> horaActual.horas = 9
>>> horaActual.minutos = 14
>>> horaActual.segundos = 30
>>> imprimeHora(horaActual)
```


Para convertir `imprimeHora` en un método, todo lo que necesitamos hacer es mover la definición de la función al interior de la definición de la clase. Fíjese en cómo cambia el sangrado.

```
class Hora:
    def imprimeHora(hora):
        print str(hora.horas) + ":" +
              str(hora.minutos) + ":" +
              str(hora.segundos)
```

Ahora podemos invocar `imprimeHora` usando la notación de punto.

```
>>> horaActual.imprimeHora()
```

Como es habitual, el objeto sobre el que se invoca el método aparece delante del punto y el nombre del método aparece tras el punto.

El objeto sobre el que se invoca el método se asigna al primer parámetro, así que en este caso `horaActual` se asigna al parámetro `hora`.

Por convenio, el primer parámetro de un método se llama `self`. La razón de esto es un tanto rebuscada, pero se basa en una metáfora útil.

La sintaxis para la llamada a una función, `imprimeHora(horaActual)`, sugiere que la función es el agente activo. Dice algo como “¡Oye `imprimeHora`! Aquí hay un objeto para que lo imprimas”.

En programación orientada a objetos, los objetos son los agentes activos. Una invocación como `horaActual.imprimeHora()` dice “¡Oye `horaActual`! ¡Imprímete!”

Este cambio de perspectiva puede ser más elegante, pero no es obvio que sea útil. En los ejemplos que hemos visto hasta ahora, puede no serlo. Pero a veces transferir la responsabilidad de las funciones a los objetos hace posible escribir funciones más versátiles, y hace más fácil mantener y reutilizar código.

14.3. Otro ejemplo

Vamos a convertir `incremento` (de la Sección 13.3) en un método. Para ahorrar espacio, dejaremos a un lado los métodos ya definidos, pero usted debería mantenerlos en su versión:

```
class Hora:
    #aquí van las definiciones anteriores de métodos...
```

```
def incremento(self, segundos):
    self.segundos = segundos + self.segundos

    while self.segundos >= 60:
        self.segundos = self.segundos - 60
        self.minutos = self.minutos + 1

    while self.minutos >= 60:
        self.minutos = self.minutos - 60
        self.horas = self.horas + 1
```

La transformación es puramente mecánica; hemos llevado la definición del método al interior de la definición de la clase y hemos cambiado el nombre del primer parámetro.

Ahora podemos invocar `incremento` como un método.

```
horaActual.incremento(500)
```

De nuevo, el objeto sobre el que invocamos el método se asigna al primer parámetro, `self`. El segundo parámetro, `segundos` toma el valor de 500.

Como ejercicio, convierta `convertirASegundos` (de la Sección 13.5) en un método de la clase `Hora`.

14.4. Un ejemplo más complicado

La función `despues` es ligeramente más complicada porque opera sobre dos objetos `Hora`, no sólo sobre uno. Sólo podemos convertir uno de los parámetros en `self`; el otro se queda como está:

```
class Hora:
    #aquí van las definiciones anteriores de métodos...

    def despues(self, hora2):
        if self.horas > hora2.horas:
            return 1
        if self.horas < hora2.horas:
            return 0

        if self.minutos > hora2.minutos:
            return 1
```

```
if self.minutos < hora2.minutos:
    return 0

if self.segundos > hora2.segundos:
    return 1
return 0
```

Invocamos este método sobre un objeto y pasamos el otro como argumento:

```
if horaHecho.despues(horaActual):
    print "El pan estará hecho después de empezar."
```

Casi puede leer la invocación como una mezcla de inglés y español: “Si la hora-hecho es después de la hora-actual, entonces...”

14.5. Argumentos opcionales

Hemos visto funciones internas que toman un número variable de argumentos. Por ejemplo, `string.find` puede tomar dos, tres o cuatro argumentos.

Es posible escribir funciones definidas por el usuario con listas de argumentos opcionales. Por ejemplo, podemos modernizar nuestra propia versión de `encuentra` para que haga lo mismo que `string.find`.

Esta es la versión original de la Sección 7.7:

```
def encuentra(cad, c):
    indice = 0
    while indice < len(cad):
        if str[indice] == c:
            return indice
        indice = indice + 1
    return -1
```

Esta es la versión aumentada y mejorada:

```
def encuentra(cad, c, comienzo=0):
    indice = comienzo
    while indice < len(cad):
        if str[indice] == c:
            return indice
        indice = indice + 1
    return -1
```

El tercer parámetro, **comienzo**, es opcional porque se proporciona un valor por omisión, 0. Si invocamos **encuentra** sólo con dos argumentos, utilizamos el valor por omisión y comenzamos por el principio de la cadena:

```
>>> encuentra("arriba", "r")
1
```

Si le damos un tercer parámetro, **anula** el predefinido:

```
>>> encuentra("arriba", "r", 2)
2
>>> encuentra("arriba", "r", 3)
-1
```

*Como ejercicio, añada un cuarto parámetro, **fin**, que especifique dónde dejar de buscar.*

*Cuidado: Este ejercicio tiene truco. El valor por omisión de **fin** debería ser **len(cad)**, pero eso no funciona. Los valores por omisión se evalúan al definir la función, no al llamarla. Cuando se define **encuentra**, **cad** aún no existe, así que no puede averiguar su longitud.*

14.6. El método de inicialización

El **método de inicialización** es un método especial que se invoca al crear un objeto. El nombre de este método es **__init__** (dos guiones bajos, seguidos de **init** y dos guiones bajos más). Un método de inicialización para la clase **Hora** es así:

```
class Hora:
    def __init__(self, horas=0, minutos=0, segundos=0):
        self.horas = horas
        self.minutos = minutos
        self.segundos = segundos
```

No hay conflicto entre el atributo **self.horas** y el parámetro **horas**. la notación de punto especifica a qué variable nos referimos.

Cuando invocamos el constructor **Hora**, los argumentos que damos se pasan a **init**:

```
>>> horaActual = Hora(9, 14, 30)
>>> horaActual.imprimeHora()
>>> 9:14:30
```

Como los parámetros son opcionales, podemos omitirlos:

```
>>> horaActual = Hora()
>>> horaActual.imprimeHora()
>>> 0:0:0
```

O dar sólo el primer parámetro:

```
>>> horaActual = Hora (9)
>>> horaActual.imprimeHora()
>>> 9:0:0
```

O los dos primeros parámetros:

```
>>> horaActual = Hora (9, 14)
>>> horaActual.imprimeHora()
>>> 9:14:0
```

Finalmente, podemos dar un subconjunto de los parámetros nombrándolos explícitamente:

```
>>> horaActual = Hora(segundos = 30, horas = 9)
>>> horaActual.imprimeHora()
>>> 9:0:30
```

14.7. Revisión de los Puntos

Vamos a reescribir la clase `Punto` de la Sección 12.1 con un estilo más orientado a objetos:

```
class Punto:
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __str__(self):
        return '(' + str(self.x) + ', ' + str(self.y) + ')'
```

El método de inicialización toma los valores de x e y como parámetros opcionales; el valor por omisión de cada parámetro es 0.

El siguiente método, `__str__`, devuelve una representación en forma de cadena de un objeto `Punto`. Si una clase ofrece un método llamado `__str__`, se impone al comportamiento por defecto de la función interna `str` de Python.

```
>>> p = Punto(3, 4)
>>> str(p)
'(3, 4)'
```

Imprimir un objeto `Punto` invoca implícitamente a `__str__` sobre el objeto, así que definir `__str__` también cambia el comportamiento de `print`:

```
>>> p = Punto(3, 4)
>>> print p
(3, 4)
```

Cuando escribimos una nueva clase, casi siempre empezamos escribiendo `__init__`, que facilita el instanciar objetos, y `__str__`, que casi siempre es útil para la depuración.

14.8. Sobrecarga de operadores

Algunos lenguajes hacen posible cambiar la definición de los operadores internos cuando se aplican a tipos definidos por el usuario. Esta característica se llama **sobrecarga de operadores**. Es especialmente útil cuando definimos nuevos tipos matemáticos.

Por ejemplo, para suplantar al operador de suma `+` necesitamos proporcionar un método llamado `__add__`:

```
class Punto:
    # aquí van los métodos que ya habíamos definido...

    def __add__(self, otro):
        return Punto(self.x + otro.x, self.y + otro.y)
```

Como es habitual, el primer parámetro es el objeto sobre el que se invoca el método. El segundo parámetro se llama convenientemente `otro` para distinguirlo del mismo (`self`). Para sumar dos `Puntos`, creamos y devolvemos un nuevo `Punto` que contiene la suma de las coordenadas x y la suma de las coordenadas y .

Ahora, cuando apliquemos el operador `+` a objetos `Punto`, Python invocará a `__add__`:

```
>>> p1 = Punto(3, 4)
>>> p2 = Punto(5, 7)
>>> p3 = p1 + p2
>>> print p3
(8, 11)
```

La expresión `p1 + p2` equivale a `p1.__add__(p2)`, pero es obviamente más elegante.

Como ejercicio, añada un método `__sub__(self, otro)` que sobrecargue el operador resta y pruébelo.

Hay varias formas de sobrecargar el comportamiento del operador multiplicación: definiendo un método llamado `__mul__`, o `__rmul__`, o ambos.

Si el operando a la izquierda de `*` es un `Punto`, Python invoca a `__mul__`, lo que presupone que el otro operando es también un `Punto`. Calcula el **producto interno** de dos puntos, definido según las reglas del álgebra lineal:

```
def __mul__(self, otro):
    return self.x * otro.x + self.y * otro.y
```

Si el operando a la izquierda de `*` es un tipo primitivo y el operando de la derecha es un `Punto`, Python invoca a `__rmul__`, lo que realiza una **multiplicación escalar**:

```
def __rmul__(self, otro):
    return Punto(otro * self.x, otro * self.y)
```

El resultado es un nuevo `Punto` cuyas coordenadas son múltiplos de las coordenadas originales. Si `otro` es un tipo que no se puede multiplicar por un número en coma flotante, entonces `__rmul__` causará un error.

Este ejemplo muestra ambos tipos de multiplicación:

```
>>> p1 = Punto(3, 4)
>>> p2 = Punto(5, 7)
>>> print p1 * p2
43
>>> print 2 * p2
(10, 14)
```

¿Qué ocurre si intentamos evaluar `p2 * 2`? Como el primer parámetro es un `Punto`, Python invoca a `__mul__` con 2 como el segundo parámetro. Dentro de `__mul__`, el programa intenta acceder a la coordenada `x` de `otro`, pero no lo consigue porque un entero no tiene atributos:

```
>>> print p2 * 2
AttributeError: 'int' object has no attribute 'x'
```

Desgraciadamente, el mensaje de error es un poco opaco. Este ejemplo muestra algunas de las dificultades de la programación orientada a objetos. A veces es difícil averiguar simplemente qué código se está ejecutando.

Para ver un ejemplo más completo de sobrecarga de operadores, vaya al Apéndice B.

14.9. Polimorfismo

La mayoría de los métodos que hemos escrito funcionan sólo para un tipo específico. Cuando usted crea un nuevo objeto, escribe métodos que operan sobre ese tipo.

Pero hay ciertas operaciones que querrá aplicar a muchos tipos, como las operaciones aritméticas de las secciones anteriores. Si muchos tipos admiten el mismo conjunto de operaciones, puede escribir funciones que trabajen sobre cualquiera de esos tipos.

Por ejemplo, la operación `multisuma` (común en álgebra lineal) toma tres parámetros; multiplica los dos primeros y luego suma el tercero. Podemos escribirla en Python así:

```
def multisuma (x, y, z):  
    return x * y + z
```

Este método trabajará con cualquier valor de `x` e `y` que se pueda multiplicar y con cualquier valor de `z` que se pueda sumar al producto.

Podemos invocarlo con valores numéricos:

```
>>> multisuma (3, 2, 1)  
7
```

O con Puntos:

```
>>> p1 = Punto(3, 4)  
>>> p2 = Punto(5, 7)  
>>> print multisuma (2, p1, p2)  
(11, 15)  
>>> print multisuma (p1, p2, 1)  
44
```

En el primer caso, el `Punto` se multiplica por un escalar y luego se suma a otro `Punto`. En el segundo caso, el producto interior produce un valor numérico, así que el tercer parámetro también debe ser un valor numérico.

Una función como ésta que puede tomar parámetros con diferentes tipos se llama **polimórfica**.

Como un ejemplo más, observe el método `delDerechoYDelReves`, que imprime dos veces una lista, hacia adelante y hacia atrás:


```
def delDerechoYDelReves(derecho):  
    import copy  
    reves = copy.copy(derecho)  
    reves.reverse()  
    print str(derecho) + str(reves)
```

Como el método `reverse` es un modificador, hacemos una copia de la lista antes de darle la vuelta. Así, este método no modifica la lista que recibe como parámetro.

He aquí un ejemplo que aplica `delDerechoYDelReves` a una lista:

```
>>> miLista = [1, 2, 3, 4]  
>>> delDerechoYDelReves(miLista)  
[1, 2, 3, 4][4, 3, 2, 1]
```

Por supuesto, pretendíamos aplicar esta función a listas, así que no es sorprendente que funcione. Lo sorprendente es que pudiéramos usarla con un **Punto**.

Para determinar si una función se puede aplicar a un nuevo tipo, aplicamos la regla fundamental del polimorfismo:

**Si todas las operaciones realizadas dentro de la función se
pueden aplicar al tipo, la función se puede aplicar al tipo.**

Las operaciones del método incluyen `copy`, `reverse` y `print`.

`copy` trabaja sobre cualquier objeto, y ya hemos escrito un método `__str__` para los **Puntos**, así que todo lo que necesitamos es un método `reverse` en la clase **Punto**:

```
def reverse(self):  
    self.x , self.y = self.y, self.x
```

Ahora podemos pasar **Puntos** a `delDerechoYDelReves`:

```
>>> p = Punto(3, 4)  
>>> delDerechoYDelReves(p)  
(3, 4)(4, 3)
```

El mejor tipo de polimorfismo es el que no se busca, cuando usted descubre que una función que había escrito se puede aplicar a un tipo para el que nunca la había planeado.

14.10. Glosario

lenguaje orientado a objetos: Un lenguaje que ofrece características, como clases definidas por el usuario y herencia, que facilitan la programación orientada a objetos.

programación orientada a objetos: Un estilo de programación en el que los datos y las operaciones que los manipulan están organizadas en clases y métodos.

método: Una función definida dentro de una definición de clase y que se invoca sobre instancias de esa clase.

imponer: Reemplazar una opción por omisión. Los ejemplos incluyen el reemplazo de un parámetro por omisión con un argumento particular y el reemplazo de un método por omisión proporcionando un nuevo método con el mismo nombre.

método de inicialización: Un método especial que se invoca automáticamente al crear un nuevo objeto y que inicializa los atributos del objeto.

sobrecarga de operadores: Ampliar los operadores internos (+, -, *, >, <, etc.) de modo que trabajen con tipos definidos por el usuario.

producto interno: Una operación definida en álgebra lineal que multiplica dos Puntos y entrega un valor numérico.

multiplicación escalar: Una operación definida en álgebra lineal que multiplica cada una de las coordenadas de un Punto por un valor numérico.

polimórfica: Una función que puede operar sobre más de un tipo. Si todas las operaciones realizadas dentro de una función se pueden aplicar a un tipo, la función se puede aplicar a ese tipo.

Capítulo 15

Conjuntos de objetos

15.1. Composición

Hasta ahora, ya ha visto varios ejemplos de composición. Uno de los primeros ejemplos fue el uso de la llamada a un método como parte de una expresión. Otro ejemplo es la estructura anidada de las sentencias; se puede escribir una sentencia `if` dentro de un bucle `while`, dentro de otra sentencia `if`, y así sucesivamente.

Una vez visto este patrón, y sabiendo acerca de listas y objetos, no le debería sorprender que pueda crear listas de objetos. También puede crear objetos que contengan listas (en forma de atributos); puede crear listas que contengan listas; objetos que contengan objetos, y así indefinidamente.

En este capítulo y el siguiente, exploraremos algunos ejemplos de estas combinaciones, y usaremos objetos `Carta` como ejemplo.

15.2. Objetos Carta

Si no está usted familiarizado con los naipes de juego comunes, puede ser un buen momento para que consiga un mazo, si no este capítulo puede que no tenga mucho sentido. Hay cincuenta y dos naipes en una baraja inglesa, cada uno de los cuales pertenece a un palo y tiene un valor; hay cuatro palos diferentes y trece valores. Los palos son Picas, Corazones, Diamantes, y Tréboles (en el orden descendente según el bridge). Los valores son As, 2, 3, 4, 5, 6, 7, 8, 9, 10, Sota, Reina, y Rey. Dependiendo del tipo de juego que se juegue, el valor del As puede ser mayor al Rey o inferior al 2.

Si queremos definir un nuevo objeto para representar un naípe, es obvio qué atributos debería tener: **valor** y **palo**. Lo que no es tan obvio es el tipo que se debe dar a los atributos. Una posibilidad es usar cadenas de caracteres que contengan palabras como "**Picas**" para los palos y "**Reina**" para los valores. Un problema de esta implementación es que no será fácil comparar naipes para ver cuál tiene mayor valor o palo.

Una alternativa es usar números enteros para **codificar** los valores y palos. Con el término "codificar" no queremos significar lo que algunas personas pueden pensar, acerca de cifrar o traducir a un código secreto. Lo que un programador entiende por "codificar" es "definir una correspondencia entre una secuencia de números y los elementos que se desea representar". Por ejemplo:

Picas	↦	3
Corazones	↦	2
Diamantes	↦	1
Tréboles	↦	0

Esta correspondencia tiene una característica obvia: los palos corresponden a números enteros en orden, o sea que podemos comparar los palos al comparar los números. La asociación de los valores es bastante obvia; cada uno de los valores numéricos se asocia con el entero correspondiente, y para las figuras:

Sota	↦	11
Reina	↦	12
Rey	↦	13

Estamos usando una notación matemática para estas asociaciones por una razón: no son parte del programa Python. Son parte del diseño del programa, pero nunca aparecen explícitamente en el código fuente. La definición de clase para el tipo **Carta** se parecerá a:

```
class Carta:
    def __init__(self, palo=0, valor=0):
        self.palo = palo
        self.valor = valor
```

Como acostumbramos, proporcionaremos un método de inicialización que toma un parámetro opcional para cada atributo.

Para crear un objeto que representa el 3 de Tréboles, usaremos la instrucción:

```
tresDeTreboles = Carta(0, 3)
```

El primer argumento, 0, representa el palo de Tréboles.

15.3. Atributos de clase y el método `__str__`

Para poder imprimir los objetos `Carta` de una manera fácil de leer para las personas, vamos a establecer una correspondencia entre los códigos enteros y las palabras. Una manera natural de hacer esto es con listas de cadenas de caracteres. Asignaremos estas listas dentro de **atributos de clase** al principio de la definición de clase:

```
class Carta:
    listaDePalos = ["Tréboles", "Diamantes", "Corazones",
                   "Picas"]
    listaDeValores = ["nada", "As", "2", "3", "4", "5", "6", "7",
                     "8", "9", "10", "Sota", "Reina", "Rey"]

    # se omite el método init

    def __str__(self):
        return (self.listaDeValores[self.valor] + " de " +
                self.listaDePalos[self.palo])
```

Un atributo de clase se define fuera de cualquier método, y puede accederse desde cualquiera de los métodos de la clase.

Dentro de `__str__`, podemos usar `listaDePalos` y `listaDeValores` para asociar los valores numéricos de `palo` y `valor` con cadenas de caracteres. Por ejemplo, la expresión `self.listaDePalos[self.palo]` significa “usa el atributo `palo` del objeto `self` como un índice dentro del atributo de clase denominado `listaDePalos`, y selecciona la cadena apropiada”.

El motivo del “`nada`” en el primer elemento de `listaDeValores` es para relleno del elemento de posición cero en la lista, que nunca se usará. Los únicos valores lícitos para el valor van de 1 a 13. No es obligatorio que desperdiciemos este primer elemento. Podríamos haber comenzado en 0 como es usual, pero es menos confuso si el 2 se codifica como 2, el 3 como 3, y así sucesivamente.

Con los métodos que tenemos hasta ahora, podemos crear e imprimir naipes:

```
>>> carta1 = Carta(1, 11)
>>> print carta1
Sota de Diamantes
```

Los atributos de clase como `listaDePalos` son compartidos por todos los objetos de tipo `Carta`. La ventaja de esto es que podemos usar cualquier objeto `Carta` para acceder a los atributos de clase:

```
>>> carta2 = Carta(1, 3)
>>> print carta2
3 de Diamantes
>>> print carta2.listaDePalos[1]
Diamantes
```

La desventaja es que si modificamos un atributo de clase, afectaremos a cada instancia de la clase. Por ejemplo, si decidimos que “Sota de Diamantes” en realidad debería llamarse “Sota de Ballenas Bailarinas”, podríamos hacer lo siguiente:

```
>>> carta1.listaDePalos[1] = "Ballenas Bailarinas"
>>> print carta1
Sota de Ballenas Bailarinas
```

El problema es que *todos* los Diamantes se transformarán en Ballenas Bailarinas:

```
>>> print carta2
3 de Ballenas Bailarinas
```

En general no es una buena idea modificar los atributos de clase.

15.4. Comparación de naipes

Para los tipos primitivos, existen operadores condicionales (< , >, ==, etc.) que comparan valores y determinan cuando uno es mayor, menor, o igual a otro. Para los tipos definidos por el usuario, podemos sustituir el comportamiento de los operadores internos si proporcionamos un método llamado `__cmp__`. Por convención, `__cmp__` toma dos parámetros, `self` y `otro`, y retorna 1 si el primer objeto es el mayor, -1 si el segundo objeto es el mayor, y 0 si ambos son iguales.

Algunos tipos están completamente ordenados, lo que significa que se pueden comparar dos elementos cualesquiera y decir cuál es el mayor. Por ejemplo, los números enteros y los números en coma flotante tienen un orden completo. Algunos conjuntos no tienen orden, o sea, que no existe ninguna manera significativa de decir que un elemento es mayor a otro. Por ejemplo, las frutas no tienen orden, lo que explica por qué no se pueden comparar peras con manzanas.

El conjunto de los naipes tiene un orden parcial, lo que significa que algunas veces se pueden comparar los naipes, y otras veces no. Por ejemplo, usted sabe que el 3 de Tréboles es mayor que el 2 de Tréboles y el 3 de Diamantes es mayor que el 3 de Tréboles. Pero, ¿cuál es mejor?, ¿el 3 de Tréboles o el 2 de Diamantes?. Uno tiene mayor valor, pero el otro tiene mayor palo.

A los fines de hacer que los naipes sean comparables, se debe decidir qué es más importante: valor o palo. Para no mentir, la selección es arbitraria. Como algo hay que elegir, diremos que el palo es más importante, porque un mazo nuevo viene ordenado con todos los Tréboles primero, luego con todos los Diamantes, y así sucesivamente.

Con esa decisión tomada, podemos escribir `__cmp__`:

```
def __cmp__(self, otro):
    # controlar el palo
    if self.palo > otro.palo: return 1
    if self.palo < otro.palo: return -1
    # si son del mismo palo, controlar el valor
    if self.valor > otro.valor: return 1
    if self.valor < otro.valor: return -1
    # los valores son iguales, es un empate
    return 0
```

En este ordenamiento, los Ases son menores que los doses.

Como ejercicio, modifique `__cmp__` de tal manera que los Ases tengan mayor valor que los Reyes.

15.5. Mazos de naipes

Ahora que ya tenemos los objetos para representar las **Cartas**, el próximo paso lógico es definir una clase para representar un **Mazo**. Por supuesto, un mazo está compuesto de naipes, así que cada objeto **Mazo** contendrá una lista de naipes como atributo.

A continuación se muestra una definición para la clase **Mazo**. El método de inicialización crea el atributo `cartas` y genera el conjunto estándar de cincuenta y dos naipes.

```
class Mazo:
    def __init__(self):
        self.cartas = []
        for palo in range(4):
            for valor in range(1, 14):
                self.cartas.append(Carta(palo, valor))
```

La forma más fácil de poblar el mazo es mediante un bucle anidado. El bucle exterior enumera los palos desde 0 hasta 3. El bucle interior enumera los valores desde 1 hasta 13. Como el bucle exterior itera cuatro veces, y el interior

itera trece veces, la cantidad total de veces que se ejecuta el cuerpo interior es cincuenta y dos (trece por cuatro). Cada iteración crea una nueva instancia de **Carta** con el palo y valor actual, y agrega dicho naípe a la lista de **cartas**.

El método **append** funciona sobre listas pero no sobre tuplas, por supuesto.

15.6. Impresión del mazo de naipes

Como es usual, cuando definimos un nuevo tipo de objeto queremos un método que imprima el contenido del objeto. Para imprimir un **Mazo**, recorremos la lista e imprimimos cada **Carta**:

```
class Mazo:
    ...
    def muestraMazo(self):
        for carta in self.cartas:
            print carta
```

Desde ahora en adelante, los puntos suspensivos (...) indicarán que hemos omitido los otros métodos en la clase.

En lugar de escribir un método **muestraMazo**, podríamos escribir un método **__str__** para la clase **Mazo**. La ventaja de **__str__** está en que es más flexible. En lugar de imprimir directamente el contenido del objeto, **__str__** genera una representación en forma de cadena de caracteres que las otras partes del programa pueden manipular antes de imprimir o almacenar para un uso posterior.

Se presenta ahora una versión de **__str__** que retorna una representación como cadena de caracteres de un **Mazo**. Para darle un toque especial, acomoda los naipes en una cascada, de tal manera que cada naípe está sangrado un espacio más que el precedente.

```
class Mazo:
    ...
    def __str__(self):
        s = ""
        for i in range(len(self.cartas)):
            s = s + " " * i + str(self.cartas[i]) + "\n"
        return s
```

Este ejemplo demuestra varias características. Primero, en lugar de recorrer **self.cartas** y asignar cada naípe a una variable, usamos **i** como variable de bucle e índice de la lista de naipes.

Segundo, utilizamos el operador de multiplicación de cadenas de caracteres para sangrar cada naípe un espacio más que el anterior. La expresión ***i** proporciona una cantidad de espacios igual al valor actual de **i**.

Tercero, en lugar de usar la instrucción `print` para imprimir los naipes, utilizamos la función `str`. El pasar un objeto como argumento a `str` es equivalente a invocar el método `__str__` sobre dicho objeto.

Finalmente, usamos la variable `s` como **acumulador**. Inicialmente, `s` es una cadena de caracteres vacía. En cada pasada a través del bucle, se genera una nueva cadena de caracteres que se concatena con el viejo valor de `s` para obtener el nuevo valor. Cuando el bucle termina, `s` contiene la representación completa en formato de cadena de caracteres del Mazo, la cual se ve como a continuación se presenta:

```
>>> mazo = Mazo()
>>> print mazo
As de Tréboles
 2 de Tréboles
 3 de Tréboles
 4 de Tréboles
 5 de Tréboles
 6 de Tréboles
 7 de Tréboles
 8 de Tréboles
 9 de Tréboles
10 de Tréboles
 Sota de Tréboles
  Reina de Tréboles
   Rey de Tréboles
    As of Diamantes
```

Y así sucesivamente. Aún cuando los resultados aparecen en 52 renglones, se trata de sólo una única larga cadena de caracteres que contiene los saltos de línea.

15.7. Barajar el mazo

Si un mazo está perfectamente barajado, cualquier naipe tiene la misma probabilidad de aparecer en cualquier posición del mazo, y cualquier lugar en el mazo tiene la misma probabilidad de contener cualquier naipe.

Para mezclar el mazo, utilizaremos la función `randrange` del módulo `random`. Esta función toma dos enteros como argumentos `a` y `b`, y elige un número entero en

forma aleatoria en el rango $a \leq x < b$. Como el límite superior es estrictamente menor a b , podemos usar la longitud de la lista como el segundo argumento y de esa manera tendremos garantizado un índice legal dentro de la lista. Por ejemplo, esta expresión selecciona el índice de un naipe al azar dentro del mazo:

```
random.randrange(0, len(self.cartas))
```

Una manera sencilla de mezclar el mazo es recorrer los naipes e intercambiar cada una con otra elegida al azar. Es posible que el naipe se intercambie consigo mismo, pero no es un problema. De hecho, si eliminamos esa posibilidad, el orden de los naipes no será completamente al azar:

```
class Mazo:
    ...
    def mezclar(self):
        import random
        nCartas = len(self.cartas)
        for i in range(nCartas):
            j = random.randrange(i, nCartas)
            self.cartas[i], self.cartas[j] = \
                self.cartas[j], self.cartas[i]
```

En lugar de presuponer que hay cincuenta y dos naipes en el mazo, obtenemos la longitud real de la lista y la almacenamos en `nCartas`.

Para cada naipe del mazo, seleccionamos un naipe al azar entre aquellos que no han sido intercambiados aún. Luego intercambiamos el naipe actual (i) con el naipe seleccionado (j). Para intercambiar los naipes usaremos la asignación de tuplas, como se describe en la Sección 9.2:

```
self.cartas[i], self.cartas[j] = self.cartas[j], self.cartas[i]
```

Como ejercicio, reescriba esta línea de código sin usar una asignación de secuencias.

15.8. Eliminación y reparto de los naipes

Otro método que podría ser útil para la clase `Mazo` es `eliminaCarta`, que toma un naipe como parámetro, lo elimina, y retorna verdadero (1) si el naipe estaba en el mazo, y falso (0) si no estaba:

```
class Mazo:
    ...
    def eliminaCarta(self, carta):
        if carta in self.cartas:
            self.cartas.remove(carta)
            return 1
        else:
            return 0
```

El operador `in` retorna verdadero si el primer operando está en el segundo, el cual debe ser una lista o tupla. Si el primer operando es un objeto, Python usa el método `__cmp__` del objeto para determinar la igualdad entre los elementos de la lista. Como el `__cmp__` en la clase `Carta` verifica la igualdad en profundidad, el método `eliminaCarta` también verifica igualdad en profundidad.

Para repartir los naipes, queremos eliminar y devolver el naipe que ocupa la posición superior en el mazo. El método `pop` de las listas proporciona una manera conveniente de realizar esto:

```
class Mazo:
    ...
    def darCarta(self):
        return self.cartas.pop()
```

En realidad, `pop` elimina el *último* naipe en la lista, así que en efecto estamos repartiendo desde el extremo inferior del mazo.

Otra operación más que es muy probable necesitemos es la función booleana `estaVacio`, la cual devuelve verdadero si el mazo no contiene ningún naipe:

```
class Deck:
    ...
    def estaVacio(self):
        return (len(self.cartas) == 0)
```

15.9. Glosario

codificar: Representar un conjunto de valores utilizando otro conjunto de valores, entre los cuales se construye una correspondencia.

atributo de clase: Una variable que se define dentro de la definición de una clase pero fuera de cualquiera de sus métodos. Los atributos de clase son accesibles desde cualquier método de la clase y están compartidos por todas las instancias de la misma.

acumulador: Una variable que se usa en un bucle para acumular una serie de valores, por ejemplo concatenándolos dentro de una cadena de caracteres o adicionándolos a una suma.

Capítulo 16

Herencia

16.1. Herencia

La característica de un lenguaje que más se asocia con la programación orientada a objetos es la **herencia**. La herencia es la capacidad de definir una nueva clase que es una versión modificada de otra ya existente.

La principal ventaja de esta característica es que se pueden agregar nuevos métodos a una clase sin modificar la clase existente. Se denomina “herencia” porque la nueva clase hereda todos los métodos de la clase existente. Si extendemos esta metáfora, a la clase existente a veces se la denomina clase **padre**. La nueva clase puede denominarse clase **hija**, o también “subclase”.

La herencia es una característica poderosa. Ciertos programas que serían complicados sin herencia pueden escribirse de manera simple y concisa gracias a ella. Además, la herencia puede facilitar la reutilización del código, pues se puede adaptar el comportamiento de la clase padre sin tener que modificarla. En algunos casos, la estructura de la herencia refleja la propia estructura del problema, lo que hace que el programa sea más fácil de comprender.

Por otro lado, la herencia puede hacer que los programas sean difíciles de leer. Cuando se llama a un método, a veces no está claro dónde debe uno encontrar su definición. El código relevante puede estar diseminado por varios módulos. Además, muchas de las cosas que se hacen mediante el uso de la herencia, se pueden lograr de forma igualmente (incluso más) elegante sin ella. Si la estructura general del problema no nos guía hacia la herencia, dicho estilo de programación puede hacer más mal que bien.

En este capítulo demostraremos el uso de la herencia como parte de un programa que juega a las cartas a la “Mona”. Una de nuestras metas será que el código que escribamos se pueda reutilizar para implementar otros juegos de naipes.

16.2. Una mano de cartas

Para casi cualquier juego de naipes, necesitamos representar una mano de cartas. Una mano es similar a un mazo, por supuesto. Ambos están compuestos de un conjunto de naipes, y ambos requieren de operaciones tales como agregar y eliminar una carta. Además, necesitaremos la capacidad de mezclar tanto un mazo como una mano de cartas.

Una mano es diferente de un mazo en ciertos aspectos. Según el juego al que se esté jugando, podemos querer realizar ciertas operaciones sobre una mano que no tienen sentido sobre un mazo. Por ejemplo, en el póker queremos clasificar una mano (straight (consecutiva), flush (de un solo palo), etc.) y compararla con otra. En bridge necesitaremos calcular el puntaje para la mano para así poder hacer la subasta.

Esta situación sugiere el uso de la herencia. Si **Mano** es una subclase de **Mazo**, entonces tendrá todos los métodos de **Mazo** y le podremos agregar otros métodos nuevos.

En la definición de clase, el nombre de la clase padre aparece entre paréntesis:

```
class Mano(Mazo):  
    pass
```

Esta sentencia indica que la nueva clase **Mano** hereda de la clase existente **Mazo**.

El constructor de **Mano** inicializa los atributos para la mano, que son **nombre** y **cartas**. La cadena de caracteres **nombre** identifica a esta mano, probablemente mediante el nombre del jugador que la sostiene. El nombre es un parámetro opcional con un valor por omisión de cadena vacía. **cartas** es la lista de cartas de la mano, inicializada como lista vacía.

```
class Mano(Mazo):  
    def __init__(self, nombre=""):  
        self.cartas = []  
        self.nombre = nombre
```

Casi para cualquier juego de naipes, es necesario agregar y quitar cartas del mazo. La eliminación de cartas ya ha sido resuelta, pues **Mano** hereda **eliminaCarta** de **Mazo**. Pero deberemos escribir **agregaCarta**:

```
class Mano(Mazo):
    ...
    def agregaCarta(self, carta) :
        self.cartas.append(carta)
```

De nuevo, los puntos suspensivos indican que hemos omitido los otros métodos. El método de lista `append` agrega la nueva carta al final de la lista de cartas.

16.3. El reparto de los naipes

Ahora que ya tenemos la clase `Mano`, queremos repartir las cartas del `Mazo` en manos. No es claramente obvio si este método debe ir en la clase `Mano` o en la clase `Mazo`, pero como opera sobre un mazo único y (posiblemente) sobre varias manos, es más natural ponerlo en el `Mazo`.

`repartir` debe ser bastante general, pues los diferentes juegos tienen distintos requerimientos. Puede que necesitemos repartir todo el mazo de una vez, o que agreguemos una carta a cada mano.

`repartir` toma dos parámetros, una lista (o tupla) de manos y la cantidad total de naipes a repartir. Si no hay suficientes cartas en el mazo, el método reparte todas las cartas y se detiene:

```
class Mazo :
    ...
    def repartir(self, manos, nCartas=999):
        nManos = len(manos)
        for i in range(nCartas):
            if self.estaVacio(): break      # fin si se acaban las cartas
            carta = self.darCarta()         # da la carta superior
            mano = manos[i % nManos]       # a quién le toca?
            mano.agregaCarta(carta)         # agrega la carta a la mano
```

El segundo parámetro, `nCartas` es opcional; el valor por omisión es un número muy grande, lo cual es lo mismo que decir que se repartirán todos los naipes del mazo.

La variable de bucle `i` va desde 0 hasta `nCartas-1`. A cada paso a través del bucle, se elimina una carta del mazo mediante el método de lista `pop`, que quita y devuelve el último elemento de la lista.

El operador módulo (%) permite que podamos repartir las cartas de una en una (una carta cada vez para cada mano). Cuando `i` es igual a la cantidad de manos en la lista, la expresión `i % nManos` salta hacia el comienzo de la lista (el índice es 0).

16.4. Mostremos la mano

Para mostrar el contenido de una mano, podemos sacar partido de la existencia de los métodos `muestraMazo` y `__str__` que se heredan de `Mazo`. Por ejemplo:

```
>>> mazo = Mazo()
>>> mazo.mezclar()
>>> mano = Mano("hugo")
>>> mazo.repartir([mano], 5)
>>> print mano
La mano de hugo contiene
2 de Picas
3 de Picas
4 de Picas
As de Corazones
9 de Tréboles
```

No es una gran mano, pero tiene lo necesario como para disponer de una escalera de color.

Aunque es conveniente usar la herencia de los métodos existentes, existe información adicional en una `Mano` que desearíamos mostrar al imprimirla. Para ello, podemos proporcionar a la clase `Mano` un método `__str__` que reemplace al de la clase `Mazo`:

```
class Mano(Mazo)
...
def __str__(self):
    s = "La mano de " + self.nombre
    if self.estaVacio():
        s = s + " está vacía\n"
    else:
        s = s + " contiene\n"
    return s + Mazo.__str__(self)
```

Al principio `s` es una cadena de caracteres que identifica a la mano. Si la mano está vacía, el programa agrega las palabras `está vacía` y devuelve `s`.

En caso contrario, el programa agrega la palabra `contiene` y la representación como cadena de caracteres del `Mazo`, que se obtiene llamando al método `__str__` de la clase `Mazo` sobre la instancia `self`.

Puede parecer extraño que enviemos a `self`, que se refiere a la `Mano` actual, como argumento de un método de la clase `Mazo`, hasta que nos damos cuenta de que una `Mano` es un tipo de `Mazo`. Los objetos `Mano` pueden hacer cualquier

cosa que pueda hacer un objeto **Mazo**, y por ello es legal que pasemos una **Mano** a un método de **Mazo**.

En general, siempre es legal usar una instancia de una subclase en el lugar de una instancia de una clase padre.

16.5. La clase JuegoDeCartas

La clase **JuegoDeCartas** asume la responsabilidad sobre algunas obligaciones básicas comunes a todos los juegos, tales como la creación del mazo y la mezcla de los naipes:

```
class JuegoDeCartas:
    def __init__(self):
        self.mazo = Mazo()
        self.mazo.mezclar()
```

Esta es la primera vez que vemos que un método de inicialización realiza una actividad computacional significativa, más allá de la inicialización de atributos.

Para implementar juegos específicos, debemos heredar de **JuegoDeCartas** y agregar las características del nuevo juego. Como ejemplo, escribiremos una simulación para La Mona.

La meta de La Mona es desembarazarse de las cartas que uno tiene en la mano. Uno se saca las cartas de encima emparejándolas por valor y color. Por ejemplo, el 4 de Tréboles se empareja con el 4 de Picas porque ambos palos son negros. La Sota de Corazones se empareja con la Sota de Diamantes porque ambos son rojos.

Para iniciar el juego, se elimina la Reina de Tréboles del mazo, de manera que la Reina de Picas no tiene con quién emparejarse. Las cincuenta y una cartas restantes se reparten entre los jugadores, de una en una. Luego del reparto, todos los jugadores emparejan y descartan tantas cartas como sea posible.

Cuando no se pueden realizar más concordancias, el juego comienza. Por turnos, cada jugador toma una carta (sin mirarla) del vecino más cercano de la izquierda que aún tiene cartas. Si la carta elegida concuerda con una de la mano del jugador, se elimina dicho par. Si no, la carta se agrega a la mano del jugador. Llega el momento en el que se realizan todas las concordancias posibles, con lo que queda sólo la Reina de Picas en la mano del perdedor.

En nuestra simulación informática del juego, la computadora juega todas las manos. Desafortunadamente, se pierden algunos de los matices del juego real. En una partida real, el jugador que tiene la Mona realiza ciertos esfuerzos para

que su vecino la tome, por ejemplo mostrándola prominentemente o al contrario, errando al intentar mostrarla abiertamente, o incluso puede fallar al tratar de errar en su intento de mostrarla prominentemente. La computadora simplemente toma una carta al azar de su vecino.

16.6. La clase ManoDeLaMona

Una mano para jugar a La Mona requiere ciertas capacidades que están más allá de las que posee una `Mano`. Definiremos una nueva clase `ManoDeLaMona`, que hereda de `Mano` y nos proporciona un método adicional denominado `eliminaCoincidencias`:

```
class ManoDeLaMona(Mano):
    def eliminaCoincidencias(self):
        cant = 0
        cartasOriginales = self.cartas[:]
        for carta in cartasOriginales:
            empareja = Carta(3 - carta.palo, carta.valor)
            if empareja in self.cartas:
                self.cartas.remove(carta)
                self.cartas.remove(empareja)
                print "Mano %s: %s con %s" % (self.nombre, carta, empareja)
                cant = cant + 1
        return cant
```

Comenzamos por hacer una copia de la lista de las cartas, de tal manera que podamos recorrer la copia mientras vamos quitando cartas de la lista original. Como `self.cartas` se modifica en el bucle, no vamos a querer usarla para controlar el recorrido. ¡Python puede quedar realmente confundido si se recorre una lista que está cambiando!

Para cada carta de la mano, averiguamos cuál es la carta que concordará con ella y la buscamos. La carta que concuerda tiene el mismo valor y el otro palo del mismo color. La expresión `3 - carta.palo` transforma un Trébol (palo 0) en una Pica (palo 3) y un Diamante (palo 1) en un Corazón (palo 2). Verifique por su cuenta que las operaciones opuestas también funcionan. Si la carta que concuerda está en la mano, ambas se eliminan.

El siguiente ejemplo demuestra el uso de `eliminaCoincidencias`:

```
>>> juego = JuegoDeCartas()
>>> mano = ManoDeLaMona("hugo")
>>> juego.mazo.repartir([mano], 13)
```

```
>>> print mano
La mano de hugo contiene
As de Picas
2 de Diamantes
7 de Picas
8 de Tréboles
6 de Corazones
8 de Picas
7 de Tréboles
Raina de Tréboles
7 de Diamantes
5 de Tréboles
Sota de Diamantes
10 de Diamantes
10 de Corazones

>>> mano.eliminaCoincidencias()
Mano hugo: 7 de Picas con 7 de Tréboles
Mano hugo: 8 de Picas con 8 de Tréboles
Mano hugo: 10 de Diamantes con 10 de Corazones
```

Debe usted notar que no existe un método `__init__` para la clase `ManoDeLaMona`. Lo heredamos de `Mano`.

16.7. La clase JuegoDeLaMona

Ahora podemos poner nuestra atención en el juego en sí mismo. `JuegoDeLaMona` es una subclase de `JuegoDeCartas` con un método nuevo denominado `jugar` que toma una lista de jugadores como parámetro.

Como el método `__init__` se hereda de `JuegoDeCartas`, el nuevo objeto `JuegoDeLaMona` contiene un mazo recientemente mezclado:

```
class JuegoDeLaMona(JuegoDeCartas):
    def jugar(self, nombres):
        # quitamos la Reina de Tréboles
        self.mazo.eliminaCarta(Carta(0,12))

        # construimos una mano para cada jugador
        self.manos = []
        for nombre in nombres :
            self.manos.append(ManoDeLaMona(nombre))
```

```

# repartimos los naipes
self.mazo.repartir(self.manos)
print "----- Se han repartido las cartas."
self.muestraManos()

# eliminamos las coincidencias iniciales
emparejadas = self.eliminaTodasLasCoincidencias()
print "----- Coincidencias eliminadas, el juego comienza."
self.muestraManos()

# se juega hasta que se han descartado las 50 cartas
turno = 0
cantManos = len(self.manos)
while emparejadas < 25:
    emparejadas = emparejadas + self.jugarUnTurno(turno)
    turno = (turno + 1) % cantManos

print "----- El juego terminó."
self.muestraManos()

```

Algunos de los pasos que componen el juego se han colocado en métodos separados. `eliminaTodasLasCoincidencias` recorre la lista de manos y llama a `eliminaCoincidencias` para cada una de ellas:

```

class JuegoDeLaMona(JuegoDeCartas):
    ...
    def eliminaTodasLasCoincidencias(self):
        cant = 0
        for mano in self.manos:
            cant = cant + mano.eliminaCoincidencias()
        return cant

```

Como ejercicio, escriba `muestraManos`, el cual recorre `self.manos` y muestra cada mano.

`cant` es un acumulador que va sumando la cantidad de concordancias en cada mano y devuelve el total.

Cuando la cantidad total de coincidencias alcanza a las veinticinco significa que se han eliminado cincuenta cartas de las manos, lo que es lo mismo que decir que sólo queda una carta y el juego ha terminado.

La variable `turno` recuerda el turno de cuál jugador se está jugando. Comienza en cero y se incrementa en uno cada vez; cuando alcanza el valor `cantManos`, el operador de módulo lo hace volver a cero.

El método `jugarUnTurno` toma un parámetro que indica de quién es el turno. El valor de retorno es la cantidad de concordancias que se han realizado durante ese turno:

```
class JuegoDeLaMona(JuegoDeCartas):
    ...
    def jugarUnTurno(self, i):
        if self.manos[i].estaVacio():
            return 0
        vecino = self.encuentraVecino(i)
        cartaElegida = self.manos[vecino].darCarta()
        self.manos[i].agregaCarta(cartaElegida)
        print "Mano", self.manos[i].nombre, "eligió", cartaElegida
        cant = self.manos[i].eliminaCoincidencias()
        self.manos[i].mezclar()
        return cant
```

Si la mano de un jugador está vacía, el jugador salió del juego, así que no hace nada y devuelve 0.

Si no, un turno consiste en encontrar el primer jugador a la izquierda que aún tiene cartas, tomar una carta de las que posee, y controlar si hay concordancias. Antes de volver se mezclan las cartas de la mano, de tal manera que la selección del siguiente jugador sea al azar.

El método `encuentraVecino` comienza con el jugador que está inmediatamente a la izquierda y continúa alrededor del círculo hasta que encuentra un jugador que aún tiene cartas.

```
class JuegoDeLaMona(JuegoDeCartas):
    ...
    def encuentraVecino(self, i):
        cantManos = len(self.manos)
        for proximo in range(1, cantManos):
            vecino = (i + proximo) % cantManos
            if not self.manos[vecino].estaVacio():
                return vecino
```

Si por cualquier motivo `encuentraVecino` llegara a dar la vuelta completa al círculo sin encontrar cartas, devolvería `None` y eso causaría un error en alguna otra parte del programa. Afortunadamente podemos probar que eso no va a suceder nunca (siempre y cuando se detecte correctamente el final del juego).

Hemos omitido el método `muestraManos`. Ése puede escribirlo usted mismo.

La siguiente salida proviene de una forma reducida del juego, en la cual solamente se reparten las quince cartas más altas (desde los dieces hacia arriba) a tres

jugadores. Con este mazo más pequeño, el juego termina tras siete coincidencias, en lugar de veinticinco.

```
>>> import cartas
>>> juego = cartas.JuegoDeLaMona()
>>> juego.jugar(["Allen","Jeff","Chris"])
----- Se han repartido las cartas.
Mano Allen contiene
Rey de Corazones
Sota de Tréboles
Reina de Picas
Rey de Picas
10 de Diamantes

Mano Jeff contiene
Reina de Corazones
Sota de Picas
Sota de Corazones
Rey de Diamantes
Reina de Diamantes

Mano Chris contiene
Sota de Diamantes
Rey de Tréboles
10 de Picas
10 de Corazones
10 de Tréboles

Mano Jeff: Reina de Corazones con Reina de Diamantes
Mano Chris: 10 de Picas con 10 de Tréboles
----- Se eliminaron las coincidencias, el juego comienza.
Mano Allen contiene
Rey de Corazones
Sota de Tréboles
Reina de Picas
Rey de Picas
10 de Diamantes

Mano Jeff contiene
Sota de Picas
Sota de Corazones
Rey de Diamantes
```

Mano Chris contiene
Sota de Diamantes
Rey de Tréboles
10 de Corazones

Mano Allen: eligió Rey de Diamantes
Mano Allen: Rey de Corazones con Rey de Diamantes
Mano Jeff: eligió 10 de Corazones
Mano Chris: eligió Sota de Tréboles
Mano Allen: eligió Sota de Corazones
Mano Jeff: eligió Sota de Diamantes
Mano Chris: eligió Reina de Picas
Mano Allen: eligió Sota de Diamantes
Mano Allen: Sota de Corazones con Sota de Diamantes
Mano Jeff: eligió Rey de Tréboles
Mano Chris: eligió Rey de Picas
Mano Allen: eligió 10 de Corazones
Mano Allen: 10 de Diamantes con 10 de Corazones
Mano Jeff: eligió Reina de Picas
Mano Chris: eligió Sota de Picas
Mano Chris: Sota de Tréboles con Sota de Picas
Mano Jeff: eligió Rey de Picas
Mano Jeff: Rey de Tréboles con Rey de Picas
----- El juego terminó.

La mano de Allen está vacía.

La mano de Jeff contiene
Reina de Picas

La mano de Chris está vacía.

Así que Jeff es quien perdió.

16.8. Glosario

herencia: La capacidad de definir una nueva clase que es una versión modificada de una clase previamente definida.

clase padre: Aquella clase de la cual la clase hija hereda.

clase hija: Una nueva clase creada heredando de una clase existente; también se la llama “subclase”.

Capítulo 17

Listas enlazadas

17.1. Referencias incrustadas

Hemos visto ejemplos de atributos que hacen referencia a otros objetos, a los que llamamos **referencias incrustadas** (véase la Sección 12.8). Una estructura de datos común, la **lista enlazada**, saca partido de esta característica.

Las listas enlazadas se componen de **nodos**, donde cada nodo contiene una referencia al próximo nodo de la lista. Además, cada nodo contiene una unidad de datos llamada **carga**

Podemos considerar una lista enlazada como una **estructura de datos recursiva** porque tiene una definición recursiva.

Una lista enlazada puede ser:

- la lista vacía, representada por `None`, o bien
- un nodo que contiene un objeto de carga y una referencia a una lista enlazada.

Las estructuras recursivas de datos nos llevan a métodos recursivos.

17.2. La clase Nodo

Como es habitual cuando se escribe una clase, comenzaremos con los métodos de inicialización y `__str__`, para poder comprobar el mecanismo básico de crear y mostrar el nuevo tipo:

```
class Nodo:
    def __init__(self, carga=None, siguiente=None):
        self.carga = carga
        self.siguiente = siguiente

    def __str__(self):
        return str(self.carga)
```

Como es habitual, los parámetros para el método de inicialización son opcionales. Por defecto, la carga y el enlace, **siguiente**, se ponen a **None**.

La representación alfanumérica de un nodo es únicamente la de la carga. Como se puede pasar cualquier valor a la función **str**, podemos guardar cualquier valor en una lista.

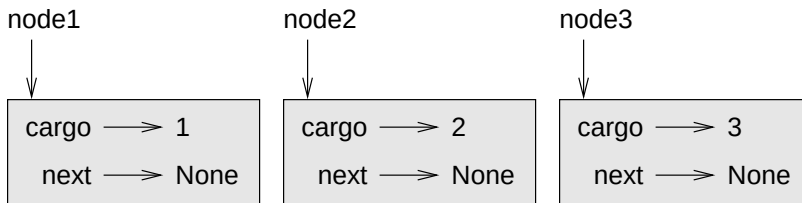
Para comprobar la implementación en este punto, podemos crear un **Nodo** e imprimirlo:

```
>>> nodo = Nodo("prueba")
>>> print nodo
prueba
```

Para hacerlo más interesante, necesitaremos una lista que contenga más de un nodo:

```
>>> nodo1 = Nodo(1)
>>> nodo2 = Nodo(2)
>>> nodo3 = Nodo(3)
```

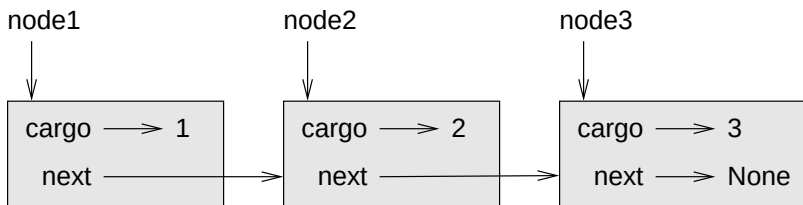
Este código crea tres nodos, pero aún no tenemos una lista porque los nodos todavía no están **enlazados**. El diagrama de estados tiene el siguiente aspecto:



Para enlazar los nodos, debemos hacer que el primer nodo haga referencia al segundo, y que éste haga referencia al tercero:

```
>>> nodo1.siguiente = nodo2
>>> nodo2.siguiente = nodo3
```

La referencia del tercer nodo será **None**, que indica que es el final de la lista. Ahora el diagrama de estados tendrá el siguiente aspecto:



Ahora ya sabe cómo crear nodos y enlazarlos en listas. Lo que podría estar menos claro es por qué.

17.3. Listas como colecciones

Las listas son útiles porque aportan un modo de ensamblar múltiples objetos dentro de una única entidad, a veces llamada **colección**. En el ejemplo, el primer nodo de la lista sirve como referencia a la lista completa.

Para pasar la lista como parámetro, sólo tenemos que hacer referencia al primer nodo. Por ejemplo, la función `imprimeLista` toma como argumento un nodo simple. Empezando con la cabeza de la lista, imprime cada nodo hasta que llega al final.

```
def imprimeLista(nodo):
    while nodo:
        print nodo,
        nodo = nodo.siguiente
    print
```

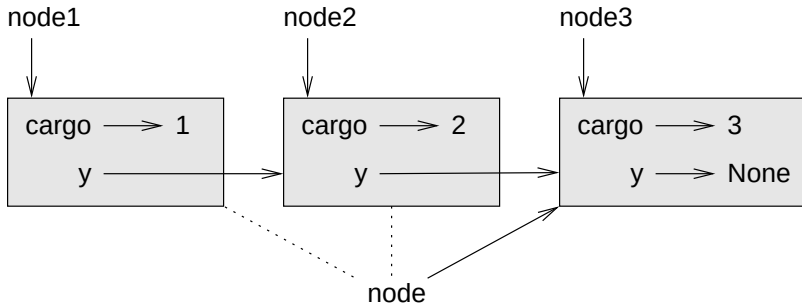
Para llamar a este método, pasamos una referencia al primer nodo:

```
>>> imprimeLista(nodo1)
1 2 3
```

Dentro de `imprimeLista` tenemos una referencia al primer nodo de la lista, pero no hay una variable que haga referencia a los otros nodos. Tendremos que usar el valor de `siguiente` de cada nodo para acceder al siguiente nodo.

Para recorrer una lista enlazada es habitual usar la variable de un bucle como `nodo` para hacer referencia sucesivamente a cada uno de los nodos.

Este diagrama muestra el valor de `lista` y los valores que va tomando `nodo`:



Por convenio, las listas suelen imprimirse entre corchetes con comas entre los elementos, como `[1, 2, 3]`. Como ejercicio, modifique `imprimeLista` para que genere una salida con este formato.

17.4. Listas y recursividad

Es natural expresar muchas operaciones con listas por medio de métodos recursivos. El siguiente ejemplo es un algoritmo recursivo para imprimir una lista hacia atrás:

1. Separar la lista en dos partes: el primer nodo (llamado cabeza) y el resto (llamado cola).
2. Imprimir la cola hacia atrás.
3. Imprimir la cabeza.

Por supuesto, el paso 2, la llamada recursiva, supone que tenemos una manera de imprimir una lista del revés. Pero si suponemos que la llamada recursiva funciona —el acto de fe— entonces podemos estar convencidos de que este algoritmo funciona.

Todo lo que necesitamos es un caso básico y una forma de demostrar que para cualquier lista podemos obtener el caso básico. Dada la definición recursiva de una lista, un caso básico natural es la lista vacía, representada por `None`:

```
def imprimeAlReves(lista):
    if lista == None: return
    cabeza = lista
    cola = lista.siguiente
    imprimeAlReves(cola)
    print cabeza,
```

La primera línea maneja el caso inicial no haciendo nada. Las dos siguientes líneas dividen la lista en **cabeza** y **cola**. Las dos últimas líneas imprimen la lista. La coma que hay al final de la última línea evita que Python salte de línea después de imprimir un nodo.

Llamamos este método tal como antes invocamos a `imprimeLista`:

```
>>> imprimeAlReves(nodo1)
3 2 1
```

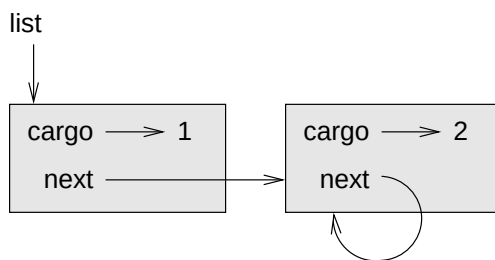
El resultado es una lista invertida.

Es posible que se pregunte por qué `imprimeLista` e `imprimeAlReves` son funciones y no métodos de la clase `Nodo`. La razón es que queremos usar `None` para representar la lista vacía y no es legal llamar un método sobre `None`. Esta limitación resulta algo incómoda a la hora de escribir código para manipular listas con un estilo orientado a objetos puro.

¿Podemos demostrar que `imprimeAlReves` siempre acabará? En otras palabras, ¿siempre alcanzaremos el caso básico? De hecho, la respuesta es no. Algunas listas harán que el método no funcione.

17.5. Listas infinitas

No hay nada que impida a un nodo hacer referencia a un nodo anterior de la lista, incluido él mismo. Por ejemplo, esta figura muestra una lista con dos nodos, uno de los cuales apunta a sí mismo:



Si llamamos a `imprimeLista` sobre esta lista, entrará en un bucle infinito. Si llamamos a `imprimeAlReves`, lo hará de forma infinitamente recursiva. Este tipo de comportamiento da lugar a que sea muy difícil trabajar con listas infinitas.

Sin embargo, en ocasiones resultan útiles. Por ejemplo, podríamos representar un número como una lista de dígitos y usar una lista infinita para representar una fracción repetida.

A pesar de todo, es un problema que no podamos probar que `imprimeLista` e `imprimeAlReves` puedan terminar correctamente. Lo mejor que podemos hacer es afirmar que “si la lista no contiene bucles, estos métodos podrán terminar”. Este tipo de afirmaciones se conocen como **condiciones previas**. Imponen una restricción sobre uno de los parámetros y describen el comportamiento del método si la restricción se satisface. Veremos más ejemplos más adelante.

17.6. Teorema fundamental de la ambigüedad

Una parte de `imprimeAlReves` podría habernos sorprendido:

```
cabeza = lista
cola = lista.siguiente
```

Después de la primera asignación, la **cabeza** y la **cola** tienen el mismo tipo y el mismo valor. Así que, ¿para qué hemos creado una nueva variable?

La razón es que las dos variables desempeñan papeles diferentes. Pensamos en la **cabeza** como una referencia al primer nodo de la lista. Estos “papeles” no forman parte del programa, sino que están en la mente del programador.

En general no podemos decir con sólo mirar un programa qué papel desempeñará una variable. Esta ambigüedad puede ser útil, pero también puede dificultar la lectura del programa. A menudo usaremos nombres para las variables como **nodo** y **lista** para explicar cómo queremos usar una variable y a veces creamos variables adicionales para eliminar ambigüedades.

Podríamos haber escrito `imprimeAlReves` sin **cabeza** ni **cola**, que lo haría más conciso, pero posiblemente menos claro:

```
def imprimeAlReves(lista) :
    if lista == None : return
    imprimeAlReves(lista.siguiente)
    print lista,
```

Mirando esas dos llamadas, hemos de recordar que `imprimeAlReves` trata sus argumentos como una colección y `print` los trata como a un sólo objeto.

El **teorema fundamental de la ambigüedad** indica que la ambigüedad que es inherente a una referencia a un nodo:

Una variable que hace apunta a nodo puede tratar a este nodo como un objeto o como el primero de una lista de nodos.

17.7. Modificar listas

Hay dos formas de modificar una lista enlazada. Obviamente, podemos cambiar la carga de uno de los nodos, pero las operaciones más interesantes son las que añaden, quitan o reordenan los nodos.

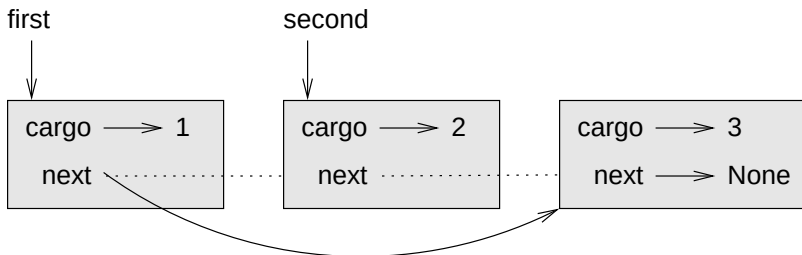
Como ejemplo, escribamos un método que quite el segundo nodo de la lista y devuelva una referencia al nodo quitado:

```
def eliminaSegundo(lista):  
    if lista == None: return  
    primero = lista  
    segundo = lista.siguiente  
    # hacer que el primer nodo apunte al tercero  
    primero.siguiente = segundo.siguiente  
    # separar el segundo nodo del resto de la lista  
    segundo.siguiente = None  
    return segundo
```

De nuevo, estamos usando variables temporales para hacer más legible el código. Así es como usamos este método:

```
>>> imprimeLista(nodo1)  
1 2 3  
>>> eliminado = eliminaSegundo(nodo1)  
>>> imprimeLista(eliminado)  
2  
>>> imprimeLista(nodo1)  
1 3
```

El diagrama de estado nos muestra el efecto de la operación:



¿Qué ocurriría si llamáramos a este método y pasáramos una lista de un único elemento (un **singleton**)? ¿Qué sucedería si pasáramos una lista vacía como argumento? ¿Hay una condición previa para este método? Si es así, sería algo razonable establecer un método para manejar una violación de la condición previa.

17.8. Envoltorios y ayudantes

A menudo es útil dividir una operación de listas en dos métodos. Por ejemplo, para imprimir una lista invertida en el formato convencional [3, 2, 1] podemos usar el método `imprimeAlReves` para imprimir 3, 2, pero necesitaremos un método aparte para imprimir los corchetes y el primer nodo. Llamémoslo `imprimeAlRevesBonito`:

```
def imprimeAlRevesBonito(lista) :
    print "[",
    if lista != None :
        cabeza = lista
        cola = lista.siguiente
        imprimeAlReves(cola)
    print cabeza,
    print "]",
```

De nuevo, vemos que es buena idea comprobar métodos como éste para ver si funcionan con casos especiales como una lista vacía o un singleton.

Cuando usamos este método en algún otro lugar del programa, llamamos directamente a `imprimeAlRevesBonito`, y éste llama a `imprimeAlReves` en nuestro lugar. En cierto modo, `imprimeAlRevesBonito` actúa como un **envoltorio**, y utiliza a `imprimeAlReves` como su **ayudante**.

17.9. La clase ListaEnlazada

Existen ciertos problemas delicados con la forma en que se implementaron las listas. Inviertiendo el orden de causa y efecto, propondremos primero una implementación alternativa y explicaremos luego los problemas que ésta resuelve.

En primer lugar crearemos una nueva clase llamada `ListaEnlazada`. Sus atributos son un entero que contiene la longitud de la lista y una referencia al primer nodo. Los objetos `ListaEnlazada` sirven de asas para la manipulación de las listas de los objetos de tipo `Nodo`:

```
class ListaEnlazada :
    def __init__(self) :
        self.longitud = 0
        self.cabeza = None
```

Una ventaja de la clase `ListaEnlazada` es que suministra un marco natural para poner funciones-envoltorio como `imprimeAlRevesBonito`, que podemos convertir en un método de la clase `ListaEnlazada`:


```
class ListaEnlazada:
    ...
    def imprimeAlReves(self):
        print "[",
        if self.cabeza != None:
            self.cabeza.imprimeAlReves()
        print "]",

class Nodo:
    ...
    def imprimeAlReves(self):
        if self.siguiente != None:
            cola = self.siguiente
            cola.imprimeAlReves()
        print self.carga,
```

Para complicar un poco más las cosas, renombramos `imprimeAlRevesBonito`. Ahora hay dos métodos llamados `imprimeAlReves`: uno en la clase `Nodo` (el ayudante), y otro en la clase `ListaEnlazada` (el envoltorio). Cuando el envoltorio llama a `self.cabeza.imprimeAlReves`, está llamando al ayudante, ya que `self.cabeza` es un objeto de tipo `Nodo`.

Otra ventaja de la clase `ListaEnlazada` es que facilita la forma de añadir o quitar el primer elemento de una lista. Por ejemplo, `agregaPrimero` es un método para `ListaEnlazada`; toma un elemento de la carga como argumento y lo coloca en el principio de la lista:

```
class ListaEnlazada:
    ...
    def agregaPrimero(self, carga):
        nodo = Nodo(carga)
        nodo.siguiente = self.cabeza
        self.cabeza = nodo
        self.longitud = self.longitud + 1
```

Como suele ser usual, deberíamos comprobar éste código para ver si maneja casos especiales. Por ejemplo, ¿qué ocurriría si la lista está unicialmente vacía?

17.10. Invariantes

Algunas listas están “bien construidas”; otras no. Por ejemplo, si una lista contiene un bucle, provocará que nuestros métodos se cuelguen, así que podríamos exigir que las listas no contengan bucles. Otro requisito es que el valor de el valor

de `longitud` en el objeto de tipo `ListaEnlazada` debería ser igual al número verdadero de nodos de la lista.

A este tipo de requisitos los llamaremos **invariantes** porque, idealmente deberían cumplirse en todos los objetos en todo momento. Especificar invariantes para objetos es una práctica útil de la programación porque hace más fácil demostrar la idoneidad del código, comprobar la integridad de las estructuras de datos y la detección de errores.

Una cosa que a veces confunde respecto a los invariantes es que en ocasiones son violados. Por ejemplo, en medio de `agregaPrimero`, después de añadir el nodo pero antes de incrementar la `longitud`, se viola el invariante. Se acepta este tipo de violación; de hecho, a menudo es imposible modificar un objeto sin violar un invariante durante al menos un corto espacio de tiempo. Normalmente, se exige que todo método que viole un invariante debe restablecerlo.

Si hay algún tramo significativo de código en el que el invariante se ve violado, es importante que los comentarios lo dejen claro, de modo que no se realicen operaciones que dependan del invariante.

17.11. Glosario

referencia incrustada: Es una referencia almacenada en un atributo de un objeto.

lista enlazada: Estructura de datos que implementa una colección por medio de una secuencia de nodos enlazados.

nodo: Elemento de una lista, normalmente implementado como un objeto que contiene una referencia a otro objeto del mismo tipo.

carga: Datos contenidos en un nodo.

enlace: Referencia incrustada usada para enlazar un objeto con otro.

condición previa: Afirmación que debe ser cierta para que un método funcione correctamente.

teorema fundamental de la ambigüedad: Una referencia a un nodo de una lista puede tratarse como un objeto individual o como el primero de una lista de nodos.

singleton: Lista enlazada con un solo nodo.

envoltorio: Método que actúa como mediador entre un método invocador y método ayudante, haciendo a menudo su invocación más fácil o menos proclive a errores.

ayudante: Método al que no se invoca directamente por un método llamante sino por otro método para formar parte de una operación.

invariante: Afirmación que debería ser cierta para un objeto en todo momento (excepto tal vez cuando se está modificando el objeto).

Capítulo 18

Pilas

18.1. Tipos abstractos de datos

Los tipos de datos vistos hasta ahora han sido todos concretos, en el sentido que se ha especificado su implementación completamente. Por ejemplo, la clase `Carta` representa un naípe por medio de dos enteros. Como dijimos, esa no es la única manera de representar una carta; existen muchas implementaciones alternativas.

Un **tipo abstracto de datos**, o TAD, especifica un conjunto de operaciones (o métodos) y la semántica de las operaciones (lo que hacen), pero no especifica la implementación de las operaciones. Esto es lo que hace lo abstracto.

¿Para qué sirve?

- Simplifica la tarea de especificar un algoritmo si se pueden indicar las operaciones necesarias sin preocuparse de cómo se ejecutarán dichas operaciones.
- Como suelen existir muchas maneras de implementar un TAD, puede ser útil desarrollar un algoritmo que se pueda usar con cualquiera de las implementaciones posibles.
- Los TADs más conocidos, como el TAD Pila de este capítulo, se implementan a menudo en bibliotecas estándares de manera que se puedan escribir una vez y usarlas luego muchos programadores.
- Las operaciones ejecutadas con TADs proporcionan un lenguaje de alto nivel común para desarrollar y hablar sobre algoritmos.

Cuando hablamos de TADs a menudo se hace la distinción entre el código que usa el TAD, el código **cliente**, y el código que implementa el TAD, el código **proveedor**.

18.2. El TAD Pila

En este capítulo se presentará un TAD común, la **pila**. Una pila es una colección, lo que significa que es una estructura de datos que contiene elementos múltiples. Otras colecciones que se han visto son los diccionarios y las listas.

Un TAD se define por medio de las operaciones que se pueden ejecutar sobre él, lo que se llama un **interfaz**. La interfaz para una pila consta de estas operaciones¹:

__init__: Inicializar una pila nueva y vacía.

push: Añadir un elemento a la pila.

pop: Extraer un elemento de la pila. El elemento devuelto siempre es el último que se añadió.

isEmpty: Probar si la pila está vacía.

A veces a una pila se la llama una estructura “último en entrar primero en salir” (“last in, first out” en inglés), o LIFO, porque el elemento añadido en último lugar es el primero que extraemos.

18.3. Cómo implementar pilas con listas de Python

Las operaciones sobre listas que posee Python son parecidas a las operaciones que definen a una pila. La interfaz no es exactamente la que debería de ser, pero se pueden desarrollar programas para convertir el TAD Pila a las operaciones predefinidas.

A este programa se lo llama **implementación** del TAD Pila. En general, una implementación es un conjunto de métodos que satisfacen los prerequisites sintácticos y semánticos de la interfaz.

He aquí una implementación de el TAD Pila que utiliza una lista Python:

¹Mantenemos los nombres ingleses porque son estándar en otros TADs en Python y otros lenguajes.

```
class Pila :
    def __init__(self) :
        self.elementos = []

    def push(self, elemento) :
        self.elementos.append(elemento)

    def pop(self) :
        return self.elementos.pop()

    def isEmpty(self) :
        return (self.elementos == [])
```

Un objeto `Pila` contiene un atributo llamado `elementos` que es una lista de elementos en la pila. El método de inicialización pone una lista vacía en `elementos`.

Para meter un elemento nuevo en la pila, `push` lo apila en `elementos`. Para quitar un elemento de la pila, `pop` utiliza el método de lista homónimo² para quitar y devolver el último elemento de la lista.

Finalmente, para probar si la pila esta vacía, `isEmpty` (está vacía) compara `elementos` con la lista vacía.

Una implementación como esta, en la cual los métodos consisten de llamadas a métodos existentes, se llama **enchapado**. En la vida real, un enchapado es una capa fina de madera de alta calidad que se utiliza en la fabricación de muebles para ocultar madera de baja calidad. Los científicos informáticos utilizan esta metáfora para describir una parte de un programa que esconde los detalles de una implementación y que provee una interfaz mas simple o mas estándar.

18.4. Uso de push y pop

Una pila es una **estructura genérica de datos**, lo significa que se puede añadir cualquier tipo de elementos a ella. El ejemplo mete en la pila dos enteros y una cadena:

```
>>> s = Stack()
>>> s.push(54)
>>> s.push(45)
>>> s.push("+")
```

Se pueden utilizar `isEmpty` y `pop` para quitar e imprimir todos los elementos en la pila:

²del mismo nombre

```
while not s.isEmpty() :  
    print s.pop(),
```

La salida es + 45 54. En otras palabras, ¡se ha utilizado una pila para imprimir los elementos en orden inverso! Reconocemos que no es el formato estándar de impresión de listas, pero fue muy fácil usar una pila para lograrlo.

Compare estas líneas con la implementación de `imprimeAlReves` que vimos en la Sección 17.4. Existe un paralelo natural entre la versión recurrente de `imprimeAlReves` y el algoritmo de pila que acabamos de ver. La diferencia es que `imprimeAlReves` utiliza la pila de tiempo de ejecución para mantenerse al tanto de los nodos mientras recorre la lista y luego los imprime cuando regresa de la recursión. El algoritmo de pila hace lo mismo, pero utiliza un objeto `Pila` en vez de la pila de tiempo de ejecución.

18.5. Usar una pila para evaluar postfijo

Las expresiones matemáticas en la mayoría de lenguajes de programación se escriben con el operador entre los dos operandos, de esta manera: $1+2$. A este formato se le llama **infijo**. Algunas calculadoras utilizan un formato alternativo llamado **postfijo**. Con postfijo, el operador va después de los operandos, así: $1\ 2\ +$.

La razón por la que el formato postfijo es útil es que existe una forma natural de evaluar una expresión en formato postfijo utilizando una pila:

- Desde el principio de la expresión, evalúe los operadores y operandos uno por uno.
 - Si el término es un operando, utilice `push` para colocarlo en la pila.
 - Si el término es un operador, utilice `pop` con dos operandos de la pila, ejecute la operación sobre ellos, y coloque el resultado en la pila con `push`.
- Cuando llegue al final de la expresión habrá un operando en la pila. Ese operando es el resultado.

*Para practicar, aplique este algoritmo a la expresión $1\ 2\ +\ 3\ *$.*

Este ejemplo demuestra una de las ventajas de el formato postfijo—no hay necesidad de usar paréntesis para controlar el orden de operaciones. Para obtener el mismo resultado con el formato infijo, se tendría que escribir $(1 + 2) * 3$.

*Para practicar, escriba una expresión en formato postfijo que sea equivalente a $1 + 2 * 3$.*

18.6. Análisis sintáctico

Para implementar el algoritmo anterior, necesitamos ser capaces de recorrer una cadena y separarla en operandos y operadores. Este proceso es un ejemplo del **análisis sintáctico**, y los resultados—la piezas individuales de la cadena—son **tokens**³. Quizás se acuerde de esas palabras del Capítulo 1.

Python posee un método llamado `split` (partir) en el módulo `string` (cadena) y en el módulo `re` (expresiones regulares). La función `string.split` parte una cadena y la convierte en una lista utilizando un sólo carácter como **delimitador**. Por ejemplo:

```
>>> import string
>>> string.split("La hora ha llegado", " ")
['La', 'hora', 'ha', 'llegado']
```

En este caso, el delimitador es el carácter espacio, y se parte la cadena en cada espacio.

La función `re.split` tiene más potente, y nos permite utilizar una expresión regular en vez de un delimitador. Una expresión regular es una manera de especificar un conjunto de cadenas. Por ejemplo, `[A-z]` es el conjunto de todas las letras y `[0-9]` es el conjunto de todas las cifras. El operador `^` niega un conjunto, y `[^0-9]` es el conjunto de todo lo que no es un número, lo cual es exactamente lo que queremos utilizar para partir expresiones en el formato postfijo:

```
>>> import re
>>> re.split("[^0-9]", "123+456*/")
['123', '+', '456', '*', '/', '']
```

Fíjese que el orden de los argumentos es diferente del de `string.split`; el delimitador viene antes de la cadena.

La lista resultante incluye los operandos 123 y 456 y los operadores `*` y `/`. También incluye dos cadenas vacías que se insertan después de los operandos.

18.7. Evaluar un postfijo

Para evaluar una expresión en formato postfijo usaremos el analizador sintáctico de la sección anterior y el algoritmo de la sección previa a ésta. Para no complicar

³Podríamos traducir “token” como “cospel” o “pieza”, en ocasiones también como “símbolo” pero la expresión inglesa está tan introducida en el vocabulario informático que sólo añadiría confusión.

las cosas, comenzaremos con un evaluador que solo implementa los operadores $+$ y $*$:

```
def evalPostfijo(expr):
    import re
    listaTokens = re.split("[^0-9]", expr)
    pila = Pila()
    for token in listaTokens:
        if token == '' or token == ' ':
            continue
        if token == '+':
            suma = pila.pop() + pila.pop()
            pila.push(suma)
        elif token == '*':
            producto = pila.pop() * pila.pop()
            pila.push(producto)
        else:
            pila.push(int(token))
    return pila.pop()
```

La primera condición se encarga de espacios y cadenas vacías. Las dos condiciones siguientes controlan los operadores. Asumimos, por ahora, que todo lo demás es un operando. Por supuesto, sería mejor verificar si la entrada tiene errores y mostrar un mensaje con el error, pero eso se hará después.

Comprobemos con una evaluación de la forma postfijo $(56+47)*2$:

```
>>> print evalPostfijo ("56 47 + 2 *")
206
```

Esto es suficiente.

18.8. Clientes y proveedores

Uno de los objetivos fundamentales de un TAD es el de separar los intereses del proveedor, quien escribe el código de programa que implementa el TAD, y los del cliente, quien utiliza el TAD. El proveedor sólo se preocupa de la implementación y de si es correcta o no—de acuerdo a las especificaciones del TAD—y no de cómo se va a utilizar.

A la inversa, el cliente *supone* que la implementación del TAD es correcta y no se preocupa de los detalles. Cuando se usa uno de los tipos predefinidos de Python, uno se puede dar el lujo de pensar exclusivamente como un cliente.

Por supuesto, cuando usted implemente un TAD, también debe desarrollar código de cliente para probarlo. En ese case, uno toma ambos papeles, lo cual puede causar confusión. Tiene que fijarse bien en del papel que está tomando en todo momento.

18.9. Glosario

tipo abstracto de datos (TAD): Un tipo de datos (a menudo una colección de objetos) que se define por un conjunto de operaciones pero que se puede implementar de varias maneras.

interfaz: El conjunto de operaciones que definen un TAD.

implementación: El código de programa que satisface los prerequisites sintácticos y semánticos de un interfaz.

cliente: Un programa (o la persona que lo escribió) que utiliza un TAD.

proveedor: Un programa (o la persona que lo escribió) que implementa un TAD.

enchapado: La definición de clase que implementa un TAD con definiciones de métodos que son las invocaciones de otros métodos, a veces con transformaciones simples. El enchapado no ejecuta nada de gran valor, pero mejora la interfaz vista por el cliente o la hace mas estándar.

estructura de datos genérica: Un tipo de estructura de datos que puede contener datos de cualquier tipo.

infijo: Un método de escribir expresiones matemáticas con los operadores entre los operandos.

postfijo: Un método de escribir expresiones matemáticas con los operadores después de los operandos.

analizar sintácticamente: Examinar una cadena de caracteres o tokens y analizar su estructura gramatical.

token: Un conjunto de caracteres que se tratan como una unidad y son analizados sintácticamente, como las palabras de un lenguaje natural.

delimitador: Un carácter utilizado para separar tokens, como la puntuación en un lenguaje natural.

Capítulo 19

Colas

Este capítulo presenta dos TADs: la Cola y la Cola Priorizada. En la vida real, una **cola** es una fila de clientes esperando un servicio de algún tipo. En la mayoría de los casos, el primer cliente de la fila es el primero al que se va a servir. Sin embargo, hay excepciones. En los aeropuertos, a veces se saca de la cola a los clientes cuyos vuelos van a salir pronto. En los supermercados, un cliente educado puede dejar que alguien que lleva pocos productos pase antes.

La regla que determina quién va primero se llama **táctica de encolamiento**. La táctica de encolamiento más simple se llama **FIFO**, de “first-in-first-out”, “el primero que entra es el primero que sale”. La táctica de encolamiento más general es el **encolamiento priorizado**, en la que a cada cliente se le asigna una prioridad y el cliente con la prioridad más alta pasa primero, sin importar el orden de llegada. Decimos que es la táctica más general porque la prioridad se puede basar en cualquier cosa: a qué hora sale el vuelo; cuántos productos lleva el cliente; cuán importante es el cliente. Por supuesto, no todas las tácticas de prioridad son “justas”, pero la justicia siempre es subjetiva.

El TAD Cola y el TAD Cola Priorizada tienen el mismo conjunto de operaciones. La diferencia está en la semántica de las operaciones: una cola usa la táctica FIFO, y una cola priorizada (como su propio nombre indica) usa una táctica de encolamiento priorizado.

19.1. El TAD Cola

El TAD Cola se define a través de las siguientes operaciones:

`__init__`: Inicializa una cola nueva vacía.

inserta: Añade un elemento a la cola.

quita: Elimina y devuelve un elemento de la cola. El elemento devuelto es el primero que se añadió.

estaVacia: Comprueba si la cola está vacía.

19.2. Cola Enlazada

La primera implementación del TAD Cola al que vamos a echar un vistazo se llama **cola enlazada** porque está hecha de objetos **Nodo** enlazados. He aquí la definición de la clase:

```
class Cola:
    def __init__(self):
        self.longitud = 0
        self.cabeza = None

    def estaVacia(self):
        return (self.longitud == 0)

    def inserta(self, carga):
        nodo = Nodo(carga)
        nodo.siguiente = None
        if self.cabeza == None:
            # si la lista está vacía el nuevo nodo va el primero
            self.cabeza = nodo
        else:
            # encuentra el último nodo de la lista
            ultimo = self.cabeza
            while ultimo.siguiente: ultimo = ultimo.siguiente
            # añadir el nuevo nodo
            ultimo.siguiente = nodo
        self.longitud = self.longitud + 1

    def quita(self):
        carga = self.cabeza.carga
        self.cabeza = self.cabeza.siguiente
        self.longitud = self.longitud - 1
        return carga
```

Los métodos **estaVacia** y **quita** son idénticos a los métodos **estaVacia** y **quitaPrimero** de **ListaEnlazada**. El método **inserta** es nuevo y un poco más

complicado.

Queremos insertar nuevos elementos al final de la lista. Si la cola está vacía, simplemente hacemos que **cabeza** se refiera al nuevo nodo.

En caso contrario, recorremos la lista hasta el último nodo y lo fijamos al final. Podemos reconocer el último nodo porque su atributo **siguiente** es **None**.

En un objeto **Cola** correctamente construido hay dos invariantes. El valor de **longitud** debería ser el número de nodos en la cola, y el último nodo debería tener **siguiente** igual a **None**. Créase que este método cumple con ambas invariantes.

19.3. Rendimiento típico

Normalmente cuando invocamos un método, no nos importan los detalles de su implementación. Pero hay un “detalle” que podría interesarnos: el rendimiento típico del método. ¿Cuánto tarda, y cómo varía el tiempo de ejecución al aumentar el número de elementos de la colección?

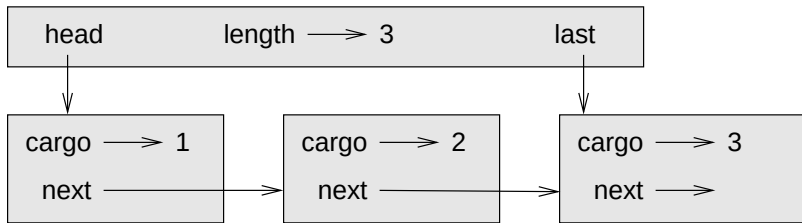
Primero mire **quita**. Ahí no hay bucles ni llamadas a funciones, dando a entender que el tiempo de ejecución de este método es siempre el mismo. Un método así se llama operación de **tiempo constante**. En realidad, el método podría ser ligeramente más rápido cuando la lista está vacía porque se salta el cuerpo de la condición, pero esa diferencia no es significativa.

El rendimiento de **inserta** es muy diferente. En el caso general, tenemos que recorrer la lista para encontrar el último elemento.

Este recorrido cuesta un tiempo proporcional a la longitud de la lista. Como el tiempo de ejecución es función lineal de la longitud, este método se llama de **tiempo lineal**. Comparado con el tiempo constante, es muy pobre.

19.4. Cola Enlazada Mejorada

Nos gustaría una implementación del TAD Cola capaz de realizar todas las operaciones en tiempo constante. Una forma de hacerlo es modificar la clase Cola de modo que mantenga una referencia tanto al primero como al último nodo, como se muestra en la figura:



La implementación de ColaMejorada es así:

```
class ColaMejorada:
    def __init__(self):
        self.longitud = 0
        self.cabeza = None
        self.ultimo = None

    def estaVacia(self):
        return (self.longitud == 0)
```

Hasta ahora, el único cambio es el atributo `ultimo`. Se usa en los métodos `inserta` y `quita`:

```
class ColaMejorada:
    ...
    def inserta(self, carga):
        nodo = Nodo(carga)
        nodo.siguiente = None
        if self.longitud == 0:
            # si la lista está vacía, el nuevo nodo es cabeza y último
            self.cabeza = self.ultimo = nodo
        else:
            # encontrar el último nodo
            ultimo = self.ultimo
            # añadir el nuevo nodo
            ultimo.siguiente = nodo
            self.ultimo = nodo
        self.longitud = self.longitud + 1
```

Como `ultimo` sigue el rastro del último nodo, no necesitamos buscarlo. A causa de esto, este método funciona en tiempo constante.

Debemos pagar un precio por esa velocidad. Tenemos que añadir un caso especial a `quita` para apuntar `ultimo` a `None` cuando quitamos el último nodo:

```
class ColaMejorada:
```



```
...
def quita(self):
    carga      = self.cabeza.carga
    self.cabeza = self.cabeza.siguiente
    self.longitud = self.longitud - 1
    if self.longitud == 0:
        self.ultimo = None
    return carga
```

Esta implementación es más complicada que la de la Lista Enlazada, y es más difícil demostrar que es correcta. La ventaja es que hemos alcanzado la meta: tanto `inserta` como `quita` son operaciones de tiempo constante.

Como ejercicio, escriba una implementación del TAD Cola usando una lista de Python. Compare el rendimiento de esta implementación con la de la ColaMejorada para varias longitudes de cola.

19.5. Cola priorizada

El TAD Cola Priorizada tiene el mismo interfaz que el TAD Cola, pero diferente semántica. De nuevo, el interfaz es:

__init__: Inicializa una cola vacía nueva.

inserta: Añade un nuevo elemento a la cola.

quita: Elimina y devuelve un elemento de la cola. El elemento devuelto es el de prioridad más alta.

estaVacía: Comprueba si la cola está vacía.

La diferencia semántica es que el elemento eliminado de la cola no es necesariamente el primero que se añadió. En su lugar, es el elemento con la prioridad más alta. Lo que son las prioridades y cómo se comparan con las otras no se especifica en la implementación de la Cola Priorizada. Depende de los elementos de la cola.

Por ejemplo, si los elementos de la cola tienen nombres, podemos elegirlos en orden alfabético. Si son puntuaciones de bolos, podemos ir de mayor a menor, pero si son puntuaciones de golf, iremos de menor a mayor. Siempre que podamos comparar los elementos de la cola, podremos encontrar y quitar el elemento con mayor prioridad.

Esta implementación de Cola Priorizada tiene como atributo una lista de Python que contiene los elementos de la cola.

```
class ColaPriorizada:
    def __init__(self):
        self.elementos = []

    def estaVacia(self):
        return self.elementos == []

    def inserta(self, elemento):
        self.elementos.append(elemento)
```

El método de inicialización, `estaVacia`, e `inserta` son todos calcados de las operaciones sobre listas. El único método interesante es `quita`:

```
class ColaPriorizada:
    ...
    def quita(self):
        maxi = 0
        for i in range(1, len(self.elementos)):
            if self.elementos[i] > self.elementos[maxi]:
                maxi = i
        elemento = self.elementos[maxi]
        self.elementos[maxi:maxi+1] = []
        return elemento
```

Al principio de cada iteración, `maxi` contiene el índice del elemento más grande (prioridad más alta) que hemos visto *hasta el momento*. Cada vez que se completa el bucle, el programa compara el iésimo elemento con el campeón. Si el nuevo elemento es mayor, el valor de `maxi` se fija a `i`.

Cuando la sentencia `for` completa su ejecución, `maxi` es el índice del elemento mayor. Este elemento se elimina de la lista y se devuelve.

Vamos a probar la implementación:

```
>>> c = ColaPriorizada()
>>> c.inserta(11)
>>> c.inserta(12)
>>> c.inserta(14)
>>> c.inserta(13)
>>> while not c.estaVacia(): print c.quita() # ver cuál se quita
14
13
12
11
```

Si la cola contiene números o cadenas simples, se eliminan en orden numérico o alfabético, de mayor a menor. Python puede encontrar el entero o la cadena mayor porque puede compararlos usando los operadores de comparación internos.

Si la cola contiene un tipo de objeto, debe proporcionar un método `__cmp__`. Cuando `quita` usa el operador `>` para comparar elementos, invoca al `__cmp__` de uno de los elementos y le pasa el otro como parámetro. Siempre que el método `__cmp__` trabaje adecuadamente, la Cola Priorizada funcionará.

19.6. La clase Golfista

Como ejemplo de un objeto con una definición inusual de prioridad, vamos a implementar una clase llamada `Golfista` que mantiene los nombres y puntuaciones de golfistas. Como es habitual, empezamos por definir `__init__` y `__str__`:

```
class Golfista:
    def __init__(self, nombre, puntos):
        self.nombre = nombre
        self.puntos = puntos

    def __str__(self):
        return "%-16s: %d" % (self.nombre, self.puntos)
```

`__str__` usa el operador de formato para poner los nombres y las puntuaciones en bonitas columnas.

A continuación definimos una versión de `__cmp__` en la que la puntuación más baja tiene la prioridad más alta. Como siempre, `__cmp__` devuelve 1 si `self` es “mayor que” `otro`, -1 si `self` es “menor que” `otro`, y 0 si son iguales.

```
class Golfista:
    ...
    def __cmp__(self, otro):
        if self.puntos < otro.puntos: return 1    # menos es más
        if self.puntos > otro.puntos: return -1
        return 0
```

Ya estamos listos para probar la cola priorizada con la clase `Golfista`:

```
>>> tiger = Golfista("Tiger Woods",    61)
>>> cabr  = Golfista("Ángel Cabrera",  72)
>>> ola   = Golfista("J.M. Olazábal",  69)
>>>
```

```
>>> cp = ColaPriorizada()
>>> cp.inserta(tiger)
>>> cp.inserta(cabr)
>>> cp.inserta(ola)
>>> while not cp.estaVacia(): print cp.quita()
Tiger Woods      : 61
J.M. Olazábal    : 69
Ángel Cabrera    : 72
```

Como ejercicio, escriba una implementación del TAD Cola Priorizada usando una lista enlazada. Debería usted mantener la lista ordenada de modo que la eliminación sea una operación de tiempo constante. Compare el rendimiento de esta implementación con la implementación con la lista de Python.

19.7. Glosario

cola: Un conjunto ordenado de objetos esperando un servicio de algún tipo.

Cola: Un TAD que ejecuta las operaciones que uno podría realizar sobre una cola.

táctica de encolamiento: Las reglas que determinan qué miembro de la cola será el próximo en eliminarse.

FIFO: “First In, First Out”, una táctica de encolamiento en la que el primer miembro en llegar es el primero en salir.

cola priorizada: Una táctica de encolamiento en la que cada miembro tiene una prioridad determinada por factores externos. El miembro con mayor prioridad es el primero en eliminarse.

Cola Priorizada: Un TAD que define las operaciones que se pueden realizar sobre una cola priorizada.

cola enlazada: Una implementación de una cola utilizando una lista enlazada.

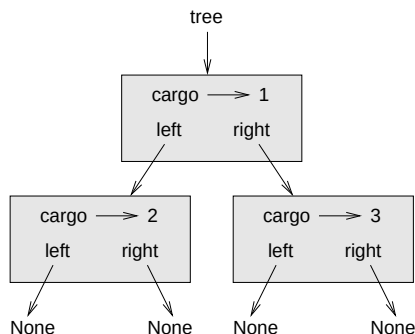
tiempo constante: Una operación cuyo tiempo de ejecución no depende del tamaño de la estructura de datos.

tiempo lineal: Una operación cuyo tiempo de ejecución es función lineal del tamaño de la estructura de datos.

Capítulo 20

Árboles

Al igual que las listas enlazadas, los árboles están hechos de nodos. Un tipo común de árbol es un **árbol binario**, en el que cada nodo contiene una referencia a otros dos nodos (posiblemente nula). Nombramos a estas referencias como subárboles izquierdo y derecho. Como los nodos de las listas, los nodos de los árboles también contienen una carga. Un diagrama de estado de un árbol es así:



Para evitar apelonar las cosas en la figura, solemos omitir los **None**s.

La parte superior del árbol (el nodo al que apunta **tree**) se llama **raíz**. Siguiendo con la metáfora del árbol, los otros nodos se llaman ramas y los nodos de los extremos con referencias nulas se llaman **hojas**. Puede parecer extraño que dibujemos la figura con la raíz arriba y las hojas abajo, pero eso no es lo más raro.

Para empeorar las cosas, los científicos informáticos añaden a la mezcla otra metáfora: el árbol genealógico. El nodo superior se llama a veces **padre** y los

nodos a los que se refiere son sus **hijos**. Los nodos con el mismo padre se llaman **hermanos**.

Para terminar, tenemos un vocabulario geométrico para hablar sobre los árboles. Ya hemos mencionado izquierda y derecha, pero también están “arriba” (hacia el padre/raíz) y “abajo” (hacia los hijos/hojas). También, todos los nodos que están a la misma distancia de la raíz forman un **nivel** del árbol.

Probablemente no sean necesarias las metáforas arbóreas para hablar de árboles, pero ahí están.

Igual que las listas enlazadas, los árboles son estructuras de datos recursivas porque se definen recursivamente.

Un árbol es:

- el árbol vacío, representado por **None**, o
- un nodo que contiene una referencia a un objeto y dos referencias a árboles.

20.1. Crear árboles

El proceso de montar un árbol es similar al proceso de montar una lista enlazada. Cada invocación del constructor crea un solo nodo.

```
class Arbol:
    def __init__(self, carga, izquierda=None, derecha=None):
        self.carga = carga
        self.izquierda = izquierda
        self.derecha = derecha

    def __str__(self):
        return str(self.carga)
```

La **carga** puede ser de cualquier tipo, pero los parámetros **izquierda** y **derecha** deben ser árboles. Tanto **izquierda** como **derecha** son opcionales; el valor por omisión es **None**.

Para imprimir un nodo, simplemente imprimimos la carga.

Una forma de construir un árbol es del fondo hacia arriba. Asigne primero los nodos hijos:

```
izquierda = Arbol(2)
derecha = Arbol(3)
```

Luego cree el nodo padre y vincúlelo a los hijos:

```
arbol = Arbol(1, izquierda, derecha);
```

Podemos escribir este código más concisamente anidando las invocaciones al constructor:

```
>>> arbol = Arbol(1, Arbol(2), Arbol(3))
```

En cualquier caso, el resultado es el árbol del principio del capítulo.

20.2. Recorrer árboles

Siempre que usted vea una nueva estructura de datos, su primera pregunta debería ser: “¿Cómo la recorro?” La forma más natural de recorrer un árbol es recursivamente. Por ejemplo, si el árbol contiene enteros como carga, esta función nos devuelve su suma:

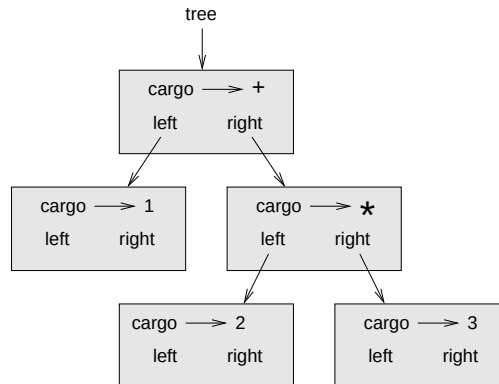
```
def total(arbol):  
    if arbol == None: return 0  
    return total(arbol.izquierda) + total(arbol.derecha) +\  
        arbol.carga
```

El caso base es el árbol vacío, que no tiene carga, así que la suma es 0. El paso recursivo hace dos llamadas recursivas para hallar la suma de los árboles hijos. Cuando terminan las llamadas recursivas, sumamos la carga del padre y devolvemos el total.

20.3. Árboles de expresión

Un árbol es una forma natural de representar la estructura de una expresión. Al contrario que otras notaciones, puede representar el cálculo de forma no ambigua. Por ejemplo, la expresión infija $1 + 2 * 3$ es ambigua a no ser que sepamos que la multiplicación se realiza antes que la suma.

Este árbol de expresión representa el mismo cálculo:



Los nodos de un árbol de expresión pueden ser operandos como 1 y 2 u operadores como + y *. Los operandos son nodos hojas; los nodos operadores contienen referencias a sus operandos. (Todos estos operadores son **binarios**, lo que significa que tienen exactamente dos operandos.)

Así podemos construir este árbol:

```
>>> arbol = Arbol('+', Arbol(1), Arbol('*', Arbol(2), Arbol(3)))
```

Mirando la figura, no hay duda del orden de las operaciones; la multiplicación se realiza antes para calcular el segundo operando de la suma.

Los árboles de expresión tienen muchos usos. El ejemplo de este capítulo usa árboles para traducir expresiones a postfijo, prefijo e infijo. Árboles similares se usan dentro de los compiladores para analizar, optimizar y traducir programas.

20.4. Recorrido de un árbol

Podemos recorrer un árbol de expresión e imprimir el contenido así:

```
def imprimeArbol(arbol):
    if arbol == None: return
    print arbol.carga,
    imprimeArbol(arbol.izquierda)
    imprimeArbol(arbol.derecha)
```

En otras palabras, para imprimir un árbol imprima primero el contenido de la raíz, luego el subárbol izquierdo entero y después el subárbol derecho entero. Esta forma de recorrer un árbol se llama **orden prefijo**, porque el contenido de la raíz aparece *antes* del contenido de los hijos. La salida del ejemplo anterior es:


```
>>> arbol = Arbol('+', Arbol(1), Arbol('*', Arbol(2), Arbol(3)))
>>> imprimeArbol(arbol)
+ 1 * 2 3
```

Este formato es diferente tanto del postfijo como del infijo; es otra notación llamada **prefija**, en la que los operadores aparecen delante de sus operandos.

Puede sospechar que si recorre el árbol en un orden diferente obtendrá la expresión en una notación diferente. Por ejemplo, si imprime primero los subárboles y luego la raíz, tendrá:

```
def imprimeArbolPostfijo(arbol):
    if arbol == None: return
    imprimeArbolPostfijo(arbol.izquierda)
    imprimeArbolPostfijo(arbol.derecha)
    print arbol.carga,
```

¡El resultado, 1 2 3 * +, está en notación posfija! Este orden de recorrido se llama **orden postfijo**.

Finalmente, para recorrer un árbol en **orden infijo**, usted imprime el árbol izquierdo, luego la raíz y después el árbol derecho:

```
def imprimeArbolInfijo(arbol):
    if arbol == None: return
    imprimeArbolInfijo(arbol.izquierda)
    print arbol.carga,
    imprimeArbolInfijo(arbol.derecha)
```

El resultado es 1 + 2 * 3, que es la expresión en notación infija.

Para ser justos debemos señalar que hemos omitido una importante complicación. A veces, al escribir una expresión infija, debemos usar paréntesis para preservar el orden de las operaciones. De modo que una exploración de orden infijo no es suficiente para generar una expresión infija.

No obstante, con unas pequeñas mejoras, el árbol de expresión y los tres recorridos nos dan una forma general de traducir expresiones de un formato a otro.

Como ejercicio, modifique `imprimeArbolInfijo` de modo que ponga entre paréntesis cada operador con sus operandos. ¿La salida es correcta y sin ambigüedades? ¿Se necesitan siempre los paréntesis?

Si hacemos un recorrido en orden infijo y tomamos nota de en qué nivel del árbol estamos, podemos generar una representación gráfica de un árbol:

```
def imprimeArbolSangrado(arbol, nivel=0):
    if arbol == None: return
    imprimeArbolSangrado(arbol.derecha, nivel+1)
    print ' '*nivel + str(arbol.carga)
    imprimeArbolSangrado(arbol.izquierda, nivel+1)
```

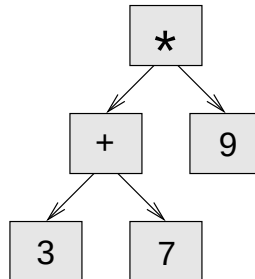
El parámetro `nivel` lleva la cuenta de dónde estamos en el árbol. Por omisión, inicialmente es 0. Cada vez que hacemos una llamada recursiva, pasamos `nivel+1` porque el nivel del hijo es siempre uno mayor que el del padre. Sangramos cada elemento con dos espacios por nivel. El resultado del árbol de ejemplo es:

```
>>> imprimeArbolSangrado(arbol)
      3
     *
    2
 +
  1
```

Si mira la salida de lado, verá una versión simplificada de la figura original.

20.5. Construir un árbol de expresión

En esta sección analizamos expresiones infijas y formamos los correspondientes árboles de expresión. Por ejemplo, la expresión $(3+7)*9$ nos da el siguiente árbol:



Fíjese en que hemos simplificado el diagrama omitiendo los nombres de los atributos.

El analizador que vamos a escribir maneja expresiones que incluyen números, paréntesis y los operadores `+` y `*`. Suponemos que la cadena de entrada ya ha sido tokenizada y almacenada en una lista de Python. La lista de tokens de $(3+7)*9$ es:

```
['(', 3, '+', 7, ')', '*', 9, 'fin']
```

El token `fin` es útil para evitar que el analizador lea más allá del final de la lista.

A modo de ejercicio, escriba una función que tome una cadena conteniendo una expresión y devuelva una lista de tokens.

La primera función que vamos a escribir es `tomaToken`, que toma como parámetros una lista de tokens y el token que esperamos. Compara el token esperado con el primer token de la lista: si coinciden, elimina el token de la lista y devuelve verdadero, en caso contrario, devuelve falso:

```
def tomaToken(listaToken, esperado):
    if listaToken[0] == esperado:
        del listaToken[0]
        return 1
    else:
        return 0
```

Como `listaToken` apunta a un objeto mutable, los cambios hechos aquí son visibles para cualquier otra variable que apunte al mismo objeto.

La siguiente función, `obtieneNumero`, maneja operandos. Si el siguiente token de `listaToken` es un número, `obtieneNumero` lo elimina y devuelve un nodo hoja que contiene el número; en caso contrario, devuelve `None`.

```
def obtieneNumero(listaToken):
    x = listaToken[0]
    if type(x) != type(0): return None
    del listaToken[0]
    return Arbol (x, None, None)
```

Antes de seguir, debemos probar `obtieneNumero` aisladamente. Asignamos una lista de números a `listaToken`, extraemos el primero, imprimimos el resultado e imprimimos lo que quede de la lista de tokens:

```
>>> listaToken = [9, 11, 'fin']
>>> x = obtieneNumero(listaToken)
>>> imprimeArbolPostfijo(x)
9
>>> print listaToken
[11, 'fin']
```

El siguiente método que necesitamos es `obtieneProducto`, que construye un árbol de expresión para productos. Un producto simple tiene dos números como operandos, como `3 * 7`.

Aquí tenemos una versión de `obtieneProducto` que maneja productos simples.

```
def obtieneProducto(listationToken):
    a = obtieneNumero(listationToken)
    if tomaToken(listationToken, '*'):
        b = obtieneNumero(listationToken)
        return Arbol('*', a, b)
    else:
        return a
```

Suponiendo que `obtieneNumero` se ejecuta con éxito y devuelve un árbol simple, asignamos el primer operando a `a`. Si el siguiente carácter es `*`, obtenemos el segundo número y construimos un árbol de expresión con `a`, `b`, y el operador.

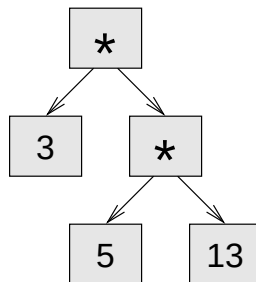
Si el siguiente carácter es cualquier otra cosa, simplemente devolvemos el nodo hoja con `a`. Veamos dos ejemplos:

```
>>> listaToken = [9, '*', 11, 'fin']
>>> arbol = obtieneProducto(listationToken)
>>> imprimeArbolPostfijo(arbol)
9 11 *
```

```
>>> listaToken = [9, '+', 11, 'fin']
>>> arbol = obtieneProducto(listationToken)
>>> imprimeArbolPostfijo(arbol)
9
```

El segundo ejemplo implica que entendemos un operando suelto como un tipo de producto. Esta definición de “producto” es contraria a la intuición, pero resulta ser útil.

Ahora tenemos que vérnoslas con productos compuestos, como $3 * 5 * 13$. Tratamos esta expresión como un producto de productos, es decir, $3 * (5 * 13)$. El árbol resultante es:



Con un pequeño cambio en `obtieneProducto` podemos manejar productos arbitrariamente largos:

```
def obtieneProducto(listaToken):
    a = obtieneNumero(listaToken)
    if tomaToken(listaToken, '*'):
        b = obtieneProducto(listaToken)      # cambiamos esta línea
        return Arbol('*', a, b)
    else:
        return a
```

En otras palabras, un producto puede ser bien un operando aislado o un árbol con * en su raíz, un número en la derecha y un producto en la izquierda. Este tipo de definición recursiva debería empezar a resultar familiar.

Comprobemos la nueva versión con un producto compuesto:

```
>>> listaToken = [2, '*', 3, '*', 5, '*', 7, 'fin']
>>> arbol = obtieneProducto(listaToken)
>>> imprimeArbolPostfijo(arbol)
2 3 5 7 * * *
```

Ahora añadiremos la capacidad de analizar sumas. De nuevo, usamos una definición poco intuitiva de “suma”. Para nosotros, una suma puede ser un árbol con + en la raíz, un producto en la izquierda y una suma en la derecha. O también, una suma puede ser simplemente un producto.

Si quiere seguir adelante con esta definición, tiene una bonita propiedad: podemos representar cualquier expresión (sin paréntesis) como una suma de productos. Esta propiedad es la base de nuestro algoritmo de análisis.

`obtieneSuma` intenta construir un árbol con un producto en la izquierda y una suma en la derecha. Pero si no encuentra un +, simplemente construye un producto.

```
def obtieneSuma(listaToken):
    a = obtieneProducto(listaToken)
    if tomaToken(listaToken, '+'):
        b = obtieneSuma(listaToken)
        return Arbol('+', a, b)
    else:
        return a
```

Vamos a probarla con $9 * 11 + 5 * 7$:

```
>>> listaToken = [9, '*', 11, '+', 5, '*', 7, 'fin']
>>> arbol = obtieneSuma(listaToken)
>>> imprimeArbolPostfijo(arbol)
9 11 * 5 7 * +
```

Ya casi hemos acabado, pero todavía tenemos que manejar los paréntesis. En culaquier lugar de una expresión donde puede haber un número, puede haber también una suma entera entre paréntesis. Sólo necesitamos modificar `obtieneNumero` para manejar **subexpresiones**:

```
def obtieneNumero(listaToken):
    if tomaToken(listaToken, '('):
        x = obtieneSuma(listaToken) # obtiene la subexpresión
        tomaToken(listaToken, ')') # quita el cierre de paréntesis
        return x
    else:
        x = listaToken[0]
        if type(x) != type(0): return None
        listaToken[0:1] = []
        return Arbol (x, None, None)
```

Vamos a probar este código con $9 * (11 + 5) * 7$:

```
>>> listaToken = [9, '*', '(', 11, '+', 5, ')', '*', 7, 'fin']
>>> arbol = obtieneSuma(listaToken)
>>> imprimeArbolPostfijo(arbol)
9 11 5 + 7 * *
```

El analizador manejó correctamente los paréntesis; la suma ocurre antes de la multiplicación.

En la versión final del programa, sería bueno dar a `obtieneNumero` un nombre más descriptivo de su nueva función.

20.6. Manejar errores

En todo momento hemos supuesto que las expresiones estaban bien formadas. Por ejemplo, cuando alcanzamos el final de una subexpresión, suponemos que el carácter siguiente es un paréntesis cerrado. Si hay un error y el siguiente carácter es cualquier otra cosa, deberíamos ocuparnos de él.

```
def obtieneNumero(listaToken):
    if tomaToken(listaToken, '('):
        x = obtieneSuma(listaToken)
        if not tomaToken(listaToken, ')'):
            raise 'ExpresionErronea', 'falta un paréntesis'
        return x
```

```
else:  
    # omitido el resto de la función
```

La sentencia **raise** crea una excepción; en este caso creamos un nuevo tipo de excepción, llamado **ExpresionErronea**. Si la función que llamé a **obtieneNumero**, o alguna de las otras funciones de la pila de llamadas, maneja la expresión, el programa podrá continuar. En caso contrario, Python imprimirá un mensaje de error y saldrá.

*Como ejercicio, encuentre otros lugares de estas funciones donde puedan ocurrir errores y añada las sentencias **raise** adecuadas. Pruebe su código con expresiones formadas incorrectamente.*

20.7. El árbol de animales

En esta sección desarrollaremos un pequeño programa que usa un árbol para representar una base de conocimientos.

El programa interactúa con el usuario para crear un árbol de preguntas y nombres de animales. Aquí tenemos un ejemplo de ejecución:

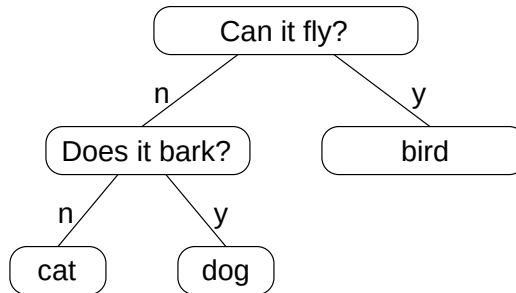
```
Estás pensando en un animal? s  
Es un pájaro? n  
Cómo se llama el animal? perro  
Qué pregunta distinguiría a un perro de un pájaro? Puede volar  
Si el animal fuera un perro, cuál sería la respuesta? n
```

```
Estás pensando en un animal? s  
Puede volar? n  
Es un perro? n  
Cómo se llama el animal? gato  
Qué pregunta distinguiría a un gato de un perro? Ladra  
Si el animal fuera un gato, cuál sería la respuesta? n
```

```
Estás pensando en un animal? s  
Puede volar? n  
Ladra? s  
Es un perro? s  
Soy el más grande!
```

Estás pensando en un animal? n

Este es el árbol que construye este diálogo:



Al principio de cada ronda, el programa empieza en lo alto del árbol y hace la primera pregunta. Dependiendo de la respuesta, se mueve al hijo de la izquierda o de la derecha y sigue hasta que llega a un nodo hoja. En ese momento, intenta adivinar. Si falla, pide al usuario el nombre del nuevo animal y una pregunta que distinga al intento fallido del nuevo animal. Entonces añade un nodo al árbol con la nueva pregunta y el nuevo animal.

Éste es el código:

```

def animal():
    # empezar con un nodo suelto
    raiz = Arbol("pájaro")

    # bucle hasta que el usuario salga
    while 1:
        print
        if not si("Estás pensando en un animal? "): break

    # recorrer el árbol
    arbol = raiz
    while arbol.tomaIzquierda() != None:
        indicador = arbol.tomaCarga() + "? "
        if si(indicador):
            arbol = arbol.tomaDerecha()
        else:
            arbol = arbol.tomaIzquierda()

    # intentar adivinar
  
```



```

adivina = arbol.tomaCarga()
indicador = "Es un " + adivina + "? "
if si(indicador):
    print "Soy el más grande!"
    continue

# obtener información nueva
indicador = "Cómo se llama el animal? "
animal = raw_input(indicador)
indicador = "Qué pregunta distinguiría a un %s de un %s? "
pregunta = raw_input(indicador % (animal,adivina))

# añadir información nueva al árbol
arbol.ponCarga(pregunta)
indicador = "Si el animal fuera un %s, cuál sería la\
    respuesta? "
if si(indicador % animal):
    arbol.ponIzquierda(Arbol(adivina))
    arbol.ponDerecha(Arbol(animal))
else:
    arbol.ponIzquierda(Arbol(animal))
    arbol.ponDerecha(Arbol(adivina))

```

La función `si` es un auxiliar; imprime un indicador y acepta una entrada del usuario. Si la respuesta comienza con `s` o `S`, la función devuelve verdadero:

```

def si(preg):
    from string import lower
    resp = lower(raw_input(preg))
    return (resp[0] == 's')

```

La condición del bucle externo es `1`, lo que significa que seguirá hasta que se ejecute la sentencia `break` cuando el usuario ya no piense en ningún animal.

El bucle `while` interno recorre el árbol de arriba a abajo, guiado por las respuestas del usuario.

Cuando se añade un nuevo nodo al árbol, la pregunta sustituye a la carga y los dos hijos son el animal nuevo y la carga original.

Una carencia del programa es que, al salir, ¡olvida todo lo que usted le había enseñado con tanto cuidado!

Como ejercicio, piense en varias formas en las que podría guardar el árbol de conocimiento en un archivo. Implemente la que piense que es más fácil.

20.8. Glosario

árbol binario: Un árbol en el que cada nodo apunta a cero, uno, o dos nodos dependientes.

raíz: El nodo superior de un árbol, sin padre.

hoja: Un nodo del extremo inferior de un árbol, sin hijos.

padre: El nodo que apunta a un nodo dado.

hijo: Uno de los nodos a los que apunta un nodo.

hermanos: Nodos que tienen un padre común.

nivel: El conjunto de nodos equidistante de la raíz.

operador binario: Un operador que toma dos operandos.

subexpresión: Una expresión entre paréntesis que actúa como un operando simple dentro de otra expresión mayor.

orden prefijo: Una forma de recorrer un árbol, visitando cada nodo antes que a sus hijos.

notación prefija: Una forma de escribir una expresión matemática en la que los operadores aparecen antes que sus operandos.

orden postfijo: Una forma de recorrer un árbol, visitando los hijos de cada nodo antes del propio nodo.

orden infijo: Una forma de recorrer un árbol, visitando el subárbol izquierdo, luego la raíz, y luego el subárbol derecho.

Apéndice A

Depuración

En un programa pueden suceder varios tipos de error, y resulta útil distinguirlos para localizarlos rápidamente:

- Python presenta errores de sintaxis mientras traduce el código fuente en código binario. Normalmente indican que hay algo erróneo en la sintaxis del programa. Ejemplo: omitir los dos puntos al final de una sentencia `def` nos da el mensaje `SyntaxError: invalid syntax`, algo redundante.
- El sistema de tiempo de ejecución presenta los errores en tiempo de ejecución si algo va mal mientras se ejecuta el programa. La mayoría de los mensajes de error en tiempo de ejecución incluyen información acerca de dónde sucedió el error y qué funciones se estaban ejecutando. Ejemplo: una recursión infinita termina por provocar un error en tiempo de ejecución del “maximum recursion depth exceeded” (superada la profundidad máxima de recursión).
- Los errores semánticos son problemas con un programa que compila y se ejecuta pero no hace lo que se espera de él. Ejemplo: una expresión puede no evaluarse en el orden esperado, dando un resultado inesperado.

El primer paso de la depuración es averiguar con qué tipo de error se enfrenta. Aunque las secciones que siguen están organizadas por tipos de error, algunas técnicas son aplicables en más de una situación.

A.1. Errores de sintaxis

Los errores de sintaxis suelen ser fáciles de arreglar una vez que averigua lo que son. Desgraciadamente, muchas veces los mensajes de error no son

muy útiles. Los mensajes más comunes son `SyntaxError: invalid syntax` y `SyntaxError: invalid token`, ninguno de los cuales es muy informativo.

Por otra parte, el mensaje le dice en qué lugar del programa sucedió el error. En realidad, le dice dónde notó el problema Python, que no es necesariamente donde está el error. A veces el error está antes de la localización del mensaje de error, muchas veces en la línea anterior.

Si está haciendo el programa incrementalmente, debería tener casi localizado el error. Estará en la última línea que añadió.

Si está usted copiando código de un libro, comience comparando con atención su código con el del libro. Compruebe cada carácter. Al mismo tiempo, recuerde que el libro podría estar equivocado, así que si ve algo que parezca un error de sintaxis, podría serlo.

He aquí algunas formas de evitar los errores de sintaxis más habituales:

1. Asegúrese de que no utiliza una palabra clave de Python como nombre de variable.
2. Compruebe que tiene los dos puntos al final de la cabecera de todas las sentencias compuestas, las `for`, `while`, `if`, y `def`.
3. Compruebe que el sangrado es consistente. Puede usted sangrar tanto con espacios como con tabuladores, pero es mejor no mezclarlos. Todos los niveles deberían estar anidados por la misma cantidad.
4. Asegúrese de que todas las cadenas del código tienen su par de comillas de apertura y cierre.
5. Si tiene cadenas que ocupan varias líneas con triples comillas (o triples apóstrofes), asegúrese de que ha terminado la cadena correctamente. Una cadena sin terminar puede provocar un error `invalid token` al final de su programa, o puede tratar la siguiente parte del programa como una cadena hasta que llegue a la siguiente cadena. ¡En el segundo caso, podría no presentar ningún mensaje de error!
6. Un paréntesis sin cerrar—`(`, `{` o `[`—hace que Python continúe con la línea siguiente como parte de la sentencia actual. Generalmente aparecerá un error casi inmediatamente en la línea siguiente.
7. Compruebe el clásico `=` donde debería haber un `==` en los condicionales.

Si nada funciona, siga con la sección que sigue...

A.1.1. No consigo ejecutar mi programa, no importa lo que haga.

Si el compilador dice que hay un error pero usted no lo ve, podría ser porque usted y el compilador no miran el mismo código. Compruebe su entorno de programación para asegurarse de que el programa que está editando es el que está intentando ejecutar Python. Si no está seguro, pruebe a poner un error de sintaxis obvio y deliberado al principio del programa. Ahora ejecute (o importe) de nuevo. Si el compilador no encuentra el nuevo error probablemente hay algo equivocado en el modo en que está configurado su entorno.

Si esto ocurre, puede enfrentarse a ello empezando de nuevo con un programa nuevo como “Hola, mundo”, y asegurarse de que puede hacer que funcione un programa conocido. Luego añada gradualmente los trozos del programa nuevo al que funciona.

A.2. Errores en tiempo de ejecución

Una vez que su programa es sintácticamente correcto, Python puede importarlo y al menos comenzar a ejecutarlo. ¿Qué podría ir mal?

A.2.1. Mi programa no hace nada de nada.

Este problema es muy común cuando su archivo consta de funciones y clases pero en realidad no invoca nada para que empiece la ejecución. Esto puede ser intencionado cuando sólo planea importar el módulo para suministrar clases y funciones.

Si no es intencionado, asegúrese de que está llamando a una función que inicie la ejecución, o ejecute una desde el indicador interactivo. Vea también la sección “Flujo de Ejecución” más adelante.

A.2.2. Mi programa se cuelga.

Si un programa se para y parece no hacer nada, decimos que “se ha colgado”. A menudo significa que se ha quedado atrapado en un bucle infinito o en una recursión infinita.

- Si hay un bucle en particular que le resulta sospechoso de provocar el problema, añada una sentencia `print` justo antes del bucle que diga “entrando al bucle” y otra inmediatamente después que diga “saliendo del bucle”.

Ejecute el programa. Si obtiene el primer mensaje pero el segundo no, tiene usted un bucle infinito. Vaya a la sección “Bucle Infinito” más adelante.

- Una recursión infinita casi siempre hará que el programa corra un rato y luego presente un error de “RuntimeError: Maximum recursion depth exceeded”. Si ocurre eso, vaya a la sección “Recursión Infinita” más adelante.

Si no ve este error pero sospecha que hay un problema con un método o función recursivos también puede utilizar las técnicas de la sección “Recursión Infinita”.

- Si no funciona ninguno de estos pasos, comience a probar otros bucles y otros métodos y funciones recursivos.
- Si eso no funciona, es posible que no comprenda el flujo de ejecución de su programa. Vaya a la sección “Flujo de Ejecución” más adelante.

Bucle Infinito

Si cree que tiene un bucle infinito y piensa que sabe qué bucle provoca el problema, añada una sentencia `print` que imprima los valores de las variables de la condición al final del bucle junto con el valor de la condición.

Por ejemplo:

```
while x > 0 and y < 0 :  
    # hacer algo con x  
    # hacer algo con y  
  
    print "x: ", x  
    print "y: ", y  
    print "condición: ", (x > 0 and y < 0)
```

Ahora, cuando ejecute el programa, verá tres líneas de salida en cada vuelta del bucle. En la última vuelta el valor de la condición debería ser `false`. Si el bucle sigue ejecutándose, podrá ver los valores de `x` e `y`, y podrá averiguar por qué no se actualizan correctamente.

Recursión Infinita

Una recursión infinita casi siempre hará que el programa se ejecute un rato y luego provoque un error de `Maximum recursion depth exceeded`.

Si sospecha que una función o un método está causando una recursión infinita, comience por asegurarse de que hay un caso básico. En otras palabras, debería

haber una condición que haga que la función devuelva un valor sin hacer otra llamada recursiva. Si no, necesita revisar el algoritmo y encontrar ese caso básico.

Si hay un caso básico pero el programa no parece llegar hasta él, añada una sentencia `print` que imprima los parámetros al principio de la función o método. Cuando ahora ejecute el programa, verá unas pocas líneas cada vez que se invoque la función o método y allí verá los parámetros. Si los parámetros no se acercan al caso básico, eso le dará alguna idea de por qué no lo hace.

Flujo de Ejecución

Si no está seguro de qué curso sigue el flujo de ejecución en su programa, añada sentencias `print` al principio de cada función con un mensaje como “entrando en la función `tururú`”, donde `tururú` es el nombre de la función.

Cuando ahora ejecute el programa, imprimirá una traza de cada función a medida que las vaya invocando.

A.2.3. Cuando ejecuto el programa recibo una excepción.

Si algo va mal durante la ejecución, Python imprime un mensaje que incluye el nombre de la excepción, la línea del programa donde sucedió el problema y una traza inversa.

La traza inversa identifica la función que se está ejecutando ahora y la función que invocó a ésta, y luego la función que invocó a *ésta*, y así sucesivamente. En otras palabras, traza la ruta de las llamadas a las funciones que le llevaron a donde se encuentra. También incluye los números de las líneas de sus archivos donde suceden todas esas llamadas.

El primer paso es examinar el lugar del programa donde sucede el error y ver si puede adivinar lo que sucedió. Estos son algunos de los errores en tiempo de ejecución más comunes:

NameError: Está intentando usar una variable que no existe en el entorno actual. Recuerde que las variables locales son locales. No puede hacer referencia a ellas desde fuera de la función en la que se definen.

TypeError: Hay varias causas posibles:

- Está intentando usar un varlor de forma inadecuada. Ejemplo: usar como índice para una cadena, lista o tupla algo que no es un entero.

- Hay una discrepancia entre los elementos de una cadena de formato y los elementos pasados para la conversión. Esto puede ocurrir tanto si el número de elementos no coincide como si se solicita una conversión no válida.
- Está pasando un número erróneo de argumentos a una función o método. Con los métodos, fíjese en la definición de los métodos y compruebe que el primer parámetro es `self`. Luego fíjese en la invocación del método; asegúrese de que está invocando el método sobre un objeto del tipo adecuado y dándole correctamente el resto de argumentos.

KeyError: Está tratando de acceder a un elemento de un diccionario con una clave que no está en el diccionario.

AttributeError: Está intentando acceder a un atributo o método que no existe.

IndexError: El índice que está usando para acceder a una lista, cadena o tupla es mayor que su longitud menos uno. Justo antes de donde aparece el error, añada una sentencia `print` que muestre el valor del índice y la longitud del vector. ¿Es correcto el tamaño del vector? ¿Tiene el índice un valor correcto?

A.2.4. Puse tantas sentencias `print` que me ahoga la salida.

Uno de los problemas de usar sentencias `print` para la depuración es que puede terminar enterrado en información. Hay dos formas de atajar el problema: simplificar la salida o simplificar el programa.

Para simplificar la salida, puede eliminar o comentar (convertir en comentarios) las sentencias `print` que no sean de ayuda, o combinarlas, o dar a la salida un formato que la haga más comprensible.

Para simplificar el programa puede hacer varias cosas. Primero, reducir la escala del problema en el que está trabajando el programa. Por ejemplo, si está ordenando un vector, ordene un vector *pequeño*. Si el programa acepta entradas del usuario, dele la entrada más simple que provoque el problema.

Segundo, limpie el programa. Elimine el código muerto y reorganice el programa para hacerlo tan legible como sea posible. Por ejemplo, si sospecha que el problema está en una parte del programa con un anidamiento muy profundo, pruebe a reescribir esa parte con una estructura más simple. Si sospecha de una función grande, trate de trocearla en funciones menores y pruébelas separadamente.

El proceso de encontrar el caso mínimo de prueba le llevará a menudo al error. Si se encuentra con que un programa funciona en una situación pero no en otra, eso le dará una pista sobre lo que ocurre.

De forma parecida, la reescritura de una porción de código puede ayudarle a encontrar errores sutiles. Si hace un cambio que a usted le parece que no afecta al programa, pero sí lo hace, le dará una pista.

A.3. Errores semánticos

En cierto modo, los errores semánticos son los más difíciles de corregir, porque el compilador y el sistema de ejecución no proporcionan información sobre lo que va mal. Sólo usted sabe lo que se supone que debe hacer el programa, y sólo usted sabe que no lo está haciendo.

El primer paso es hacer una conexión entre el texto del programa y el comportamiento que está usted viendo. Necesita una hipótesis sobre lo que realmente está haciendo el programa. Una de las dificultades que nos encontramos para ello es la alta velocidad de los computadores.

A menudo desearía ralentizar el programa a una velocidad humana, y con algunos programas depuradores podrá hacerlo. Pero el tiempo que lleva colocar unas sentencias `print` en los lugares adecuados suele ser menor que el que lleva configurar el depurador, poner y quitar puntos de interrupción y “hacer caminar” el programa hasta donde se produce el error.

A.3.1. Mi programa no funciona.

Debería hacerse estas preguntas:

- ¿Hay algo que se supone que debería hacer el programa pero que no parece suceder? Busque la sección del código que realiza esa función y asegúrese de que se ejecuta cuando debería.
- ¿Ocurre algo que no debería? Busque el programa que realiza esa función y vea si se ejecuta cuando no debe.
- ¿Hay una sección de código que causa un efecto que no esperaba? asegúrese de que entiende el código en cuestión, especialmente si incluye invocaciones de funciones o métodos de otros módulos de Python. Lea la documentación de las funciones que invoca. Pruébelas escribiendo casos de prueba simples y comprobando el resultado.

Para programar necesitará tener un modelo mental de cómo funcionan los programas. Si escribe un programa que no hace lo que espera de él, muchas veces el problema no estará en el programa, sino en su modelo mental.

La mejor manera de corregir su modelo mental es dividiendo el programa en sus componentes (normalmente las funciones y métodos) y probando cada componente de forma independiente. Una vez que encuentre la discrepancia entre su modelo y la realidad, podrá solucionar el problema.

Por supuesto, debería ir haciendo y probando componentes tal como desarrolla el programa. Si encuentra un problema, sólo habrá una pequeña cantidad de código nuevo del que no sabe si está correcto.

A.3.2. Tengo una expresión grande y peliaguda y no hace lo que espero.

Está bien escribir expresión complejas mientras sean legibles, pero pueden ser difíciles de depurar. Suele ser una buena idea dividir una expresión compleja en una serie de asignaciones de variables temporales.

Por ejemplo:

```
self.manos[i].agregaCarta (self.manos[\
    self.encuentraVecino(i)].darCarta())
```

Puede reescribirse como:

```
vecino = self.encuentraVecino (i)
cartaElegida = self.manos[vecino].darCarta()
self.manos[i].agregaCarta (cartaElegida)
```

La versión explícita es más fácil de leer porque los nombres de variable nos facilitan documentación adicional, y es más fácil de depurar porque puede comprobar los tipos de las variables intermedias y mostrar sus valores.

Otro problema que puede suceder con las expresiones grandes es que el orden de evaluación puede no ser el que usted esperaba. Por ejemplo, si está traduciendo la expresión $\frac{x}{2\pi}$ a Python, podría escribir:

```
y = x / 2 * math.pi;
```

Eso no es correcto, porque la multiplicación y la división tienen la misma precedencia y se evalúan de izquierda a derecha. Así que esa expresión calcula $x\pi/2$.

Una buena forma de depurar expresiones es añadir paréntesis para hacer explícito el orden de evaluación:

```
y = x / (2 * math.pi);
```

Siempre que no esté seguro del orden de evaluación, utilice paréntesis. El programa no sólo será correcto (en el sentido de hacer lo que usted pretendía), sino que además será más legible para otras personas que no hayan memorizado las reglas de precedencia.

A.3.3. Tengo una función o método que no devuelve lo que esperaba.

Si tiene una sentencia `return` con una expresión compleja no tendrá la oportunidad de imprimir el valor de retorno antes de volver. De nuevo, puede usar una variable temporal. Por ejemplo, en lugar de:

```
return self.manos[i].eliminaCoincidencias()
```

podría escribir:

```
cant = self.manos[i].eliminaCoincidencias()
return cant
```

Ahora ya tiene la oportunidad de mostrar el valor de `cant` antes de regresar.

A.3.4. Estoy atascado de verdad y necesito ayuda.

Primero, intente alejarse del computador durante unos minutos. Los computadores emiten unas ondas que afectan al cerebro provocando estos efectos:

- Frustración y/o furia.
- Creencias supersticiosas (“el computador me odia”) y pensamiento mágico (“el programa sólo funciona cuando me pongo la gorra hacia atrás”).
- Programar dando palos de ciego (el empeño de programar escribiendo todos los programas posibles y eligiendo el que hace lo correcto).

Si se encuentra afectado por alguno de estos síntomas, levántese y dé un paseo. Cuando esté calmado, piense en el programa. ¿Qué es lo que hace? ¿Cuáles pueden ser las causas de tal comportamiento? ¿Cuándo fue la última vez que tenía un programa que funcionaba y qué fue lo siguiente que hizo?

A veces lleva tiempo encontrar un error. Muchas veces encontramos errores cuando estamos lejos del computador y divagamos. Algunos de los mejores lugares para encontrar errores son los trenes, las duchas y la cama, justo antes de quedarse dormido.

A.3.5. No, de verdad necesito ayuda.

Sucede. Incluso los mejores programadores se atascan de vez en cuando. A veces trabaja durante tanto tiempo en un programa que no puede ver el error. Lo que necesita es un par de ojos nuevos.

Antes de llamar a andie, asegúrese de que ha agotado las técnicas explicadas aquí. Su programa debería ser tan simple como sea posible, y usted debería estar trabajando con la entrada mínima que provoca el error. Debería tener sentencias `print` en los lugares adecuados (y lo que dicen debería ser comprensible). Debería entender el problema lo bastante bien como para describirlo sucintamente.

Cuando llame a alguien para que le ayude, asegúrese de darles la información que necesitan:

- Si hay un mensaje de error, ¿cuál es y qué parte del programa señala?
- ¿Qué fue lo último que hizo antes de que apareciera el error? ¿Cuáles son las últimas líneas de código que escribió, o cuál es el nuevo caso de prueba que no cumple?
- ¿Qué ha intentado hasta ahora y qué ha averiguado?

Cuando encuentre el error, tómese un momento para pensar acerca de lo que podría haber hecho para encontrarlo más rápido. La siguiente vez que vea algo parecido, será capaz de encontrar el error antes.

Recuerde, el objetivo no es sólo hacer que el programa funcione. El objetivo es aprender cómo hacer funcionar al programa.

Apéndice B

Crear un nuevo tipo de datos

Los lenguajes de programación orientados a objetos permiten a los programadores crear nuevos tipos de datos que se comporten de manera muy parecida a los tipos de datos nativos. Exploraremos esta posibilidad construyendo una clase **Fraccion** que funcione de manera muy similar a los tipos numéricos nativos, enteros, enteros largos y flotantes.

Las fracciones, también conocidas como números racionales, son valores que pueden expresarse como la proporción entre dos números enteros, tal como $5/6$. Al número superior se le llama numerador y al inferior se le llama denominador.

Comenzamos definiendo la clase **Fraccion** con un método de inicialización que nos surta de un numerador y un denominador enteros:

```
class Fraccion:
    def __init__(self, numerador, denominador=1):
        self.numerador = numerador
        self.denominador = denominador
```

El denominador es opcional. Una **Fraccion** con un sólo parámetro representa un número entero. Si el numerador es n , construimos la fracción $n/1$.

El siguiente paso es escribir un método `__str__` para que imprima las fracciones de forma que tenga sentido. La forma natural de hacerlo es “numerador/denominador”:

```
class Fraccion:
    ...
    def __str__(self):
        return "%d/%d" % (self.numerador, self.denominador)
```

Para probar lo que tenemos hasta ahora, lo ponemos en un fichero llamado `Fraccion.py` y lo importamos desde el intérprete de Python. Entonces creamos un objeto fracción y lo imprimimos.

```
>>> from Fraccion import fraccion
>>> mortadela = Fraccion(5,6)
>>> print "La fracción es", mortadela
La fracción es 5/6
```

Como siempre, la función `print` invoca implícitamente al método `__str__`.

B.1. Multiplicación de fracciones

Nos gustaría poder aplicar las operaciones normales de suma, resta, multiplicación y división a las fracciones. Para ello, podemos sobrecargar los operadores matemáticos para los objetos de clase `Fraccion`.

Comenzaremos con la multiplicación porque es la más fácil de implementar. Para multiplicar dos fracciones, creamos una nueva fracción cuyo numerador es el producto de los numeradores de los operandos y cuyo denominador es el producto de los denominadores de los operandos. `__mul__` es el nombre que Python utiliza para el método que sobrecarga al operador `*`:

```
class Fraccion:
    ...
    def __mul__(self, otro):
        return Fraccion(self.numerador*otro.numerador,
                        self.denominador*otro.denominador)
```

Podemos probar este método calculando el producto de dos fracciones:

```
>>> print Fraccion(5,6) * Fraccion(3,4)
15/24
```

Funciona, pero ¡podemos hacerlo mejor! Podemos ampliar el método para manejar la multiplicación por un entero. Usamos la función `type` para ver si `otro` es un entero y convertirlo en una fracción en tal caso.

```
class Fraccion:
    ...
```

```
def __mul__(self, otro):
    if type(otro) == type(5):
        otro = Fraccion(otro)
    return Fraccion(self.numerador * otro.numerador,
                    self.denominador * otro.denominador)
```

Ahora funciona la multiplicación para fracciones y enteros, pero sólo si la fracción es el operando de la izquierda.

```
>>> print Fraccion(5,6) * 4
20/6
>>> print 4 * Fraccion(5,6)
TypeError: __mul__ nor __rmul__ defined for these operands
```

Para evaluar un operador binario como la multiplicación, Python comprueba primero el operando de la izquierda para ver si proporciona un método `__mul__` que soporte el tipo del segundo operando. En este caso, el operador nativo de multiplicación del entero no soporta fracciones.

Después, Python comprueba el segundo operando para ver si provee un método `__rmul__` que soporte el tipo del primer operando. En este caso, no hemos provisto el método `__rmul__`, por lo que falla.

Por otra parte, hay una forma sencilla de obtener `__rmul__`:

```
class Fraccion:
    ...
    __rmul__ = __mul__
```

Esta asignación hace que el método `__rmul__` sea el mismo que `__mul__`. Si ahora evaluamos `4 * Fraccion(5,6)`, Python llamará al método `__rmul__` del objeto `Fraccion` y le pasará 4 como parámetro:

```
>>> print 4 * Fraccion(5,6)
20/6
```

Dado que `__rmul__` es lo mismo que `__mul__`, y `__mul__` puede manejar un parámetro entero, ya está hecho.

B.2. Suma de fracciones

La suma es más complicada que la multiplicación, pero aún es llevadera. La suma de a/b y c/d es la fracción $(a*d+c*b)/b*d$.

Usando como modelo el código de la multiplicación, podemos escribir `__add__` y `__radd__`:

```
class Fraccion:
    ...
    def __add__(self, otro):
        if type(otro) == type(5):
            otro = Fraccion(otro)
        return Fraccion(self.numerador * otro.denominador +
                        self.denominador * otro.numerador,
                        self.denominador * otro.denominador)

    __radd__ = __add__
```

Podemos probar estos métodos con Fracciones y enteros.

```
>>> print Fraccion(5,6) + Fraccion(5,6)
60/36
>>> print Fraccion(5,6) + 3
23/6
>>> print 2 + Fraccion(5,6)
17/6
```

Los dos primeros ejemplos llaman a `__add__`; el último llama a `__radd__`.

B.3. Algoritmo de Euclides

En el ejemplo anterior, computamos la suma de $5/6 + 5/6$ y obtuvimos $60/36$. Es correcto, pero no es la mejor forma de representar la respuesta. Para **reducir** la fracción a su expresión más simple, hemos de dividir el numerador y el denominador por el **máximo común divisor (MCD)** de ambos, que es 12. El resultado sería $5/3$.

En general, siempre que creamos un nuevo objeto `Fraccion`, deberíamos reducirlo dividiendo el numerador y el denominador por el MCD de ambos. Si la fracción ya está reducida, el MCD es 1.

Euclides de Alejandría (aprox. 325–265 a. C.) presentó un algoritmo para encontrar el MCD de dos números enteros m y n :

Si n divide a m sin resto, entonces n es el MCD. De lo contrario, el MCD es el MCD de n y el resto de dividir m entre n .

Esta definición recursiva puede expresarse concisamente como una función:

```
def mcd (m, n):
    if m % n == 0:
```



```

    return n
else:
    return mcd(n, m%n)

```

En la primera línea del cuerpo, usamos el operador de módulo para comprobar la divisibilidad. En la última línea, lo usamos para calcular el resto de la división.

Dado que todas las operaciones que hemos escrito creaban un nuevo objeto `Fraccion` para devolver el resultado, podemos reducir todos los resultados modificando el método de inicialización.

```

class Fraccion:
    def __init__(self, numerador, denominador=1):
        m = mcd(numerador, denominador)
        self.numerador = numerador / m
        self.denominador = denominador / m

```

Ahora siempre que creamos una `Fraccion` quedará reducida a su forma canónica:

```

>>> Fraccion(100,-36)
-25/9

```

Una característica estupenda de `mcd` es que si la fracción es negativa, el signo menos siempre se trasladará al numerador.

B.4. Comparar fracciones

Supongamos que tenemos dos objetos `Fraccion`, `a` y `b`, y evaluamos `a == b`. La implementación por defecto de `==` comprueba la igualdad superficial, por lo que sólo devuelve `true` si `a` y `b` son el mismo objeto.

Queremos más bien devolver verdadero si `a` y `b` tienen el mismo valor —eso es, igualdad en profundidad.

Hemos de enseñar a las fracciones cómo compararse entre sí. Como vimos en la Sección 15.4, podemos sobrecargar todos los operadores de comparación de una vez proporcionando un método `__cmp__`.

Por convenio, el método `__cmp__` devuelve un número negativo si `self` es menor que `otro`, zero si son lo mismo, y un número positivo si `self` es mayor que `otro`.

La forma más simple de comparar dos fracciones es la multiplicación cruzada. Si $a/b > c/d$, entonces $ad > bc$. Con esto en mente, aquí está el código para `__cmp__`:

```
class Fraccion:
    ...
    def __cmp__(self, otro):
        dif = (self.numerador * otro.denominador -
               otro.numerador * self.denominador)
        return dif
```

Si `self` es mayor que `otro`, entonces `dif` será positivo. Si `otro` es mayor, entonces `dif` será negativo. Si son iguales, `dif` es cero.

B.5. Forzando la máquina

Por supuesto, aún no hemos terminado. Todavía hemos de implementar la resta sobrecargando `__sub__` y la división sobrecargando `__div__`.

Una manera de manejar estas operaciones es implementar la negación sobrecargando `__neg__` y la inversión sobrecargando `__invert__`. Entonces podemos restar negando el segundo operando y sumando, y podemos dividir invirtiendo el segundo operando y multiplicando.

Luego, hemos de suministrar los métodos `__rsub__` y `__rdiv__`. Desgraciadamente, no podemos usar el mismo truco que usamos para la suma y la multiplicación, porque la resta y la división no son conmutativas. No podemos igualar `__rsub__` y `__rdiv__` a `__sub__` y `__div__`. En estas operaciones, el orden de los operandos tiene importancia.

Para manejar la **negación unitaria**, que es el uso del signo menos con un único operando, sobrecargamos el método `__neg__`.

Podemos computar potencias sobrecargando `__pow__`, pero la implementación tiene truco. Si el exponente no es un número entero podría no ser posible representar el resultado como una `Fraccion`. Por ejemplo, `Fraccion(2) ** Fraccion(1,2)` es la raíz cuadrada de 2, que es un número irracional (no se puede representar como una fracción). Por lo tanto, no es fácil escribir la versión más general de `__pow__`.

Existe otra extensión a la clase `Fraccion` que cabría considerar. Hasta ahora, hemos asumido que el numerador y el denominador son enteros. Podríamos considerar la posibilidad de permitirles que sean enteros largos.

Como ejercicio, termine la implementación de la clase `Fraccion` de forma que pueda manejar resta, división, exponenciación y enteros largos como numerador y denominador.

B.6. Glosario

máximo común divisor (MCD): El mayor entero positivo que divide al numerador y al denominador de una fracción sin que quede un resto.

reducir: Cambiar la fracción a su forma equivalente con un MCD igual a 1.

negación unitaria: Operación que computa el elemento simétrico aditivo, normalmente denotada con un signo menos delante. Se denomina “unitaria” en contraste con la operación binaria menos, que es la resta.

Apéndice C

Listados Completos de Python

C.1. Clase Punto

```
class Punto:
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __str__(self):
        return '(' + str(self.x) + ', ' + str(self.y) + ')'

    def __add__(self, otro):
        return Punto(self.x + otro.x, self.y + otro.y)

    def __sub__(self, otro):
        return Punto(self.x - otro.x, self.y - otro.y)

    def __mul__(self, otro):
        return self.x * otro.x + self.y * otro.y

    def __rmul__(self, otro):
        return Punto(otro * self.x, otro * self.y)

    def reverse(self):
```

```
self.x, self.y = self.y, self.x

def delDerechoYDelReves(derecho):
    from copy import copy
    reves = copy(derecho)
    reves.reverse()
    print str(derecho) + str(reves)
```

C.2. Clase Hora

```
class Hora:
    def __init__(self, horas=0, minutos=0, segundos=0):
        self.horas = horas
        self.minutos = minutos
        self.segundos = segundos

    def __str__(self):
        return str(self.horas) + ":" + str(self.minutos) \
            + ":" + str(self.segundos)

    def convierteASegundos(self):
        minutos = self.horas * 60 + self.minutos
        segundos = self.minutos * 60 + self.segundos
        return segundos

    def incrementa(self, segs):
        segs = segs + self.segundos

        self.horas = self.horas + segs/3600
        segs = segs % 3600
        self.minutos = self.minutos + segs/60
        segs = segs % 60
        self.segundos = segs

    def haceHora(segs):
        hora = Hora()
        hora.horas = segs/3600
        segs = segs - hora.horas * 3600
        hora.minutos = segs/60
        segs = segs - hora.minutos * 60
```

```
hora.segundos = segs
return hora
```

C.3. Cartas, mazos y juegos

```
import random

class Carta:
    listaDePalos = ["Tréboles", "Diamantes", "Corazones",
                    "Picas"]
    listaDeValores = ["nada", "As", "2", "3", "4", "5", "6", "7",
                      "8", "9", "10", "Sota", "Reina", "Rey"]

    def __init__(self, palo=0, valor=0):
        self.palo = palo
        self.valor = valor

    def __str__(self):
        return (self.listaDeValores[self.valor] + " de " +\
                self.listaDePalos[self.palo])

    def __cmp__(self, otro):
        # controlar el palo
        if self.palo > otro.palo: return 1
        if self.palo < otro.palo: return -1
        # si son del mismo palo, controlar el valor
        if self.valor > otro.valor: return 1
        if self.valor < otro.valor: return -1
        # los valores son iguales, es un empate
        return 0

class Mazo:
    def __init__(self):
        self.cartas = []
        for palo in range(4):
            for valor in range(1, 14):
                self.cartas.append(Carta(palo, valor))

    def muestraMazo(self):
        for carta in self.cartas:
            print carta
```

```

def __str__(self):
    s = ""
    for i in range(len(self.cartas)):
        s = s + " " * i + str(self.cartas[i]) + "\n"
    return s

def mezclar(self):
    import random
    nCartas = len(self.cartas)
    for i in range(nCartas):
        j = random.randrange(i, nCartas)
        self.cartas[i], self.cartas[j] =\
            self.cartas[j], self.cartas[i]

def eliminaCarta(self, carta):
    if carta in self.cartas:
        self.cartas.remove(carta)
        return 1
    else: return 0

def darCarta(self):
    return self.cartas.pop()

def estaVacio(self):
    return (len(self.cartas) == 0)

def repartir(self, manos, nCartas=999):
    nManos = len(manos)
    for i in range(nCartas):
        if self.estaVacio(): break      # fin si se acaban las cartas
        carta = self.darCarta()         # da la carta superior
        mano = manos[i % nManos]       # a quién le toca?
        mano.agregaCarta(carta)         # agrega la carta a la mano

class Mano(Mazo):
    def __init__(self, nombre=""):
        self.cartas = []
        self.nombre = nombre

    def agregaCarta(self, carta) :
        self.cartas.append(carta)

```



```
def __str__(self):
    s = "La mano de " + self.nombre
    if self.estaVacio():
        s = s + " está vacía\n"
    else:
        s = s + " contiene\n"
    return s + Mazo.__str__(self)

class JuegoDeCartas:
    def __init__(self):
        self.mazo = Mazo()
        self.mazo.mezclar()

class ManoDeLaMona(Mano):
    def eliminaCoincidencias(self):
        cant = 0
        cartasOriginales = self.cartas[:]
        for carta in cartasOriginales:
            empareja = Carta(3 - carta.palo, carta.valor)
            if empareja in self.cartas:
                self.cartas.remove(carta)
                self.cartas.remove(empareja)
                print "Mano %s: %s con %s" % (self.nombre, carta, empareja)
                cant = cant + 1
        return cant

class JuegoDeLaMona(JuegoDeCartas):
    def jugar(self, nombres):
        # quitamos la Reina de Tréboles
        self.mazo.eliminaCarta(Carta(0,12))

        # construimos una mano para cada jugador
        self.manos = []
        for nombre in nombres :
            self.manos.append(ManoDeLaMona(nombre))

        # repartimos los naipes
        self.mazo.repartir(self.manos)
        print "----- Se han repartido las cartas."
        self.muestraManos()
```

```
# eliminamos las coincidencias iniciales
emparejadas = self.eliminaTodasLasCoincidencias()
print "----- Coincidencias eliminadas, el juego comienza."
self.muestraManos()

# se juega hasta que se han descartado las 50 cartas
turno = 0
cantManos = len(self.manos)
while emparejadas < 25:
    emparejadas = emparejadas + self.jugarUnTurno(turno)
    turno = (turno + 1) % cantManos

print "----- El juego terminó."
self.muestraManos()

def eliminaTodasLasCoincidencias(self):
    cant = 0
    for mano in self.manos:
        cant = cant + mano.eliminaCoincidencias()
    return cant

def jugarUnTurno(self, i):
    if self.manos[i].estaVacio():
        return 0
    vecino = self.encuentraVecino(i)
    cartaElegida = self.manos[vecino].darCarta()
    self.manos[i].agregaCarta(cartaElegida)
    print "Mano", self.manos[i].nombre, "eligió", cartaElegida
    cant = self.manos[i].eliminaCoincidencias()
    self.manos[i].mezclar()
    return cant

def encuentraVecino(self, i):
    cantManos = len(self.manos)
    for proximo in range(1, cantManos):
        vecino = (i + proximo) % cantManos
        if not self.manos[vecino].estaVacio():
            return vecino

def muestraManos(self) :
    for mano in self.manos :
        print mano
```

C.4. Lists Enlazadas

```
def imprimeLista(nodo):
    while nodo:
        print nodo,
        nodo = nodo.siguiente
    print

def imprimeAlReves(lista):
    if lista == None: return
    cabeza = lista
    cola = lista.siguiente
    imprimeAlReves(cola)
    print cabeza,

def imprimeAlRevesBonito(lista) :
    print "[",
    if lista != None :
        cabeza = lista
        cola = lista.siguiente
        imprimeAlReves(cola)
    print cabeza,
    print "]",

def eliminaSegundo(lista):
    if lista == None: return
    primero = lista
    segundo = lista.siguiente
    primero.siguiente = segundo.siguiente
    segundo.siguiente = None
    return segundo

class Nodo:
    def __init__(self, carga=None, siguiente=None):
        self.carga = carga
        self.siguiente = siguiente

    def __str__(self):
        return str(self.carga)
```

```
def imprimeAlReves(self):
    if self.siguiente != None:
        cola = self.siguiente
        cola.imprimeAlReves()
    print self.carga,

class ListaEnlazada :
    def __init__(self) :
        self.longitud = 0
        self.cabeza = None

    def imprimeAlReves(self):
        print "[",
        if self.cabeza != None:
            self.cabeza.imprimeAlReves()
        print "]",

    def agregaPrimero(self, carga):
        nodo = Nodo(carga)
        nodo.siguiente = self.cabeza
        self.cabeza = nodo
        self.longitud = self.longitud + 1
```

C.5. Clase Pila

```
class Pila :                                # implem. con listas de Python
    def __init__(self) :
        self.elementos = []

    def push(self, elemento) :
        self.elementos.append(elemento)

    def pop(self) :
        return self.elementos.pop()

    def isEmpty(self) :
        return (self.elementos == [])

    def evalPostfijo(expr):
```

```
import re
listaTokens = re.split("([0-9])", expr)
pila = Pila()
for token in listaTokens:
    if token == '' or token == ' ':
        continue
    if token == '+':
        suma = pila.pop() + pila.pop()
        pila.push(suma)
    elif token == '*':
        producto = pila.pop() * pila.pop()
        pila.push(producto)
    else:
        pila.push(int(token))
return pila.pop()
```

C.6. Colas y colas priorizadas

```
class Cola :
    def __init__(self) :
        self.longitud = 0
        self.cabeza = None

    def vacia(self) :
        return (self.longitud == 0)

    def inserta(self, carga) :
        nodo = Nodo(carga)
        nodo.siguiente = None
        if self.cabeza == None :
            # Si la lista está vacía nuestro nuevo nodo es el primero
            self.cabeza = nodo
        else :
            # Encuentra el último nodo de la lista
            ultimo = self.cabeza
            while ultimo.siguiente : ultimo = ultimo.siguiente
            # Añada el nuevo nodo
            ultimo.siguiente = nodo
        self.longitud = self.longitud + 1
```

```
def quita(self) :
    carga = self.cabeza.carga
    self.cabeza = self.cabeza.next
    self.longitud = self.longitud - 1
    return carga

class ColaMejorada :
    def __init__(self) :
        self.longitud = 0
        self.cabeza = None
        self.ultimo = None

    def vacia(self) :
        return (self.longitud == 0)

    def inserta(self, carga) :
        nodo = Nodo(carga)
        nodo.siguiente = None
        if self.longitud == 0 :
            # Si la lista está vacía nuestro nuevo nodo es el primero
            self.cabeza = self.ultimo = nodo
        else :
            # Encuentra el ultimo nodo de la lista
            ultimo = self.ultimo
            # Añade nuestro nodo nuevo
            ultimo.siguiente = nodo
            self.ultimo = nodo
        self.longitud = self.longitud + 1

    def quita(self) :
        carga = self.cabeza.carga
        self.cabeza = self.cabeza.siguiente
        self.longitud = self.longitud - 1
        if self.longitud == 0 : self.ultimo = None
        return carga

class ColaPriorizada :
    def __init__(self) :
        self.elementos = []

    def vacia(self) :
```

```
    return self.elementos == []

def inserta(self, elemento) :
    self.elementos.append(elemento)

def quita(self) :
    maxi = 0
    for i in range(1,len(self.elementos)) :
        if self.elementos[i] > self.elementos[maxi] :
            maxi = i
    elemento = self.elementos[maxi]
    self.elementos[maxi:maxi+1] = []
    return elemento

class Golfista :
    def __init__(self, nombre, puntos) :
        self.nombre = nombre
        self.puntos = puntos

    def __str__(self) :
        return "%-15s: %d" % (self.nombre, self.puntos)

    def __cmp__(self, otro) :
        if self.puntos < otro.puntos : return 1    # menos es más
        if self.puntos > otro.puntos : return -1
        return 0
```

C.7. Árboles

```
class Arbol :
    def __init__(self, carga, izquierda=None, derecha=None) :
        self.carga = carga
        self.izquierda = izquierda
        self.derecha = derecha

    def __str__(self) :
        return str(self.carga)

    def tomaCarga(self): return self.carga
    def tomaIzquierda(self): return self.izquierda
```

```
def tomaDerecha(self): return self.derecha

def ajustaCarga(self, carga): self.carga = carga
def ajustaIzquierda(self, izquierda): self.left = izquierda
def ajustaDerecha(self, derecha): self.derecha = derecha

def total(arbol) :
    if arbol == None : return 0
    return total(arbol.izquierda) + total(arbol.derecha) + arbol.carga

def imprimeArbol(arbol):
    if arbol == None: return
    print arbol.carga,
    imprimeArbol(arbol.izquierda)
    imprimeArbol(arbol.derecha)

def imprimeArbolPosfijo(arbol):
    if arbol == None: return
    imprimeArbolPosfijo(arbol.izquierda)
    imprimeArbolPosfijo(arbol.derecha)
    print arbol.carga,

def imprimeArbolInfijo(arbol):
    if arbol == None: return
    imprimeArbolInfijo(arbol.izquierda)
    print arbol.carga,
    imprimeArbolInfijo(arbol.derecha)

def imprimeArbolSangrado(arbol, nivel=0):
    if arbol == None: return
    imprimeArbolSangrado(arbol.derecha, nivel+1)
    print ' '*nivel + str(arbol.carga)
    imprimeArbolSangrado(arbol.izquierda, nivel+1)
```

C.8. Árboles de expresión

```
def tomaToken(listaToken, esperado):
    if listaToken[0] == esperado:
        listaToken[0:1] = [] # quita el token
        return 1
    else:
```



```
    return 0

def obtieneProducto(listaToken) :
    a = obtieneNumero(listaToken)
    if tomaToken(listaToken, '*') :
        b = obtieneProducto(listaToken)
        return Arbol('*', a, b)
    else :
        return a

def obtieneSuma(listaToken) :
    a = obtieneProducto(listaToken)
    if tomaToken(listaToken, '+') :
        b = obtieneSuma(listaToken)
        return Arbol('+', a, b)
    else :
        return a

def obtieneNumero(listaToken):
    if tomaToken(listaToken, '(') :
        x = obtieneSuma(listaToken)      # obtiene subexpresión
        tomaToken(listaToken, ')')      # se come el cierre de paréntesis
        return x
    else :
        x = listaToken[0]
        if type(x) != type(0) : return None
        listaToken[0:1] = []           # quita el token
        return Arbol(x, None, None)     # devuelve una hoja sin el número
```

C.9. Adivina el animal

```
def animal():
    # empezar con un nodo suelto
    raiz = Arbol("pájaro")

    # bucle hasta que el usuario salga
    while 1:
        print
        if not si("Estás pensando en un animal? "): break

    # recorrer el árbol
```

```

arbol = raiz
while arbol.tomaIzquierda() != None:
    indicador = arbol.tomaCarga() + "? "
    if si(indicador):
        arbol = arbol.tomaDerecha()
    else:
        arbol = arbol.tomaIzquierda()

# intentar adivinar
adivina = arbol.tomaCarga()
indicador = "Es un " + adivina + "? "
if si(indicador):
    print "¡Soy el más grande!"
    continue

# obtener información nueva
indicador = "Cómo se llama el animal? "
animal = raw_input(indicador)
indicador = "Qué pregunta distinguiría a un %s de un %s? "
pregunta = raw_input(indicador % (animal, adivina))

# añadir información nueva al árbol
arbol.ponCarga(pregunta)
indicador = "Si el animal fuera un %s, cuál sería la respuesta? "
if si(indicador % animal):
    arbol.ponIzquierda(Arbol(adivina))
    arbol.ponDerecha(Arbol(animal))
else:
    arbol.ponIzquierda(Arbol(animal))
    arbol.ponDerecha(Arbol(adivina))

def si(preg):
    from string import lower
    resp = lower(raw_input(preg))
    return (resp[0:1] == 's')

```

C.10. Fraction class

```

class Fraccion:
    def __init__(self, numerador, denominador=1):
        m = mcd (numerador, denominador)

```

```
self.numerador = numerador / m
self.denominador = denominador / m

def __mul__(self, otro):
    if type(otro) == type(5):
        otro = Fraccion(otro)
    return Fraccion(self.numerador * otro.numerador,
                    self.denominador * otro.denominador)

__rmul__ = __mul__

def __add__(self, otro):
    if type(otro) == type(5):
        otro = Fraccion(otro)
    return Fraccion(self.numerador * otro.denominador +
                    self.denominador * otro.numerador,
                    self.denominador * otro.denominador)

__radd__ = __add__

def __cmp__(self, otro):
    if type(otro) == type(5):
        otro = Fraccion(otro)

    dif = (self.numerador * otro.denominador -
           otro.numerador * self.denominador)
    return dif

def __repr__(self):
    return self.__str__()

def __str__(self):
    return "%d/%d" % (self.numerador, self.denominador)

def mcd(m,n):
    "devuelve el máximo común denominador de dos enteros"
    if m % n == 0:
        return n
    else:
        return mcd(n,m%n)
```


Apéndice D

Lecturas recomendadas

Y ahora, ¿hacia dónde ir desde aquí? Hay muchas direcciones en las que seguir, ampliando sus conocimientos de Python específicamente y de informática en general.

Los ejemplos en este libro han sido deliberadamente simples, por lo que pueden no haber mostrado las capacidades más excitantes de Python. A continuación exponemos una muestra de las extensiones de Python y sugerencias sobre sus usos.

- La programación de GUIs (interfaces gráficas de usuario, *graphic user interface* en inglés) permite que su programa utilice un entorno de ventanas para interactuar con el usuario y mostrar gráficos.

El primer paquete que ha tenido Python para esto es Tkinter, basado en los lenguajes interpretados Tcl y Tk de Jon Ousterhout. Tkinter está incluido en la distribución de Python.

Otra plataforma popular es wxPython, que es esencialmente un enchapado sobre wxWindows, un paquete de C++ que implementa ventanas utilizando la interfaces nativas las plataformas Windows y Unix (incluido Linux). Las ventanas y los controles con wxPython tienen una apariencia más nativa que Tkinter y son un poco más sencillos de programar.

Cualquier tipo de programación de GUIs le llevará a programación basada en eventos, donde es el usuario y no el programador quien determina el flujo de la ejecución. Este estilo de programación requiere de algo de tiempo para acostumbrarse, y a veces le forzará a replantearse toda la estructura del programa.

- La programación web integra Python en la Internet. Por ejemplo, puede construir programas de cliente web que abran y lean una página remota (casi) tan fácilmente como si fuera un fichero en disco. También hay módulos de Python que le permiten acceder a ficheros remotamente vía ftp, y módulos que le permiten enviar y recibir correos electrónicos. Python también es ampliamente utilizado en el lado del servidor de la programación web para manejar los datos de entrada de los formularios.
- Las bases de datos son un poco como super ficheros en donde los datos están almacenados en esquemas predefinidos, y las relaciones entre los datos le permiten acceder a ellos de varias maneras. Python tiene varios módulos para permitir a los usuarios conectarse a varios motores de bases de datos, tanto Open Source como comerciales.
- La programación multi-procesos (multi-hilos) le permite ejecutar varios procesos (hilos) de ejecución dentro de un único programa. Si ha tenido la experiencia de usar un navegador web para desplazarse por una página web mientras el navegador continúa cargando el resto de la misma, entonces tiene una idea de lo que los hilos pueden hacer.
- Cuando la velocidad es más importante se pueden escribir extensiones para Python en un lenguaje compilado como C o C++. Tales extensiones forman la base de la mayoría de módulos en la librería de Python. El mecanismo de enlazar funciones y datos es un poco complejo. SWIG (Simplified Wrapper and Interface Generator) es una herramienta para hacer este proceso mucho más sencillo.

D.1. Libros y sitios web sobre Python

Aquí tiene las recomendaciones de los autores sobre recursos para Python en la web:

- La página de inicio de Python en www.python.org es el lugar para empezar su búsqueda de material sobre Python. Encontrará ayuda, documentación, enlaces a otros libros y listas de correo de SIGs (Special Interest Group) a las que se puede unir.
- El proyecto Open Book Project www.ibiblio.com/obp contiene no sólo este libro en línea sino también otros libros similares para Java y C++ de Allen Downey. Además está *Lessons in Electric Circuits* de Tony R. Kuphaldt, *Getting down with ...*, un conjunto de tutoriales de varios temas sobre informática, escritos y editados por estudiantes de instituto, *Python for Fun*, un conjunto de estudios de casos en Python de Chris Meyers, y *The Linux Cookbook* de Michael Stultz, con 300 páginas de trucos y técnicas.

- Finalmente si acude a Google y busca con la cadena “python -snake -monty” obtendrá cerca de 750.000 resultados.

Y aquí algunos libros que contienen más material sobre el lenguaje Python:

- *Core Python Programming* de Wesley Chun es un libro largo, más de 750 páginas. La primera parte del libro cubre las características básicas del lenguaje Python. La segunda parte proporciona una introducción paso a paso a temas más avanzados incluyendo muchos de los mencionados anteriormente.
- *Python Essential Reference* de David M. Beazley es un libro pequeño, pero contiene información sobre el lenguaje en sí mismo y los módulos de la librería estándar. También está muy bien indexado.
- *Python Pocket Reference* de Mark Lutz realmente cabe en el bolsillo. Aunque no es tan extensivo como *Python Essential Reference* es una referencia útil para los módulos y funciones más comunmente usadas. Mark Lutz también es autor de *Programming Python*, uno de los primeros (y más largos) libros de Python y no está dirigido al programador principiante. Su siguiente libro *Learning Python* es más pequeño y más accesible.
- *Python Programming on Win32* de Mark Hammond y Andy Robinson es un libro que “debe tener” cualquiera que que utilice seriamente Python para desarrollar aplicaciones para Windows. Entre otras cosas cubre la integración de Python y COM, construye una pequeña aplicación con wxPython, e incluso utiliza Python para escribir scripts para aplicaciones tales como Word y Excel.

D.2. Libros recomendados sobre informática en general

Las siguientes sugerencias sobre lecturas adicionales incluyen muchos de los libros favoritos de los autores. Estos tratan sobre buenas prácticas de programación e informática en general.

- *The Practice of Programming* de Kernighan y Pike cubre no sólo el diseño y modificación de algoritmos y estructuras de datos, sino también depuración, testeo y mejora de rendimiento de los programas. Los ejemplos están principalmente en C++ y Java, sin nada de Python.

- *The Elements of Java Style* editado por Al Vermeulen es otro libro pequeño que discute algunos de los puntos más sutiles de la buena programación, tales como el buen uso de las convenciones de nombres, comentarios e indentación (un poco irrelevante en Python). El libro también cubre la programación por contrato, usando aserciones para encontrar los errores probando precondiciones y postcondiciones, y programación correcta con hilos y su sincronización.
- *Programming Pearls* de Jon Bentley es un libro clásico. Consiste en estudios de caso que aparecieron originalmente en la columna del autor en *Communications of the ACM*. Los estudios tratan sobre toma y daca en programación y por qué suele ser mala idea desarrollar con la primera idea de un programa. El libro es un poco más antiguo que los anteriores (1986), por lo que los ejemplos están en lenguajes más antiguos. Hay muchos problemas para resolver, algunos con soluciones y otros con pistas. Este libro fue muy popular y le siguió un segundo volumen.
- *The New Turing Omnibus* de A.K. Dewdney proporciona una introducción amigable a 66 temas de informática desde computación en paralelo hasta virus informáticos, desde TACs (tomografías computerizadas) hasta algoritmos genéticos. Todos los temas son cortos y entretenidos. Un libro anterior de Dewdney *Aventuras Informáticas* es una colección de su columna *Juegos de ordenador en Investigación y Ciencia*. Ambos libros son ricas fuentes de ideas para proyectos.
- *Tortugas, Termitas y Atascos de Tráfico* de Mitchel Resnick trata sobre el poder de la descentralización y de como pueden obtenerse comportamientos complejos a partir de las actividades simples de una multitud de agentes coordinados. Introduce el lenguaje StarLogo, que permite al usuario escribir programas para agentes. La ejecución del programa demuestra comportamientos complejos agregados, que suelen ser intuitivos. La mayoría de los programas en el libro fueron desarrollados por estudiantes de colegio e instituto. Programas similares pueden escribirse en Python usando gráficos e hilos.
- *Gödel, Escher, Bach* de Douglas Hofstadter. Simplemente, si encuentra magia en la recursión también la encontrará en este libro superventas. Uno de los temas de Hofstadter concierne a los “lazos extraños” donde los patrones se desenvuelven y ascienden hasta que se encuentran a sí mismos de nuevo. Es una disputa de Hofstadter que tales “lazos extraños” son una parte esencial de lo que separa lo animado de lo no animado. Él demuestra tales patrones en la música de Bach, las ilustraciones de Escher y el teorema de incompletitud de Gödel.

Apéndice E

GNU Free Documentation License

Version 1.1, March 2000

Copyright © 2000 Free Software Foundation, Inc.

59 Temple Place, Suite 330, Boston, MA 02111-1307 USA

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

Preamble

The purpose of this License is to make a manual, textbook, or other written document “free” in the sense of freedom: to assure everyone the effective freedom to copy and redistribute it, with or without modifying it, either commercially or noncommercially. Secondly, this License preserves for the author and publisher a way to get credit for their work, while not being considered responsible for modifications made by others.

This License is a kind of “copyleft,” which means that derivative works of the document must themselves be free in the same sense. It complements the GNU General Public License, which is a copyleft license designed for free software.

We have designed this License in order to use it for manuals for free software, because free software needs free documentation: a free program should come with manuals providing the same freedoms that the software does. But this

License is not limited to software manuals; it can be used for any textual work, regardless of subject matter or whether it is published as a printed book. We recommend this License principally for works whose purpose is instruction or reference.

E.1. Applicability and Definitions

This License applies to any manual or other work that contains a notice placed by the copyright holder saying it can be distributed under the terms of this License. The “Document,” below, refers to any such manual or work. Any member of the public is a licensee, and is addressed as “you.”

A “Modified Version” of the Document means any work containing the Document or a portion of it, either copied verbatim, or with modifications and/or translated into another language.

A “Secondary Section” is a named appendix or a front-matter section of the Document that deals exclusively with the relationship of the publishers or authors of the Document to the Document’s overall subject (or to related matters) and contains nothing that could fall directly within that overall subject. (For example, if the Document is in part a textbook of mathematics, a Secondary Section may not explain any mathematics.) The relationship could be a matter of historical connection with the subject or with related matters, or of legal, commercial, philosophical, ethical, or political position regarding them.

The “Invariant Sections” are certain Secondary Sections whose titles are designated, as being those of Invariant Sections, in the notice that says that the Document is released under this License.

The “Cover Texts” are certain short passages of text that are listed, as Front-Cover Texts or Back-Cover Texts, in the notice that says that the Document is released under this License.

A “Transparent” copy of the Document means a machine-readable copy, represented in a format whose specification is available to the general public, whose contents can be viewed and edited directly and straightforwardly with generic text editors or (for images composed of pixels) generic paint programs or (for drawings) some widely available drawing editor, and that is suitable for input to text formatters or for automatic translation to a variety of formats suitable for input to text formatters. A copy made in an otherwise Transparent file format whose markup has been designed to thwart or discourage subsequent modification by readers is not Transparent. A copy that is not “Transparent” is called “Opaque.”

Examples of suitable formats for Transparent copies include plain ASCII without markup, Texinfo input format, L^AT_EX input format, SGML or XML using a publicly available DTD, and standard-conforming simple HTML designed for human modification. Opaque formats include PostScript, PDF, proprietary formats that can be read and edited only by proprietary word processors, SGML or XML for which the DTD and/or processing tools are not generally available, and the machine-generated HTML produced by some word processors for output purposes only.

The “Title Page” means, for a printed book, the title page itself, plus such following pages as are needed to hold, legibly, the material this License requires to appear in the title page. For works in formats which do not have any title page as such, “Title Page” means the text near the most prominent appearance of the work’s title, preceding the beginning of the body of the text.

E.2. Verbatim Copying

You may copy and distribute the Document in any medium, either commercially or noncommercially, provided that this License, the copyright notices, and the license notice saying this License applies to the Document are reproduced in all copies, and that you add no other conditions whatsoever to those of this License. You may not use technical measures to obstruct or control the reading or further copying of the copies you make or distribute. However, you may accept compensation in exchange for copies. If you distribute a large enough number of copies you must also follow the conditions in Section 3.

You may also lend copies, under the same conditions stated above, and you may publicly display copies.

E.3. Copying in Quantity

If you publish printed copies of the Document numbering more than 100, and the Document’s license notice requires Cover Texts, you must enclose the copies in covers that carry, clearly and legibly, all these Cover Texts: Front-Cover Texts on the front cover, and Back-Cover Texts on the back cover. Both covers must also clearly and legibly identify you as the publisher of these copies. The front cover must present the full title with all words of the title equally prominent and visible. You may add other material on the covers in addition. Copying with changes limited to the covers, as long as they preserve the title of the Document and satisfy these conditions, can be treated as verbatim copying in other respects.

If the required texts for either cover are too voluminous to fit legibly, you should put the first ones listed (as many as fit reasonably) on the actual cover, and continue the rest onto adjacent pages.

If you publish or distribute Opaque copies of the Document numbering more than 100, you must either include a machine-readable Transparent copy along with each Opaque copy, or state in or with each Opaque copy a publicly accessible computer-network location containing a complete Transparent copy of the Document, free of added material, which the general network-using public has access to download anonymously at no charge using public-standard network protocols. If you use the latter option, you must take reasonably prudent steps, when you begin distribution of Opaque copies in quantity, to ensure that this Transparent copy will remain thus accessible at the stated location until at least one year after the last time you distribute an Opaque copy (directly or through your agents or retailers) of that edition to the public.

It is requested, but not required, that you contact the authors of the Document well before redistributing any large number of copies, to give them a chance to provide you with an updated version of the Document.

E.4. Modifications

You may copy and distribute a Modified Version of the Document under the conditions of Sections 2 and 3 above, provided that you release the Modified Version under precisely this License, with the Modified Version filling the role of the Document, thus licensing distribution and modification of the Modified Version to whoever possesses a copy of it. In addition, you must do these things in the Modified Version:

- Use in the Title Page (and on the covers, if any) a title distinct from that of the Document, and from those of previous versions (which should, if there were any, be listed in the History section of the Document). You may use the same title as a previous version if the original publisher of that version gives permission.
- List on the Title Page, as authors, one or more persons or entities responsible for authorship of the modifications in the Modified Version, together with at least five of the principal authors of the Document (all of its principal authors, if it has less than five).
- State on the Title page the name of the publisher of the Modified Version, as the publisher.
- Preserve all the copyright notices of the Document.

- Add an appropriate copyright notice for your modifications adjacent to the other copyright notices.
- Include, immediately after the copyright notices, a license notice giving the public permission to use the Modified Version under the terms of this License, in the form shown in the Addendum below.
- Preserve in that license notice the full lists of Invariant Sections and required Cover Texts given in the Document's license notice.
- Include an unaltered copy of this License.
- Preserve the section entitled "History," and its title, and add to it an item stating at least the title, year, new authors, and publisher of the Modified Version as given on the Title Page. If there is no section entitled "History" in the Document, create one stating the title, year, authors, and publisher of the Document as given on its Title Page, then add an item describing the Modified Version as stated in the previous sentence.
- Preserve the network location, if any, given in the Document for public access to a Transparent copy of the Document, and likewise the network locations given in the Document for previous versions it was based on. These may be placed in the "History" section. You may omit a network location for a work that was published at least four years before the Document itself, or if the original publisher of the version it refers to gives permission.
- In any section entitled "Acknowledgements" or "Dedications," preserve the section's title, and preserve in the section all the substance and tone of each of the contributor acknowledgements and/or dedications given therein.
- Preserve all the Invariant Sections of the Document, unaltered in their text and in their titles. Section numbers or the equivalent are not considered part of the section titles.
- Delete any section entitled "Endorsements." Such a section may not be included in the Modified Version.
- Do not retile any existing section as "Endorsements" or to conflict in title with any Invariant Section.

If the Modified Version includes new front-matter sections or appendices that qualify as Secondary Sections and contain no material copied from the Document, you may at your option designate some or all of these sections as invariant.

To do this, add their titles to the list of Invariant Sections in the Modified Version's license notice. These titles must be distinct from any other section titles.

You may add a section entitled “Endorsements,” provided it contains nothing but endorsements of your Modified Version by various parties—for example, statements of peer review or that the text has been approved by an organization as the authoritative definition of a standard.

You may add a passage of up to five words as a Front-Cover Text, and a passage of up to 25 words as a Back-Cover Text, to the end of the list of Cover Texts in the Modified Version. Only one passage of Front-Cover Text and one of Back-Cover Text may be added by (or through arrangements made by) any one entity. If the Document already includes a cover text for the same cover, previously added by you or by arrangement made by the same entity you are acting on behalf of, you may not add another; but you may replace the old one, on explicit permission from the previous publisher that added the old one.

The author(s) and publisher(s) of the Document do not by this License give permission to use their names for publicity for or to assert or imply endorsement of any Modified Version.

E.5. Combining Documents

You may combine the Document with other documents released under this License, under the terms defined in Section 4 above for modified versions, provided that you include in the combination all of the Invariant Sections of all of the original documents, unmodified, and list them all as Invariant Sections of your combined work in its license notice.

The combined work need only contain one copy of this License, and multiple identical Invariant Sections may be replaced with a single copy. If there are multiple Invariant Sections with the same name but different contents, make the title of each such section unique by adding at the end of it, in parentheses, the name of the original author or publisher of that section if known, or else a unique number. Make the same adjustment to the section titles in the list of Invariant Sections in the license notice of the combined work.

In the combination, you must combine any sections entitled “History” in the various original documents, forming one section entitled “History”; likewise combine any sections entitled “Acknowledgements,” and any sections entitled “Dedications.” You must delete all sections entitled “Endorsements.”

E.6. Collections of Documents

You may make a collection consisting of the Document and other documents released under this License, and replace the individual copies of this License in the various documents with a single copy that is included in the collection, provided that you follow the rules of this License for verbatim copying of each of the documents in all other respects.

You may extract a single document from such a collection, and distribute it individually under this License, provided you insert a copy of this License into the extracted document, and follow this License in all other respects regarding verbatim copying of that document.

E.7. Aggregation with Independent Works

A compilation of the Document or its derivatives with other separate and independent documents or works, in or on a volume of a storage or distribution medium, does not as a whole count as a Modified Version of the Document, provided no compilation copyright is claimed for the compilation. Such a compilation is called an “aggregate,” and this License does not apply to the other self-contained works thus compiled with the Document, on account of their being thus compiled, if they are not themselves derivative works of the Document.

If the Cover Text requirement of Section 3 is applicable to these copies of the Document, then if the Document is less than one quarter of the entire aggregate, the Document’s Cover Texts may be placed on covers that surround only the Document within the aggregate. Otherwise they must appear on covers around the whole aggregate.

E.8. Translation

Translation is considered a kind of modification, so you may distribute translations of the Document under the terms of Section 4. Replacing Invariant Sections with translations requires special permission from their copyright holders, but you may include translations of some or all Invariant Sections in addition to the original versions of these Invariant Sections. You may include a translation of this License provided that you also include the original English version of this License. In case of a disagreement between the translation and the original English version of this License, the original English version will prevail.

E.9. Termination

You may not copy, modify, sublicense, or distribute the Document except as expressly provided for under this License. Any other attempt to copy, modify, sublicense, or distribute the Document is void, and will automatically terminate your rights under this License. However, parties who have received copies, or rights, from you under this License will not have their licenses terminated so long as such parties remain in full compliance.

E.10. Future Revisions of This License

The Free Software Foundation may publish new, revised versions of the GNU Free Documentation License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns. See <http://www.gnu.org/copyleft/>.

Each version of the License is given a distinguishing version number. If the Document specifies that a particular numbered version of this License “or any later version” applies to it, you have the option of following the terms and conditions either of that specified version or of any later version that has been published (not as a draft) by the Free Software Foundation. If the Document does not specify a version number of this License, you may choose any version ever published (not as a draft) by the Free Software Foundation.

E.11. Addendum: How to Use This License for Your Documents

To use this License in a document you have written, include a copy of the License in the document and put the following copyright and license notices just after the title page:

Copyright © YEAR YOUR NAME. Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.1 or any later version published by the Free Software Foundation; with the Invariant Sections being LIST THEIR TITLES, with the Front-Cover Texts being LIST, and with the Back-Cover Texts being LIST. A copy of the license is included in the section entitled “GNU Free Documentation License.”

If you have no Invariant Sections, write “with no Invariant Sections” instead of saying which ones are invariant. If you have no Front-Cover Texts, write “no Front-Cover Texts” instead of “Front-Cover Texts being LIST”; likewise for Back-Cover Texts.

If your document contains nontrivial examples of program code, we recommend releasing these examples in parallel under your choice of free software license, such as the GNU General Public License, to permit their use in free software.

Índice alfabético

- árbol, 211
 - expresión, 213, 216
 - recorrido, 213, 214
 - vacío, 212
- árbol binario, 211, 224
- árbol de expresión, 213, 216
- índice, 76, 84, 97, 109, 229
 - negativo, 76
- Make Way for Ducklings*, 77
- Referencia de la Biblioteca de Python*, 83
- , 33
- abecedario, 77
- acceso, 86
- acto de fe, 55, 186
- acumulador, 167, 170, 178
- aleatorio, 167
- algoritmo, 10, 146, 148
- alias, 92, 97
- ambigüedad, 7, 134
 - teorema fundamental, 188
- análisis sintáctico, 10
- analizar, 199, 216
- analizar sintácticamente, 7, 201
- andamiaje, 48, 59
- anidamiento, 45
- archivo, 128
 - texto, 121
- archivo de texto, 121, 128
- archivos, 119
- argumento, 21, 33
- argumentos, 28
- asignación, 12, 20, 61
 - tupla, 168
 - tuplas, 100
- asignación de alias, 112
- asignación de tuplas, 100, 107, 168
- asignación múltiple, 61, 73
- asignación
 - tupla, 107
- atributo, 139
 - clase, 163, 170
- atributo de clase, 163, 170
- atributos, 132
- AttributeError, 230
- barajar, 167
- base de conocimiento, 221
- bifurcación condicional, 37
- bloque, 37, 45
- booleana
 - expresión, 36
- booleanas
 - funciones, 52
- borrado
 - lista, 91
- borrado en una lista, 91
- bucle, 63, 73
 - anidado, 165
 - condición, 228
 - cuerpo, 63, 73
 - en una lista, 187
 - for, 76
 - infinito, 63, 73, 228
 - recorrido, 76

- variable de, 73
- while, 62
- bucle for, 76, 88
- bucle infinito, 227, 228
- buffer circular, 210
- código de objeto, 10
- código ejecutable, 10
- código fuente, 10
- código muerto, 48, 59
- cadena, 11, 12
 - inmutable, 79
 - longitud, 76
 - porción, 78
- cadena de formato, 123, 128
- cadena inmutable, 79
- caja, 114
- caja de función, 114
- carácter, 75
- carga, 183, 192, 211
- Carta, 161
- caso base, 43, 45
- cifrar, 162
- clase, 131, 139
 - Carta, 161
 - Golfista, 209
 - JuegoDeLaMona, 177
 - ListaEnlazada, 190
 - ManoDeLaMona, 176
 - Nodo, 183
 - padre, 172, 175
 - Pila, 196
 - Punto, 155
- clase abstracta, 210
- clase hija, 171, 182
- clase padre, 171, 172, 175, 182
- clase Punto, 155
- clasificación
 - carácter, 82
- clasificación de caracteres, 82
- clave, 109, 117
- cliente, 196, 201
- clonado, 93, 112
- clonar, 97
- codificar, 162, 170
- coerción, 33
 - tipo, 115
- coerción de tipos, 22
- coercion, 23
- coincidencia de esquemas, 107
- cola, 203, 210
 - implementación con Lista, 203
 - implementación enlazada, 204
 - implementación mejorada, 205
- cola enlazada, 204, 210
- cola mejorada, 205
- cola priorizada, 203, 210
 - TAD, 207
- colección, 185, 196
- columna, 96
- coma flotante, 20, 131
- comentario, 20
- comentarios, 19
- comparable, 164
- comparación
 - cadena, 78
 - fracción, 239
- comparación de cadenas, 78
- compilador, 225
- compilar, 2, 10
- composición, 18, 20, 24, 51, 161, 165
- compresión, 116
- comprobación de errores, 57
- concatenación, 77, 80
- concatenar, 20
- concatenation, 18
- condición, 63
- condición, 45, 228
- condición previa, 188, 192
- condiciones
 - encadenadas, 38
- condiciones encadenadas, 38
- constructor, 131, 139, 162
- contador, 80, 84
- conteo, 103

- ul style="list-style-type: none;">
- conversión de tipo, 33
- conversión de tipos, 22
- copia profunda, 139
- copia superficial, 139
- copiado, 112, 137
- corresponder, 162
- cuelgue, 227
- cuerpo, 37, 45
 - bucle, 63
- cursor, 73
- decrementar, 84
- definición
 - circular, 53
 - recursiva, 219
- definición circular, 53
- definición de función, 25, 33
- definición recursiva, 219
- delimitador, 97, 125, 199, 201
- denominador, 235
- depuración, 10, 225
- depuración (debugging), 4
- desarrollo
 - incremental, 148
 - planificado, 148
- desarrollo de progamas
 - encapsulado, 67
- desarrollo de programas, 73
 - generalización, 67
- desarrollo de prototipos, 145
- desarrollo incremental, 49, 59, 148
- desarrollo incremental de progra-
 - mas, 226
- desarrollo planificado, 148
- desbordamiento, 115
- determinista, 107
- diagrama de estado, 13, 20
- diagrama de pila, 33
- diagramas de pila, 30, 42
- diccionario, 96, 109, 117, 124, 230
 - métodos, 111
 - operaciones, 110
- diccionarios, 109
 - métodos, 111
 - operaciones sobre, 110
- directorio, 125, 128
- diseño orientado a objetos, 171
- división de enteros, 16, 20, 22
- documentación, 192
- Doyle, Arthur Conan, 6
- ejecución
 - flujo, 229
- ejecución condicional, 37
- elemento, 85, 97
- eliminación de naipes, 168
- encapsulación, 136, 195, 200
- encapsulado, 67
- encapsular, 73
- enchapado, 197
- encolamiento priorizado, 203
- encurtido, 125, 128
- enlace, 192
- enteros
 - división, 22
 - largos, 115
- enteros largos, 115
- envoltorio, 192
- error
 - de sintaxis, 225
 - en tiempo de compilación, 225
 - en tiempo de ejecución, 43, 225
 - semántico, 225, 231
 - sintaxis, 5
 - tiempo de ejecución, 5
- error (bug), 4
- error de sintaxis, 225
- error en tiempo de compilación, 225
- error en tiempo de ejecución, 5, 10,
 - 43, 76, 79, 87, 100, 111,
 - 113, 115, 120, 124, 225,
 - 229
- error semántico, 5, 10, 101, 225, 231
- error sintáctico, 5, 10
- error(bug), 10
- escalera de color, 174

- escape
 - secuencia de, 73
- espacio en blanco, 84
- estilo funcional de programación,
 - 144, 148
- estructura anidada, 161
- estructura de datos
 - genérica, 196, 197
 - recursiva, 183, 192, 212
- estructura de datos recursiva, 212
- estructura genérica de datos, 196,
 - 197
- estructura recursiva de datos, 183,
 - 192
- Euclides, 238
- excepción, 5, 10, 126, 128, 225, 229
- expresión, 16, 20, 198
 - booleana, 45
 - grande y peliaguda, 232
- expresión booleana, 36, 45
- expresión regular, 199
- Fibonacci function, 56
- FIFO, 203, 210
- fila, 96
- float, 12
- flujo de ejecución, 27, 33, 229
- formal
 - lenguaje, 6
- forzado de tipo de datos, 115
- fracción, 235
 - comparación, 239
 - multiplicación, 236
 - suma, 237
- frangoso, 53
- función, 71
- función join, 96
- función split, 96
- función, 25, 33, 141, 150
 - booleana, 169
 - definición, 25
 - factorial, 54
- función booleana, 169
- función de Fibonacci, 113
- función factorial, 54, 57
- función gamma, 57
- función pura, 142, 148
- funciones
 - argumentos, 28
 - composición, 24, 51
 - llamadas, 21
 - parámetros, 28
 - recursivas, 42
 - tuplas como valor de retorno,
 - 101
- funciones booleanas, 52
- funciones matemáticas, 23
- generalización, 67
- generalización, 136, 146
- generalizar, 73
- Golfista, 209
- gráfico de llamadas, 114
- guardián, 59
- guión, 10
- guión bajo, 14
- herencia, 171, 182
- histograma, 106, 107, 116
- Holmes, Sherlock, 6
- identidad, 134
- igualdad, 134
- igualdad profunda, 134, 139
- igualdad superficial, 134, 139
- implementación
 - Cola, 203
- imponer, 160
- impresión
 - mano de cartas, 174
 - objeto mazo, 166
- imprimir
 - objeto, 133, 150
- incrementar, 84
- IndexError, 230
- indicador, 44, 45

- ul style="list-style-type: none; padding-left: 0;">
- infixo, 198, 201
- infinito
 - bucle, 63
- infio, 213
- inmutable, 99
- instancia, 133, 136, 139
 - objeto, 132, 150, 164
- instancia de objeto, 164
- instancia de un objeto, 132, 150
- instanciación, 132
- instanciar, 139
- instrucción, 4
- int, 12
- Intel, 64
- intercambio, 168
- interfaz, 196, 210
- interpretar, 2, 10
- invariante, 191, 192
- invocar, 117
- invocar métodos, 111
- irracional, 240
- iteración, 61, 62, 73
-
- juego
 - animales, 221
- juego de los animales, 221
-
- KeyError, 230
-
- lógico
 - operador, 36
- lanzar una excepción, 126, 128
- lenguaje, 134
 - alto nivel, 2
 - bajo nivel, 2
 - completo, 53
 - programación, 1
- lenguaje completo, 53
- lenguaje de alto nivel, 2, 10
- lenguaje de bajo nivel, 2, 10
- lenguaje de programación, 1
- lenguaje de programación orientado
 - a objetos, 149, 160
 - lenguaje formal, 6, 10
 - lenguaje natural, 6, 10, 134
 - lenguaje seguro, 5
- Linux, 6
- lista, 85, 97, 183
 - anidada, 95, 112
 - bien construida, 191
 - bucle, 187
 - bucle for, 88
 - como parámetro, 185
 - de objetos, 165
 - elemento, 86
 - enlazada, 183, 192
 - impresión, 185
 - imprimir hacia atrás, 186
 - infinita, 187
 - longitud, 87
 - modificar, 189
 - mutable, 90
 - pertenencia, 88
 - porciones, 90
 - recorrer recursivamente, 186
 - recorrido, 87, 185
- lista anidada, 97, 112
- lista enlazada, 183, 192
- lista infinita, 187
- ListaEnlazada, 190
- listas
 - anidadas, 85, 95
 - clonado, 93
 - como parámetros, 94
 - operaciones con, 89
- listas anidadas, 95
- literalidad, 7
- llamada a función, 33
- llamadas a funciones, 21
- local
 - variable, 69
- locales
 - variables, 29
- logaritmo, 64
- longitud, 87

- lowercase, 82
- máximo común divisor, 238, 241
- método, 111, 117, 141, 150, 160
 - ayudante, 190
 - envoltorio, 190
 - inicialización, 154, 165
 - invocación, 111
 - lista, 116, 166
- método append, 166
- método ayudante, 190, 192
- método de inicialización, 154, 160, 165
- método de lista, 116
- método envoltorio, 190
- métodos de desarrollo
 - incremental, 49
- métodos de lista, 166
- métodos sobre diccionarios, 111
- módulo, 23, 33, 81
 - copy, 137
 - operador, 35
 - string, 83
- módulo copy, 137
- módulo string, 81, 83
- múltiple
 - asignación, 73
- manejar errores, 220
- manejar una excepción, 126, 128
- manejo de errores, 220
- marco, 42
- marco de función, 42
- matemáticas
 - funciones, 23
- matriz, 95
 - dispersa, 112
- mazo, 165
- McCloskey, Robert, 77
- mensajes de error, 225
- mismidad, 134
- modelo
 - mental, 232
- modelo mental, 232
- modificador, 148
- modificador, 143
- modificar listas, 189
- multiplicación
 - fracción, 236
- multiplicación escalar, 156, 160
- mutable, 79, 84, 99
 - lista, 90
- número
 - aleatorio, 101
- número aleatorio, 101
- NameError, 229
- natural
 - lenguaje, 6
- negación, 240
- negación unitaria, 241
- nivel, 211, 224
- nodo, 183, 192, 211, 224
- Nodo clase, 183
- nodo de un árbol, 211
- nodo hermano, 224
- nodo hijo, 211, 224
- nodo hoja, 211, 224
- nodo padre, 211, 224
- nodo raíz, 211, 224
- None, 48, 59
- notación de punto, 111, 151, 154
- nueva línea, 73
- numerador, 235
- objeto, 91, 97, 131, 139
 - mudable, 136
- objeto invariante, 191
- objeto mudable, 136
- objetos
 - lista de, 165
- obstáculo al rendiminto, 210
- operaciones
 - con listas, 89
- operaciones con listas, 89
- operaciones sobre cadenas, 17
- operador, 16, 20

- binario, 213, 224
- condicional, 164
- corchete, 75
- formato, 123, 128, 209, 230
- in, 88, 169
- módulo, 173
- sobrecarga, 156, 236
- operador binario, 213, 224
- operador condicional, 164
- operador corchete, 75
- operador de formato, 123, 128, 209, 230
- operador in, 88, 169
- operador lógico, 36
- operador módulo, 35, 45, 173
- operador matemático, 236
- operador unitario, 240
- operadores
 - para listas, 89
- operando, 16, 20
- orden, 164
- orden completo, 164
- orden de evaluación, 232
- orden de las operaciones, 17
- orden infijo, 214, 215, 224
- orden parcial, 164
- orden postfijo, 214, 215, 224
- orden prefijo, 214, 224
- palabra reservada, 13, 20
- palabras reservadas, 14
- palo, 161
- papel
 - variable, 188
- par clave-valor, 109, 117
- parámetros, 28
- parámetro, 33, 94, 133
 - lista, 94
- pass
 - sentencia, 37
- patrón, 80
- patrón computacional, 80
- Pentium, 64
- Pila, 196
- pila, 196
- pista, 113, 117
- plan de desarrollo, 73
- poesía, 8
- polimórfica, 160
- polimorfismo, 158
- pop, 197
- porción, 78, 84
- porciones, 90
- portabilidad, 10
- portable, 2
- postfijo, 198, 201, 213
- precedencia, 20, 232
- prefijo, 214, 224
- print
 - sentencia, 9, 10
- prioridad, 209
- producto, 219
- producto interior, 156, 160
- programa, 10
 - desarrollo de, 73
- programación orientada a objetos, 149, 171
- prosa, 8
- proveedor, 196, 201
- pseudoaleatorio, 107
- pseudocódigo, 238
- push, 197
- racional, 235
- rama, 38, 45
- random, 167
- randrange, 167
- recorrer, 84, 185, 186, 213
- recorrido, 76, 80, 88, 176, 208, 214
 - lista, 87
- recorrido de lista, 97
- recorrido eureka, 80
- rectángulo, 135
- recuento, 116
- recursión, 213, 214
 - infinita, 228

- recursión infinita, 227, 228
- recursividad, 40, 42, 45, 53, 55
 - caso base, 43
 - infinita, 43, 57
- recursividad infinita, 43, 45, 57
- redimiento, 205
- reducir, 238, 241
- redundancia, 7
- referencia, 183
 - incrustada, 183, 192
- referencia empotrada, 211
- referencia incrustada, 183, 192
- referencias
 - alias, 92
- reglas de precedencia, 17, 20
- reparto de naipes, 173
- repetición
 - lista, 89
- runtime error, 76
- ruta, 125
- secuencia, 97
- secuencia aritmética, 66
- secuencia de escape, 66, 73
- secuencia geométrica, 66
- seguro
 - lenguaje, 5
- semántica, 5, 10
 - error, 5
- sentencia, 20
 - asignación, 12, 61
 - break, 121, 128
 - condicional, 45
 - continue, 122, 128
 - except, 126
 - print, 230
 - return, 40, 233
 - try, 126
 - while, 62
- sentencia break, 121, 128
- sentencia compuesta, 45
- sentencia condicional, 45
- sentencia continue, 122, 128
- sentencia except, 126, 128
- sentencia pass, 37
- sentencia print, 9, 10, 230
- sentencia return, 40, 233
- sentencia try, 126
- sentencias
 - bloque, 37
 - compuestas, 37
- sentencias compuestas, 37
 - bloque de sentencias, 37
 - cabecera, 37
 - cuerpo, 37
- sequencia, 85
- singleton, 189, 190, 192
- sintaxis, 5, 10, 226
- sobrecarga, 160, 236
 - operador, 209
- sobrecarga de operadores, 156, 160, 164, 209
- solución de problemas, 10
- subclase, 171, 175, 182
- subexpresión, 220
- suma, 219
 - fracción, 237
- sustituir, 164
- táctica de encolamiento, 203, 210
- tablas, 64
 - dos dimensiones, 66
- tabulador, 73
- TAD, 195, 200, 201
 - Cola, 203
 - cola, 203
 - Cola Priorizada, 203, 207
 - Pila, 196
- TAD Cola, 203
- temporal
 - variable, 59
- temporales
 - variables, 48
- teorema
 - fundamental de la ambigüedad, 188

- ul style="list-style-type: none; padding-left: 0;">
- teorema fundamental de la ambigüedad, 192
- tesis de Turing, 53
- tiempo constante, 205, 210
- tiempo lineal, 205, 210
- tipo, 11, 12, 20
 - cadena, 12
 - float, 12
 - int, 12
- tipo abstracto de datos |verTAD, 195
- tipo compuesto de datos, 84
- tipo de datos
 - compuesto, 75, 131
 - definido por el usuario, 131, 235
 - diccionario, 109
- tipo de datos compuesto, 75, 131
- tipo de datos definido por el usuario, 131
- tipo de función
 - modificador, 143
 - pura, 142
- tipo inmutable, 107
- tipo mutable, 107
- tipos
 - coerción, 22
 - comprobación, 57
 - conversión, 22
- tipos de datos
 - enteros largos, 115
 - inmutables, 99
 - tuplas, 99
- token, 199, 201, 216
- traceback, 31
- traza, 127
- traza inversa, 43, 229
- try, 128
- tupla, 99, 107
- tuplas, 101
- Turing, Alan, 53
- TypeError, 229
- unidad, 10
- uppercase, 82
- uso de alias, 137
- valor, 20, 91, 161
- valor de retorno, 21, 33, 47, 59, 136
- valores
 - tuplas, 101
- valores de retorno
 - tuplas, 101
- value, 11
- variable, 12, 20
 - bucle, 173
 - papeles, 188
 - temporal, 59, 232
- variable de bucle, 73, 173, 185
- variable local, 33
- variable temporal, 232
- variables
 - locales, 69
- variables locales, 29
- variables temporales, 48
- while
 - sentencia, 62
- whitespace, 82

